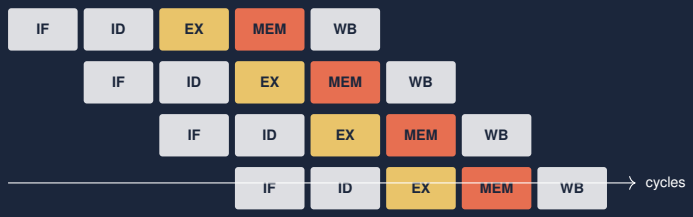
BUET • CSE 209

Computer Architecture

Topic-wise questions with complete worked solutions



12 topics

342 question cards

16 papers & class tests

2 course codes

CSE 209 term finals (2022–2023, 2023–2024) and class tests CT-3, CT-4, plus the CSE 305 (former course code) term finals, sessions 2011–2012 to 2022–2023.

Every question is re-typeset and followed by its solution. A question repeated word for word in several papers (or a theory question with the same answer) is stacked and solved once; numerical variants with different data are solved separately. Datapaths, pipelines, caches, flowcharts and FSMs are drawn in Ti*k*Z. Topics follow the course slides (ART and RS); questions on material outside the slides are kept in separate, flagged topics.

MAHDI HASAN

Credit: Claude Opus 5.5

Contents

01 Computer Abstractions and Performance . . . 1
02 Instructions: Language of the Computer . . . 22
03 Arithmetic Logic Unit (ALU) Design . . . 66
04 Arithmetic for Computers . . . 77
05 The Processor: Datapath and Control . . . 109
06 The Processor: Pipelining, Hazards and ILP . . . 131
07 Memory Hierarchy: Caches, Virtual Memory and Storage . . . 180
08 Graphics and Computing GPUs . . . 233
09 CPU Organization, Control Unit Design and Exceptions NOT IN SLIDES . . . 236
10 Multiprocessors and Parallel Architectures NOT IN SLIDES . . . 247
11 Memory Systems, Storage, I/O and Buses NOT IN SLIDES . . . 262
12 Performance Benchmarks: SPEC and FLOPS NOT IN SLIDES . . . 272

How to read this book

2099–2100 • Q0(a) Course • exam date • unit ID MARKS 5

A question card. The left part of the title gives the session and the original question number; the grey part gives the course code, the exam date and the unit ID. The coral badge shows the printed marks. Figures, tables and code from the paper are redrawn inside the card.

SOLUTION

The solution panel always follows its question. Key results are highlighted:

Answer. like this.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

Paper A Q1 MARKS 8

A question that was asked again…

Paper B Q4 MARKS 10

…and again…

Paper C Q2 MARKS 6

…is stacked, and solved *once* below.

SOLUTION

One shared solution covers every stacked card. Numerical / problem questions are stacked only when they are exactly the same (same data, same answer); a variant with different numbers, code or tables is a separate question with its own solution. Theory questions are stacked when their answers are the same; if one card asks for a little extra, that part is answered and labelled.

2011–2012 • Q0 NOT IN SLIDES Course • date • ID MARKS 5

A gold *NOT IN SLIDES* tag marks a question whose material is not covered by the course slides. All such questions are collected in Topics 09–12.

NOTE Gold boxes flag an issue in the source scan (a faint figure, a missing value, a misprint) and state which reading or assumption the solution uses.

Ordering

- Inside each topic the most recent session comes first. Class tests (CT-3, CT-4) are placed first in their topics.
- For the same session, the lower course number comes first (CSE 209 before CSE 305). Inside a stack

the cards follow the same rule.

Conventions

- MIPS-32 as in Patterson & Hennessy (the course slides): registers `$zero`, `$t0`–`$t9`, `$s0`–`$s7`, `$a0`–`$a3`, `$v0`–`$v1`, `$sp`, `$ra`; word = 4 bytes; byte addressing.
- Pipeline diagrams use the 5-stage pipeline IF ID EX MEM WB; stall is a bubble and flush a flushed slot. Unless a question says otherwise, n instructions with no hazards take $5 + (n - 1)$ cycles.
- In cache tables ✓ = hit and × = miss; addresses are byte addresses unless stated as block addresses. $1\,\mathrm{K} = 2^{10}$, $1\,\mathrm{M} = 2^{20}$, $1\,\mathrm{G} = 2^{30}$ for memory sizes.
- Added or modified hardware in datapath figures is drawn in **coral**.

01

Computer Abstractions and Performance

2023–2024 • Q6(a) CSE 209 • 29/04/2026 • PF01 **MARKS 13**

Our favorite program runs in 10 seconds on computer A, which has a 4 GHz clock. We are trying to help a computer designer build a computer, B, which will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program. What clock rate should we tell the designer to target?

SOLUTION

$$\text{CPU time} = \frac{\text{Clock cycles}}{\text{Clock rate}} \Rightarrow \text{Clock cycles} = \text{CPU time} \times \text{Clock rate}.$$

Cycles on A: $\text{Cycles}_A = 10\text{ s} \times 4 \times 10^9\text{ Hz} = 40 \times 10^9$ cycles.

Cycles on B: $\text{Cycles}_B = 1.2 \times 40 \times 10^9 = 48 \times 10^9$ cycles.

Required clock rate of B:

$$\text{Clock rate}_B = \frac{\text{Cycles}_B}{\text{CPU time}_B} = \frac{48 \times 10^9}{6\text{ s}} = 8 \times 10^9\text{ Hz}.$$

Answer. Computer B must be clocked at **8 GHz** (twice A's clock rate). That buys a $10/6 \approx 1.67\times$ speedup, even though B needs 20% more cycles.

2022–2023 • Q6(a) CSE 305 • 25/04/2024 • PF02 **MARKS 5**

Joey claims that the number of cycles is identical to the number of instructions. Do you agree with Joey's statement? Justify your answer.

SOLUTION

Disagree. The two are related through the CPI:

$$\text{Clock cycles} = \sum_i \text{IC}_i \times \text{CPI}_i = \text{IC} \times \overline{\text{CPI}}.$$

- Cycles equal instructions only when the average CPI is exactly 1, i.e. every instruction takes exactly one cycle.
- In real processors, instruction classes take different numbers of cycles: multiply/divide, loads that miss in the cache, floating-point operations, and so on. A pipeline also loses cycles to stalls. So the average CPI is usually $\neq 1$. Superscalar (multiple-issue) machines can even reach CPI < 1 (IPC > 1).
- Slide example (sequence 1): IC $= 5$ instructions, but cycles $= 2 \times 1 + 1 \times 2 + 2 \times 3 = 10$, so CPI $= 2.0$.

Answer. No. Cycles $=$ IC $\times$ CPI, so they are equal only in the special case CPI $= 1$.

2022–2023 • Q6(d) CSE 305 • 25/04/2024 • PF03 **MARKS 5 + 5 + 5**

Consider two different implementations (P1 and P2) of the same instruction-set architecture. The instructions can be divided into four classes according to their CPI (class A, B, C, and D). P1 has a clock rate of 2.7 GHz and CPIs of 2, 1, 3 and 2, and P2 has a clock rate of 3.2 GHz and CPIs of 2, 3, 2, and 3.
Leonard develops a program with a dynamic instruction count of 1.2E6 instructions, which is divided into the following classes: 20% class A, 25% class B, 40% class C, and 15% class D.

(i) Find the number of clock cycles required in both cases.

(ii) Which implementation is faster and how much faster compared to the other one?

(iii) If Sheldon has improved 40 percent of the code for the program, determine the maximum possible improvement factor Sheldon can obtain for that program.

SOLUTION

Method. $\overline{\text{CPI}} = \sum_i f_i \,\text{CPI}_i$, $\text{Cycles} = \text{IC} \times \overline{\text{CPI}}$, $T = \text{Cycles}/\text{Clock rate}$. "X is n times faster than Y" means $n = T_Y / T_X$.
Instruction counts: A $= 0.24$ M, B $= 0.30$ M, C $= 0.48$ M, D $= 0.18$ M.
(i) $\overline{\text{CPI}}_{P1} = 0.2(2) + 0.25(1) + 0.4(3) + 0.15(2) = 2.15$, so $\text{Cycles}_{P1} = 2.15 \times 1.2 \times 10^6 = 2.58 \times 10^6$.
$\overline{\text{CPI}}_{P2} = 0.2(2) + 0.25(3) + 0.4(2) + 0.15(3) = 2.40$, so $\text{Cycles}_{P2} = 2.40 \times 1.2 \times 10^6 = 2.88 \times 10^6$.
(ii) $T_{P1} = \dfrac{2.58 \times 10^6}{2.7 \times 10^9} = 0.956\,\text{ms}$; $T_{P2} = \dfrac{2.88 \times 10^6}{3.2 \times 10^9} = 0.900\,\text{ms}$; $\dfrac{T_{P1}}{T_{P2}} = \dfrac{0.956}{0.900} = 1.06$.
(iii) Amdahl's law with an improved fraction $F = 0.4$ and an unbounded speedup $S \to \infty$ on that part:

$$\text{Speedup}_{\max} = \lim_{S\to\infty} \frac{1}{(1-F) + F/S} = \frac{1}{1-0.4} = 1.67.$$

Answer. (i) P1: 2.58×10^6 cycles; P2: 2.88×10^6 cycles. (ii) P2 is faster, by $\approx 1.06\times$ (6%). (iii) At most $1/(1-0.4) \approx \mathbf{1.67}\times$, however much the 40% is improved.

> **NOTE** In (iii), "improved 40 percent of the code" is read as 40% of the program's execution time, as Amdahl's law requires. The untouched 60% bounds the overall gain.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q7(a) CSE 305 • 25/04/2024 • PF04 **MARKS 5**

Define response time and throughput. Are these two terms synonymous?

2013–2014 • Q5(a) CSE 305 • 05/07/2015 • PF04 **MARKS 6**

What is Moore's Law? What do you mean by response time and throughput?

SOLUTION

Response time vs. throughput

- **Response time** (execution time, latency): how long it takes to do *one* task, from start to completion. This includes disk and memory accesses, I/O, OS overhead and so on. A user cares about this.

- **Throughput** (bandwidth): the total work done per unit time, e.g. tasks, transactions or jobs per hour. A data-centre manager cares about this.

Not synonymous. They measure different things and do not always move together:

- Replacing the processor with a faster one improves *both*: each task finishes sooner, so more tasks finish per hour.
- Adding more processors (each task still runs on one processor) improves *throughput* only. Response time drops only if tasks had been queueing.
- Pipelining raises instruction throughput, but the latency of an individual instruction does not fall (it may even rise slightly).

Performance is defined from response time: Performance $= 1/\text{Execution time}$.

Moore's Law (2013–2014 card)

Gordon Moore (1965) observed that the number of transistors on an integrated circuit **doubles roughly every 18–24 months**. Integrated-circuit resources therefore grow exponentially. This gives cheaper, faster and more capable chips, and it underlies the computer revolution. Because a design may take years to finish, architects must "design for Moore's Law": they target the technology expected at completion, not at the start.

Answer. Response time is the latency of one job; throughput is jobs per unit time. They are **not** synonyms: a change can improve one without the other. Moore's Law: transistor count per chip doubles about every 1.5–2 years.

2021–2022 • Q5(b) CSE 305 • 06/04/2023 • PF05 **MARKS 4 + 6**

Assume that 2% of the runtime of a program is not parallelizable. This program runs on 24 cores of an Intel Xeon Gold processor. Under the assumption that the program runs at the same speed on all of those cores and there are no additional overheads,

(i) What is the parallel speedup for 24 cores?

(ii) Is it possible to make the program 60 times faster? If yes, how can that be achieved? If not, mention the reasons.

SOLUTION

(i) Amdahl's law

With serial fraction $s = 0.02$, parallel fraction $1 - s = 0.98$ and $N = 24$ cores:

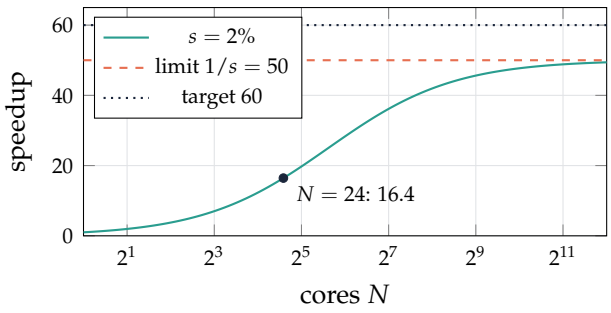
$$\text{Speedup} = \frac{1}{s + \dfrac{1-s}{N}} = \frac{1}{0.02 + \dfrac{0.98}{24}} = \frac{1}{0.02 + 0.04083} = \frac{1}{0.06083} = 16.44.$$

Answer. Speedup on 24 cores $\approx \mathbf{16.4}\times$ (only 68% of the ideal $24\times$).

(ii) Can it be $60\times$ faster?

Not as specified. The serial 2% cannot be sped up by adding cores, so even with infinitely many cores

$$\text{Speedup}_{\max} = \lim_{N\to\infty} \frac{1}{0.02 + 0.98/N} = \frac{1}{0.02} = 50 < 60.$$



To reach 60× the serial part itself must change:

- **Shrink the serial fraction** below $1/60 = 1.67\%$ (better algorithm, parallelise more of the code). For example, with $s = 1\%$: $0.01 + 0.99/N = 1/60 \Rightarrow N \approx 149$ cores.
- **Speed up the serial part** with faster single-thread hardware (a higher clock or better microarchitecture). This breaks the "same speed on all cores" assumption.
- Add far more cores, beyond 24 (needed in either case).

Answer. No. With 2% serial code the speedup can never exceed $1/0.02 = 50\times$ (Amdahl's law). $60\times$ needs the serial fraction reduced below 1.67%, plus well over 24 cores.

2021–2022 • Q8(a) CSE 305 • 06/04/2023 • PF06 **MARKS 3 + 3 + 5**

Consider two different implementations (P1 and P2) of the same instruction set architecture. There are three classes of instructions A, B, and C. Suppose, a program has 200,000 instructions divided into three classes as 25% class A, 60% class B, and 15% class C. The clock rate and CPI of each implementation are given in the following table (Table: 8.a).

Implementation	Clock Rate	CPI of Class A	CPI of Class B	CPI of Class C
P1	2.7 GHz	3	3	2
P2	3.5 GHz	2	4	1

Table: 8.a

(i) Find the total clock cycle required in both implementations.

(ii) Calculate the execution time required for both implementations and determine which implementation is faster.

(iii) How much must we improve the CPI of Class C instructions if we want the program to run two times faster in the better implementation determined in the previous question?

SOLUTION

Method. Cycles $= \sum_i \text{IC}_i \text{CPI}_i$, $T = \text{Cycles}/\text{Clock rate}$. For a target time, solve $\text{Cycles}_{\text{target}} = \text{Cycles}_{\text{other classes}} + \text{IC}_C \cdot \text{CPI}_C^{\text{new}}$.
Counts: A = 50,000, B = 120,000, C = 30,000.
(i) $\text{Cycles}_{P1} = 50\text{k}(3) + 120\text{k}(3) + 30\text{k}(2) = 150\text{k} + 360\text{k} + 60\text{k} = 570{,}000$.
$\text{Cycles}_{P2} = 50\text{k}(2) + 120\text{k}(4) + 30\text{k}(1) = 100\text{k} + 480\text{k} + 30\text{k} = 610{,}000$.
(ii) $T_{P1} = 570{,}000/2.7 \times 10^9 = 211.1\,\mu\text{s}$; $T_{P2} = 610{,}000/3.5 \times 10^9 = 174.3\,\mu\text{s}$. So P2 is faster, by $211.1/174.3 = 1.21\times$.

(iii) In P2, twice as fast needs $T = 87.14\,\mu$s, i.e. 305,000 cycles. The class A and B cycles alone are $100\text{k} + 480\text{k} = 580{,}000 > 305{,}000$:

$$580{,}000 + 30{,}000 \cdot \text{CPI}_C^{\text{new}} = 305{,}000 \;\Rightarrow\; \text{CPI}_C^{\text{new}} = -9.17 \quad \text{(impossible)}.$$

Even with $\text{CPI}_C = 0$, the speedup is only $610/580 = 1.05\times$ (Amdahl's law).

Answer. (i) 570,000 and 610,000 cycles. (ii) 211.1 μs vs. 174.3 μs, so **P2** is faster (1.21×). (iii) **Not possible.** Class C contributes only 30,000 of P2's 610,000 cycles.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2020–2021 • Q1 CSE 305 • 24/09/2022 • PF07 **MARKS 15**

You have surely learnt about the eight big ideas of computer architecture. They are again mentioned below to brush up your memory:

- Design for Moore's Law
- Use abstraction to simplify design
- Make the common case fast
- Performance via parallelism
- Performance via pipelining
- Performance via prediction
- Hierarchy of memories
- Dependability via redundancy

Now identify the statements below best relate to which of the above ideas. Each statement should relate to a single idea. Elaborate the justification behind your answer.

(a) Code reordering is done to avoid the use of load result in the next instruction.

(b) Different caches for data and memory can effectively increase bandwidth.

(c) As CPUs have hit their physical performance barrier due to power limit issues, GPUs have started gaining increased popularity among the programming community.

2020–2021 • Q10(a) CSE 305 • 24/09/2022 • PF07 **MARKS 12**

Briefly explain the eight great ideas in Computer Architecture.

2016–2017 • Q5(a) CSE 305 • 02/08/2017 • PF07 **MARKS 10**

"Computer architectures have invented several great ideas in the last 60 years of computer design. These ideas are so powerful that they have lasted long after the first computer that used them, with newer architects demonstrating their admiration by imitating their predecessors." These ideas include: (i) Design for Moore's Law (ii) Use abstraction to simplify design (iii) Make the common case faster (iv) Performance via parallelism (v) Performance via pipelining (vi) Performance via prediction (vii) Hierarchy of memories (viii) Dependability via redundancy. You are given several design features of MIPS and other modern computing systems in the following. Now, please **identify** that which of the above great ideas are behind these design

decisions.

(i) While implementing instructions at hardware level, it is possible to implement all the instructions as single cycle instructions. However, modern designers do not use this single-cycle design as it is inefficient. (hint: for single cycle implementation, the clock cycle must have the same length for every instruction and the longest instruction determine the clock cycle length.)

(ii) Modern computers use a technique called prefetching. "In prefetching, a block of data is brought into the cache before it is actually referenced. Many microprocessors use hardware perfectching to access the block needed in the future that may be difficult for software to notice".

2014–2015 • Q5(a) CSE 305 • 26/01/2016 • PF07 MARKS 20

"Computer architects have invented several great ideas in the last 60 years of computer design. These ideas are so powerful that they have lasted long after the first computer that used them, with newer architects demonstrating their admiration by imitating their predecessors". These ideas include: 1. Design for Moore's Law 2. Use abstraction to simplify design 3. Make the common case faster 4. Performance via parallelism 5. Performance via pipelining 6. Performance via prediction 7. Hierarchy of memories 8. Dependability via redundancy.
You are given several design features of MIPS and other modern computing systems in the following. Now, please **identify** that which of the above great ideas are behind these design decisions.

1. In MIPS datapath, there is an adder which computes the value of **(PC+4)**, where PC means Program Counter. For branch addressing the MIPS address is actually relative to this **(PC+4)** rather than **(PC)**. This form of branch addressing is called PC-relative addressing.
2. In the past, most PCs (personal computers) and server systems used separate SRAM (static random access memory) chips for either their primary, secondary, or even tertiary caches. Today, all levels of caches are integrated onto the processor chip, so the market for separate SRAM chips has nearly evaporated.
3. A fast multiplication hardware, "unrolls the loop" by using 31 adders rather than using a single 32-bit adder 31 times.
4. Procedures or Functions allow programmers to concentrate on just one portion of the task at a time, parameters act as an interface between the procedure and the rest of the program and data, since they can pass values and return results.
5. Modern computers use a technique called prefetching. In prefetching, a block of data is brought into the cache before it is actually referenced. Many microprocessors use hardware prefetching to access the block needed in the future that may be difficult for software to notice.

SOLUTION

The eight great ideas (answer to 2020–2021 Q10(a); used by every card)

Idea	What it means
Design for Moore's Law	Transistor counts double every 18–24 months. A design takes years, so architects target the technology that will exist when it ships, not when it starts.
Use abstraction to simplify design	Hide lower-level detail behind clean interfaces (ISA, ABI, procedures, OS) so that each level can be designed and understood separately. This raises productivity.
Make the common case fast	Optimise what happens most often; this pays off more than optimising rare cases (Amdahl's law).
Performance via parallelism	Do several operations at the same time: multiple ALUs or adders, multicore, SIMD, multiple issue.
Performance via pipelining	A special form of parallelism. Overlap the stages of successive operations like an assembly line, so throughput rises.
Performance via prediction	It can be faster on average to guess and start working than to wait, provided recovery from a wrong guess is cheap and guesses are usually right (branch prediction, prefetching).
Hierarchy of memories	Fastest, smallest, costliest memory at the top (caches), slower, larger, cheaper memory below (DRAM, disk). This gives the illusion of memory that is both fast and large.
Dependability via redundancy	Include spare or duplicate components so the system keeps working when one fails (RAID, ECC, duplicated servers).

2020–2021 Q1

(a) **Performance via pipelining.** In a pipeline, a loaded value is available only after MEM, so an instruction that uses it immediately causes a load-use hazard and a stall. Reordering independent instructions into the slot after the load keeps the pipeline full (no bubble). This problem exists only because the instructions are pipelined.

(b) **Hierarchy of memories.** Splitting the first cache level into an I-cache and a D-cache refines the top of the memory hierarchy: an instruction fetch and a data access can be served by the fast small memories in the same cycle, which doubles the bandwidth seen by the processor. (The statement means separate caches for data and *instructions*.)

(c) **Performance via parallelism.** The power wall stopped clock-rate growth, so single-core performance stalled. GPUs gain performance by running thousands of simple threads in parallel, which is why programmers moved to them.

2016–2017 Q5(a)

(i) **Performance via pipelining**, motivated by *make the common case fast*. In a single-cycle design the slowest instruction (`lw`) sets the clock period for every instruction, so common fast instructions are slowed down. Splitting execution into short equal stages and overlapping them (pipelining) gives a short clock and a CPI near 1.

(ii) **Performance via prediction**, within the hierarchy of memories. The hardware guesses which block will be needed next and brings it into the cache early. A correct guess turns a miss into a hit; a wrong guess costs only some bandwidth.

2014–2015 Q5(a)

1. **Make the common case fast.** PC+4 has already been computed for the sequential (most common) path, and most branch targets are near the branch. Making the offset relative to PC+4 reuses that adder output and reaches nearby targets with a 16-bit offset.

2. **Design for Moore's Law.** The growing number of transistors per chip let whole cache levels move on die. Designers planned for this transistor budget, and separate SRAM chips disappeared.
3. **Performance via parallelism.** 31 adders work simultaneously (a tree of adders) instead of one adder being reused sequentially 31 times.
4. **Use abstraction to simplify design.** A procedure hides its implementation behind an interface (parameters and return value), so the programmer deals with one piece at a time.
5. **Performance via prediction.** The hardware predicts future accesses and fetches blocks before they are referenced. (The prefetched block then sits in the cache, part of the memory hierarchy.)

Answer. Q1: (a) pipelining, (b) hierarchy of memories, (c) parallelism. 2016–17: (i) pipelining (make the common case fast), (ii) prediction. 2014–15: 1 common case fast, 2 Moore's Law, 3 parallelism, 4 abstraction, 5 prediction.

2020–2021 • Q2 CSE 305 • 24/09/2022 • PF08 **MARKS 15**

What is your most favorite topic learnt from processor, memory hierarchy, and multiprocessors? Describe briefly.

SOLUTION

NOTE This is an open question; any well-explained topic earns credit. Below is a model answer on **pipelining** (from the processor part). A good answer names the topic, explains the core idea, gives a small example or diagram, and mentions its issues or limits.

Favourite topic: pipelining in the MIPS processor

Idea. Instruction execution is split into five stages, each taking one clock cycle: IF (fetch), ID (decode / register read), EX (ALU), MEM (data memory), WB (register write-back). Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) separate the stages, so a new instruction can enter every cycle, like an assembly line.

	1	2	3	4	5	6	7
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB		
`add $t1,$t2,$t3`		IF	ID	EX	MEM	WB	
`sub $t4,$t5,$t6`			IF	ID	EX	MEM	WB

Why I like it.

- Throughput rises without making any single instruction faster. Ideal speedup ≈ number of stages. With stage times 200/100/200/200/100 ps, a single-cycle CPU needs 800 ps per instruction; the pipeline completes one every 200 ps, i.e. 4× for long programs.
- It shows a whole chain of design trade-offs, the **hazards**:
  - *Structural*: two stages need the same hardware. MIPS avoids this with separate I- and D-memory.
  - *Data*: an instruction needs a result that is not yet written back. Fixes: **forwarding** (EX/MEM, MEM/WB → ALU inputs), a hazard-detection unit that stalls one cycle for a load-use hazard, and compiler code scheduling.
  - *Control*: the branch outcome is not known in time. Fixes: move the branch comparison into ID, predict (static, or dynamic with 1-/2-bit predictors), or use delayed branches.

- It extends naturally to deeper pipelines and to multiple issue (superscalar/VLIW), which is the instruction-level parallelism used by modern CPUs.

Takeaway: Time $=$ IC $\times$ CPI $\times$ T_c. Pipelining cuts T_c while keeping CPI close to 1; hazards are what push CPI above 1.

2020–2021 • Q12(a) CSE 305 • 24/09/2022 • PF09 **MARKS 8**

The data for the two programs A and B are given below.

Instruction Type	Clock Cycle per Instruction (CPI)	Instruction Count in Program A	Instruction Count in Program B
Multiplication	3	2	3
Addition	1	1	2
Subtraction	2	4	3
Division	4	2	2

Which program has the higher average clock cycle per instruction?

SOLUTION

Average CPI $= \dfrac{\sum_i \mathrm{IC}_i \times \mathrm{CPI}_i}{\sum_i \mathrm{IC}_i}$.

	CPI	IC_A	cycles_A	IC_B	cycles_B
Multiplication	3	2	6	3	9
Addition	1	1	1	2	2
Subtraction	2	4	8	3	6
Division	4	2	8	2	8
Total		9	23	10	25

$$\overline{\mathrm{CPI}}_A = \frac{23}{9} = 2.56, \qquad \overline{\mathrm{CPI}}_B = \frac{25}{10} = 2.50.$$

Answer. **Program A** has the higher average CPI (2.56 vs. 2.50). Program B still takes more total cycles (25 vs. 23) because it executes more instructions.

2019–2020 • Q1(a) CSE 305 • 18/09/2021 • PF10 **MARKS 15**

Suppose we want to compare the performance of *Computer System A* with *Computer System B*. For this purpose, *Program 1* is run in both the computer systems and corresponding information (logs) are captured. The logs are presented below in Table 1.1 and Table 1.2.

Operation	Required Clock Cycle(s)
Multiplication 1 of Program 1	4
Addition 1 of Program 2	2
I/O overhead time required by OS	5
Multiplication 2 of Program 1	4
Division 1 of Program 1	5
Multiplication 1 of Program 2	4
I/O overhead time required by OS	2
Division 2 of Program 1	5
I/O overhead time required by OS	4
Addition 1 of Program 3	2
Addition 1 of Program 1	2

Table 1.1: Logs from Computer System A

Operation	Required Clock Cycle(s)
Multiplication 1 of Program 4	5
Multiplication 1 of Program 1	5
I/O overhead time required by OS	1
Multiplication 2 of Program 1	5
Division 1 of Program 1	6
Multiplication 2 of Program 4	5
Addition 1 of Program 4	3
I/O overhead time required by OS	3
Addition 2 of Program 4	3
Division 2 of Program 1	6
Addition 1 of Program 1	3

Table 1.2: Logs from Computer System B

If the clock rates of *Computer System A* and *Computer System B* are 2 GHz and 2.5 GHz respectively, then compare their performances based on the execution time of *Program 1*.

SOLUTION

Key idea. The performance of a program is judged by its own **CPU time**. CPU time counts only the cycles spent executing that program's work; it *discounts* OS I/O overhead and other jobs' shares (slide: Measuring Execution Time). So keep only the "of Program 1" rows.

Program 1 operation	System A cycles	System B cycles
Multiplication 1	4	5
Multiplication 2	4	5
Division 1	5	6
Division 2	5	6
Addition 1	2	3
Total	**20**	**25**

$$T_A = \frac{20}{2 \times 10^9} = 10\text{ ns}, \qquad T_B = \frac{25}{2.5 \times 10^9} = 10\text{ ns}, \qquad \frac{\text{Perf}_A}{\text{Perf}_B} = \frac{T_B}{T_A} = 1.$$

Answer. CPU time of Program 1 is **10 ns on both** systems, so A and B have **equal performance** for Program 1. B's higher clock rate (2.5 vs. 2 GHz) is exactly cancelled by its 25% more cycles.

NOTE If one wrongly adds the OS I/O rows (A: $5+2+4=11$, B: $1+3=4$ cycles), the totals become A = 31 cycles = 15.5 ns and B = 29 cycles = 11.6 ns. That measures system/elapsed behaviour, not Program 1's CPU time. Rows of Programs 2, 3 and 4 belong to other jobs and are always excluded.

2019–2020 • Q1(b) CSE 305 • 18/09/2021 • PF11 MARKS 4 + 8

What is the importance of Moore's Law? "To increase the overall performance of a computer by x amount, it is required to identify the most important aspect of the computer and improve its performance by x amount." Do you agree or disagree with the statement? Justify your answer with proper mathematical explanation.

SOLUTION

Importance of Moore's Law

- Transistors per chip double about every 18–24 months. This exponential growth has driven the steady gains in cost and performance, and made new applications possible (mobile devices, the web, genome work).
- It is a **design target**: a processor takes years to design, so architects must plan for the transistor budget available when the product ships ("Design for Moore's Law").
- It turns extra transistors into features: on-chip caches, multiple cores, wider issue.

The statement: disagree (Amdahl's law)

Improving one aspect by a factor x speeds up only the part of the time that aspect uses:

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{x} + T_{\text{unaffected}}, \qquad \text{Speedup} = \frac{1}{(1-F) + F/x}$$

where F is the fraction of time affected. Overall speedup $= x$ only if $F = 1$. For any $F < 1$ the speedup is $< x$, and it can never exceed $1/(1-F)$, however large x is.
Example (slides). Multiply takes 80 s of a 100 s program. To get $5\times$ overall we need 20 s:

$$20 = \frac{80}{n} + 20 \;\Rightarrow\; \frac{80}{n} = 0 \quad \text{(impossible).}$$

Even improving multiply by $5\times$ gives only $100/(16+20) = 2.78\times$, not $5\times$.

Answer. Disagree. Improving one aspect by x gives overall speedup $1/((1-F)+F/x) < x$ unless that aspect is 100% of the time. The right strategy is *make the common case fast* and improve every significant part, not one part alone.

2018–2019 • Q7(b) CSE 305 • 29/10/2019 • PF12 MARKS 20

Assume that a processor with 4 GHz clock rate is executing a program that requires the following instruction types.

Instructions	FP	INT	L/S	BRANCH
Instruction Count ($\times 10^6$)	160	110	10	16
CPI	2	1	5	2

(i) Calculate the execution time of the program.

(ii) If you want the program to run two times faster, then how much should you improve the CPI of **FP** instructions? Explain your finding.

(iii) If you want the program to run two times faster, then how much should you improve the CPI of **L/S** instructions? Explain your finding.

(iv) By how much is the execution time changed if the **CPI** of **INT** and **BRANCH** instructions are increased by 15% and the **CPI** of **FP** and **L/S** instructions are decreased by 20%?

SOLUTION

Method. Cycles $= \sum \text{IC}_i \cdot \text{CPI}_i$, $T = \text{Cycles}/f$. "Two times faster" at the same clock means half the cycles. A class can deliver this only if $\text{Cycles}_{\text{others}} + \text{IC}_k \cdot \text{CPI}_k^{\text{new}} = \frac{1}{2}\text{Cycles}$ has a solution with

$\mathrm{CPI}_k^{\mathrm{new}} > 0$.

	FP	INT	L/S	BR	Total
cycles (10^6)	$160 \times 2 = 320$	$110 \times 1 = 110$	$10 \times 5 = 50$	$16 \times 2 = 32$	**512**

(i) $T = 512 \times 10^6 / 4 \times 10^9 = 0.128\,\mathrm{s}$.
(ii) Target $= 256\,\mathrm{M}$ cycles. Non-FP cycles $= 110 + 50 + 32 = 192\,\mathrm{M}$, so

$$192 + 160 \cdot \mathrm{CPI}_{FP}^{\mathrm{new}} = 256 \;\Rightarrow\; \mathrm{CPI}_{FP}^{\mathrm{new}} = \frac{64}{160} = 0.4.$$

FP is the dominant class (62.5% of the cycles), so improving it can reach the target: its CPI must drop from 2 to 0.4, a $5\times$ **improvement** (80% reduction).
(iii) Non-L/S cycles $= 320 + 110 + 32 = 462\,\mathrm{M} > 256\,\mathrm{M}$:

$$462 + 10 \cdot \mathrm{CPI}_{L/S}^{\mathrm{new}} = 256 \;\Rightarrow\; \mathrm{CPI}_{L/S}^{\mathrm{new}} = -20.6 \quad \text{(impossible)}.$$

L/S is only $50/512 \approx 9.8\%$ of the cycles. Even $\mathrm{CPI}_{L/S} = 0$ gives at most $512/462 = 1.11\times$ (Amdahl's law).
(iv) New CPIs: INT 1.15, BR 2.30, FP 1.6, L/S 4.0.

$$\mathrm{Cycles}' = 160(1.6) + 110(1.15) + 10(4) + 16(2.3) = 256 + 126.5 + 40 + 36.8 = 459.3\,\mathrm{M}$$

$T' = 459.3 \times 10^6 / 4 \times 10^9 = 0.1148\,\mathrm{s}$. Change $= (512 - 459.3)/512 = 10.3\%$ less time (speedup $1.115\times$).

Answer. (i) 0.128 s. (ii) CPI_{FP}: $2 \to 0.4$ ($5\times$ better). (iii) **Impossible** through L/S alone. (iv) Execution time falls by $\approx$10.3% (0.128 s $\to$ 0.1148 s).

2017–2018 • Q5(a) CSE 305 • 23/09/2018 • PF13 **MARKS 8 + 7**

Suppose there are three classes of instructions A, B and C in a particular instruction set architecture with CPIs 1.2, 2 and 2.5, respectively. The number of instructions from each class in two separate programs P1 and P2 are as follows:

Programs	Instruction Classes		
	A	B	C
P1	40	10	16
P2	12	13	40

(i) Which program will execute faster and by how much?

(ii) By how much should we improve the CPI of Instruction Class B to make P1 execute two times faster? Explain your finding.

SOLUTION

Method. Cycles $= \sum_i \mathrm{IC}_i \,\mathrm{CPI}_i$. Both programs run on the same machine (same clock), so execution time $\propto$ cycles.
(i) $\mathrm{Cycles}_{P1} = 40(1.2) + 10(2) + 16(2.5) = 48 + 20 + 40 = 108$; $\mathrm{Cycles}_{P2} = 12(1.2) + 13(2) + 40(2.5) = 14.4 + 26 + 100 = 140.4$.

$140.4/108 = 1.3$.
(ii) For P1 to run twice as fast it needs $108/2 = 54$ cycles. Classes A and C alone need $48 + 40 = 88 > 54$:

$$88 + 10 \cdot \mathrm{CPI}_B^{\text{new}} = 54 \;\Rightarrow\; \mathrm{CPI}_B^{\text{new}} = -3.4 \quad \text{(impossible)}.$$

Class B accounts for only $20/108 = 18.5\%$ of P1's time. By Amdahl's law the best possible gain is $108/88 = 1.23\times$ (with $\mathrm{CPI}_B = 0$).

Answer. (i) **P1** is faster, by **1.3**× (108 vs. 140.4 cycles). (ii) **Impossible**: no CPI_B improvement can give 2×; the maximum is 1.23×.

2016–2017 • Q5(c) CSE 305 • 02/08/2017 • PF14 **MARKS 15**

Assume a processor with 3 GHz clock rate is executing a program that requires following instructions,

Instructions →	**FP**	**INT**	**L/S**	**BRANCH**
Instruction Count ($\times 10^6$)	40	80	90	15
CPI	1	1	3	2

(i) Calculate the execution time of the program.

(ii) By how much must we improve the CPI of **FP** instructions if we want the program to run two times faster?

(iii) By how much is the execution time program changed (improved or degraded) if the **CPI** of **INT** and **FP** instruction is increased by 20% and the **CPI** of **L/S** and Branch is reduced by 10%?

SOLUTION

Method. Cycles $= \sum \mathrm{IC}_i \cdot \mathrm{CPI}_i$, $T = \text{Cycles}/f$. "Two times faster" at the same clock means half the cycles, which must be reachable with $\mathrm{CPI}_{FP}^{\text{new}} > 0$.
(i) Cycles $= 40(1) + 80(1) + 90(3) + 15(2) = 40 + 80 + 270 + 30 = 420$ M, so $T = 420 \times 10^6/3 \times 10^9 = 0.14$ s.
(ii) Target 210 M cycles. Non-FP cycles $= 80 + 270 + 30 = 380\,\text{M} > 210\,\text{M}$, so $\mathrm{CPI}_{FP}^{\text{new}} = (210 - 380)/40 < 0$, which is **impossible**. FP is only $40/420 = 9.5\%$ of the cycles; the best possible is $420/380 = 1.105\times$. (The heavy class here is L/S, 64% of the cycles.)
(iii) New CPIs: FP 1.2, INT 1.2, L/S 2.7, BR 1.8.

$$\text{Cycles}' = 40(1.2) + 80(1.2) + 90(2.7) + 15(1.8) = 48 + 96 + 243 + 27 = 414\ \text{M}, \qquad T' = 0.138\ \text{s}.$$

Answer. (i) 0.14 s. (ii) **Not possible** by improving FP alone. (iii) **Improved** slightly: 0.14 → 0.138 s, i.e. 1.43% less time (speedup 1.014×). The 10% L/S saving outweighs the 20% FP/INT penalty.

2015–2016 • Q1(a) CSE 305 • 18/01/2017 • PF15 **MARKS 10**

Consider three different processors P1, P2 and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.

(i) Which processor has the highest performance expressed in instructions per second?

(ii) If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.

(iii) We are trying to reduce the execution time by 30% but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

SOLUTION

(i) Instructions per second = clock rate / CPI

P1: $3 \times 10^9 / 1.5 = 2.0 \times 10^9$; P2: $2.5 \times 10^9 / 1.0 = 2.5 \times 10^9$; P3: $4 \times 10^9 / 2.2 = 1.82 \times 10^9$.

Answer. **P2** has the highest performance: 2.5×10^9 instructions/s.

(ii) Program runs 10 s: cycles = $T \times f$, instructions = cycles / CPI

	P1	P2	P3
Cycles	3×10^{10}	2.5×10^{10}	4×10^{10}
Instructions	2×10^{10}	2.5×10^{10}	1.82×10^{10}

(iii) New time $0.7T$, new CPI 1.2 CPI, same IC

$$0.7\,\frac{\text{IC}\cdot\text{CPI}}{f} = \frac{\text{IC}\cdot 1.2\,\text{CPI}}{f_{\text{new}}} \;\Rightarrow\; f_{\text{new}} = \frac{1.2}{0.7} f = 1.714\, f.$$

Answer. Required clock rates: P1 3 → **5.14** GHz, P2 2.5 → **4.29** GHz, P3 4 → **6.86** GHz (each 1.714× the old rate).

2015–2016 • Q1(b) CSE 305 • 18/01/2017 • PF16 **MARKS 10**

The following table shows the numbers of instructions for a program.

Arithmetic	Store	Load	Branch
500	50	100	50

The following table shows the number of cycles taken by each instruction.

Arithmetic	Store	Load	Branch
1	5	5	2

(i) What is the execution time of the program in a 4 GHz processor?

(ii) What is the CPI of the program?

(iii) If the number of load instructions can be reduced by half, what is the speed-up and CPI?

SOLUTION

	Arithmetic	Store	Load	Branch	Total
IC	500	50	100	50	700
Cycles	500	250	500	100	1350
IC (loads halved)	500	50	50	50	650
Cycles (loads halved)	500	250	250	100	1100

(i) $T = \dfrac{1350}{4 \times 10^9} = 337.5\,\text{ns}$.

(ii) $\text{CPI} = \dfrac{1350}{700} = 1.93$.

(iii) $T' = 1100/4 \times 10^9 = 275\,\text{ns}$, so Speedup $= \dfrac{1350}{1100} = 1.23$ and $\text{CPI}' = \dfrac{1100}{650} = 1.69$.

Answer. (i) 337.5 ns. (ii) CPI $= 1.93$. (iii) Speedup $= \mathbf{1.23}\times$ and new CPI $= \mathbf{1.69}$. Removing slow 5-cycle loads lowers both the cycle count and the average CPI.

2015–2016 • Q1(c) CSE 305 • 18/01/2017 • PF17 **MARKS 15**

Assume for arithmetic, load/store, and branch instructions, a processor has CPIs of 1, 12, and 5, respectively. Also, assume that on a single processor a program requires the execution of 2.56 billion arithmetic instructions, 1.28 billion load/store instructions, and 256 million branch instructions. Assume that each processor has a 2 GHz clock frequency. Assume that, as the program is parallelized to run over multiple cores, the number of arithmetic and load/store instructions per processor is divided by $0.7 \times p$ (where p is the number of processors) but the number of branch instructions per processor remains the same. (1 billion $= 10^9$, 1 million $= 10^6$).

(i) Find the total execution time for this program on 1, 2, and 4 processors, and show the relative speedup of the 2, and 4 processors compared to the single processor.

(ii) If the CPI of the arithmetic instructions was doubled, what would the impact be on the execution time of the program on 1, 2, and 4 processors?

(iii) How much should the CPI of load/store instructions be reduced for a single processor to match the performance of 4 processors using the original CPI values?

SOLUTION

Model. Cycles per processor:

$$C(p) = \frac{\text{IC}_{ar}\text{CPI}_{ar} + \text{IC}_{ls}\text{CPI}_{ls}}{0.7\,p} + \text{IC}_{br}\text{CPI}_{br} \quad (p \geq 2), \qquad C(1) = \text{undivided sum}.$$

On one processor nothing is divided (the "0.7" factor models parallelisation overhead, which applies only when $p > 1$). This is the standard P&H interpretation. Time $= C(p)/2 \times 10^9$.

(i) Original CPIs (1, 12, 5)

Arithmetic 2.56×10^9 cycles, L/S $1.28 \times 10^9 \times 12 = 15.36 \times 10^9$, branch $0.256 \times 10^9 \times 5 = 1.28 \times 10^9$.

p	cycles per processor	time	speedup vs. $p = 1$
1	$2.56 + 15.36 + 1.28 = 19.2 \times 10^9$	9.60 s	1
2	$17.92/1.4 + 1.28 = 14.08 \times 10^9$	7.04 s	1.36
4	$17.92/2.8 + 1.28 = 7.68 \times 10^9$	3.84 s	2.50

(ii) Arithmetic CPI doubled (1 → 2)

Arithmetic cycles become 5.12×10^9, so the parallel part is $5.12 + 15.36 = 20.48 \times 10^9$.

p	cycles per processor	new time	increase
1	$20.48 + 1.28 = 21.76 \times 10^9$	10.88 s	+13.3%
2	$20.48/1.4 + 1.28 = 15.91 \times 10^9$	7.95 s	+13.0%
4	$20.48/2.8 + 1.28 = 8.59 \times 10^9$	4.30 s	+11.9%

The impact is modest because arithmetic is only $2.56/19.2 = 13.3\%$ of the single-processor cycles; L/S dominates.

(iii) Single processor matching 4 processors (3.84 s, i.e. 7.68×10^9 cycles)

$$2.56 \times 10^9 + 1.28 \times 10^9 \cdot \mathrm{CPI}_{ls} + 1.28 \times 10^9 = 7.68 \times 10^9 \;\Rightarrow\; \mathrm{CPI}_{ls} = \frac{3.84}{1.28} = 3.$$

Answer. (i) 9.6 s, 7.04 s, 3.84 s; speedups $1.36\times$ ($p = 2$) and $2.5\times$ ($p = 4$). (ii) Times rise to 10.88, 7.95 and 4.30 s ($\approx$12–13% slower). (iii) $\mathrm{CPI}_{L/S}$ must fall from 12 to **3**, a 75% reduction ($4\times$).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2015–2016 • Q2(a) CSE 305 • 18/01/2017 • PF18 **MARKS 5**

Consider the following information for two different computers.

Computer	Number of Instructions	Clock Rate	CPI
A	10 Billion	4 GHz	1
B	8 Billion	4 GHz	1.1

(i) Which has the highest MIPS rating?

(ii) Which is faster?

2013–2014 • Q5(d) CSE 305 • 05/07/2015 • PF18 **MARKS 6**

Consider the following information for two different computers.

Computer	Number of Instructions	Clock Rate	CPI
A	10 Billion	4 GHz	1
B	8 Billion	4 GHz	1.1

(i) Which one has the highest MIPS rating?

(ii) Which one is faster?

SOLUTION

Both cards are identical.

(i) MIPS rating

$$\mathrm{MIPS} = \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} = \frac{\text{Clock rate}}{\mathrm{CPI} \times 10^6}$$

$$\mathrm{MIPS}_A = \frac{4 \times 10^9}{1 \times 10^6} = 4000; \qquad \mathrm{MIPS}_B = \frac{4 \times 10^9}{1.1 \times 10^6} = 3636.$$

(ii) Execution time

$$T_A = \frac{10 \times 10^9 \times 1}{4 \times 10^9} = 2.5\,\mathrm{s}; \qquad T_B = \frac{8 \times 10^9 \times 1.1}{4 \times 10^9} = 2.2\,\mathrm{s}; \qquad \frac{T_A}{T_B} = 1.14.$$

Answer. (i) **A** has the higher MIPS rating (4000 vs. 3636). (ii) **B** is faster, by $1.14\times$ (2.2 s vs. 2.5 s). This shows why MIPS is a misleading metric: it ignores instruction count.

2014–2015 • Q5(b) CSE 305 • 26/01/2016 • PF19 **MARKS 15**

Assume a processor with 2 GHz clock rate is executing a program that requires the following instructions.

Instructions	**FP**	**INT**	**L/S**	**BRANCH**
Instruction Count ($\times 10^6$)	160	110	10	16
CPI	2	1	5	2

(i) Calculate the execution time of the program.

(ii) By how much must we improve the CPI of **FP** instructions if we want the program to run two times faster?

(iii) By how much must we improve the CPI of **L/S** instructions if we want the program to run two times faster?

SOLUTION

Method. Cycles $= \sum \mathrm{IC}_i \cdot \mathrm{CPI}_i$, $T = \mathrm{Cycles}/f$. "Two times faster" at the same clock means half the cycles. A class can deliver this only if $\mathrm{Cycles}_{\text{others}} + \mathrm{IC}_k \cdot \mathrm{CPI}_k^{\text{new}} = \frac{1}{2}\mathrm{Cycles}$ has a solution with $\mathrm{CPI}_k^{\text{new}} > 0$.

	FP	INT	L/S	BR	Total
cycles (10^6)	$160 \times 2 = 320$	$110 \times 1 = 110$	$10 \times 5 = 50$	$16 \times 2 = 32$	**512**

(i) $T = 512 \times 10^6 / 2 \times 10^9 = 0.256\,\mathrm{s}$.
(ii) Target $= 256\,\mathrm{M}$ cycles. Non-FP cycles $= 110 + 50 + 32 = 192\,\mathrm{M}$, so

$$192 + 160 \cdot \mathrm{CPI}_{FP}^{\text{new}} = 256 \;\Rightarrow\; \mathrm{CPI}_{FP}^{\text{new}} = \frac{64}{160} = 0.4.$$

FP's CPI must drop from 2 to 0.4, a $5\times$ **improvement** (80% reduction). This is achievable because FP is 62.5% of the cycles.
(iii) Non-L/S cycles $= 320 + 110 + 32 = 462\,\mathrm{M} > 256\,\mathrm{M}$:

$$462 + 10 \cdot \mathrm{CPI}_{L/S}^{\text{new}} = 256 \;\Rightarrow\; \mathrm{CPI}_{L/S}^{\text{new}} = -20.6 \quad \text{(impossible)}.$$

L/S is only $\approx 9.8\%$ of the cycles. Even $\mathrm{CPI}_{L/S} = 0$ gives at most $512/462 = 1.11\times$ (Amdahl's law).

Answer. (i) 0.256 s. (ii) CPI_{FP}: 2 → 0.4 (5× better). (iii) **Impossible**: no L/S CPI improvement can make the program 2× faster.

2013–2014 • Q5(b) CSE 305 • 05/07/2015 • PF20 **MARKS 15**

Consider two different implementations of the same instruction set architecture. There are four classes of instructions, A, B, C and D. The clock rate and CPI of each implementation are given in the following table:

Implementation	Clock Rate	CPI Class A	CPI Class B	CPI Class C	CPI Class D
P1	3 GHz	1	2	3	4
P2	2 GHz	2	2	2	2

Given a program with 200 instruction divided into classes as follows: 10% class A, 20% class B, 50% class C and 20% class D.

(i) Find the clock cycles required in both cases.

(ii) Which implementation is faster?

(iii) What is the global CPI for each implementation?

SOLUTION

Method. Cycles $= \sum_i \mathrm{IC}_i \,\mathrm{CPI}_i$, $T = \text{Cycles}/\text{Clock rate}$, global CPI $= \text{Cycles}/\text{IC}$.
Counts: A $= 20$, B $= 40$, C $= 100$, D $= 40$.
(i) $\text{Cycles}_{P1} = 20(1) + 40(2) + 100(3) + 40(4) = 20 + 80 + 300 + 160 = 560$; $\text{Cycles}_{P2} = 200 \times 2 = 400$.
(ii) $T_{P1} = 560/3 \times 10^9 = 186.7\,\text{ns}$; $T_{P2} = 400/2 \times 10^9 = 200\,\text{ns}$, so P1 is faster by $200/186.7 = 1.07$.
(iii) $\text{CPI}_{P1} = 560/200 = 2.8$; $\text{CPI}_{P2} = 400/200 = 2.0$.

Answer. (i) 560 and 400 cycles. (ii) **P1** is faster (186.7 ns vs. 200 ns, 1.07×), despite its higher CPI. (iii) Global CPI: P1 = 2.8, P2 = 2.0.

2013–2014 • Q5(c) CSE 305 • 05/07/2015 • PF21 MARKS 8

What do you mean by Amdahl's law? Write down the equation. Suppose a sample program takes 100 seconds to complete. The division operation accounts for 80 seconds. How much improvement in division operations' performance can give 4x overall improvement?

SOLUTION

Amdahl's law

The performance gain from an improvement is limited by the fraction of time the improved part is actually used:

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}, \qquad \text{Speedup}_{\text{overall}} = \frac{1}{(1-F) + \dfrac{F}{S}}$$

where F is the fraction of time affected and S is the speedup of that part. Corollary: *make the common case fast*.

Division improvement for $4\times$ overall

Target time $= 100/4 = 25\,\text{s}$:

$$25 = \frac{80}{n} + 20 \;\Rightarrow\; \frac{80}{n} = 5 \;\Rightarrow\; n = 16.$$

Answer. Division must become **16 times** faster (80 s → 5 s). Note that $5\times$ overall would be impossible, since it needs 20 s and the non-division part alone is 20 s.

2012–2013 • Q6(a) CSE 305 • 10/05/2014 • PF22 MARKS 3 + 3 + 3

Consider a computer running program with CPU times shown in the following table:

FP intr.	INT instr.	Load/Store instr.	Branch intr.	Total Time
50 sec	80 sec	50 sec	30 sec	210 sec

(i) By how much (in %) is the total time reduced if the time for FP operations is reduced by 20%?

(ii) Assume that the time for INT operations is improved. By how much (in %) is the time for INT operations reduced if the total time is reduced by 20%?

(iii) Can the total time be reduced by 20% by reducing only the time for branch instructions?

SOLUTION

(i) FP time: $50 \rightarrow 40$ s (saves 10 s). New total $= 200$ s.

$$\text{Reduction} = \frac{10}{210} = 4.76\%.$$

(ii) A 20% total reduction must save $0.2 \times 210 = 42$ s, all from INT: $80 \rightarrow 38$ s.

$$\text{INT reduction} = \frac{42}{80} = 52.5\%.$$

(iii) We again need to save 42 s, but branches take only 30 s. Even eliminating them entirely saves $30/210 = 14.3\% < 20\%$.

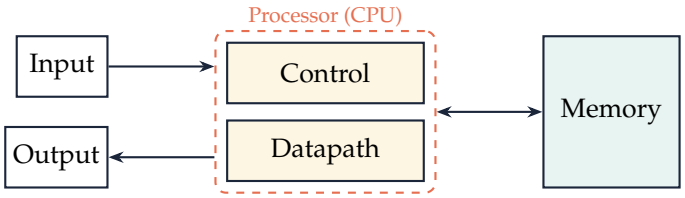
Answer. (i) Total time falls by **4.76%** (210 s $\rightarrow$ 200 s). (ii) INT time must fall by **52.5%** (80 s $\rightarrow$ 38 s). (iii) **No**: the 42 s needed exceeds the whole 30 s of branch time (Amdahl's law).

2011–2012 • Q1(a) CSE 305 • 27/07/2013 • PF23 MARKS 7.5

What are the components of a computer? Mention the task of each of these components.

SOLUTION

Every computer (desktop, server or embedded) has the same **five classic components**: input, output, memory, and the processor, which consists of datapath and control.



- **Input**: brings data and programs into the computer (keyboard, mouse, touchscreen, microphone, network adapter, storage devices).
- **Output**: sends results to the user or to other machines (display, printer, speaker, network).
- **Memory**: stores programs while they run and the data they need. This includes main memory (DRAM), caches (SRAM), and secondary storage (disk, flash) for non-volatile storage.
- **Datapath**: performs the arithmetic and logical operations. It holds the ALU, registers and the buses between them.
- **Control**: tells the datapath, memory and I/O what to do, according to the instruction being executed (it generates the control signals).

Answer. Input, Output, Memory, Datapath and Control; the datapath and control together form the processor.

2011–2012 • Q1(c) CSE 305 • 27/07/2013 • PF24 **MARKS 7**

Suppose we have two implementations of the same instruction set architecture. Computer A has a clock cycle time of 250 ps and a CPI of 2.0 for some program, and computer B has a clock cycle time of 500 ps and a CPI of 1.2 for the same program. Which computer is faster for this program and by how much? (Show all steps of calculations.)

SOLUTION

Same ISA and same program, so both computers execute the same instruction count I.

$$\text{CPU time} = I \times \text{CPI} \times T_c$$

$$\text{CPU time}_A = I \times 2.0 \times 250\ \text{ps} = 500\,I\ \text{ps}, \qquad \text{CPU time}_B = I \times 1.2 \times 500\ \text{ps} = 600\,I\ \text{ps}.$$

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{CPU time}_B}{\text{CPU time}_A} = \frac{600\,I}{500\,I} = 1.2.$$

Answer. **Computer A** is faster, by a factor of **1.2**: it takes 500 ps per instruction against B's 600 ps.

2011–2012 • Q1(d) CSE 305 • 27/07/2013 • PF25 **MARKS 6**

Suppose a program runs in 10 seconds and the program spends 50% of its time executing floating point instructions. We improve the floating point unit by a factor of five. How big is the speedup?

SOLUTION

Amdahl's law: $T_{\text{improved}} = \dfrac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$, i.e. $\text{Speedup} = \dfrac{1}{(1-F) + F/S}$.
Here $F = 0.5$ (5 s of FP) and $S = 5$:

$$T_{\text{new}} = \frac{5}{5} + 5 = 6\ \text{s}, \qquad \text{Speedup} = \frac{10}{6} = \frac{1}{0.5 + 0.5/5} = 1.67.$$

Answer. Overall speedup = **1.67**× (10 s → 6 s), far below the FP unit's 5×. The upper bound is $1/(1-0.5) = 2\times$.

2011–2012 • Q1(e) CSE 305 • 27/07/2013 • PF26 **MARKS 4.5**

What are the problems with measuring performance in terms of MIPS (million instructions per second)?

SOLUTION

$$\text{MIPS} = \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}$$

MIPS is an instruction *execution rate*. It does not account for:

- **Differences in ISAs**: you cannot compare computers with different instruction sets, because one instruction does different amounts of work on each.
- **Differences in instruction complexity**: many simple instructions can give a high MIPS rating but a longer run time than fewer complex ones. Example (PF15): A rates 4000 MIPS against B's 3636, yet B runs the program faster (2.2 s vs. 2.5 s).

- **Variation of CPI between programs**: on the same CPU, MIPS changes from program to program, so a computer has no single MIPS rating.

In fact MIPS can even vary *inversely* with performance, e.g. when a compiler adds cheap instructions or an optimiser removes them.

Answer. MIPS ignores instruction count (ISA and complexity differences) and varies with the program's CPI. Use execution time as the only reliable measure.

02

Instructions: Language of the Computer

2023–2024 • Q6(b) CSE 209 • 29/04/2026 • IS01 **MARKS 13**

Write down the MIPS assembly code for the following program.

```
int main() {
    int a = 5, b = 10, c = 2;   // $t0 = a, $t1 = b, $t2 = c
    int N = 30;                 // $t3 = N
    do {
        if ((a << 2) > b) {
            a += 2;
            b += a;
        }
        else {
            a += c;
            b += c;
        }
    } while (a < N);
    return 0;
}
```

SOLUTION

Plan

- `a«2` is one `sll`. MIPS has no "branch if greater", so $(a \ll 2) > b$ is written as $b < (a \ll 2)$ with `slt`, followed by `beq`/`bne` against `$zero`.
- A **do–while** tests its condition at the bottom, so the body always runs at least once.
- `main` is a leaf procedure that only uses `$t` registers, so it needs no stack frame. `return 0` puts 0 in `$v0` and returns with `jr $ra`.

```
main:   addi $t0, $zero, 5      # a = 5
        addi $t1, $zero, 10     # b = 10
        addi $t2, $zero, 2      # c = 2
        addi $t3, $zero, 30     # N = 30
Loop:   sll  $t4, $t0, 2        # $t4 = a << 2
        slt  $t5, $t1, $t4      # $t5 = 1 if b < (a<<2), i.e. (a<<2) > b
        beq  $t5, $zero, Else   # condition false -> else part
        addi $t0, $t0, 2        # a += 2
        add  $t1, $t1, $t0      # b += a   (uses the new a)
        j    Cond
Else:   add  $t0, $t0, $t2      # a += c
        add  $t1, $t1, $t2      # b += c
Cond:   slt  $t5, $t0, $t3      # $t5 = 1 if a < N
        bne  $t5, $zero, Loop   # repeat while (a < N)
        add  $v0, $zero, $zero  # return value 0
        jr   $ra                # return to caller
```

Check by tracing. (a, b) goes $(5,10) \to (7,17) \to (9,26) \to (11,37) \to (13,50) \to (15,65) \to (17,67) \to (19,86) \to (21,88) \to (23,90) \to (25,115) \to (27,117) \to (29,119) \to (31,121)$. At that point $a < N$ is false and the loop stops.

Answer. 16 instructions (4 initialisations + 10-instruction loop + 2 for the return). On exit $a = 31$, $b = 121$ and $\$v0 = 0$.

REPEATED ×10 Same question in 10 papers — stacked below, solved once.

2023–2024 • Q6(c) CSE 209 • 29/04/2026 • IS02 **MARKS 9**

Explain PC-relative addressing mode, Immediate addressing mode, and Base addressing mode with examples.

2022–2023 • Q11(c) CSE 209 • 13/01/2025 • IS02 **MARKS 10**

What are the different addressing modes of MIPS? Illustrate the calculation process for each addressing mode.

2021–2022 • Q8(c) CSE 305 • 06/04/2023 • IS02 **MARKS 10**

What are the different addressing modes of MIPS? Illustrate the calculation process for each addressing mode.

2020–2021 • Q13(b) CSE 305 • 24/09/2022 • IS02 **MARKS 7**

How is address calculation performed in PC-relative addressing?

2018–2019 • Q6(a) CSE 305 • 29/10/2019 • IS02 **MARKS 15**

What are the different addressing modes of MIPS? Describe and illustrate the calculation process for each addressing mode.

2016–2017 • Q7(a) CSE 305 • 02/08/2017 • IS02 **MARKS 10 + 10**

Please draw the illustrations of the five MIPS addressing modes. Also give an example instruction for each of the five addressing modes.

2015–2016 • Q6(b) CSE 305 • 18/01/2017 • IS02 **MARKS 15**

What are the different addressing modes of MIPS? Describe and illustrate the calculation process for each addressing mode.

2014–2015 • Q7(a) CSE 305 • 26/01/2016 • IS02 **MARKS 15**

Please draw the illustrations of the five MIPS addressing modes.

2013–2014 • Q7(b) CSE 305 • 05/07/2015 • IS02 **MARKS 4**

Describe MIPS addressing modes.

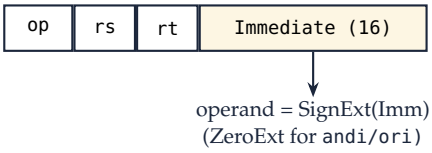
2011–2012 • Q2(d) CSE 305 • 27/07/2013 • IS02 **MARKS 10**

Describe MIPS addressing modes.

SOLUTION

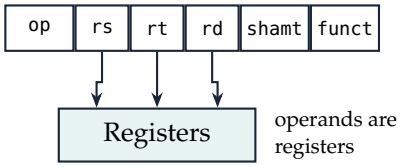
An **addressing mode** is the rule that tells the hardware where an instruction's operand, or its branch/jump target, is found. MIPS has **five** of them, shown below in the style of the slides' "Addressing Mode Summary". The 2023–24 card asks for modes 1, 3 and 4 only, and the 2020–21 card for mode 4 only, which is worked out in detail further down.

1. Immediate addressing



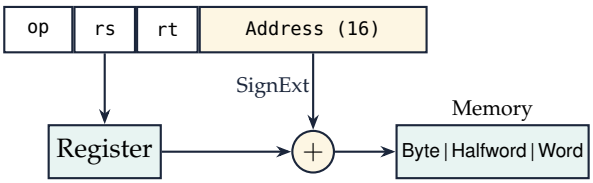

```
addi $s3, $s3, 4 # $s3 = $s3 + 4
```

2. Register addressing



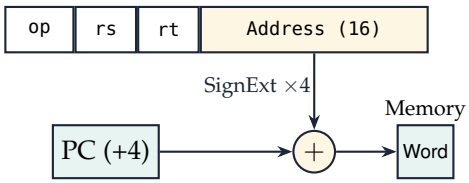

```
add $t0, $s1, $s2 # $t0 = $s1 + $s2
```

3. Base (displacement) addressing



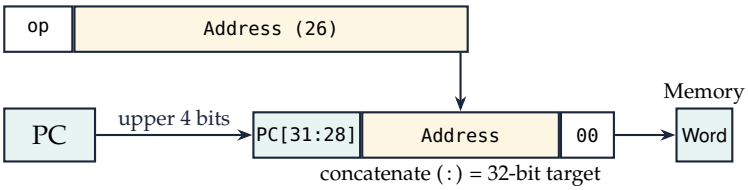

```
lw $t0, 32($s3) # addr = $s3 + 32
```

4. PC-relative addressing




```
beq $s0, $s1, L1 # PC+4+off*4
```

5. Pseudodirect addressing




```
j 20000 # target = PC[31:28] :  (20000 × 4) = 80000
```

Summary and calculation process

Mode	Where the operand / target is	Example	Used by
Immediate	The constant is inside the instruction (16 bits, sign-extended)	`addi $s3,$s3,4`	`addi, slti, andi, ori, lui`
Register	The operand is a register named by rs/rt/rd (5-bit numbers)	`add $t0,$s1,$s2`	R-type, `jr`
Base	$EA = \text{Reg}[rs] + \text{SignExt}(\text{offset}_{16})$	`lw $t0,32($s3)`	`lw, sw, lb, lh, sb, ll, sc`
PC-relative	$\text{Target} = (PC{+}4) + \text{SignExt}(\text{offset}_{16}){\times}4$	`bne $t0,$s5,Exit`	`beq, bne`
Pseudodirect	$\text{Target} = PC_{31..28} : \text{address}_{26} : 00$	`j Loop`, `jal f`	`j, jal`

PC-relative addressing in detail (2020–21 card; also 2023–24)

(i) While the branch is fetched, the PC has already been incremented, so the reference point is **PC**+4.

(ii) The 16-bit offset field is a signed count of *words*. It is sign-extended to 32 bits and shifted left 2 (that is, $\times 4$), because instructions are word-aligned.

(iii) Target $=$ (PC+4) $+$ SignExt(offset)$\times 4$. Since offset $\in [-2^{15}, 2^{15}-1]$, a branch reaches roughly $\pm 2^{15}$ instructions ($\approx \pm 128$ KB) around itself. That is enough because most branch targets are near the branch (loops, if-else).

Example (slides' loop at 80000): `bne $t0,$s5,Exit` sits at 80012 and its offset field is 2, so the target is $80012 + 4 + 2 \times 4 = 80024$, which is `Exit`.
Base example: for `lw $t0, 32($s3)` with `$s3` $=$ `0x10010000`, EA $=$ `0x10010000`$+32 =$ `0x10010020`. This is how `A[8]` is accessed.
Immediate example: `addi $s2, $s1, -1`. The constant -1 (`0xFFFF`) is sign-extended to `0xFFFFFFFF` and added, so no load is needed. For a 32-bit constant, `lui` first puts the upper 16 bits in place.

Answer. The five MIPS modes are immediate, register, base (displacement), PC-relative and pseudodirect. Branches use PC-relative addressing: target $=$ PC$+4+$SignExt(offset)$\times 4$. Loads and stores use base addressing: EA $=$ rs $+$ SignExt(offset). Jumps use pseudodirect addressing: $\text{PC}_{31..28} : \text{addr}_{26} : 00$.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q8(c) CSE 209 • 29/04/2026 • IS03 **MARKS 5**

Given the following branch instruction where register `$s0` value equals to register `$s1` value and label `L1` indicates an out of range branching distance. `beq $s0, $s1, L1`. Replace the `beq` instruction by a pair of instructions that offers a much greater branching distance and resolves the out of range branching issue.

2013–2014 • Q6(d) CSE 305 • 05/07/2015 • IS03 **MARKS 3**

If branch target is too far to encode with 16-bit offset, how does the assembler rewrite the following code: `beq $s0, $s1, L1`

2011–2012 • Q2(c) CSE 305 • 27/07/2013 • IS03 **MARKS 3.5**

If branch target is too far to encode with 16-bit offset, how does the assembler rewrite the following code: `BEQ $s0, $s1, L1`?

SOLUTION

Why the branch can fail

`beq`/`bne` are I-format instructions: | op (6) | rs (5) | rt (5) | offset (16) |. They use PC-relative addressing:

$$\text{target} = (\text{PC} + 4) + \text{SignExt}(\text{offset}) \times 4, \qquad -2^{15} \le \text{offset} \le 2^{15} - 1.$$

So a branch reaches only -131072 to $+131068$ bytes ($\pm 2^{15}$ words) from PC+4. If `L1` lies farther away, the offset does not fit in 16 bits.

The rewrite (slides: "Branching far away")

Invert the condition and branch a short distance over an unconditional `j`. The `j` instruction has a 26-bit word address, so it reaches anywhere in the current 256 MB region (target = $PC_{31..28}$:$addr_{26}$:00).

```
beq $s0, $s1, L1
```

$\Longrightarrow$

```
    bne $s0, $s1, L2   # short hop if NOT equal
    j   L1             # far jump, 26-bit field
L2:  ...
```

When `$s0`=`$s1` (the case in the 2023–24 card), `bne` falls through and `j` takes control to `L1`, exactly as the original `beq` would. When they differ, execution continues at `L2`, the instruction that followed the original `beq`.

Answer. `bne $s0, $s1, L2; j L1; L2: ...`. The inverted short branch skips over a pseudodirect jump that can reach the whole 256 MB region.

2022–2023 • Q6 CSE 209 • 13/01/2025 • IS04 **MARKS 15**

The current implementation of MIPS hardware doesn't support memory values as operand for R-type instructions. You can perform R-type instructions only if both of the operand values are stored in a register. Now you want to add a new instruction type (i.e. M-type instruction).
In M-type instruction you should perform only addition and subtraction, where one of the operand value would be stored in the memory (i.e. memory operand).
Now you need to design the M-type instruction format (i.e. which bits will represent which value) and the conversion protocol for transforming MIPS assembly to machine code for M-type instruction. You have the full freedom to design the new format and you can set any rules you wish; you can add new control signal(s) if needed. Explain your designed instruction format with proper justification. While designing your new protocol you **mustn't** hamper the working of the three original instruction formats.
Also, there will be major change(s) needed in hardware for the above-described M-type instruction. Identify the change(s) and explain them with proper justification.

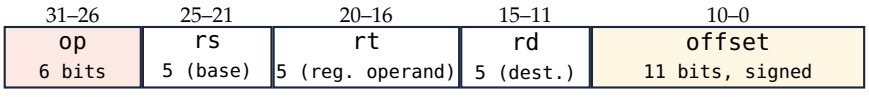
SOLUTION

1. Semantics and assembly syntax (one possible design)

$$\texttt{addm rd, rt, offset(rs)}: \quad \texttt{rd} \leftarrow \texttt{rt} + \text{Mem}[\texttt{rs} + \text{SignExt}(\text{offset})]$$
$$\texttt{subm rd, rt, offset(rs)}: \quad \texttt{rd} \leftarrow \texttt{rt} - \text{Mem}[\texttt{rs} + \text{SignExt}(\text{offset})]$$

This keeps the three-operand style (*simplicity favours regularity*) and uses the same `offset(base)` notation as `lw/sw`.

2. M-type format (32 bits)



Justification

- **rs and rt sit in bits 25–21 and 20–16**, as in R- and I-format, and **rd in 15–11**, as in R-format. The register file can therefore be read before decoding finishes, and the existing RegDst mux already selects rd (*good design demands good compromises*: keep formats as similar as possible).

- **add vs. sub is told apart by the opcode** (two new, currently unused opcodes, e.g. 24 = `addm`, 25 = `subm`) rather than by a funct field. This leaves $32 - 6 - 15 = 11$ bits for the offset (range $-1024 \ldots + 1023$ bytes).
- **Original formats are untouched**: R (op=0), I and J instructions keep their opcodes and fields. Only the control unit learns two more opcodes.

3. Conversion protocol (assembler rules)

(i) Mnemonic → opcode: `addm`→`011000`, `subm`→`011001`.

(ii) The 1st register is rd, the 2nd is rt, and the register in parentheses is rs. Each is written as its 5-bit number.

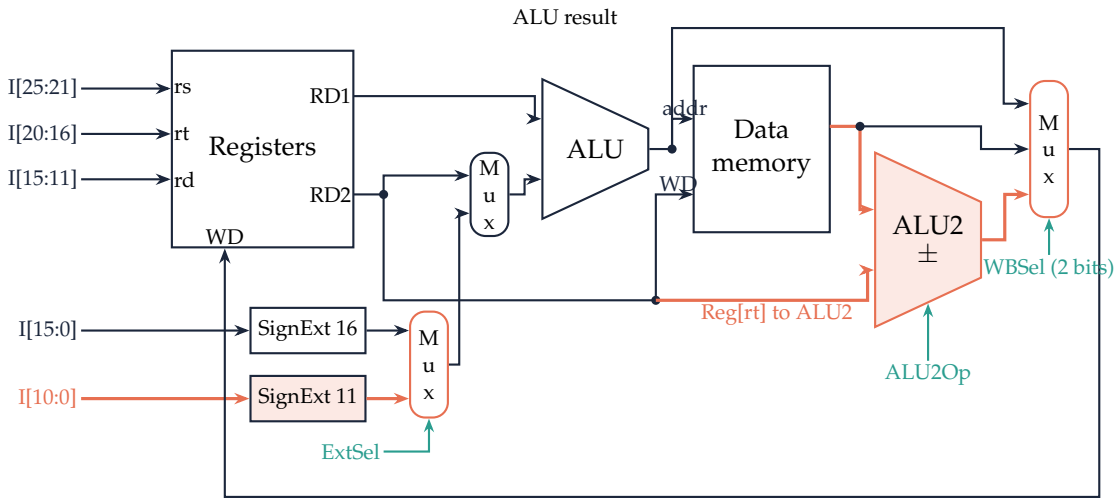
(iii) The offset is a byte offset written as 11-bit two's complement. It must be a multiple of 4 (word operand). If it lies outside $[-1024, 1023]$, the assembler expands the instruction into `lw $at,off(rs)` and `add/sub rd,rt,$at`, the same way pseudoinstructions are handled.

Example: `addm $t0, $s1, 8($s2)` → op= 24, rs= 18, rt= 17, rd= 8, offset= 8:

`011000 10010 10001 01000 00000001000` = `0x62514008`.

Similarly, `subm $t0,$s1,-4($s2)` has offset field `11111111100` and encodes as `0x665147FC`.

4. Hardware changes



(i) Second arithmetic unit (ALU2, add/sub) after data memory. In a single-cycle datapath the main ALU is busy computing the address rs+offset in the same cycle, so the memory operand needs its own adder/subtractor. Its inputs are Mem[EA] and Reg[rt] (Read data 2, which until now went only to memory for `sw`).

(ii) Write-back mux widened to 3 inputs (ALU result / memory data / ALU2 result). MemtoReg becomes a 2-bit signal, WBSel.

(iii) 11-bit sign extender plus a mux (ExtSel), so the address offset comes from bits 10–0 for M-type and from bits 15–0 for I-type.

(iv) Control unit: it decodes the two new opcodes and generates RegDst= 1, ALUSrc= 1, ALUOp=add, MemRead= 1, MemWrite= 0, RegWrite= 1, ExtSel= 1, WBSel=ALU2, ALU2Op= 0 (`addm`) or 1 (`subm`). All existing instructions get ExtSel= 0 and their old WBSel values, so their behaviour does not change.

(v) Timing: the critical path becomes IM → Reg → ALU → DM → ALU2 → Reg, longer than `lw`'s. In a single-cycle design *every* instruction pays for this longer clock. In a pipeline it would need an extra stage (or a memory-then-execute ordering, as in x86-style pipelines) and extra forwarding and stall logic. This is exactly why load/store RISC ISAs avoid memory operands.

Answer. Format: op(6) | rs(5, base) | rt(5, register operand) | rd(5, destination) | offset(11, signed), with opcodes 24/25 for `addm`/`subm`. Meaning: rd = rt ± Mem[rs + SignExt(offset)]. Hardware: a second add/sub unit after data memory, a 3-input write-back mux (WBSel), an 11-bit sign extender with its select mux (ExtSel), new control outputs, and a longer critical path.

2022–2023 • Q9(c) CSE 209 • 13/01/2025 • IS05 **MARKS 10**

Convert the code snippet below to the MIPS assembly code.

Listing 1: Code Snippet for Question 9(c)

```
1 int fun(int a, int b){
2     if (b == 0) return 0;
3     else if (b / 2 == 0) return fun(a*4, b/8);
4     return fun(a+a, b/4) + a;
5 }
```

SOLUTION

Method

- Arguments: `a` in `$a0`, `b` in `$a1`. The result goes in `$v0`.
- Multiplying or dividing by a power of two becomes a shift: `a*4` → `sll 2`, `b/2`, `b/4`, `b/8` → `sra 1/2/3`. We assume $b \geq 0$, so `sra` gives the same result as C's truncating division.
- `return fun(a*4, b/8)` returns the callee's value *unchanged*. It can therefore be a **tail call**: set up the new arguments and `j fun`. `$ra` still holds the original caller's address, so no stack frame is needed.
- `fun(a+a, b/4) + a` still needs `a` after the call, so that path is non-leaf. It saves `$ra` and `a` on the stack, calls with `jal`, then restores both (slides: non-leaf procedure).

```
fun:    bne  $a1, $zero, L1      # if (b == 0)
        add  $v0, $zero, $zero   #     return 0
        jr   $ra
L1:     sra  $t0, $a1, 1         # $t0 = b / 2
        bne  $t0, $zero, L2      # else if (b/2 == 0) ...
        sll  $a0, $a0, 2         #   a = a*4
        sra  $a1, $a1, 3         #   b = b/8
        j    fun                 #   return fun(a*4, b/8)   (tail call)
L2:     addi $sp, $sp, -8        # non-leaf part: make a 2-word frame
        sw   $ra, 4($sp)         # save return address
        sw   $a0, 0($sp)         # save a (needed after the call)
        add  $a0, $a0, $a0       # 1st arg = a + a
        sra  $a1, $a1, 2         # 2nd arg = b / 4
        jal  fun                 # $v0 = fun(a+a, b/4)
        lw   $a0, 0($sp)         # restore a
        lw   $ra, 4($sp)         # restore return address
```

```
        addi $sp, $sp, 8          # pop frame
        add  $v0, $v0, $a0        # return fun(a+a, b/4) + a
        jr   $ra
```

(A conventional `jal fun` with its own stack frame in place of the tail `j fun` is also correct, just longer.)
Check: `fun(3,5)` = `fun(6,1)`+3. `fun(6,1)`: $1/2 = 0$, so it returns `fun(24,0)` = 0. Hence the result is 3, which the code also gives.

Answer. The 19-instruction procedure above. Shifts replace `*4`, `/2`, `/4` and `/8`. `fun(a*4,b/8)` is a tail jump. Only the `fun(a+a,b/4)+a` path builds a 2-word stack frame holding `$ra` and `a`.

2022–2023 • Q11(b) CSE 209 • 13/01/2025 • IS06 **MARKS 8**

How does "`jal`" instruction differ from an unconditional jump instruction? How is the NOT operation performed in the MIPS assembly language?

SOLUTION

`jal` vs. `j`

	`j Label` (jump)	`jal Label` (jump and link)
Format / opcode	J-format, op = 2, 26-bit address	J-format, op = 3, 26-bit address
Target	PC ← $PC_{31..28}$: addr : 00	same target computation
Side effect	none	`$ra` (reg 31) ← PC+4, the return address
Purpose	`goto`, end of if-else, loop back-edge	**procedure call**. The callee returns with `jr $ra`

So `jal` is an unconditional jump that also *links*: it saves the address of the next instruction so control can come back to the caller. A non-leaf procedure must save `$ra` on the stack before its own `jal`, because the new `jal` overwrites it.

```
      jal  leaf          # $ra = address of next instr; PC = leaf
      ...                # execution resumes here after the return
leaf: ...
      jr   $ra           # PC = $ra  -> back to caller
```

NOT in MIPS

MIPS has no `not` instruction. It uses the 3-operand `nor`, since a NOR $0 = \overline{a+0} = \overline{a}$ (slides: "NOT Operations"):

```
nor $t0, $t1, $zero      # $t0 = ~$t1
```

Example:

$$\begin{aligned} \texttt{\$t1} &= \quad \texttt{0000 0000 0000 0000 0011 1100 0000 0000} \\ \texttt{\$t0} = \texttt{nor(\$t1, 0)} &= \quad \texttt{1111 1111 1111 1111 1100 0011 1111 1111} \end{aligned}$$

The assembler pseudoinstruction `not $t0,$t1` expands to exactly this `nor`. `xori $t0,$t1,0xFFFF` would *not* work, because `xori` zero-extends its immediate and so inverts only the low 16 bits.

Answer. `jal` jumps like `j` but also stores PC+4 in `$ra`, so a procedure can return with `jr $ra`. `j` only changes the PC. NOT is done with `nor rd, rs, $zero`.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2022–2023 • Q12(a–b) CSE 209 • 13/01/2025 • IS07 **MARKS 5 + 10**

(a) How does Just-In-Time (JIT) compilation improve the performance of a Java application? (5)
(b) What is the main difference between static linking and dynamic linking? What are the advantages of using dynamic linking? (10)

2022–2023 • Q7(d) CSE 305 • 25/04/2024 • IS07 **MARKS 15**

Write about the main difference between static linking and dynamic linking. With the necessary diagrams, explain how dynamically linked libraries (DLLs) are called.

2019–2020 • Q1(c) CSE 305 • 18/09/2021 • IS07 **MARKS 4 + 4**

What are the advantages of using Dynamically Linked Libraries (DLL) during a C program execution? What is the role of JIT in Java program execution?

SOLUTION

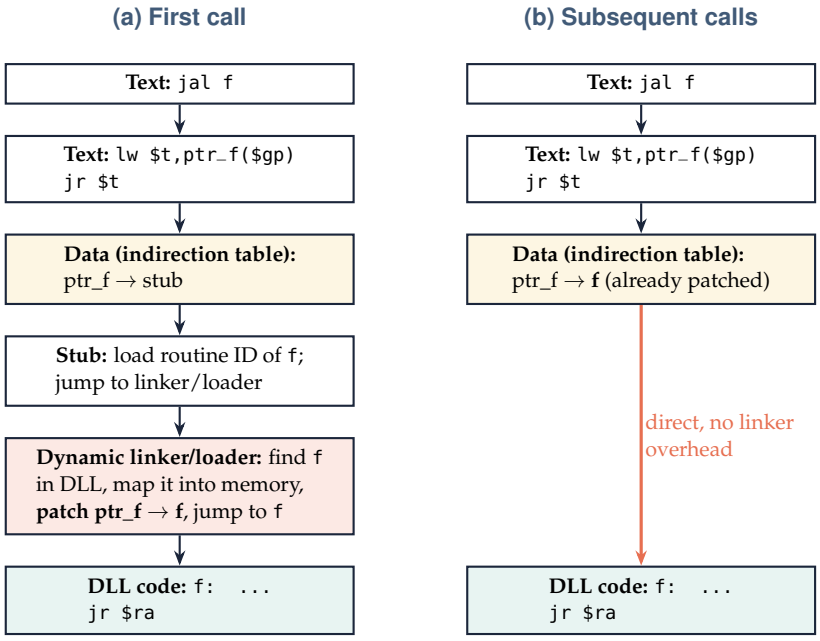
Static vs. dynamic linking (main difference)

	Static linking	Dynamic linking (DLL / shared library)
When	Before execution. The linker merges all object modules *and* every referenced library routine into one executable.	At load time or at run time. A library routine is linked and loaded only when it is (first) called.
Executable	Large: it contains copies of all library code, called or not.	Small: it contains only references (stubs) to the library.
Library update	Program keeps the old version until it is re-linked.	The new library version is picked up automatically.
Requirement	none	Library code must be relocatable. There is some extra overhead on the first call.

Advantages of dynamic linking / DLLs (2022–23 Q12(b), 2019–20)

- **No image bloat:** only routines that are actually called are loaded. Static linking pulls in all (transitively) referenced libraries.
- **Automatic upgrades and bug fixes:** a new library version is used without re-linking the program.
- **Sharing:** one copy of a DLL in memory can serve many running programs, which saves memory and disk.
- **Faster start-up with lazy linkage:** a routine is resolved only on its first call, so unused routines cost nothing.

How a DLL routine is called: lazy procedure linkage (2022–23 Q7(d))

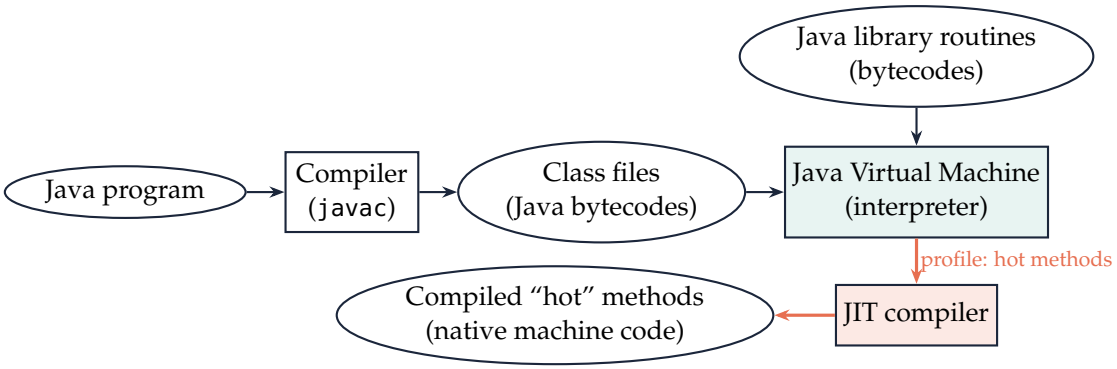


(i) The program calls a DLL routine through an **indirection table** entry (one word per external routine). At first every entry points to a small **stub**.

(ii) **First call:** `jal` → load the pointer and `jr` → stub. The stub loads the routine's ID and jumps to the **dynamic linker/loader**. The linker finds the routine, maps it into the address space, *overwrites the indirection entry with the routine's real address*, and jumps to it.

(iii) **Every later call:** the indirection word already holds the routine's address, so the call reaches the DLL code with only one extra indirect jump.

Role of JIT in Java (2022–23 Q12(a), 2019–20)



- Java is compiled to portable **bytecodes**, which the JVM normally *interprets*. Interpretation is portable but slow, because every bytecode is decoded in software each time it runs.
- A **Just-In-Time compiler** profiles the running program, finds the frequently executed ("hot") methods, and compiles their bytecodes into **native machine code** for the host machine while the program runs. The compiled code is cached and reused on later calls.
- Result: portability is kept, and the hot code runs at nearly compiled speed. Programs get faster the longer they run, since more of the run time is spent in native code. Only rarely used code stays interpreted.

Answer. Static linking copies all library code into the executable before it runs. Dynamic linking links each library routine only when it is loaded or first called, using an indirection table, a stub and the dynamic linker (lazy linkage). The result is smaller images, shared memory and automatic library updates. JIT speeds up Java by compiling hot bytecode methods into native code at run time instead of interpreting them.

2022–2023 • Q12(c) CSE 209 • 13/01/2025 • IS08 **MARKS 10**

(i) Write the MIPS assembly code that creates the 32-bit constant `0010 0000 0000 0001 0100 1001 0010 0100`$_2$ and stores that value to register `$t1`.

(ii) If the current value of the PC is `0x00000000` can you use a single jump instruction to go to the address equal to the constant shown in question 12(c)(i)?

(iii) If the current value of the PC is `0x1FFFF000`, can you use a single branch instruction to go to the address equal to the constant shown in Question 12(c)(i)?

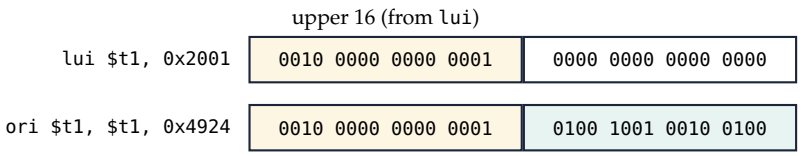
SOLUTION

Facts used

- 32-bit constant: `lui` loads the upper 16 bits (and clears the lower half). Then `ori` ORs in the lower 16 bits. `ori` zero-extends its immediate, so the upper half is not disturbed.
- `j` (pseudodirect): target $= (\text{PC}+4)_{31..28} : \text{addr}_{26} : 00$. It can only reach addresses in the same 256 MB region, i.e. with the same upper 4 bits.
- `beq`/`bne` (PC-relative): target $= \text{PC}+4+\text{SignExt}(\text{off}_{16})\times 4$ with $-32768 \le \text{off} \le 32767$.

(i) `0010 0000 0000 0001 | 0100 1001 0010 0100` = `0x20014924`. The upper half is `0x2001` = 8193 and the lower half is `0x4924` = 18724.

```
lui  $t1, 0x2001          # $t1 = 0x2001 0000
ori  $t1, $t1, 0x4924     # $t1 = 0x2001 4924
```



(ii) PC = `0x00000000`, so PC+4 = `0x00000004`, whose upper 4 bits are `0000`. The target `0x20014924` has upper 4 bits `0010`. A `j` can only produce addresses whose top 4 bits are `0000`, so **no**. (The target is word-aligned; the region is the problem.) Instead use the `lui`/`ori` of (i) followed by `jr $t1`.

(iii) PC+4 = `0x1FFFF004`.

$$\text{offset} = \frac{\texttt{0x20014924} - \texttt{0x1FFFF004}}{4} = \frac{\texttt{0x15920}}{4} = \frac{88352}{4} = 22088\ (\texttt{0x5648}) \le 2^{15} - 1 = 32767.$$

So **yes**: a single `beq` with offset field 22088 reaches it. (A `j` would fail here, because the upper bits `0001` and `0010` differ.)

Answer. (i) `lui $t1,0x2001; ori $t1,$t1,0x4924`. (ii) No: the jump cannot change the upper 4 bits from 0000 to 0010. (iii) Yes: the offset is 22088, which is at most 32767.

2022–2023 • Q5(a) CSE 305 • 25/04/2024 • IS09 **MARKS 7**

Rachel has a basic MIPS instruction set without support for multiple languages. In an attempt to add such support, she introduces four new instructions: `lh` (load halfword), `lhu` (load halfword unsigned), `sh` (store halfword), and `shu` (store halfword unsigned). However, Ross argued that she doesn't need all these instructions to achieve multi-language support. Do you agree with Ross? Justify your answer.

SOLUTION

Yes, I agree with Ross. Multi-language text uses **Unicode** (UTF-16), whose characters are 16-bit *halfwords* (slides: "Character Data", "Byte/Halfword Operations"). Loads and stores behave differently:

- **Loads widen** a 16-bit value into a 32-bit register, so the upper 16 bits must be filled somehow. `lh` *sign*-extends and `lhu` *zero*-extends. Both variants are meaningful:

  memory halfword `0x8041` `lh` → `0xFFFF8041` (−32703) `lhu` → `0x00008041` (32833)

- **Stores narrow:** `sh` simply writes the rightmost 16 bits of the register to memory. No extension happens, so signed and unsigned stores would write identical bits. **`shu` is redundant**, which is why real MIPS has `sh` but no `shu`.
- Character codes are unsigned, so text handling really needs only `lhu` (read a character) and `sh` (write a character). `lh` is useful for signed 16-bit integer data, not for characters. It could even be emulated as `lhu`, `sll 16`, `sra 16`.

Answer. Agree. `shu` is pointless because a store writes the same 16 bits whether the value is signed or unsigned. For Unicode text, `lhu` and `sh` suffice. `lh` is optional and only needed for signed halfword data. So she needs 2 (at most 3) of the 4 instructions, as in real MIPS: `lh`, `lhu`, `sh`.

2022–2023 • Q5(c) CSE 305 • 25/04/2024 • IS10 **MARKS 14**

Assemble the two object files of Figure for Question 5(c)-1 and Figure for Question 5(c)-2. Find the updated addresses of the first few instructions of the completed executable file. Note that all the necessary addresses and symbols must be updated in the link process. [Use Figure for Question 5(c)-3 for reference]

Object file header			
	Name Text size Data size	Procedure A 150_{hex} 40_{hex}	

Text segment	Address
	0
	4
	…
		Instruction
		`lw $a0, 0($gp)`
		`jal 0`
		…	

Data segment	0 …	(X) …	

Relocation information	Address
	0
	4
		Instruction type
		`lw`
		`jal`
			Dependency
			X
			B

Symbol table	Label X B	Address – –	

Figure for Question 5(c)-1: Object File Header for Procedure A

Object file header			
	Name Text size Data size	Procedure B 350_{hex} 50_{hex}	

Text segment	Address
	0
	4
	…
		Instruction
		`sw $a1, 0($gp)`
		`jal 0`
		…	

Data segment	0 …	(Y) …	

Relocation information	Address
	0
	4
		Instruction type
		`sw`
		`jal`
			Dependency
			Y
			A

Symbol table	Label Y A	Address – –	

Figure for Question 5(c)-2: Object File Header for Procedure B

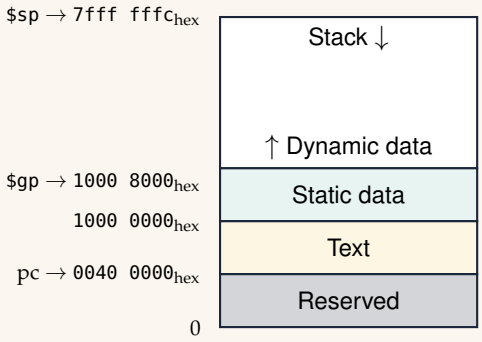


Figure for Question 5(c)-3: The MIPS memory allocation for program and data.

SOLUTION

What the linker does (slides: "Linking Object Modules")

(1) **merge** the text segments and the data segments, (2) **resolve** every label to an absolute address, (3) **patch** each location listed in the relocation information.

Step 1: place the segments (Figure 5(c)-3)

- Text starts at `0x00400000`. Procedure A's text (`0x150` bytes) occupies `0x00400000`–`0x0040014C`. Procedure B's text (`0x350` bytes) follows at `0x00400150`–`0x0040049C`.
- Static data starts at `0x10000000`. A's data (`0x40` bytes) is at `0x10000000`–`0x1000003C`, and B's data (`0x50` bytes) follows at `0x10000040`–`0x1000008C`.

Step 2: resolve the symbols

Label	Belongs to	Address	Used by
A	start of A's text	`0x00400000`	`jal` in B
B	start of B's text	`0x00400000`+`0x150` = `0x00400150`	`jal` in A
X	A's data word 0	`0x10000000`	`lw` in A
Y	B's data word 0	`0x10000000`+`0x40` = `0x10000040`	`sw` in B

Step 3: patch the relocated instructions

- `lw`/`sw` use base addressing off `$gp` = `0x10008000`, so the 16-bit offset is (address − `$gp`):
  X: `0x10000000`−`0x10008000` = −`0x8000` = −32768, so the field is `0x8000`.
  Y: `0x10000040`−`0x10008000` = −`0x7FC0` = −32704, so the field is `0x8040`.
- `jal` uses pseudodirect addressing, so its 26-bit field is (target ÷4):
  `jal B`: `0x00400150`/4 = `0x0100054`. `jal A`: `0x00400000`/4 = `0x0100000`. Both lie in the same 256 MB region as the calls (upper 4 bits 0000).

The executable file

Executable file header: Text size `0x4A0` (=`0x150`+`0x350`), Data size `0x90` (=`0x40`+`0x50`)		
Text segment	**Address**	**Instruction**
	`0x00400000`	`lw $a0, 8000`$_{\text{hex}}$`($gp)` *(i.e. −32768($gp) → X)*
	`0x00400004`	`jal 400150`$_{\text{hex}}$ *(field 0x0100054)*
	…	…
	`0x00400150`	`sw $a1, 8040`$_{\text{hex}}$`($gp)` *(i.e. −32704($gp) → Y)*
	`0x00400154`	`jal 400000`$_{\text{hex}}$ *(field 0x0100000)*
	…	…
Data segment	`0x10000000`	(X)
	…	…
	`0x10000040`	(Y)
	…	…

Answer. A's text is at `0x00400000` and B's at `0x00400150`; X is at `0x10000000` and Y at `0x10000040`. The patched instructions are: `0x00400000`: `lw $a0,0x8000($gp)`; `0x00400004`: `jal 0x00400150`; `0x00400150`: `sw $a1,0x8040($gp)`; `0x00400154`: `jal 0x00400000`. The executable has text size `0x4A0` and data size `0x90`.

REPEATED ×5 Same question in 5 papers — stacked below, solved once.

2022–2023 • Q6(b) CSE 305 • 25/04/2024 • IS11 **MARKS 7**

What support does the MIPS instruction-set architecture provide for synchronization? Explain with an example.

2016–2017 • Q6(b) CSE 305 • 02/08/2017 • IS11 **MARKS 10**

What support does MIPS instruction set architecture provide for synchronization? Explain with an example.

2015–2016 • Q6(c) CSE 305 • 18/01/2017 • IS11 **MARKS 8**

What is data race? What instructions are available in MIPS to address this issue? How these instructions work?

2014–2015 • Q6(b) CSE 305 • 26/01/2016 • IS11 **MARKS 5**

What support does MIPS instruction set architecture provide for synchronization?

2013–2014 • Q7(a) CSE 305 • 05/07/2015 • IS11 **MARKS 8**

Briefly describe the two instructions used for synchronization in MIPS. Write down the MIPS code to do the following atomically:

```
A[0] = X
```

Consider the base address of A is in `$s1` and X is in `$s2`.

SOLUTION

Data race and why hardware support is needed

Two processors (or threads) share a memory location, and P1 writes it while P2 reads or writes it. If they do not synchronize, the result depends on the *order of the accesses*. This is a **data race**. Example: both execute `count = count + 1` as `lw/addi/sw`. If both `lw` instructions happen before either `sw`, one increment is lost. Locks need an **atomic read-modify-write**: no other access to the location may come between the read and the write. Hardware can provide this as one instruction (e.g. atomic swap) or as an **atomic pair of instructions**, which is what MIPS does.

MIPS support: `ll` / `sc`

- `ll rt, offset(rs)`, **load linked**: loads the word like `lw` and remembers (*links*) its address in the hardware.
- `sc rt, offset(rs)`, **store conditional**: stores rt to that address *only if the location has not been changed since the matching* `ll`. On success it writes **1** into rt; on failure (another store to the location, or a context switch) it does not store and writes **0** into rt.
- Software loops back and retries when `sc` returns 0, so the `ll`–`sc` pair behaves as one atomic operation without locking the memory bus.

Example: atomic swap of $s4 with Mem[$s1] (slides), used to build a lock

```
try: add  $t0, $zero, $s4    # copy exchange value
     ll   $t1, 0($s1)        # load linked:   $t1 = Mem[$s1]
```

```
      sc   $t0, 0($s1)       # store conditional: Mem[$s1] = $t0 if unchanged
      beq  $t0, $zero, try   # sc failed (0) -> retry
      add  $s4, $zero, $t1   # put loaded (old) value in $s4
```

With `$s4`= 1 this is a *test-and-set*. If the old value returned is 0, the lock was free and is now ours. If it is 1, someone else holds it, so spin and try again. The lock is released by an ordinary `sw $zero,0($s1)`.

time	P1	P2
1	`ll $t1,0($s1)` (reads 0)	
2		`ll $t1,0($s1)` (reads 0)
3		`sc $t0,0($s1)` → `$t0`= 1, lock acquired
4	`sc $t0,0($s1)` → `$t0`= 0 (location changed)	
5	`beq` → retry, now reads 1: busy	

2013–2014 card: `A[0] = X` atomically (&A[0] in `$s1`, X in `$s2`)

```
try: add  $t0, $zero, $s2   # $t0 = X (sc overwrites its register)
     ll   $t1, 0($s1)       # load-link A[0] (old value)
     sc   $t0, 0($s1)       # A[0] = X only if no one wrote A[0] since ll
     beq  $t0, $zero, try   # failed -> repeat
```

After the loop, A[0] = X has been written atomically, and the old A[0] is in `$t1` if it is needed.

Answer. MIPS provides the atomic pair `ll` (load linked) and `sc` (store conditional). `sc` succeeds and returns 1 only if the location was not modified since the `ll`; otherwise it returns 0 and the code retries. Together they implement atomic swap and locks, which prevent data races.

2022–2023 • Q6(c) CSE 305 • 25/04/2024 • IS12 MARKS 8

Write down the equivalent MIPS code for the following C code, considering registers `$t1` and `$t2` for variables `x` and `y`, respectively:

```
if (x > y + 131075)
    x = 8;
```

SOLUTION

Key points

- I-format immediates are 16-bit signed ($-32768 \ldots 32767$). $131075 = 2 \times 65536 + 3 =$ `0x00020003` does not fit, so it is built with `lui` (upper half 2) and `ori` (lower half 3).
- MIPS has no "branch if greater". Rewrite $x > y + 131075$ as $(y + 131075) < x$ and use `slt` followed by `beq` against `$zero`.

```
      lui  $t0, 2             # $t0 = 2 << 16 = 131072
      ori  $t0, $t0, 3        # $t0 = 131075
      add  $t0, $t2, $t0      # $t0 = y + 131075
      slt  $t3, $t0, $t1      # $t3 = 1 if (y+131075) < x
      beq  $t3, $zero, Exit   # condition false -> skip
      addi $t1, $zero, 8      # x = 8
Exit:
```

Answer. Build 131075 with `lui $t0,2`; `ori $t0,$t0,3`. Then `add $t0,$t2,$t0`; `slt $t3,$t0,$t1`; `beq $t3,$zero,Exit`; `addi $t1,$zero,8`; `Exit:`.

2022–2023 • Q7(b) CSE 305 • 25/04/2024 • IS13 **MARKS 7**

Chandler has used 40 variables in his program, whereas the MIPS architecture has only 32 registers. What does the compiler do in such a scenario?

SOLUTION

The compiler performs **register allocation** and **spilling** (slides: "Registers vs. Memory": *only spill to memory for less frequently used variables*).

- Not all 32 registers are free for variables: `$zero`, `$at`, `$k0/$k1`, `$gp`, `$sp`, `$fp` and `$ra` are reserved. About 24 (`$s0–$s7`, `$t0–$t9`, `$a0–$a3`, `$v0/$v1`) can hold values.
- **Live-range analysis:** a register is needed only while a variable is *live*, i.e. between its assignment and its last use. Variables whose live ranges do not overlap can share one register (graph-colouring allocation). So 40 variables often need far fewer than 40 registers at any one moment.
- **Spilling:** when more values are live than there are registers, the compiler keeps the most frequently used ones (e.g. loop counters, values in inner loops) in registers. The rest live in memory, normally in the procedure's stack frame. Before a spilled variable is used it is loaded with `lw`; after it is changed it is stored back with `sw`:

```
lw   $t0, 24($sp)     # bring spilled variable v into a register
addi $t0, $t0, 1      # v = v + 1
sw   $t0, 24($sp)     # spill it back to the stack frame
```

- Callee-saved `$s` registers that it reuses are themselves saved and restored on the stack.

Spilled variables cost extra instructions and slower memory accesses, so good allocation ("register optimization is important") directly improves performance.

Answer. The compiler allocates registers to the most frequently used, simultaneously live variables (reusing registers when live ranges do not overlap) and *spills* the rest to memory, the stack frame, accessing them with `lw`/`sw` when needed.

2022–2023 • Q7(c) CSE 305 • 25/04/2024 • IS14 **MARKS 8**

Translate function `f` (defined below) into MIPS assembly language. If you need to use registers `$t0` through `$t7`, use the lower-numbered registers first. Assume the function declaration for `func` is "`int func(int a, int b);`". The code for function `f` is as follows:

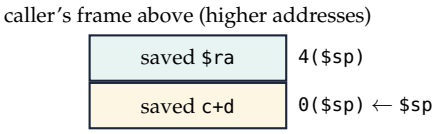
```
int f (int a, int b, int c, int d) {
    return func(func(a,b), c+d);
}
```

SOLUTION

Analysis

- By convention `a`, `b`, `c`, `d` arrive in `$a0–$a3`, and results come back in `$v0`.
- `f` is **non-leaf**: it calls `func` twice, so it must save `$ra`.
- `c+d` is needed *after* the first call, but `$a2`, `$a3` and `$t0–$t9` may be destroyed by `func` (caller-saved). So we compute `c+d` into `$t0` (lowest temporary first) and save it on the stack across the call.
- The first call needs `(a,b)`, which are already in `$a0`, `$a1`. The second call needs `$a0=func(a,b)`

and $a1=c+d.




```
f:   addi $sp, $sp, -8      # make room for 2 words
     sw   $ra, 4($sp)       # save return address
     add  $t0, $a2, $a3     # $t0 = c + d
     sw   $t0, 0($sp)       # save c+d ($t0 is not preserved by func)
     jal  func              # $v0 = func(a, b)   (a,b already in $a0,$a1)
     add  $a0, $v0, $zero   # 1st argument = func(a, b)
     lw   $a1, 0($sp)       # 2nd argument = c + d
     jal  func              # $v0 = func(func(a,b), c+d)
     lw   $ra, 4($sp)       # restore return address
     addi $sp, $sp, 8       # pop the frame
     jr   $ra               # return; result already in $v0
```

NOTE Optimisation (not required): the second call is a tail call. One can restore `$ra`, pop the frame, and then end with `j func` instead of `jal func ... jr $ra`. This saves one instruction.

Answer. The 11-instruction procedure above. It uses a 2-word stack frame holding `$ra` and `c+d` (computed in `$t0`). The first `jal func` uses `$a0`, `$a1` as given. Then `$a0`←`$v0` and `$a1`←`c+d`, followed by the second `jal func`. The result is returned in `$v0`.

REPEATED ×5 Same question in 5 papers — stacked below, solved once.

2021–2022 • Q5(a) CSE 305 • 06/04/2023 • IS15 **MARKS 3×5 = 15**

The following four design principles have been guiding the instruction-set designers to find a balance between the number of instructions needed to execute a program, the number of clock cycles needed by an instruction, and the speed of the clock.

(i) Simplicity favors regularity

(ii) Smaller is faster

(iii) Good design demands good compromise

(vi) Make the common case faster

You are given several design features of MIPS instruction-set architecture in the following. Now, please identify which of the above principle(s) is behind each of these design decisions.

(i) Keeping conditional branch's offset value in a range of 16 bits.

(ii) Keeping 32 registers rather than many more.

(iii) Consistent instruction format, i.e., always requiring three registers for R-type Instructions.

(iv) Multiple instruction formats to allow flexibility.

(v) Reduced instruction set, Complex instructions (i.e., pseudo instructions like `bgt`, `bge`, `mov` etc.) are translated into one or more basic instructions.

2020–2021 • Q11(d) CSE 305 • 24/09/2022 • IS15 **MARKS 5**

Provide an illustrative example for showing how MIPS architecture follows the design principle: "Good design demands good compromises."

2016–2017 • Q6(c) CSE 305 • 02/08/2017 • IS15 **MARKS 15**

The following four design principles have been guiding the instruction-set designers to find a balance between the number of instructions needed to execute a program, the number of clock cycles needed by an instruction, and the speed of the clock.

(i) Simplicity favors regularity

(ii) Smaller is faster

(iii) Good design demands good compromise

(iv) Make the common case faster

- You are given several design features of MIPS instruction-set architecture in the following. Now, please identify that which of the above principles are behind these design decisions.

(i) keeping instruction immediate value in a range of 16-bits

(ii) always requiring three register operands in arithmetic instructions

(iii) using PC-relative addressing for conditional branching

(iv) keeping 32 registers rather than many more

(v) keeping the register fields in the same place in each instruction format

2013–2014 • Q6(a) CSE 305 • 05/07/2015 • IS15 **MARKS 8**

What are the four design principles used for MIPS instruction set design? What are the steps for starting a Java application?

2011–2012 • Q2(a) CSE 305 • 27/07/2013 • IS15 **MARKS 4 + 4**

What are the MIPS design principles? Mention one use of each of these design principles in MIPS ISA.

SOLUTION

The four design principles, with one use of each (2013–14, 2011–12)

Principle	Use in MIPS
1. Simplicity favours regularity	Every arithmetic instruction has exactly 3 register operands (`add a,b,c`). All instructions are 32 bits, and fields keep the same positions across formats. Regular hardware is simpler and faster.
2. Smaller is faster	Only 32 registers. A small register file has short wires and fast access, and 5-bit register fields keep instructions short.
3. Make the common case fast	Small constants are common, so there are immediate instructions (`addi $s3,$s3,4`) that avoid a load. `$zero` makes moves cheap. Branches use PC-relative addressing because most targets are near.
4. Good design demands good compromises	Three formats (R, I, J) instead of one: decoding is a little more complex, but every instruction stays 32 bits. As a result immediates and branch offsets are limited to 16 bits.

2021–2022 Q5(a)

The paper numbers the fourth principle "(vi)", a misprint for (iv). The numbering below keeps the paper's labels.

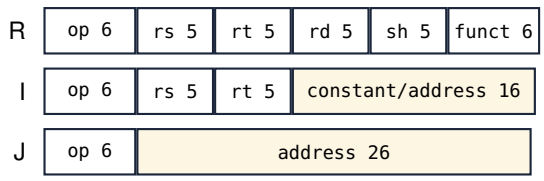
Decision	Principle	Reason
(i) 16-bit branch offset	(iii) Good compromise (also (vi))	Keeps a fixed 32-bit I-format with op, rs, rt. 16 bits suffice for the common near branches. Far targets use `bne`+`j`.
(ii) 32 registers	(ii) Smaller is faster	More registers mean a slower register file, longer clock and more field bits.
(iii) Always 3 registers in R-type	(i) Simplicity favours regularity	One uniform form makes decoding and the datapath simple.
(iv) Multiple formats	(iii) Good compromise	Different needs (registers, constants, long jump addresses) are met while the length stays at 32 bits and formats stay similar.
(v) Reduced set; pseudo-instructions	(vi) Common case fast (also (i))	Hardware implements only simple, frequent operations, so they run fast. Rare "complex" ones (`bgt`, `bge`, `move`) are synthesized by the assembler (`slt`+`bne`, `add $t0,$zero,$t1`).

2016–2017 Q6(c)

Decision	Principle	Reason
(i) 16-bit immediates	(iii) Good compromise (also (iv))	Fits the fixed 32-bit format. Most constants are small, and `lui`+`ori` handle the rest.
(ii) 3 register operands in arithmetic	(i) Simplicity favours regularity	Same form for all arithmetic instructions.
(iii) PC-relative conditional branches	(iv) Make the common case fast	Most branch targets are near (loops, ifs), so a 16-bit relative offset is enough.
(iv) 32 registers	(ii) Smaller is faster	A larger register file would be slower.
(v) Register fields in the same place in every format	(i) Simplicity favours regularity (also (iii))	Registers can be read before the opcode is decoded, and the hardware is simpler.

2020–2021 Q11(d): an illustration of "good design demands good compromises"

MIPS wants *all instructions the same length (32 bits)* for simple fetch and decode, and *large constants and addresses* for convenience. These goals conflict, and the compromise is to use **several formats while keeping them as similar as possible**:



op, rs and rt stay in the same positions, so decoding stays simple. The price is that I-format immediates and branch offsets are only 16 bits, which is why `lui` is needed for 32-bit constants and `bne+j` for far branches. A second example is not providing `blt/bge`: a combined compare-and-branch would slow the clock for every instruction, so MIPS uses `slt+beq/bne`.

Steps for starting a Java application (2013–14)

(i) The Java compiler translates the Java program into **class files of Java bytecodes**, a simple, portable instruction set for the JVM.

(ii) The **Java Virtual Machine** loads the class files, together with the Java library routines, which are also bytecodes.

(iii) The JVM **interprets** the bytecodes, which gives portability.

(iv) A **JIT compiler** profiles the program and compiles the bytecodes of "hot" methods into **native machine code** for the host, which is then executed directly and reused (slides: "Starting Java Applications").

Answer. The four principles are simplicity favours regularity, smaller is faster, make the common case fast, and good design demands good compromises. 2021–22: (i) compromise, (ii) smaller is faster, (iii) regularity, (iv) compromise, (v) common case fast. 2016–17: (i) compromise, (ii) regularity, (iii) common case fast, (iv) smaller is faster, (v) regularity.

2021–2022 • Q6(a) CSE 305 • 06/04/2023 • IS16 **MARKS 15**

Write the MIPS assembly code for the C code shown in the following figure (Figure: 6.a).

```
int fib(int n) {
    if (n==0) return 0;
    else if (n == 1) return 1;
    else return fib(n-1) + fib(n-2);
}
```

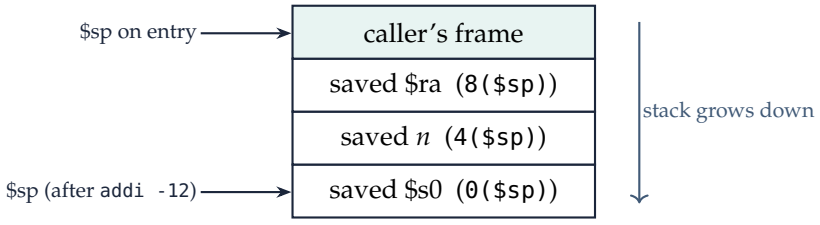
Figure: 6.a

SOLUTION

Conventions (slides): argument `$a0` $= n$, result in `$v0`, return address `$ra`. `fib` is a **non-leaf** procedure that calls itself *twice*, so across the calls it must keep `$ra`, the original n (needed to form $n-2$) and the value of `fib(n-1)` (needed for the final add). We keep `fib(n-1)` in the callee-saved register `$s0`, so `$s0` itself must also be saved. Both base cases return n itself ($0 \to 0$, $1 \to 1$), so they share one exit.

```
fib:    beq  $a0, $zero, Base     # n == 0 ?
        addi $t0, $zero, 1
        beq  $a0, $t0, Base       # n == 1 ?
        addi $sp, $sp, -12        # make room for 3 words
        sw   $ra, 8($sp)          # save return address
        sw   $a0, 4($sp)          # save n
        sw   $s0, 0($sp)          # save caller's $s0
```

```
        addi $a0, $a0, -1          # argument n-1
        jal  fib                   # $v0 = fib(n-1)
        add  $s0, $v0, $zero       # keep fib(n-1) in $s0
        lw   $a0, 4($sp)           # restore n
        addi $a0, $a0, -2          # argument n-2
        jal  fib                   # $v0 = fib(n-2)
        add  $v0, $s0, $v0         # fib(n-1) + fib(n-2)
        lw   $s0, 0($sp)           # restore $s0
        lw   $ra, 8($sp)           # restore return address
        addi $sp, $sp, 12          # pop the frame
        jr   $ra                   # return
Base:   add  $v0, $a0, $zero       # return 0 or 1 (= n)
        jr   $ra
```



The base cases are tested exactly as in the C code ($n = 0$, $n = 1$). Each call that does not hit a base case pushes a 3-word frame and pops it before `jr $ra`, so every caller finds `$sp`, `$s0` and `$ra` as it left them.

2021–2022 • Q6(b) CSE 305 • 06/04/2023 • IS17 **MARKS 5 + 5**

Suppose the program counter (PC) is set to `0x2000 0000`. Explain your answer for the following questions.

(i) Is it possible to use the jump (`j`) MIPS assembly instruction to set the PC to the address `0x4000 0000`?

(ii) Is it possible to use single branch (`beq/bneq`) instruction to set the PC to this same address?

SOLUTION

Facts used

- `j` (pseudodirect, J-format): target $= (\text{PC}+4)_{31..28} : \text{addr}_{26} : 00$. Only the low 28 bits come from the instruction; the upper 4 bits are copied from PC+4.
- `beq/bne` (PC-relative, I-format): target $= \text{PC}+4+\text{SignExt}(\text{off}_{16})\times 4$. The reach is $-2^{17} \ldots +2^{17} - 4$ bytes ($\approx \pm 128$ KB).

(i) PC+4 = `0x20000004`, upper 4 bits `0010`. The target `0x40000000` has upper 4 bits `0100`. These differ, so `j` **cannot** reach it: the target is in another 256 MB region.
(ii) The required distance is `0x40000000`−`0x20000004` = `0x1FFFFFFC` = 536,870,908 bytes, i.e. $\approx 1.3 \times 10^8$ words. That is far beyond the 16-bit offset limit of 32767 words. So **no**.
How to do it: load the full address into a register and use `jr`:

```
lui  $t0, 0x4000           # $t0 = 0x4000 0000
jr   $t0                   # PC = 0x4000 0000
```

Answer. (i) No: `j` keeps $PC_{31..28} = 0010$, but the target needs 0100. (ii) No: the distance `0x1FFFFFFC` bytes far exceeds the $\pm 2^{17}$-byte branch range. Use `lui $t0,0x4000; jr $t0`.

2020–2021 • Q11(c) CSE 305 • 24/09/2022 • IS18 **MARKS 12**

Write the equivalent efficient sequence of MIPS instructions for the following C code fragment.

```
int testFunc (int a, int b, int c)
{
    int i=0, result=0;
    do
    {
        if(i>b)
            result += c;
        i++;
    } while (i<a);
    return result;
}
```

Assume, `$s1` and `$s2` will carry the values of variable *i* and variable *result*, respectively.

SOLUTION

Register use: a, b, c arrive in `$a0`, `$a1`, `$a2` (argument registers); $i =$ `$s1`, $result =$ `$s2`; the return value goes in `$v0`. Because `$s1`, `$s2` are **callee-saved**, the procedure saves/restores them on the stack.

Efficiency points: a `do-while` loop needs *no* test at the top — the body runs once and the loop-closing branch is at the bottom (one branch per iteration). MIPS has no `bgt/blt`, so $i > b \Leftrightarrow b < i$ is computed with `slt` and combined with `beq/bne` against `$zero`.

```
testFunc: addi $sp, $sp, -8       # save callee-saved registers
          sw   $s1, 4($sp)
          sw   $s2, 0($sp)
          add  $s1, $zero, $zero  # i = 0
          add  $s2, $zero, $zero  # result = 0
Loop:     slt  $t0, $a1, $s1      # $t0 = (b < i)  i.e. (i > b)
          beq  $t0, $zero, Skip   # if !(i > b) skip the add
          add  $s2, $s2, $a2      # result += c
Skip:     addi $s1, $s1, 1        # i++
          slt  $t0, $s1, $a0      # $t0 = (i < a)
          bne  $t0, $zero, Loop   # repeat while (i < a)
          add  $v0, $s2, $zero    # return result
          lw   $s2, 0($sp)        # restore $s2, $s1
          lw   $s1, 4($sp)
          addi $sp, $sp, 8
          jr   $ra
```

The loop body is only 6 instructions (5 when the `if` is false). If the caller does not need `$s1`, `$s2` preserved, the 6 save/restore instructions can be dropped.

2020–2021 • Q12(b) CSE 305 • 24/09/2022 • IS19 **MARKS 10**

Consider the following MIPS code:

```
L1: bne $t0, $t1, L3
```

```
L2: j LX
L3: addi $t0, $t1, 1
```

Now, if the opcode of *j*, *bne* and *addi* are 2, 5 and 8 respectively, the register `$t0` and `$t1` are indicated by register values 8 and 9 respectively, and the addresses are

$$\text{L1} \to 1000, \qquad \text{L2} \to 1004, \qquad \text{L3} \to 1008, \qquad \text{LX} \to 131072,$$

then, write down the instruction format name and the instruction values for each of the above-mentioned instructions. The following example is provided as a hint.
Example: Consider the following MIPS instruction: `add $s1, $s2, $s3`. The instruction format is R-format and instruction values should be:

Address	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
0400	0	18	19	17	0	0

SOLUTION

Method

R	op (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	op (6)	rs (5)	rt (5)	constant / offset (16)		
J	op (6)	address (26)				

- `bne rs, rt, L` and `addi rt, rs, imm` are **I-format**; `j` is **J-format**.
- Branch (PC-relative): offset $= \dfrac{\text{target} - (\text{PC}_{\text{branch}} + 4)}{4}$ words. Jump (pseudo-direct): field $= \dfrac{\text{target}}{4}$; the upper 4 bits of the target come from PC+4.

Encoding

Addr	Instruction	Format	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
1000	`bne $t0,$t1,L3`	I-format	5	8	9	1		
1004	`j LX`	J-format	2	32768				
1008	`addi $t0,$t1,1`	I-format	8	9	8	1		

- `bne`: rs $=$ `$t0` $= 8$, rt $=$ `$t1` $= 9$; offset $= (1008 - (1000 + 4))/4 = 1$.
- `j`: field $= 131072/4 = 32768$. Since $131072 < 2^{28}$, the upper 4 bits are 0, the same as those of PC+4.
- `addi $t0,$t1,1`: rs $=$ `$t1` $= 9$ (source), rt $=$ `$t0` $= 8$ (destination), immediate 1.

Answer. bne: I-format (5, 8, 9, 1) $=$ `0x15090001`; j: J-format (2, 32768) $=$ `0x08008000`; addi: I-format (8, 9, 8, 1) $=$ `0x21280001`.

NOTE In the hint, the last (funct) field of `add $s1,$s2,$s3` is printed as 0; the correct funct for `add` is 32 (the other fields 0, 18, 19, 17, 0 are right).

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2020–2021 • Q13(d) CSE 305 • 24/09/2022 • IS20 **MARKS 5**

What are the main differences between x86 architecture and MIPS architecture?

2016–2017 • Q4(c) CSE 305 • 02/08/2017 • IS20 **MARKS 6**

Compare the characteristics of typical CISC and RISC architectures.

2014–2015 • Q1(c) CSE 305 • 26/01/2016 • IS20 **MARKS 8**

Compare the characteristics of typical CISC and RISC architectures.

2011–2012 • Q2(e) CSE 305 • 27/07/2013 • IS20 **MARKS 6**

Give three examples of complexity of IA-32.

SOLUTION

x86 (IA-32) is the classic **CISC** ISA; MIPS is a typical **RISC** ISA (slides: "MIPS: typical of RISC ISAs, c.f. x86"). One table answers all three comparison cards:

Feature	CISC / x86 (IA-32)	RISC / MIPS
Instruction length	variable, 1 to 15 bytes	fixed, 32 bits (one word)
Formats	many, irregular (prefix, opcode, ModR/M, SIB, displacement, immediate)	3 regular formats (R, I, J), fields in fixed places
Operands of ALU ops	register–memory: an ALU instruction may read/write memory directly	load–store: only `lw`/`sw` touch memory; ALU ops use registers only
Number of operands	2 (destination is also a source: `add eax, ebx`)	3 (`add $t0,$s1,$s2`)
Registers	few: 8 general-purpose (16 in x86-64), many special uses	32 general-purpose registers
Addressing modes	many (base + scaled index + displacement, absolute, . . .)	few (register, immediate, base+offset, PC-relative, pseudo-direct)
Instruction set	large, includes complex ops (string move `rep movs`, `enter`/`leave`)	small set of simple instructions
Conditions	condition codes (flags) set as side effects; `cmp` + `jl`	no flags: `slt` writes a register, then `beq`/`bne`
Control unit	usually microcoded; modern x86 converts instructions into RISC-like micro-ops	hardwired, simple decoding
CPI / pipelining	many cycles, varies by instruction; hard to pipeline	about 1 cycle per instruction; easy to pipeline
Code size	compact (fewer, denser instructions)	larger (more, simpler instructions)
Who does the work	hardware (complex instructions)	compiler (combines simple instructions)

2011–2012 Q2(e) • three examples of IA-32 complexity

(i) **Variable-length, irregular encoding.** An instruction is 1–15 bytes: optional prefixes, a 1–2 byte opcode, a ModR/M byte, an optional SIB byte, a 0/1/2/4-byte displacement and a

0/1/2/4-byte immediate. The decoder cannot know where the next instruction starts until it has partly decoded this one.

(ii) **Memory operands and complex addressing modes.** Most ALU instructions accept one operand in memory (e.g. `add [ebx+esi*4+12], eax`). The address is base + scaled index + displacement, and the same instruction can read memory, compute and write memory back. This makes pipelining and exception handling hard.

(iii) **Backward compatibility keeps adding to the ISA** (slides: "Backward compatibility ⇒ instruction set doesn't change, but they do accrete more instructions"). 8086 16-bit modes and segment registers are still there, and on top came 32-bit and 64-bit extensions, x87 floating point, MMX, SSE and AVX: thousands of instructions, special-purpose registers and flags that are set as side effects.

Because of this, modern x86 processors translate each instruction into simple RISC-like *micro-operations* in hardware, then pipeline those.

2019–2020 • Q3(a) CSE 305 • 18/09/2021 • IS21 **MARKS 6 + 5 + 4**

Write down the equivalent MIPS code for each of the following the C code fragments.

(i) Code Fragment 1:

```
sum = 0;
x = 100;
while (x>=-5)
{
    sum = sum + x;
    x = x-1;
}
```

(ii) Code Fragment 2:

```
if(x>y+131075)
    x=1;
```

(iii) Code Fragment 3:

```
x = A[131075]+5;
```

SOLUTION

Register assignment (the question leaves it to us)

`sum` = `$s0`, `x` = `$s1`, `y` = `$s2`, base address of `A` = `$s3`. Immediates are 16-bit signed, so 131075 (=`0x00020003`) and large array offsets need `lui`.

(i) while loop. x starts at $100 \geq -5$, so the first test is always true. The loop can therefore be compiled with the test at the bottom (one branch per iteration instead of a branch plus a jump):

```
      add  $s0, $zero, $zero  # sum = 0
      addi $s1, $zero, 100    # x = 100
Loop: add  $s0, $s0, $s1      # sum = sum + x
      addi $s1, $s1, -1       # x = x - 1
      slti $t0, $s1, -5       # $t0 = 1 if x < -5
      beq  $t0, $zero, Loop   # repeat while x >= -5
```

(The general top-tested form is `Loop: slti $t0,$s1,-5; bne $t0,$zero,Exit;` body; `j Loop; Exit:`.) On exit, $x = -6$ and sum $= \sum_{k=-5}^{100} k = 5050 - 15 = 5035$.

(ii) $x > y + 131075 \iff (y + 131075) < x$, which is tested with `slt`:

```
        lui  $t0, 2             # $t0 = 131072
        ori  $t0, $t0, 3        # $t0 = 131075
        add  $t0, $s2, $t0      # y + 131075
        slt  $t1, $t0, $s1      # (y+131075) < x ?
        beq  $t1, $zero, Exit
        addi $s1, $zero, 1      # x = 1
Exit:
```

(iii) The byte offset of `A[131075]` is $4 \times 131075 = 524300 =$ `0x0008000C`, far beyond the 16-bit displacement. Put the upper part in a register with `lui` and keep the small remainder (`0x000C` $= 12$) as the displacement:

```
        lui  $t0, 8             # $t0 = 0x0008 0000 = 524288
        add  $t0, $t0, $s3      # $t0 = A + 524288
        lw   $t1, 12($t0)       # $t1 = Mem[A + 524300] = A[131075]
        addi $s1, $t1, 5        # x = A[131075] + 5
```

Answer. (i) A bottom-tested loop using `slti $t0,$s1,-5` and `beq $t0,$zero,Loop`; it ends with sum $= 5035$. (ii) `lui 2/ori 3` build 131075, then add, `slt`, `beq` and `addi $s1,$zero,1`. (iii) `lui $t0,8`; `add $t0,$t0,$s3`; `lw $t1,12($t0)`; `addi $s1,$t1,5`.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q3(b) CSE 305 • 18/09/2021 • IS22 **MARKS 12**

With an appropriate example, briefly describe how pseudoinstructions work. Suppose, there is a critical section in your code where performance matters. Do you think it is always a good idea to write the assembly code by yourself for such type of scenario? Justify your answer. (6+2+4)

2015–2016 • Q5(c) CSE 305 • 18/01/2017 • IS22 **MARKS 4**

What do you understand by *pseudoinstructions*? Give two examples of *pseudoinstructions* that MIPS assembler accepts.

SOLUTION

What a pseudoinstruction is and how it works

Most assembly instructions map one-to-one onto machine instructions. A **pseudoinstruction** is a "figment of the assembler's imagination" (slides): it is accepted in assembly language but has *no* opcode in the hardware. The **assembler** replaces it with one or more real MIPS instructions. When it needs a scratch register for this, it uses `$at` (register 1, *assembler temporary*), which is reserved for that purpose. The programmer gets a richer instruction set, and the hardware stays small and regular.

Pseudoinstruction	Meaning	Real instructions generated by the assembler
`move $t0, $t1`	`$t0` ← `$t1`	`add $t0, $zero, $t1`
`blt $t0, $t1, L`	branch if `$t0` < `$t1`	`slt $at, $t0, $t1; bne $at, $zero, L`
`bge $t0, $t1, L`	branch if `$t0` ≥ `$t1`	`slt $at, $t0, $t1; beq $at, $zero, L`
`li $t0, 0x12345678`	load 32-bit constant	`lui $at, 0x1234; ori $t0, $at, 0x5678`
`la $a0, label`	load address	`lui` + `ori` with the upper/lower halves of the address

Example of the process (2019–2020 card): the programmer writes `blt $s1, $s2, Less`. There is no `blt` opcode, because a $<$ comparator inside a branch would slow every instruction. So during translation the assembler emits the two real instructions `slt $at,$s1,$s2` and `bne $at,$zero,Less`, and computes the branch offset as usual. The object file contains only real machine instructions.

Answer. (2015–2016 card) Two examples: `move $t0,$t1` → `add $t0,$zero,$t1`, and `blt $t0,$t1,L` → `slt $at,$t0,$t1` + `bne $at,$zero,L`.

Hand-written assembly for a performance-critical section? (2019–2020 card)

No, not always. The slides list "Use assembly code for high performance" as a *fallacy*:

- Modern optimizing compilers are *better* than most people at making fast code for modern processors. They schedule instructions around pipeline hazards, allocate registers, unroll loops and use the cache well, and these trade-offs change with every processor generation.
- More lines of code mean more errors and less productivity. Assembly is hard to read, debug and maintain.
- Assembly is tied to one ISA. It is not portable, and it cannot automatically benefit from a newer compiler or processor.

Hand-coding makes sense only for a tiny, measured hot spot, after profiling shows that the compiler's output really is slower (or for special instructions such as `ll/sc` or SIMD that the compiler does not use).

2019–2020 • Q3(c) CSE 305 • 18/09/2021 • IS23 **MARKS 8**

Why does MIPS multiplication operation always strictly demand for three operands? Why has the number of registers in MIPS architecture been restricted to 32? (4+4)

SOLUTION

Why three operands — Design Principle 1: Simplicity favours regularity

- Every MIPS arithmetic/logical instruction has the *same* form: two sources and one destination, `op dest, src1, src2` (`add`, `sub`, `and`, ..., and `mul $v0,$a0,$v0` as used in `fact` in the slides). Multiplication follows the same rule, so it fits the R-format (op, rs, rt, rd, shamt, funct) like every other ALU instruction.
- **Regularity makes the hardware simpler.** The register file always reads two registers (rs, rt) and writes one (rd), and decoding needs no special cases for instructions with 1, 2 or variable operand counts. Simpler hardware gives higher performance at lower cost.
- Three operands also mean the sources are not destroyed (unlike the x86 two-operand form `imul eax, ebx`), which saves extra `move` instructions. A longer expression is just a sequence of 3-operand instructions.

(The hardware `mult rs, rt` writes its 64-bit product into the special `Hi/Lo` pair, which acts as the implicit third operand. The three-register form `mul rd, rs, rt` keeps the regular 3-operand format.)

Why only 32 registers — Design Principle 2: Smaller is faster

- A larger register file is **slower**: more entries mean longer word/bit lines and bigger decoders and multiplexers, so the access time, and with it the clock cycle, grows. Registers must be read and written every cycle, so this cost hits every instruction.

- **Encoding:** each register field is $\log_2 32 = 5$ bits. An R-format instruction already uses $3 \times 5 = 15$ bits for register numbers. With 64 registers it would need 18 bits, leaving fewer bits for opcode, funct and the 16-bit immediate in a fixed 32-bit instruction.
- 32 registers are *enough* for compilers to keep the frequently used variables in registers and spill the rest (c.f. main memory with millions of locations). The designers balanced cycle time, instruction size and spill traffic.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2018–2019 • Q6(c) CSE 305 • 29/10/2019 • IS24 **MARKS 5**

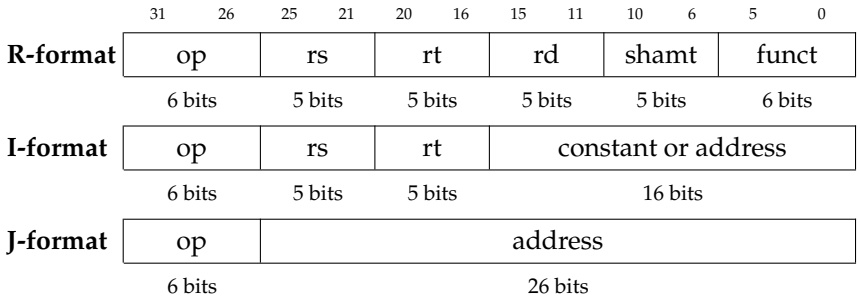
How many instruction formats are there in MIPS? Write down the name of each instruction format along with the length of each field therein.

2017–2018 • Q8(c) CSE 305 • 23/09/2018 • IS24 **MARKS 10**

How many instruction formats are there in MIPS? Write down the name of each instruction format along with the length of each field in it.

SOLUTION

MIPS has **three** instruction formats. Every instruction is 32 bits long, and the formats keep the opcode and the register fields in the same positions (*good design demands good compromises*).



Format	Fields and their meaning	Example instructions
R (register)	op = 0 (opcode); rs, rt = source registers; rd = destination; shamt = shift amount; funct = selects the exact operation	`add`, `sub`, `and`, `slt`, `sll`, `jr`
I (immediate)	op; rs = source / base register; rt = destination (or source for `sw`, `beq`); 16-bit signed constant, load/store offset or branch offset (-2^{15} to $2^{15}-1$)	`addi`, `lw`, `sw`, `beq`, `bne`, `lui`
J (jump)	op; 26-bit word address, target = $PC_{31..28}$: (address $\times 4$)	`j`, `jal`

Answer. 3 formats: R = 6+5+5+5+5+6, I = 6+5+5+16, J = 6+26 bits (each 32 bits in total).

2018–2019 • Q7(a) CSE 305 • 29/10/2019 • IS25 **MARKS 10**

Consider the following MIPS assembly language code and assume that the code fragment is loaded at location 60000 in the memory:

```
Loop:   sll  $t1, $s2, 2
        add  $t1, $t1, $s6
        lw   $t0, 0($t1)
        beq  $t0, $s5, Lp1
        j    Exit
Lp1:    addi $s2, $s2, 1
        j    Loop
Exit:
```

Write down the MIPS machine code with appropriate decimal values for the above assembly code.

SOLUTION

Method

- Formats: R = op(6) rs(5) rt(5) rd(5) shamt(5) funct(6); I = op(6) rs(5) rt(5) imm(16); J = op(6) address(26).
- Registers: `$t0`= 8, `$t1`= 9, `$s2`= 18, `$s5`= 21, `$s6`= 22. Opcodes: R-type 0 (funct `sll` 0, `add` 32), `lw` 35, `beq` 4, `addi` 8, `j` 2. For `sll rd, rt, shamt`, rs = 0.
- Branch offset = $(\text{target} - (\text{PC}+4))/4$; jump field = target/4.

Instructions are 4 bytes apart, so `Loop` = 60000, `Lp1` = 60020, `Exit` = 60028.

Addr	Instruction	op	rs	rt	rd	shamt	funct
60000	`Loop: sll $t1,$s2,2`	0	0	18	9	2	0
60004	`add $t1,$t1,$s6`	0	9	22	9	0	32
60008	`lw $t0,0($t1)`	35	9	8	0		
60012	`beq $t0,$s5,Lp1`	4	8	21	1		
60016	`j Exit`	2	15007				
60020	`Lp1: addi $s2,$s2,1`	8	18	18	1		
60024	`j Loop`	2	15000				
60028	`Exit:`	...					

Answer. beq offset = $(60020 - 60016)/4 = 1$; `j Exit` field = $60028/4 = 15007$; `j Loop` field = $60000/4 = 15000$.

2018–2019 • Q8(a) CSE 305 • 29/10/2019 • IS26 **MARKS 20**

Consider the following C code which calculates `a^b` (a raised power b),

```
int fun (int a, int b)
{
    if (b == 0)
        return 0,
    else if (b % 2 == 0)
        return fun (a*a, b/2);
    return fun (a* a, b/2) *a;
}
```

Write down the equivalent MIPS assembly code for it. Use a minimal number of MIPS instructions.

SOLUTION

Method

- Arguments: `a` in `$a0`, `b` in `$a1`. The result goes in `$v0`. `b%2` → `andi 1`. `b/2` → `sra 1` (assuming $b \geq 0$, an exponent). `a*a` needs `mul` (as in the slides' `fact`).
- Both recursive branches use the *same* new arguments `(a*a, b/2)`, so they are computed once, before the parity test.
- Even b: `return fun(a*a,b/2)` returns the callee's value unchanged, so it is a **tail call**. A plain branch to `fun` is enough, because `$ra` still points to the original caller.
- Odd b: the result must still be multiplied by the original `a`, so this path is non-leaf. It saves `$ra` and `a`, calls with `jal`, then restores both.

```
fun:    bne  $a1, $zero, L1      # if (b == 0)
        add  $v0, $zero, $zero   #     return 0      (see note)
        jr   $ra
L1:     andi $t0, $a1, 1         # $t0 = b % 2
        add  $t1, $a0, $zero     # $t1 = original a
        mul  $a0, $a0, $a0       # 1st arg = a*a
        sra  $a1, $a1, 1         # 2nd arg = b/2
        beq  $t0, $zero, fun     # b even: return fun(a*a, b/2)  (tail call)
        addi $sp, $sp, -8        # b odd: non-leaf, save $ra and a
        sw   $ra, 4($sp)
        sw   $t1, 0($sp)
        jal  fun                 # $v0 = fun(a*a, b/2)
        lw   $t1, 0($sp)         # restore a
        lw   $ra, 4($sp)
        addi $sp, $sp, 8
        mul  $v0, $v0, $t1       # return fun(a*a, b/2) * a
        jr   $ra
```

NOTE As printed, the base case is `return 0,`, with a comma (a misprint for `;`). With `return 0` the function returns 0 for every input. For a true a^b the base case must be `return 1`, so change the second line to `addi $v0, $zero, 1`; nothing else changes. Check with `return 1`: `fun(3,5)` $=$ `fun(9,2)`$\cdot 3 =$ `fun(81,1)`$\cdot 3 =$ (`fun(6561,0)`$\cdot 81) \cdot 3 = 1 \cdot 81 \cdot 3 = 243 = 3^5$.

Answer. The 17-instruction procedure above. `a*a` and `b/2` are computed once. The even case is a tail branch (`beq $t0,$zero,fun`). Only the odd case saves `$ra` and `a` in a 2-word frame and multiplies the returned value by `a`.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2018–2019 • Q8(c) CSE 305 • 29/10/2019 • IS27 **MARKS 5**

What is the problem in the following code snippet? How can you solve the problem?

Address	Instruction
005000	`beq $s0, $s1, 30500`

2014–2015 • Q7(c) CSE 305 • 26/01/2016 • IS27 **MARKS 5 + 5**

What is the problem in the following code snippet? How can you solve this problem?

Address	Instruction
005000	`beq, $s0, $s1, 30500`

SOLUTION

Branch reach

`beq` is I-format with a 16-bit signed word offset: target $=$ (PC$+4$) $+$ SignExt(offset)$\times 4$, where $-32768 \le$ offset ≤ 32767.

Problem

Addresses in a MIPS memory map are written in hex, so the instruction is at `0x00005000` and the target is `0x00030500`.

$$\text{offset} = \frac{\texttt{0x30500} - (\texttt{0x5000} + 4)}{4} = \frac{197888 - 20484}{4} = \frac{177404}{4} = 44351 > 2^{15} - 1 = 32767.$$

The offset needs 17 bits, so the target is out of `beq`'s range and the instruction cannot be encoded. (The 2014–15 card also has a stray comma, `beq,`, which is a syntax error; remove it.)

Solution (what the assembler does)

Invert the test, branch over the next instruction, and reach the target with `j`. `j` has a 26-bit word address and reaches the whole 256 MB region:

```
005000:        bne $s0, $s1, Next   # offset field = (0x5008-0x5004)/4 = 1
005004:        j   0x30500          # address field = 0x30500/4 = 0xC140 (49472)
005008: Next:  ...                  # PC+4 = 0x5008, upper 4 bits 0000 = target's
```

NOTE If 005000 and 30500 were read as decimal, the offset would be $(30500 - 5004)/4 = 6374$, which fits in 16 bits and leaves nothing to fix. The question's "what is the problem" therefore implies the hex reading used above.

Answer. The required offset is 44351, which exceeds the 16-bit limit of 32767, so `beq` cannot reach `0x30500`. Replace it with `bne $s0,$s1,Next; j 0x30500; Next:`.

2017–2018 • Q7(a) CSE 305 • 23/09/2018 • IS28 **MARKS 25**

The following C code segment can sort an array of integers in ascending order. Write down the equivalent MIPS assembly code for it. Assume appropriate registers for the variables.

```
void swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}

void sort (int v[], int n)
{
    int i, j;
```

```
    for (i = 0; i < n; i += 1) {
      for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1) {
         swap(v,j);
      }
    }
}
```

SOLUTION

Register allocation (MIPS conventions):

Procedure	Variable	Register
`swap` (leaf)	`v`, `k`; `temp`	`$a0`, `$a1`; `$t0` (temporaries are enough, nothing to save)
`sort` (non-leaf)	`v`, `n` (copies kept across `jal`)	`$s2`, `$s3` (`$a0`, `$a1` are needed to call `swap`)
	`i`, `j`	`$s0`, `$s1`

swap — leaf procedure

```
swap:   sll  $t1, $a1, 2        # $t1 = k * 4
        add  $t1, $a0, $t1      # $t1 = &v[k]
        lw   $t0, 0($t1)        # temp = v[k]
        lw   $t2, 4($t1)        # $t2 = v[k+1]
        sw   $t2, 0($t1)        # v[k] = v[k+1]
        sw   $t0, 4($t1)        # v[k+1] = temp
        jr   $ra                # return
```

sort — non-leaf procedure (calls swap)

```
sort:   addi $sp, $sp, -20      # room for 5 registers
        sw   $ra, 16($sp)       # save $ra and $s0-$s3
        sw   $s3, 12($sp)
        sw   $s2, 8($sp)
        sw   $s1, 4($sp)
        sw   $s0, 0($sp)
        move $s2, $a0           # $s2 = v   (keep across calls)
        move $s3, $a1           # $s3 = n
        move $s0, $zero         # i = 0
for1tst: slt $t0, $s0, $s3      # $t0 = (i < n)
        beq  $t0, $zero, exit1  # exit outer loop if i >= n
        addi $s1, $s0, -1       # j = i - 1
for2tst: slti $t0, $s1, 0       # $t0 = (j < 0)
        bne  $t0, $zero, exit2  # exit inner loop if j < 0
        sll  $t1, $s1, 2        # $t1 = j * 4
        add  $t2, $s2, $t1      # $t2 = &v[j]
        lw   $t3, 0($t2)        # $t3 = v[j]
        lw   $t4, 4($t2)        # $t4 = v[j+1]
        slt  $t0, $t4, $t3      # $t0 = (v[j+1] < v[j])
        beq  $t0, $zero, exit2  # exit inner loop if v[j] <= v[j+1]
        move $a0, $s2           # 1st argument: v
        move $a1, $s1           # 2nd argument: j
        jal  swap               # swap(v, j)
        addi $s1, $s1, -1       # j -= 1
        j    for2tst            # test inner loop again
exit2:  addi $s0, $s0, 1        # i += 1
        j    for1tst            # test outer loop again
```

```
exit1:  lw   $s0, 0($sp)        # restore saved registers
        lw   $s1, 4($sp)
        lw   $s2, 8($sp)
        lw   $s3, 12($sp)
        lw   $ra, 16($sp)
        addi $sp, $sp, 20       # pop the frame
        jr   $ra                # return to caller
```

Notes. The condition `j >= 0 && v[j] > v[j+1]` is evaluated with *short-circuit* order: if $j < 0$ we leave before touching `v[j]`. MIPS has no `bge/bgt`, so `slt/slti` followed by `beq/bne` against `$zero` is used. `move` is the pseudoinstruction `add rd,$zero,rs`. `$s0`–`$s3` are callee-saved, so `sort` saves them, and `$ra` too because the `jal swap` overwrites it.

2017–2018 • Q7(b) CSE 305 • 23/09/2018 • IS29 **MARKS 10**

Suppose you are required to jump to the instruction at address $(\texttt{4000 0001})_{\text{hex}}$. Write down the MIPS assembly code for this task.

SOLUTION

Why a plain `j` is not enough

`j` uses pseudodirect addressing: target $= (\text{PC}+4)_{31..28} : \text{addr}_{26} : 00$. It can only reach the current 256 MB region, and only word-aligned addresses. Code normally runs in the text segment at `0x004xxxxx` (upper bits `0000`), but `0x40000001` has upper bits `0100`. So the full 32-bit address must be built in a register and used with `jr`.

Code

An immediate has only 16 bits, so build the address with `lui` (upper half) and `ori` (lower half; zero-extends):

```
lui  $t0, 0x4000          # $t0 = 0x4000 0000
ori  $t0, $t0, 0x0001     # $t0 = 0x4000 0001
jr   $t0                  # PC = $t0
```

NOTE `0x40000001` is **not word-aligned**: instruction addresses must be multiples of 4. `j` cannot even encode it, since its low 2 bits are always 00, and `jr` to it raises an address-error exception. The code above is what the question literally asks. If the intended target is the aligned word `0x40000000`, drop the `ori` (`lui $t0,0x4000; jr $t0`).

Answer. `lui $t0,0x4000; ori $t0,$t0,0x0001; jr $t0`. A register jump is needed because the target lies outside the current 256 MB region. The odd address would trap as unaligned in real hardware.

2016–2017 • Q6(a) CSE 305 • 02/08/2017 • IS30 **MARKS 10**

A simple MIPS assembly code snippet and it's corresponding MIPS machine code is given below. Note that, the former one is not complete. Please complete it with appropriate MIPS assembly code.

	sll
	add
	lw
	bne
	addi
	j

1000	0	00000	10000	01111	00010	000000
1004	0	01111	10001	01111	00000	100000
1008	35	01111	01000	00000	00000	000000
1012	5	01000	10111	00000	00000	000001
1016	8	10000	10000	00000	00000	000001
1020	2	00000	00000	00000	00011	111010

SOLUTION

Method

The opcode (2nd column) tells us the format. With op 0 the row is **R-format**: rs, rt, rd, shamt, funct. With op 5, 8 or 35 it is **I-format**: rs, rt, then a 16-bit immediate made of the last three columns. With op 2 it is **J-format**: a 26-bit address made of the last five columns. Register numbers: `$t0`–`$t7` = 8–15, `$s0`–`$s7` = 16–23. Branch target = PC+4 + 4×offset; jump target = $PC_{31..28}$: (field ×4).

Addr	Decoded fields (decimal)	Instruction
1000	R: op 0, rs 0, rt 16, rd 15, shamt 2, funct 0 (`sll`)	`sll $t7, $s0, 2`
1004	R: op 0, rs 15, rt 17, rd 15, shamt 0, funct 32 (`add`)	`add $t7, $t7, $s1`
1008	I: op 35, rs 15, rt 8, imm 0	`lw $t0, 0($t7)`
1012	I: op 5, rs 8, rt 23, imm 1	`bne $t0, $s7, L1` ($1016 + 4 \cdot 1 = 1020$)
1016	I: op 8, rs 16, rt 16, imm 1	`addi $s0, $s0, 1`
1020	J: op 2, address $11111010_2 = 250$	`j Loop` ($250 \times 4 = 1000$)

Completed code:

```
Loop: sll  $t7, $s0, 2        # 1000
      add  $t7, $t7, $s1      # 1004
      lw   $t0, 0($t7)        # 1008
      bne  $t0, $s7, L1       # 1012   offset 1 -> 1020
      addi $s0, $s0, 1        # 1016
L1:   j    Loop               # 1020   250 x 4 = 1000
```

NOTE The 16-bit offset of `bne` is printed as 1, so its target is $1016 + 4 = 1020$ (the `j`). In the textbook version of this loop the `bne` goes to an `Exit` label after the `j`, which would need offset 2 (target 1024). The code above follows the machine code as printed.

2015–2016 • Q5(a) CSE 305 • 18/01/2017 • IS31 MARKS 25

Produce the compiled MIPS code for the following C function. Assume that addresses of `dest`, `source` in `$a0`, and `$a1`. Use a minimal number of MIPS assembly instructions.

```
void rstrcopy (char *dest, char *source)
{
    if (*source)
    {
        *dest++=*source++;
        rcopy (dest, source);
    }
    else
        *dest='\0';
}
```

SOLUTION

> **NOTE** The recursive call is printed as `rcopy(dest, source);` it is taken to be the recursive call `rstrcopy(dest, source)`.

What the function does: if the current source character is non-zero, copy it, advance both pointers and recurse; otherwise write the terminating `'\0'`. Characters are bytes, so we use `lbu`/`sb`, and `dest++`, `source++` add 1 (not 4).

(1) Direct recursive translation

`rstrcopy` is non-leaf, so it must save `$ra` before `jal`. `dest` and `source` are *not* used after the call, so they need not be saved.

```
rstrcopy: lbu  $t0, 0($a1)          # $t0 = *source
          beq  $t0, $zero, Else     # if (*source == 0) goto else-part
          addi $sp, $sp, -4         # push $ra
          sw   $ra, 0($sp)
          sb   $t0, 0($a0)          # *dest = *source
          addi $a0, $a0, 1          # dest++
          addi $a1, $a1, 1          # source++
          jal  rstrcopy             # rstrcopy(dest, source)
          lw   $ra, 0($sp)          # pop $ra
          addi $sp, $sp, 4
          jr   $ra
Else:     sb   $zero, 0($a0)        # *dest = '\0'
          jr   $ra
```

(2) Minimal version — tail-call elimination (the answer)

Three observations shrink the code:

- The recursive call is the **last action** of the function (a *tail call*). Nothing is done after it returns, so instead of `jal` + saving/restoring `$ra` we can simply `j` back to the top with the new arguments. The final `jr $ra` then returns straight to the original caller. No stack frame is needed at all.
- In the else-part, `*source` = 0, so `*dest = '\0'` is the same as `*dest = *source`. One `sb` placed *before* the test serves both branches.
- The test can then go at the *bottom*: after copying and advancing, "if the character was non-zero, call `rstrcopy` again" is a single `bne` back to the entry. (Advancing the pointers after the final `'\0'` is harmless, because they are local copies.)

```
rstrcopy: lbu  $t0, 0($a1)          # $t0 = *source
          addi $a1, $a1, 1          # source++
          sb   $t0, 0($a0)          # *dest = *source  (copies '\0' too)
          addi $a0, $a0, 1          # dest++
          bne  $t0, $zero, rstrcopy # char != 0 : tail call rstrcopy(dest, source)
          jr   $ra                  # '\0' copied: return to the original caller
```

Answer. 6 instructions (5 executed per character), with no stack usage. It gives the same result as the 13-instruction recursive version, which needs one stack frame per character. Putting `addi $a1` between `lbu` and `sb` also avoids a load-use stall in the pipeline.

2015–2016 • Q5(b) CSE 305 • 18/01/2017 • IS32 MARKS 6

What is the MIPS assembly code to load this 32-bit constant into register `$s0`?
`0001 0000 0011 1101 0000 1001 0000 0000`

SOLUTION

A 16-bit immediate cannot hold a 32-bit constant, so it is built in two steps (slides: "32-bit Constants"). `lui` copies a 16-bit constant into the upper half and clears the lower half. `ori` then ORs in the lower half; it zero-extends, so the upper half is unchanged.

- Upper half `0001 0000 0011 1101` = `0x103D` = $4096 + 61 = 4157$.
- Lower half `0000 1001 0000 0000` = `0x0900` = 2304.

```
lui  $s0, 4157          # 0x103D -> $s0 = 0001 0000 0011 1101 0000 0000 0000 0000
ori  $s0, $s0, 2304     # 0x0900 -> $s0 = 0001 0000 0011 1101 0000 1001 0000 0000
```

Answer. `lui $s0, 4157` (i.e. `0x103D`) then `ori $s0, $s0, 2304` (i.e. `0x0900`). Result: `$s0` = `0x103D0900`.

2014–2015 • Q6(a) CSE 305 • 26/01/2016 • IS33 **MARKS 15**

A simple recursive C function and it's corresponding MIPS assembly code is given below. Note that, the later one is not complete. Please complete it with appropriate expressions.

```
int recurseSum(int n){
   if(n<1) return 0;
   else return n+recurseSum(n-1);
}
```

```
recurseSum:
[      ] $t0, $a0, 1
beq $t0, $zero, Recurse
add $v0, $zero, $zero
jr [      ]
Recurse:
addi $sp, $sp, [      ]
sw $ra, [      ]
sw [      ], 0($sp)
addi $a0, $a0, -1
jal recurseSum
lw [      ], 0($sp)
lw $ra, [      ]
[                    ]
add [      ], $a0, $v0
jr [      ]
```

SOLUTION

This is the same pattern as `fact` in the slides: test $n < 1$ with `slti`. In the recursive case, save `$ra` and n (a 2-word frame), call with $n - 1$, restore, pop, then add n to the returned value.

```
recurseSum:
        slti $t0, $a0, 1          # [slti]   $t0 = (n < 1)
        beq  $t0, $zero, Recurse  # n >= 1 -> recursive case
        add  $v0, $zero, $zero    # return 0
        jr   $ra                  # [$ra]
Recurse:
        addi $sp, $sp, -8         # [-8]       room for 2 words
        sw   $ra, 4($sp)          # [4($sp)]   save return address
        sw   $a0, 0($sp)          # [$a0]      save n
        addi $a0, $a0, -1         # argument n-1
        jal  recurseSum           # $v0 = recurseSum(n-1)
```

```
lw   $a0, 0($sp)          # [$a0]      restore n
lw   $ra, 4($sp)          # [4($sp)]   restore return address
addi $sp, $sp, 8          # [addi $sp, $sp, 8]  pop the frame
add  $v0, $a0, $v0        # [$v0]      n + recurseSum(n-1)
jr   $ra                  # [$ra]      return
```

Answer. Blanks in order: `slti`; `$ra`; `-8`; `4($sp)`; `$a0`; `$a0`; `4($sp)`; `addi $sp, $sp, 8`; `$v0`; `$ra`.
(The two offsets only need to be consistent: since `$a0` is stored at `0($sp)`, `$ra` must go in the other word, `4($sp)`, of the 8-byte frame.)

2014–2015 • Q6(c) CSE 305 • 26/01/2016 • IS34 **MARKS 15**

A simple MIPS assembly code snippet and it's corresponding MIPS machine code are given below. Note that, the later one is not complete. Please complete it with appropriate decimal values.

```
Loop: sll $t1, $s2, 2
      add $t1, $t1, $s6
      lw $t0, 0($t1)
      beq $t0, $s5, L1
      j Exit
L1:   addi $s2, $s2, 1
      j Loop
Exit:
```

6000	0	0	18	9		0
6004	0				0	32
6008	35			0		
6012	4	8	21			
6016						
6020	8	18	18	1		
6024	2					
6028						

SOLUTION

Method

- Formats: R = op rs rt rd shamt funct (6-5-5-5-5-6); I = op rs rt imm (6-5-5-16); J = op address (6-26).
- Registers: `$t0`= 8, `$t1`= 9, `$s2`= 18, `$s5`= 21, `$s6`= 22. `j` opcode = 2.
- `sll rd, rt, shamt`: shamt = 2. `add rd, rs, rt`: rs = `$t1`, rt = `$s6`, rd = `$t1`. `lw rt, imm(rs)`: rs = `$t1`, rt = `$t0`.
- Branch offset = $(\text{target} - (\text{PC}+4))/4$; jump field = target/4.

Blanks filled in **teal**:

```
sll $t1,$s2,2
add $t1,$t1,$s6
lw $t0,0($t1)
beq $t0,$s5,L1
j Exit
L1:  addi $s2,$s2,1
j Loop
Exit:
```

6000	0	0	18	9	2	0
6004	0	9	22	9	0	32
6008	35	9	8	0		
6012	4	8	21	1		
6016	2	1507				
6020	8	18	18	1		
6024	2	1500				
6028	...					

Answer. beq offset = $(6020 - 6016)/4 = 1$; `j Exit` field = $6028/4 = 1507$; `j Loop` field = $6000/4 = 1500$
(the other blanks: shamt 2; add 9, 22, 9; lw 9, 8; j opcode 2).

2014–2015 • Q7(b) CSE 305 • 26/01/2016 • IS35 **MARKS 10**

MIPS instruction set architecture supports *beq* and *bne* instructions, however, does not support other branching instructions like *blt*, *bge*, etc. What is the reason behind this design decision? How could you program these instructions (*blt*, *bge*, etc.) in MIPS instructions? (5+5)

SOLUTION

Reason — "Good design demands good compromises" (slides: Branch Instruction Design)

- Testing $=$ / $\neq$ needs only an XOR/zero-detect of the two registers, which is fast. Testing $<, \geq$ needs a full **subtraction with carry propagation** (a magnitude comparison), which is **slower**.
- Combining a slow comparison with the branch in one instruction means more work per instruction, so the **clock cycle must be longer**, and *all* instructions are penalized, not only the branches.
- `beq/bne` are the **common case** (comparisons against zero and equality tests dominate). The rarer $<, \geq$ cases are done in two fast instructions instead: `slt` (an ordinary ALU operation) + `beq/bne`.

Programming them — `slt/slti` + `beq/bne` (the assembler does this with `$at`)

Pseudo-branch	Condition	Real MIPS instructions
`blt $s0, $s1, L`	$s0 < s1$	`slt $at, $s0, $s1` `bne $at, $zero, L`
`bge $s0, $s1, L`	$s0 \geq s1$, i.e. NOT($s0 < s1$)	`slt $at, $s0, $s1` `beq $at, $zero, L`
`bgt $s0, $s1, L`	$s0 > s1$, i.e. $s1 < s0$	`slt $at, $s1, $s0` `bne $at, $zero, L`
`ble $s0, $s1, L`	$s0 \leq s1$, i.e. NOT($s1 < s0$)	`slt $at, $s1, $s0` `beq $at, $zero, L`
`blt $s0, 10, L`	$s0 < 10$ (constant)	`slti $at, $s0, 10` `bne $at, $zero, L`

`slt rd, rs, rt` sets rd $= 1$ if rs $<$ rt, else 0. For unsigned operands use `sltu/sltiu` instead.

2013–2014 • Q6(b) CSE 305 • 05/07/2015 • IS36 **MARKS 12**

Write down the MIPS code and corresponding machine code for the following C code.

```
A[3] = A[1] - A[2];
```

Consider the base address of A is in `$s0`. The op-code for load word is 35, and for store word is 43. The (op-code, function-code) for addition is (0, 32), and for subtraction is (0, 34). Registers `$t0` to `$t7` are numbered from 8 to 15, and `$s0` to `$s7` are numbered from 16 to 23.

SOLUTION

MIPS code

Memory is byte addressed and an `int` is 4 bytes, so `A[k]` is at offset $4k$: `A[1]`$\rightarrow$ 4, `A[2]`$\rightarrow$ 8, `A[3]`$\rightarrow$ 12.

```
lw  $t0, 4($s0)      # $t0 = A[1]
lw  $t1, 8($s0)      # $t1 = A[2]
sub $t0, $t0, $t1    # $t0 = A[1] - A[2]
sw  $t0, 12($s0)     # A[3] = $t0
```

Machine code

`lw`/`sw` are I-format: op, rs = base (`$s0`= 16), rt = data register, 16-bit offset. `sub rd, rs, rt` is R-format: op 0, rs, rt, rd, shamt 0, funct 34.

MIPS code	op	rs	rt	rd	shamt	funct	hex
`lw $t0, 4($s0)`	35	16	8		4		`0x8E080004`
`lw $t1, 8($s0)`	35	16	9		8		`0x8E090008`
`sub $t0, $t0, $t1`	0	8	9	8	0	34	`0x01094022`
`sw $t0, 12($s0)`	43	16	8		12		`0xAE08000C`

Binary:
```
100011 10000 01000 0000000000000100
100011 10000 01001 0000000000001000
000000 01000 01001 01000 00000 100010
101011 10000 01000 0000000000001100
```

2013–2014 • Q6(c) CSE 305 • 05/07/2015 • IS37 **MARKS 12**

Write down the MIPS code for the following C procedure:

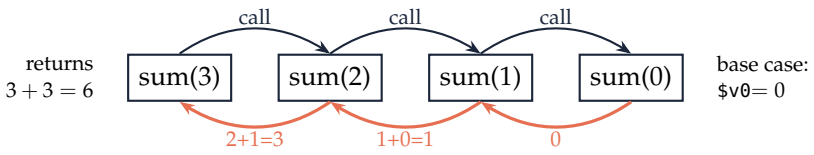
```
int sum (int n)
{
    if (n <=0) return 0;
    else return n + sum (n - 1);
}
```

Consider n in `$a0` and result in `$v0`.

SOLUTION

`sum` is a recursive (non-leaf) procedure, structured like `fact` in the slides. For integers, $n \leq 0 \Leftrightarrow n < 1$, so a single `slti $t0,$a0,1` gives the test. Each recursive call saves `$ra` (overwritten by `jal`) and n (needed after the call) in a 2-word stack frame.

```
sum:    slti $t0, $a0, 1        # $t0 = (n < 1)  i.e. (n <= 0)
        beq  $t0, $zero, L1     # n > 0 -> recursive case
        add  $v0, $zero, $zero  # return 0
        jr   $ra
L1:     addi $sp, $sp, -8       # push 2 words
        sw   $ra, 4($sp)        # save return address
        sw   $a0, 0($sp)        # save n
        addi $a0, $a0, -1       # argument n - 1
        jal  sum                # $v0 = sum(n-1)
        lw   $a0, 0($sp)        # restore n
        lw   $ra, 4($sp)        # restore return address
        addi $sp, $sp, 8        # pop 2 words
        add  $v0, $a0, $v0      # return n + sum(n-1)
        jr   $ra
```



2012–2013 • Q1(a) CSE 305 • 10/05/2014 • IS38 **MARKS 15**

What is a jump address table? Discuss with an appropriate example how Case/Switch Statements can be implemented with a jump address table.

SOLUTION

Jump address table

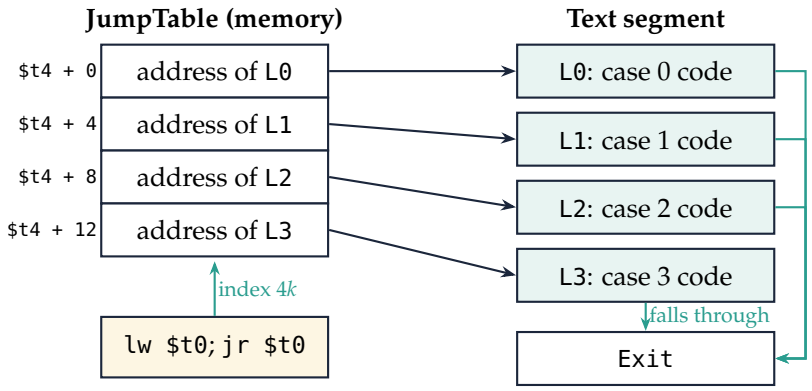
A **jump address table** (jump table) is an array of words in memory. Entry k holds the *address* of the code for alternative k. To select alternative k, the program indexes the table (`base` $+4k$), loads that address into a register and jumps to it with `jr` (slides: `jr` "can also be used for computed jumps, e.g. for case/switch statements"). This costs a constant number of instructions for any k, whereas a chain of `if-else` tests costs up to one compare-and-branch per case.

Example

```
switch (k) {
  case 0: f = i + j; break;
  case 1: f = g + h; break;
  case 2: f = g - h; break;
  case 3: f = i - j; break;
}
```

Assume f, g, h, i, j, k are in `$s0`–`$s5`, and `$t4` holds the base address of `JumpTable`, whose four words contain the addresses of labels `L0`–`L3`.

```
        sltiu $t3, $s5, 4       # $t3 = (0 <= k < 4) ; unsigned test also rejects k < 0
        beq   $t3, $zero, Exit  # k out of range -> no case matches
        sll   $t1, $s5, 2       # $t1 = 4 * k  (byte offset of entry k)
        add   $t1, $t1, $t4     # $t1 = &JumpTable[k]
        lw    $t0, 0($t1)       # $t0 = JumpTable[k] = address of Lk
        jr    $t0               # jump to the selected case
L0:     add   $s0, $s3, $s4     # k = 0 : f = i + j
        j     Exit              # break
L1:     add   $s0, $s1, $s2     # k = 1 : f = g + h
        j     Exit
L2:     sub   $s0, $s1, $s2     # k = 2 : f = g - h
        j     Exit
L3:     sub   $s0, $s3, $s4     # k = 3 : f = i - j  (last case: no jump needed)
Exit:
```



Every case ends with `j Exit`, which is the `break`. The range check is required, because an out-of-range k would load a garbage address. (Two signed tests `slt $t3,$s5,$zero/bne` and `slti $t3,$s5,4/beq` do the same job as the single `sltiu`.)

2012–2013 • Q1(b) CSE 305 • 10/05/2014 • IS39 **MARKS 5**

How you can create a 32 bit constant in MIPS Assembly Language?

SOLUTION

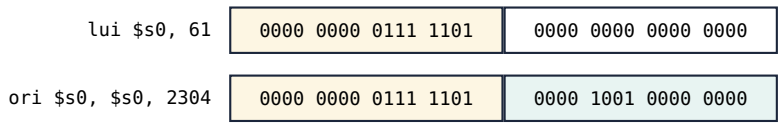
I-format instructions hold only a 16-bit immediate, which covers the common case of small constants. For the occasional 32-bit constant MIPS uses two instructions (slides: "32-bit Constants"):

(i) `lui rt, upper16` (*load upper immediate*): copies the 16-bit constant into the *left* 16 bits of rt and clears the right 16 bits to 0.

(ii) `ori rt, rt, lower16`: ORs the lower 16 bits in. `ori` **zero-extends** its immediate, so the upper half is untouched. `addi` should not be used here: it sign-extends, which corrupts the upper half whenever bit 15 is 1.

Example (slides): load `0000 0000 0111 1101 0000 1001 0000 0000` (=`0x007D0900`) into `$s0`:

```
lui  $s0, 61              # $s0 = 0000 0000 0111 1101 0000 0000 0000 0000
ori  $s0, $s0, 2304       # $s0 = 0000 0000 0111 1101 0000 1001 0000 0000
```



The assembler does this automatically for pseudoinstructions such as `li $s0, 0x007D0900` or `la`, using `$at` where needed.

Answer. Use `lui` to put the upper 16 bits in place, then `ori` to insert the lower 16 bits. Example: `lui $s0,61`; `ori $s0,$s0,2304` gives `$s0` = `0x007D0900`.

2012–2013 • Q1(c) CSE 305 • 10/05/2014 • IS40 **MARKS 8**

Consider the following MIPS assembly language code and assume that the code fragment is loaded at location 80000 in the memory. What should be the values of the immediate fields of *bne* instruction and *j* instruction?

```
Loop:   sll  $t1,$s3,2
        add  $t1,$t2,$s6
        Lw   $t0, o($t1)
        bnr  $to,$s5, Exit
        addi $s3, $s3, 1
        j    Loop
Exit:
```

SOLUTION

Method

- `bne` is I-format and uses **PC-relative** addressing: the 16-bit field holds offset $= (\text{target} - (\text{PC}_{\text{bne}} + 4))/4$ words, because PC has already been incremented by 4.
- `j` is J-format and uses **pseudo-direct** addressing: the 26-bit field holds target/4, and the upper 4 bits of the target come from PC+4.

Each instruction is 4 bytes, so: `sll` 80000, `add` 80004, `lw` 80008, `bne` 80012, `addi` 80016, `j` 80020, `Exit` 80024.

Addr	Instruction
80000	`Loop: sll $t1,$s3,2`
80004	`add $t1,$t1,$s6`
80008	`lw $t0,0($t1)`
80012	`bne $t0,$s5,Exit`
80016	`addi $s3,$s3,1`
80020	`j Loop`
80024	`Exit:`

op	rs	rt	rd	shamt	funct
0	0	19	9	2	0
0	9	22	9	0	32
35	9	8	0		
5	8	21	2		
8	19	19	1		
2	20000				
...					

Answer. bne immediate $= (80024 - (80012 + 4))/4 = 2$; j address field $= 80000/4 = 20000$.

NOTE The printed code has typos: `bnr $to` is read as `bne $t0`, `Lw $t0, o($t1)` as `lw $t0, 0($t1)`, and `add $t1,$t2,$s6` is taken as `add $t1,$t1,$s6` (as in the slides). None of these changes the two immediate fields.

2012–2013 • Q1(d) CSE 305 • 10/05/2014 • IS41 **MARKS 7**

Explain why the assembler might have problems directly implementing the branch instruction in the following code fragment. How can this problem be fixed?

```
Here:  beq $s0, $s2, there
       ...
       ...
there: add $s0, $s0, $s0
```

SOLUTION

Why there may be a problem

`beq` is I-format, op (6) | rs (5) | rt (5) | offset (16), and uses PC-relative addressing:

$$\text{target} = (\text{PC} + 4) + \text{SignExt}(\text{offset}) \times 4, \qquad -2^{15} \le \text{offset} \le 2^{15} - 1.$$

The "..." between `Here` and `there` may contain more than 2^{15} (32768) instructions, i.e. more than about 128 KB. In that case the word offset does not fit in 16 bits, and the assembler cannot encode the branch directly.

Fix (slides: "Branching far away")

Invert the condition, hop over one instruction, and use an unconditional `j`, whose 26-bit field reaches anywhere in the current 256 MB region:

```
Here:  bne  $s0, $s2, Skip    # opposite test; offset field = 1 (always fits)
       j    there             # pseudodirect: PC[31:28] : addr26 : 00
Skip:  ...                    # continues with the code after the old beq
       ...
there: add  $s0, $s0, $s0
```

If `there` were even in a different 256 MB region (different $PC_{31..28}$), the `j` would be replaced by a register jump: `lui $at, hi(there); ori $at, $at, lo(there); jr $at`.

Answer. `there` may be more than 2^{15} words from PC+4, so the 16-bit branch offset overflows. The assembler rewrites the branch as `bne $s0,$s2,Skip; j there; Skip:`. For a target outside the 256 MB region it uses `lui/ori/jr`.

2011–2012 • Q2(b) CSE 305 • 27/07/2013 • IS42 MARKS 7.5

Which instructions and registers are used in procedure call in MIPS hardware? What are the purpose of using these registers and instructions?

SOLUTION

Instructions

- `jal ProcedureLabel` (*jump and link*) puts the address of the following instruction (PC+4) in `$ra` and jumps to the procedure, so it does both the call and remembering where to come back.
- `jr $ra` (*jump register*) copies `$ra` into the PC, which returns to the caller. (`jr` with another register is also used for computed jumps, e.g. switch tables.)
- `sw`/`lw` and `addi $sp,$sp,`$\pm$`n` push and pop registers on the stack (procedure frame).

Registers

Register	Number	Purpose
`$a0`–`$a3`	4–7	pass the first four arguments to the procedure
`$v0`, `$v1`	2, 3	return the result values to the caller
`$ra`	31	return address, written by `jal` and used by `jr`
`$sp`	29	stack pointer: top of the stack, where registers/locals are saved (stack grows down)
`$fp`	30	frame pointer: fixed base of the current procedure frame
`$gp`	28	global pointer to the static data segment
`$t0`–`$t9`	8–15, 24–25	temporaries, which the callee may overwrite (the caller saves them if it needs them)
`$s0`–`$s7`	16–23	saved registers, which the callee must save and restore if it uses them

Call sequence (slides): place parameters in `$a0`–`$a3` $\rightarrow$ `jal` transfers control $\rightarrow$ callee acquires storage on the stack (moves `$sp`, saves `$ra`/`$s`-registers) $\rightarrow$ performs its task $\rightarrow$ puts the result in `$v0`/`$v1` $\rightarrow$ restores registers and `$sp` $\rightarrow$ `jr $ra` returns to the caller.

03

Arithmetic Logic Unit (ALU) Design

2023–2024 • Q5(b) CSE 209 • 29/04/2026 • AL01 **MARKS 15**

Suppose you want to design a 4-bit ALU where the inputs to the ALU are A $(A_3A_2A_1A_0)$ and B $(B_3B_2B_1B_0)$, and the selection variables are S_2, S_1, and S_0. Also, consider X_i, Y_i and Z_i as the inputs to the i-th parallel adder of the ALU. The ALU must satisfy the following functional design specification given in Table for Question 5(b) that has to be realized through the parallel adders.

S_2	S_1	S_0	Required Functions
0	0	0	$F = A - B - 1$
0	0	1	$F = A + B + 1$
0	1	0	$F =$ Transfer A
0	1	1	$F =$ Decrement A
1	0	x	$F = A$ OR B
1	1	x	$F = A'$

Table for Question 5(b)

Devise the input equations (X_i, Y_i, Z_i) for the parallel adders of the ALU.

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_2 = 0$): $X_i = A_i$. Choose $Y_i \in \{0, B_i, B_i', 1\}$ and the input carry C_0 so that $F = A + Y + C_0$. For example $A - B - 1 = A + B'$, $A - 1 = A + 1111$, and $A = A + 1111 + 1$.
- **Logic** ($S_2 = 1$): kill every carry with $Z_i = S_2'C_i$, so $F_i = X_i \oplus Y_i$. Keep Y_i close to the arithmetic Y_i and modify X_i. OR: $X = A + B$, $Y = 0$. NOT: $X = A$, $Y = 1$.

$S_2S_1S_0$	F	X_i	Y_i	C_0	
000	$A - B - 1 = A + B'$	A_i	B_i'	0	
001	$A + B + 1$	A_i	B_i	1	
010	Transfer A	A_i	1	1	$(A + 1111 + 1 = A)$
011	Decrement A	A_i	1	0	
10x	A OR B	$A_i + B_i$	0	– $(Z = 0)$	
11x	A'	A_i	1	– $(Z = 0)$	

Transfer A uses $Y = 1111$ with carry 1 instead of $Y = 0$ with carry 0. This makes $Y_i = 1$ whenever $S_1 = 1$. For $S_2S_1 = 00$, Y_i is B_i' when $S_0 = 0$ and B_i when $S_0 = 1$, i.e. $S_0 \odot B_i$. For $S_2S_1 = 10$, $Y_i = 0$. X_i differs from A_i only for OR. $C_0 = 1$ only for rows 001 and 010.

Answer. $X_i = A_i + S_2S_1'B_i$, $Y_i = S_1 + S_2'(S_0 \odot B_i)$, $C_0 = S_1 \oplus S_0$, $Z_0 = S_2'(S_1 \oplus S_0)$, $Z_i = S_2'C_i$ $(i = 1, 2, 3)$.

The design was checked by simulating all (S, A, B) combinations.

2022–2023 • Q11(a) CSE 209 • 13/01/2025 • AL02 **MARKS 7**

Why is the overflow bit (V) computed by XOR-ing the last two carry bits in binary arithmetic operations?

SOLUTION

In n-bit 2's-complement addition, overflow means the true result does not fit in n bits, which can only happen when both operands have the *same* sign and the result sign differs (slides: $+$ve $+$ $+$ve gives sign 1, or $-$ve $+$ $-$ve gives sign 0). Look at the sign-bit (MSB) full adder: inputs a_{n-1}, b_{n-1}, carry-in c_{n-1}; outputs sum (sign) s_{n-1} and carry-out c_n.

a_{n-1}	b_{n-1}	c_{n-1}	s_{n-1}	c_n	Case
0	0	0	0	0	$+ + \to +$: fine
0	0	1	1	0	$+ + \to -$: **overflow**, $c_{n-1} \neq c_n$
0/1	1/0	0	1	0	mixed signs: never overflows, $c_{n-1} = c_n$
0/1	1/0	1	0	1	mixed signs: never overflows, $c_{n-1} = c_n$
1	1	0	0	1	$- - \to +$: **overflow**, $c_{n-1} \neq c_n$
1	1	1	1	1	$- - \to -$: fine

- Both operands positive $\Rightarrow c_n = 0$ always; the sign becomes wrongly 1 exactly when a carry *enters* the sign bit ($c_{n-1} = 1$).
- Both negative $\Rightarrow c_n = 1$ always; the sign becomes wrongly 0 exactly when *no* carry enters ($c_{n-1} = 0$).
- Mixed signs $\Rightarrow c_n = c_{n-1}$ always, and no overflow is possible.

So overflow $\iff$ carry into the sign bit $\neq$ carry out of it, i.e. $V = c_{n-1} \oplus c_n$ (in the slides' 8-bit ALU, $V = C_8 \oplus C_9$). One XOR gate replaces a comparison of three sign bits, and it works for subtraction too because $A - B$ is done as $A + \overline{B} + 1$. Example (4-bit): $0111 + 0001 = 1000$ ($7 + 1 \neq -8$): $c_3 = 1, c_4 = 0 \Rightarrow V = 1$.

Answer. Because overflow in 2's-complement occurs exactly when the carry into the MSB differs from the carry out of the MSB: $V = C_{n-1} \oplus C_n$.

2022–2023 • Q11(d) CSE 209 • 13/01/2025 • AL03 **MARKS 10**

Design a 4-bit Arithmetic Logic Unit (ALU) with three selection variables, S_0, S_1, and S_2. These selecting variables control the ALU to perform various operations on two 4-bit inputs, labeled as A and B. Derive the input equations (X_i, Y_i, and Z_i) for the parallel adders used in the ALU that satisfy the following functional design specifications, (where X means don't care).

S_2	S_1	S_0	**Required Operation**
0	0	0	$-A - B - 2$
0	0	1	Add 1's complement of A to B
0	1	0	$A + B$
0	1	1	$A - B$
1	0	x	A AND B
1	1	x	A XOR B

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_2 = 0$): $F = X + Y + C_0$ with $X_i \in \{A_i, A_i'\}$, $Y_i \in \{B_i, B_i'\}$ and input carry C_0. In 2's complement $-A = A' + 1$, so $-A - B - 2 = (A' + 1) + (B' + 1) - 2 = A' + B'$ and $A - B = A + B' + 1$.
- **Logic** ($S_2 = 1$): kill every carry with $Z_i = S_2'C_i$, so $F_i = X_i \oplus Y_i$. XOR: $X = A$, $Y = B$. AND: $X = A + B'$, $Y = B'$, since $(A + B') \oplus B' = AB$.

$S_2S_1S_0$	F	X_i	Y_i	C_0
000	$-A - B - 2 = A' + B'$	A_i'	B_i'	0
001	$A' + B$	A_i'	B_i	0
010	$A + B$	A_i	B_i	0
011	$A - B = A + B' + 1$	A_i	B_i'	1
10x	AND	$A_i + B_i'$	B_i'	$-(Z = 0)$
11x	XOR	A_i	B_i	$-(Z = 0)$

X_i is A_i' when $S_2S_1 = 00$ and A_i otherwise, plus the AND term. So $X_i = (A_i \oplus S_2'S_1') + S_2S_1'B_i'$. Y_i, in the order $000, 001, 010, 011, 10x, 11x$, is B', B, B, B', B', B. So B_i is complemented exactly when $S_1' \oplus S_2'S_0 = 1$.

Answer. $X_i = (A_i \oplus S_2'S_1') + S_2S_1'B_i'$, $Y_i = B_i \oplus S_1' \oplus S_2'S_0$, $C_0 = S_1S_0$, $Z_0 = S_2'S_1S_0$, $Z_i = S_2'C_i$ $(i = 1, 2, 3)$.

The design was checked by simulating all (S, A, B) combinations.

2022–2023 • Q5(b) CSE 305 • 25/04/2024 • AL04 **MARKS 14**

Howard wants to demonstrate his engineering skills, but Sheldon sets a challenging constraint for him. Howard is tasked with designing a 4-bit Arithmetic Logic Unit (ALU), and the constraint is that he cannot use any parallel adder. Instead, he is only allowed to use parallel subtractors and logic gates for the ALU design. The ALU should have three selection variables, denoted as S0, S1, and S2. These selection variables control the ALU to perform various operations on two 4-bit inputs, Labelled as A and B. Your task is to help Howard design the 4-bit Arithmetic Logic Unit (ALU) so that it can do the following operations:

S_2	S_1	S_0	*Operation*
0	0	0	A − B
0	0	1	A + B
0	1	0	−A −B − 2
0	1	1	B − A
1	0	0	A XOR B
1	1	0	A AND B

SOLUTION

Method: the ALU-design approach (slides) with a parallel *subtractor*

This follows the slides' Xi/Yi/Zi method, with a parallel subtractor in place of the adder. Each bit i

is a full-subtractor stage with inputs X_i, Y_i and borrow-in Z_i:

$$D_i = X_i \oplus Y_i \oplus Z_i, \qquad b_{i+1} = X_i'Y_i + X_i'Z_i + Y_iZ_i, \qquad \text{4-bit result } D = X - Y - b_0.$$

- **Arithmetic** ($S_2 = 0$): rewrite each operation as $X - Y - b_0$ with $X \in \{A, A'\}$, $Y \in \{B, B'\}$, using $-A = A' + 1$ (2's complement):

$$A + B = A - (B' + 1) = A - B' - 1, \qquad -A - B - 2 = (A' + 1) - B - 2 = A' - B - 1,$$
$$B - A = (-A - 1) - (-B - 1) = A' - B'.$$

- **Logic** ($S_2 = 1$): the difference bit is the same XOR as an adder's sum bit. Killing every borrow ($Z_i = S_2'b_i$) gives $D_i = X_i \oplus Y_i$. XOR: $X = A$, $Y = B$. AND: $X = A + B'$, $Y = B'$, since $(A + B') \oplus B' = AB$.

$S_2S_1S_0$	F	X_i	Y_i	b_0
000	$A - B$	A_i	B_i	0
001	$A + B = A - B' - 1$	A_i	B_i'	1
010	$-A - B - 2 = A' - B - 1$	A_i'	B_i	1
011	$B - A = A' - B'$	A_i'	B_i'	0
100	XOR	A_i	B_i	$-(Z = 0)$
110	AND	$A_i + B_i'$	B_i'	$-(Z = 0)$
101, 111	unused (don't care)			

X_i is complemented when $S_2S_1 = 01$, and B_i' is ORed in for AND ($S_2S_1 = 11$). Y_i is B_i' in rows 001, 011 and 110. Using the don't-care rows, the complement condition becomes $S_0 + S_2S_1$.

Answer. $X_i = (A_i \oplus S_2'S_1) + S_2S_1B_i'$, $Y_i = B_i \oplus (S_0 + S_2S_1)$, $b_0 = S_1 \oplus S_0$, $Z_0 = S_2'(S_1 \oplus S_0)$, $Z_i = S_2'\, b_i$ (b_i = borrow-out of stage $i - 1$).

> **NOTE** The question does not say how the subtractor's borrow is modelled. We take the standard full subtractor, $D = X - Y - b_{\text{in}}$. Rows 101 and 111 are not in the table, so we treat them as don't-cares.

The design was checked by simulating all (S, A, B) combinations.

2020–2021 • Q10(b) CSE 305 • 24/09/2022 • AL05 **MARKS 18**

Suppose you want to design a 4-bit ALU where the inputs to the ALU are A $(A_3A_2A_1A_0)$ and B $(B_3B_2B_1B_0)$, and the selection variables are S_3, S_2, S_1 and S_0. Also, consider X_i, Y_i and Z_i as the inputs to the i-th parallel adder of the ALU. The ALU must satisfy the following functional design specification (given in Table 10b) that has to be realized through the parallel adders. Your task is to derive the input equations (X_i, Y_i and Z_i) for the parallel adders of the ALU. You must design using the least possible number of gates.

S_3	S_2	S_1	S_0	Required Functions
0	0	0	0	$F = A$
0	0	0	1	$F = A - 1$
0	0	1	0	$F = A + B + 1$
0	0	1	1	$F = A + B$
0	1	0	0	$F = A - B$
0	1	0	1	$F = A - B - 1$
0	1	1	0	$F = A + 1$
0	1	1	1	$F = A$
1	0	0	X	$F = A'$
1	0	1	X	$F = AB'$
1	1	0	X	$F = A \odot B$ [XNOR]
1	1	1	X	$F = A \vee B$ [OR]

Table 10(b): Required functional design specification

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_3 = 0$): $X_i = A_i$. Choose $Y_i \in \{0, B_i, B_i', 1\}$ and the input carry C_0 so that $F = A + Y + C_0$ (e.g. $A - B = A + B' + 1$, $A - 1 = A + 1111$).
- **Logic** ($S_3 = 1$): kill every carry with $Z_i = S_3'C_i$, so $F_i = X_i \oplus Y_i$. Choose the (X_i, Y_i) pair that *reuses* the arithmetic Y_i for the same S_2S_1, to save gates. NOT: $(A, 1)$. AB': $(A+B,\ B)$, since $(A + B) \oplus B = AB'$. XNOR: (A, B'). OR: $(A+B,\ 0)$.

Every arithmetic row uses $X_i = A_i$. Y_i depends on S_2S_1: $00 \to S_0$ $(A\ /\ A - 1)$, $01 \to B_i$, $10 \to B_i'$, $11 \to 0$. For row 0111 ($F = A$) we choose $Y = 0$, $C_0 = 0$ so that $11 \to 0$ holds for both values of S_0.

$S_3S_2S_1S_0$	F	X_i	Y_i	C_0
0000 / 0001	$A\ /\ A - 1$	A_i	0 / 1	0
0010 / 0011	$A + B + 1\ /\ A + B$	A_i	B_i	1 / 0
0100 / 0101	$A - B\ /\ A - B - 1$	A_i	B_i'	1 / 0
0110 / 0111	$A + 1\ /\ A$	A_i	0	1 / 0
100x	A'	A_i	1	–
101x	AB'	$A_i + B_i$	B_i	–
110x	XNOR	A_i	B_i'	–
111x	OR	$A_i + B_i$	0	–

X_i changes only when $S_3S_1 = 11$. Y_i is a 4-to-1 selection on S_2S_1 of $(S_0 + S_3,\ B_i,\ B_i',\ 0)$. $C_0 = 1$ only for 0010, 0100 and 0110.

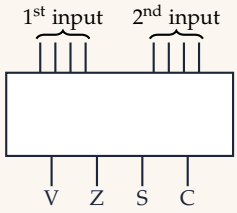
Answer. $X_i = A_i + S_3S_1B_i$, $Y_i = S_2'S_1'(S_0 + S_3) + S_2'S_1B_i + S_2S_1'B_i'$, $C_0 = S_0'(S_1 + S_2)$, $Z_0 = S_3'S_0'(S_1 + S_2)$, $Z_i = S_3'C_i$ $(i = 1, 2, 3)$.

NOTE In the scan of Table 10(b), the "B" in rows $A + B$ and $A - B$ is a degraded glyph that looks like "6". We read it as B. The S_3 column header is handwritten in the scan.

The design was checked by simulating all (S, A, B) combinations.

2020–2021 • Q12(d) CSE 305 • 24/09/2022 • AL06 **MARKS 8**

Consider the following module.



This module takes two 4-bit inputs, and shows only the four status bits (V: Overflow, Z: Zero, S: Sign and C: Carry) considering the subtraction operation of the second input from the first input. If the first 4-bit input is expressed by A and second 4-bit input is expressed by B, draw the diagram of the complete circuit that can determine whether A>B, A=B, or A<B.

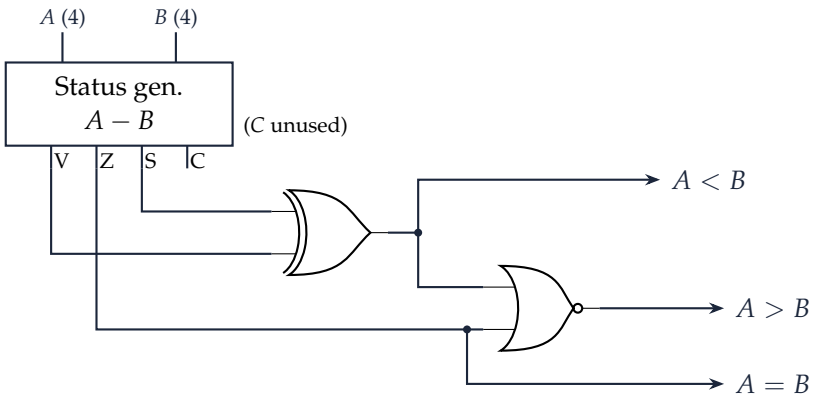
SOLUTION

Reading the status bits after $A - B$ (Mano, ALU status register)

The module computes $A + \overline{B} + 1$. $Z = 1$ iff $A = B$. For **signed** (2's-complement) numbers the true sign of $A - B$ is $S \oplus V$ (the sign bit is wrong exactly when overflow occurred); for **unsigned** numbers $C = 1$ means no borrow, i.e. $A \geq B$.

Relation	signed (2's complement)	unsigned
$A < B$	$S \oplus V = 1$	$C = 0$ (borrow)
$A = B$	$Z = 1$	$Z = 1$
$A > B$	$(S \oplus V)' \cdot Z' = (S \oplus V + Z)'$	$C \cdot Z'$

Assuming signed 4-bit operands: $A < B = S \oplus V$, $A = B = Z$, $A > B = \overline{(S \oplus V) + Z}$ (one XOR + one NOR).



Answer. $A < B = S \oplus V$; $A = B = Z$; $A > B = \overline{(S \oplus V) + Z}$ (signed). For unsigned inputs: $A < B = \overline{C}$, $A = B = Z$, $A > B = C \cdot \overline{Z}$.

2019–2020 • Q2(a) CSE 305 • 18/09/2021 • AL07 **MARKS 20**

Suppose you want to design a 4-bit ALU where the inputs to the ALU are A $(A_3A_2A_1A_0)$ and B $(B_3B_2B_1B_0)$, and the selection variables are S_3, S_2, S_1 and S_0. Also, consider X_i, Y_i and Z_i as the

inputs to the ith parallel adder of the ALU. The ALU must satisfy the following functional design specification (given in Table 2a) that has to be realized through the parallel adders. Your task is to derive the input equations (X_i, Y_i and Z_i) for the parallel adders of the ALU. You must design using the least possible number of gates.

S_3	S_2	S_1	S_0	**Required Functions**
0	0	0	0	$F = A'$
0	0	0	1	$F = A' + 1$
0	0	1	0	$F = A' + B$
0	0	1	1	$F = A' + B + 1$
0	1	0	0	$F = A' - B - 1$
0	1	0	1	$F = A' - B$
0	1	1	0	$F = A' - 1$
0	1	1	1	$F = A'$
1	0	0	x	$F = (AB)'$ [NAND]
1	0	1	x	$F = A \odot B$ [XNOR]
1	1	0	x	$F = A \oplus B$ [XOR]
1	1	1	x	$F = A \vee B$ [OR]

Table 2a: Required Functional design specification

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_3 = 0$): this is exactly the slides' Table 9-1, but with $X_i = A_i'$. Y_i is selected by S_2S_1: $00 \to 0$, $01 \to B$, $10 \to B'$, $11 \to 1111$, giving $Y_i = S_1B_i + S_2B_i'$. S_0 is the input carry: $C_0 = S_0$. (Check: $A' - B - 1 = A' + B'$, $A' - B = A' + B' + 1$, $A' - 1 = A' + 1111$, $A' = A' + 1111 + 1$.)
- **Logic** ($S_3 = 1$): kill every carry with $Z_i = S_3'C_i$, so $F_i = X_i \oplus Y_i$. To save gates, keep the same Y_i and solve $X_i \oplus Y_i = F_i$ for X_i.

$S_3S_2S_1$	F	Y_i (fixed)	required X_i	check
100	NAND	0	$A_i' + B_i'$	$= (AB)'$
101	XNOR	B_i	A_i'	$A' \oplus B = A \odot B$
110	XOR	B_i'	A_i'	$A' \oplus B' = A \oplus B$
111	OR	1	$A_i'B_i'$	$(A'B')' = A + B$

So $X_i = A_i'$ everywhere except two rows. Let $K = S_3S_2S_1$ and $M = S_3S_2'S_1'$; each is one gate, shared by all four stages. Then $X_i = A_i'(KB_i)' + MB_i'$.

Answer. $X_i = A_i'(S_3S_2S_1B_i)' + S_3S_2'S_1'B_i'$, $Y_i = S_1B_i + S_2B_i'$, $C_0 = S_0$, $Z_0 = S_3'S_0$, $Z_i = S_3'C_i$ $(i = 1, 2, 3)$.

The design was checked by simulating all (S, A, B) combinations.

2019–2020 • Q2(b) CSE 305 • 18/09/2021 • AL08 **MARKS 15**

Suppose, you want to implement a hardware unit that would produce the value of F using the following logic (the logic is given in C syntax).

```
if ((A>=B && C!=D) || X==0)
      F=1;
else
      F=0
```

Here, each of the variables A, B, C, D and X is of 4-bits where the variable F is of 1-bit. After searching the hardware stationaries, you bought some basic gates. Also, you bought some status-generator modules (see Figure 2b) each of which takes two 4-bit inputs, and shows only the four status bits (V: Overflow, Z: Zero, S: Sign and C: Carry) considering the subtraction operation of the 2nd input from the 1st input.

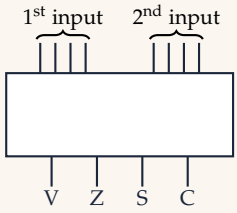


Figure 2b: Status-generator module

Now, (just) draw the circuit diagram of the full hardware unit using basic gates and status-generator modules.

SOLUTION

Reading the status bits after $P - Q$ (Mano, ALU status register)

A module computes $P + \overline{Q} + 1$. $Z = 1$ iff $P = Q$, so $P \neq Q = Z'$. For **signed** (2's-complement) numbers the true sign of $P - Q$ is $S \oplus V$, so $P \geq Q \iff S \oplus V = 0$; for **unsigned** numbers $P \geq Q \iff C = 1$ (no borrow). Comparing X with the constant 0000 gives $X == 0 \iff Z = 1$.

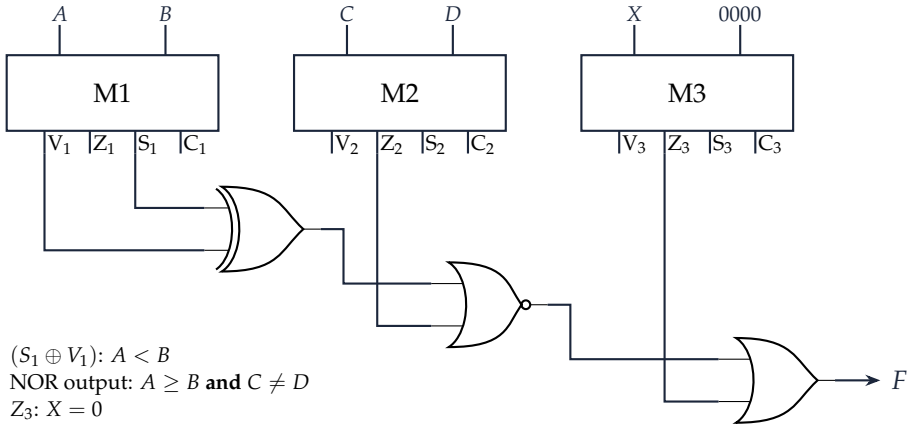
Design (signed 4-bit operands assumed)

Use three modules: M1 on (A, B), M2 on (C, D), M3 on $(X, 0000)$.

$$A \geq B = (S_1 \oplus V_1)', \qquad C \neq D = Z_2', \qquad X == 0 \;=\; Z_3$$

$$F = (S_1 \oplus V_1)' \cdot Z_2' + Z_3 = \overline{(S_1 \oplus V_1) + Z_2} + Z_3$$

So one XOR, one NOR and one OR gate suffice.



Answer. $F = \overline{(S_1 \oplus V_1) + Z_2} + Z_3$ (signed). For unsigned A, B: $F = C_1 \cdot \overline{Z_2} + Z_3$.

2018–2019 • Q5(d) CSE 305 • 29/10/2019 • AL09 **MARKS 10**

Design a 4-bit ALU with three selection variables S_0, S_1 and S_2 that performs the following operations on two 4-bit inputs (A and B):

S_2	S_1	S_0	**Operation**
0	0	0	Addition
0	0	1	Transfer A
0	1	0	Add 1's complement of B to A
0	1	1	Increment A
1	0	0	XOR
1	0	1	NOT
1	1	0	AND
1	1	1	OR

Draw the logic diagram of one typical stage of your designed ALU.

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

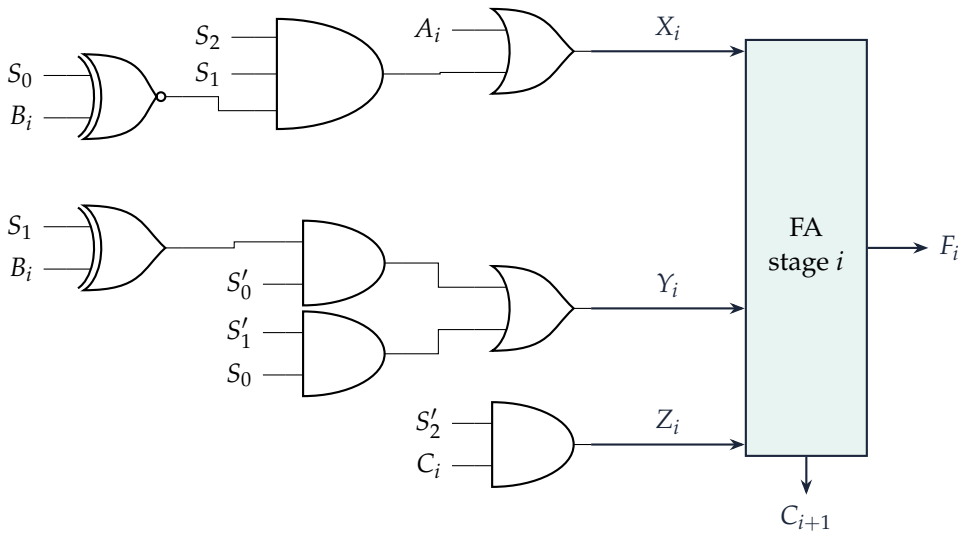
- **Arithmetic** ($S_2 = 0$): $X_i = A_i$. Choose $Y_i \in \{0, B_i, B_i', 1\}$ and the input carry C_0 so that $F = A + Y + C_0$.
- **Logic** ($S_2 = 1$): kill every carry with $Z_i = S_2'C_i$, so $F_i = X_i \oplus Y_i$. Keep Y_i close to the arithmetic Y_i and modify X_i. XOR: (A, B). NOT: $(A, 1)$. AND: $(A{+}B', B')$. OR: $(A{+}B, 0)$.

$S_2S_1S_0$	F	X_i	Y_i	C_0	
000	$A + B$	A_i	B_i	0	
001	Transfer A	A_i	1	1	$(A + 1111 + 1 = A)$
010	$A + B'$	A_i	B_i'	0	
011	$A + 1$	A_i	0	1	
100	XOR	A_i	B_i	–	
101	NOT	A_i	1	–	
110	AND	$A_i + B_i'$	B_i'	–	
111	OR	$A_i + B_i$	0	–	

Transfer A uses (Y=1, C_0=1) instead of (Y=0, C_0=0). This makes Y_i independent of S_2: $S_1S_0 = 00 \to B$, $01 \to 1$, $10 \to B'$, $11 \to 0$. $C_0 = 1$ exactly when $S_2'S_0 = 1$.

Answer. $X_i = A_i + S_2S_1(S_0 \odot B_i)$, $Y_i = S_0'(S_1 \oplus B_i) + S_1'S_0$, $C_0 = S_0$, $Z_i = S_2'C_i$ (so $Z_0 = S_2'S_0$).

One typical stage:



The complements S_0', S_1', S_2' come from one inverter per select line, shared by all stages. In stage 0, $C_0 = S_0$ is fed in place of C_i. The design was checked by simulating all (S, A, B) combinations.

2017–2018 • Q6(a–b) CSE 305 • 23/09/2018 • AL10 **MARKS 25 + 10**

(a) Design an ALU with three selection variables S_2, S_1 and S_0 that performs the following operations on two 4-bit inputs (A and B): (25)

S_2	S_1	S_0	**Operation**
0	0	0	Addition
0	0	1	Transfer A
0	1	0	Decrement A
0	1	1	Add 1's complement of B to A
1	0	0	XOR
1	0	1	AND
1	1	0	NOT
1	1	1	OR

Draw the logic diagram of one typical stage of your designed ALU.
(b) For each arithmetic operation listed in question 6(a), find out the conditions under which the output carry will be equal to 1. (10)

SOLUTION

Method (slides: *ALU Design*)
Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_2 = 0$): $X_i = A_i$. Choose $Y_i \in \{0, B_i, B_i', 1\}$ and the input carry C_0 so that $F = A + Y + C_0$ (decrement: $A + 1111$).
- **Logic** ($S_2 = 1$): kill every carry with $Z_i = S_2'C_i$, so $F_i = X_i \oplus Y_i$. Keep Y_i close to the arithmetic Y_i and modify X_i. XOR: (A, B). AND: $(A{+}B', B')$. NOT: $(A, 1)$. OR: $(A{+}B, 0)$.

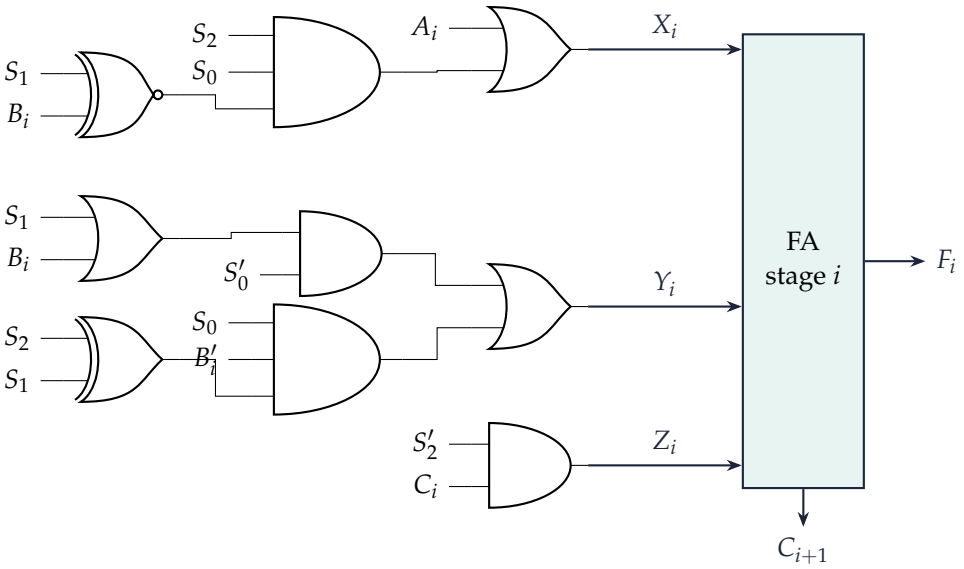
(a) Design

$S_2S_1S_0$	F	X_i	Y_i	C_0
000	$A+B$	A_i	B_i	0
001	Transfer A	A_i	0	0
010	$A-1$	A_i	1	0
011	$A+B'$	A_i	B_i'	0
100	XOR	A_i	B_i	–
101	AND	A_i+B_i'	B_i'	–
110	NOT	A_i	1	–
111	OR	A_i+B_i	0	–

Every arithmetic row has $C_0 = 0$, so no carry-in logic is needed. For $S_0 = 0$: $Y_i = S_1 + B_i$ (rows $x00 \to B$, $x10 \to 1$). For $S_0 = 1$: $Y_i = B_i'$ when $S_2 \neq S_1$ (rows 011, 101) and 0 otherwise (rows 001, 111).

Answer. $X_i = A_i + S_2S_0(S_1 \odot B_i)$, $Y_i = S_0'(S_1 + B_i) + S_0B_i'(S_2 \oplus S_1)$, $C_0 = 0$, $Z_i = S_2'C_i$ (so $Z_0 = 0$).

One typical stage:



$S_2 \oplus S_1$ is computed once and shared by all four stages. In stage 0 the C_i input is tied to 0. The design was checked by simulating all (S, A, B) combinations.

(b) When is the output carry $C_{out} = 1$? ($n = 4$; cf. slide Table 9-2)

Operation (adder does)	$C_{out} = 1$ when	Comment
Addition: $A + B$	$A + B \geq 2^n = 16$	unsigned overflow; F holds only the low 4 bits
Transfer A: $A + 0 + 0$	never	C_{out} is always 0
Decrement: $A + 1111$	$A \neq 0$	$A + 15 \geq 16 \iff A \geq 1$; for $A = 0$, $F = 1111$, $C_{out} = 0$
$A + B'$ $(= A - B - 1)$	$A > B$	$A + 15 - B \geq 16 \iff A > B$; else F = 1's comp. of $(B - A)$

04

Arithmetic for Computers

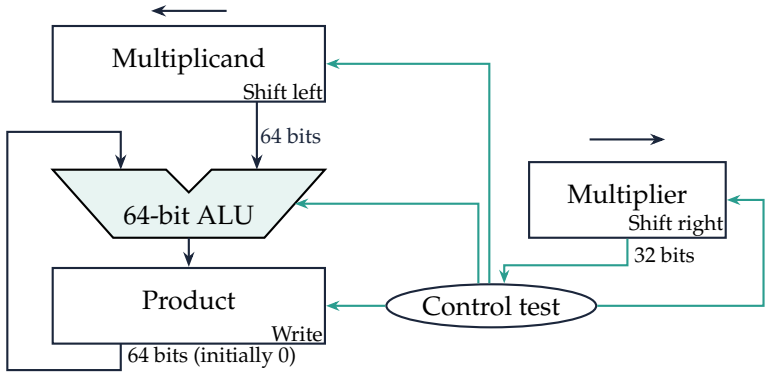
2023–2024 • Q7(a) CSE 209 • 29/04/2026 • AR01 **MARKS 20**

Draw the diagram of the integer multiplication hardware following (i) long multiplication approach, and (ii) parallel add/shift approach assuming 32 bit multiplier and 32 bit multiplicand. For each approach, also simulate the steps of the multiplication with an example for two 4 bit integer numbers: 1000 (multiplicand), 1011 (multiplier).

SOLUTION

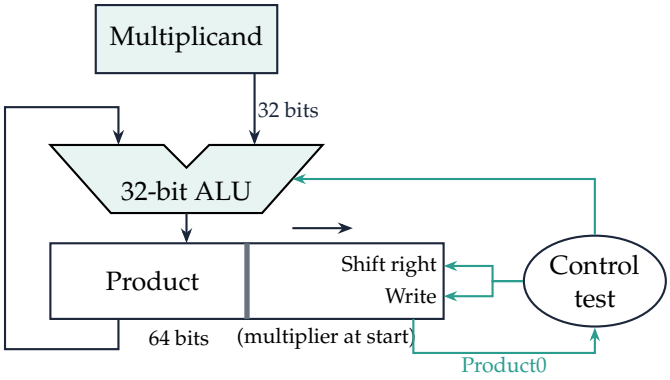
(i) Long-multiplication hardware (first version)

This copies pencil-and-paper multiplication. Each step tests Multiplier0, the multiplier's LSB. If it is 1, the multiplicand is added to the product. Then the multiplicand shifts left and the multiplier shifts right. For 32×32 bits this needs a 64-bit Multiplicand register (multiplicand in its right half), a 64-bit ALU, a 64-bit Product register (initially 0) and a 32-bit Multiplier register. It runs 32 iterations of up to 3 steps each.



(ii) Parallel add/shift hardware (refined version)

The multiplicand stays still and the *product shifts right*, so only a 32-bit Multiplicand register and a 32-bit ALU are needed. The ALU adds into the left half of the product. The multiplier is loaded into the right half of the 64-bit Product register, and the bit tested is Product0. The add and the shift are done in the same cycle, so the multiply takes 32 cycles.



Simulation of 1000×1011 • **expected** $8 \times 11 = 88 = 0101\,1000_2$

Pencil-and-paper view: $1000 \times 1011 = 1000 + 10000 + 0 + 1000000 = 1011000_2$.

(i) Long-multiplication hardware (scaled to 4 bits: 8-bit Multiplicand and Product, 4-bit Multiplier):

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	1011	0000 1000	0000 0000
1	1a: 1 ⇒ Prod = Prod + Mcand	101**1**	0000 1000	0000 1000
	2: Shift left Multiplicand	1011	0001 0000	0000 1000
	3: Shift right Multiplier	0101	0001 0000	0000 1000
2	1a: 1 ⇒ Prod = Prod + Mcand	010**1**	0001 0000	0001 1000
	2: Shift left Multiplicand	0101	0010 0000	0001 1000
	3: Shift right Multiplier	0010	0010 0000	0001 1000
3	1: 0 ⇒ no operation	001**0**	0010 0000	0001 1000
	2: Shift left Multiplicand	0010	0100 0000	0001 1000
	3: Shift right Multiplier	0001	0100 0000	0001 1000
4	1a: 1 ⇒ Prod = Prod + Mcand	000**1**	0100 0000	0101 1000
	2: Shift left Multiplicand	0001	1000 0000	0101 1000
	3: Shift right Multiplier	0000	1000 0000	**0101 1000**

(ii) Parallel add/shift hardware (4-bit Multiplicand, 4-bit ALU, 8-bit Product with the multiplier in its right half):

Iteration	Step	Multiplicand	Product
0	Initial values (multiplier in right half)	1000	0000 1011
1	1a: Prod0 = 1 ⇒ left half + = Mcand	1000	1000 101**1**
	2: Shift right Product	1000	0100 0101
2	1a: Prod0 = 1 ⇒ 0100 + 1000	1000	1100 010**1**
	2: Shift right Product	1000	0110 0010
3	1: Prod0 = 0 ⇒ no operation	1000	0110 001**0**
	2: Shift right Product	1000	0011 0001
4	1a: Prod0 = 1 ⇒ 0011 + 1000	1000	1011 000**1**
	2: Shift right Product	1000	**0101 1000**

Answer. Both approaches give $1000 \times 1011 = 0101\,1000_2 = 88_{10}$ after 4 iterations (32 iterations for 32-bit operands). The first version needs up to 3 steps per iteration; the add/shift version needs 1 cycle per iteration.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q7(b) CSE 209 • 29/04/2026 • AR02 **MARKS 15**

What is the purpose of using a positive BIAS value in IEEE 754 floating point representation? Devise the smallest and largest values that can be expressed in (i) single precision (32 bit), and (ii) double precision (64 bits) using IEEE 754 floating point representation.

2018–2019 • Q7(c) CSE 305 • 29/10/2019 • AR02 **MARKS 5**

What is the use of Bias in floating point representation? Give the encoding of the floating point "infinity".

2015–2016 • Q7(c) CSE 305 • 18/01/2017 • AR02 **MARKS 10**

What is the use of *Bias* in floating point representation? Give the encoding of floating point `infinity`.

2012–2013 • Q8(a) CSE 305 • 10/05/2014 • AR02 **MARKS 5**

What is the purpose of using biased notation for representing exponents of floating point numbers?

SOLUTION

Purpose of the bias (all four cards)

IEEE 754 stores the exponent in **excess (biased) form**: stored E = actual exponent + Bias, with Bias $= 127$ (single, 8-bit E) or 1023 (double, 11-bit E).

- **The exponent field is unsigned.** Negative exponents need no sign bit or 2's complement. The most negative exponent is stored as $00\ldots01$ and the most positive as $11\ldots10$.
- **Fast comparison and sorting.** The layout is sign | exponent | fraction. With a biased exponent, a larger exponent is always a larger unsigned bit pattern, so two FP numbers can be compared (and sorted) with ordinary *integer* compare hardware. With a 2's-complement exponent, 1.0×2^{-1} (exponent `1111 1111`) would look larger than $1.0 \times 2^{+1}$ (exponent `0000 0001`). With bias 127 they are `0111 1110` $<$ `1000 0000`, which is correct.
- The two end codes are left free for special values: $E = 0$ for zero and denormals, $E =$ all 1s for $\pm\infty$ and NaN.

Value of a normalized number: $x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(E-\text{Bias})}$.

Smallest and largest values (2023–24 Q7(b))

	Bit pattern	Value	Approx.
Single (1 + 8 + 23 bits, Bias 127)			
Largest	$E = 1111\,1110$ (254 − 127 = +127), F = 1…1 (`0x7F7FFFFF`)	$(2 - 2^{-23}) \times 2^{127}$	3.40×10^{38}
Smallest normalized	$E = 0000\,0001$ (1 − 127 = −126), F = 0…0 (`0x00800000`)	1.0×2^{-126}	1.18×10^{-38}
Smallest denormal	$E = 0$, F = 0…01 (`0x00000001`)	$2^{-23} \times 2^{-127} = 2^{-150}$	7.0×10^{-46}
Double (1 + 11 + 52 bits, Bias 1023)			
Largest	$E = 111\,1111\,1110$ (2046 − 1023 = +1023), F all 1s	$(2 - 2^{-52}) \times 2^{1023}$	1.80×10^{308}
Smallest normalized	$E = 000\,0000\,0001$ (1 − 1023 = −1022), F all 0s	1.0×2^{-1022}	2.23×10^{-308}
Smallest denormal	$E = 0$, F = 0…01	$2^{-52} \times 2^{-1023} = 2^{-1075}$	2.5×10^{-324}

Denormals use the slides' formula $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$.

> **NOTE** Real IEEE-754 hardware scales denormals by $2^{1-\text{Bias}}$ instead, so the actual hardware minimums are $2^{-149} \approx 1.4 \times 10^{-45}$ (single) and $2^{-1074} \approx 4.9 \times 10^{-324}$ (double). The table follows the course slides.

The sign bit is separate, so each range is symmetric. The most negative values are $-(2 - 2^{-23}) \times 2^{127} \approx -3.4 \times 10^{38}$ (single) and $-(2 - 2^{-52}) \times 2^{1023} \approx -1.8 \times 10^{308}$ (double).

Answer. Single: $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$ to $\pm(2 - 2^{-23}) \times 2^{127} \approx \pm 3.4 \times 10^{38}$. Double: $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$ to $\pm(2 - 2^{-52}) \times 2^{1023} \approx \pm 1.8 \times 10^{308}$.

Encoding of infinity (2018–19 Q7(c), 2015–16 Q7(c))

Infinity is **exponent all 1s, fraction all 0s**. The sign bit tells $+\infty$ from $-\infty$. If the exponent is all 1s and the fraction is *not* zero, the value is NaN (e.g. $0/0$).

	S	Exponent	Fraction	(hex)
$+\infty$ (single)	0	1111 1111	000…0	`(0x7F800000)`
$-\infty$ (single)	1	1111 1111	000…0	`(0xFF800000)`
$+\infty$ (double)	0	111 1111 1111	000…0	`(0x7FF0000000000000)`

$\pm\infty$ is produced on overflow (e.g. $1/0$). It can be used in later calculations without a separate overflow check.

REPEATED ×5 Same question in 5 papers — stacked below, solved once.

2022–2023 • Q1(a) CSE 209 • 13/01/2025 • AR03 **MARKS 10**

Express the steps of floating-point addition using a flow chart. Also, write down the conditions of overflow, underflow, NaN and infinity during a floating-point addition.

2018–2019 • Q8(b) CSE 305 • 29/10/2019 • AR03 **MARKS 10**

What are the steps of floating point addition? Describe with an example. Also, write down the condition of overflow and underflow during a floating point addition.

2015–2016 • Q6(a) CSE 305 • 18/01/2017 • AR03 **MARKS 12**

What are the steps of floating point addition? Describe with an example.

2013–2014 • Q3(a) CSE 305 • 05/07/2015 • AR03 **MARKS 10**

Write down the steps for the addition of two binary floating point numbers.

2011–2012 • Q5(a) CSE 305 • 27/07/2013 • AR03 **MARKS 8 + 2**

Draw the flow diagram of floating point addition. Also write down the condition of overflow or underflow during a floating point addition.

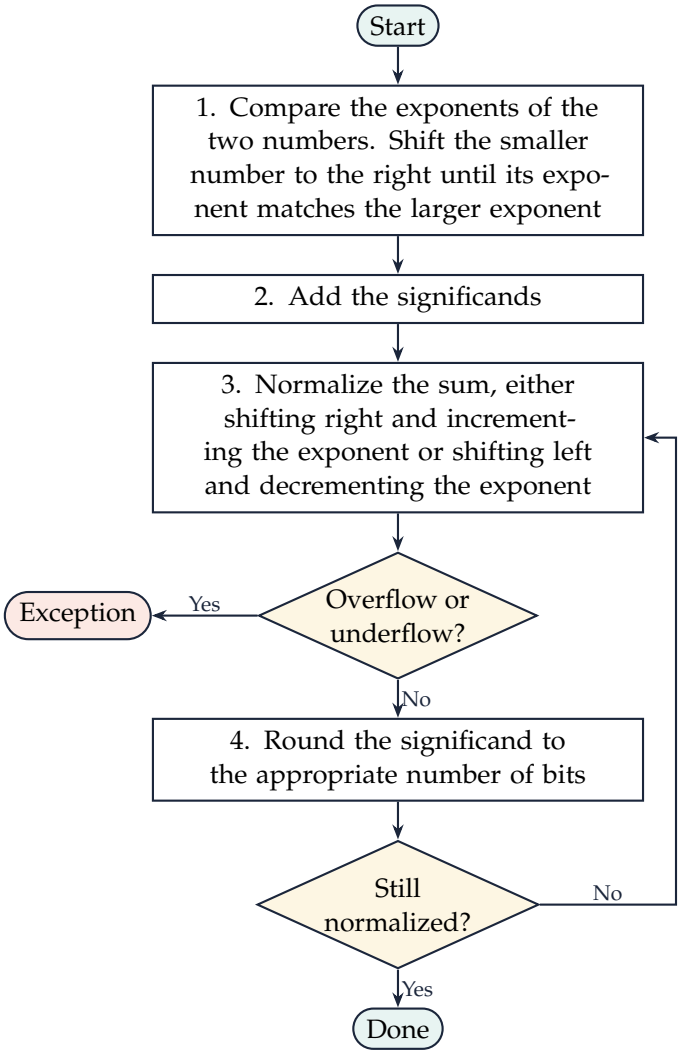
SOLUTION

Steps / flow chart of FP addition (all cards)

For $x = (-1)^{s_x}\, 1.f_x \times 2^{e_x}$ and $y = (-1)^{s_y}\, 1.f_y \times 2^{e_y}$:

1. **Align**: compare exponents (small ALU computes $e_x - e_y$); shift the significand of the number with the *smaller* exponent right by $|e_x - e_y|$ places; the result takes the larger exponent.
2. **Add** the (signed) significands in the big ALU.

3. **Normalize**: shift the sum right and increment the exponent, or shift left and decrement it, until it is $1.xxx$; then **check for overflow/underflow**.
4. **Round** the significand to the available bits (using guard/round/sticky); if rounding denormalizes it (e.g. $1.111 \to 10.000$), go back to step 3.



Example (2018–19 Q8(b), 2015–16 Q6(a)) — 4-bit significands: $0.5 + (-0.4375)$

Operands	$1.000_2 \times 2^{-1}$ and $-1.110_2 \times 2^{-2}$
1. Align (shift smaller right by 1)	$1.000 \times 2^{-1} + (-0.111 \times 2^{-1})$
2. Add significands	$1.000 - 0.111 = 0.001 \times 2^{-1}$
3. Normalize (shift left 3, exponent $-1-3$)	1.000×2^{-4}; $-126 \le -4 \le 127 \Rightarrow$ no over/underflow
4. Round	1.000×2^{-4} (no change) $= 0.0625$ ✓

Exception conditions (IEEE single; biased exponent E, 1 .. 254 for normal numbers)

- **Overflow**: after normalization (right shift, exponent incremented) the exponent exceeds the largest value, i.e. $E > 254$ (true exponent $> +127$; double: $> +1023$). Result is set to $\pm\infty$.
- **Underflow**: after normalization (left shift, exponent decremented) the exponent falls below the smallest value, i.e. $E < 1$ (true exponent < -126; double: < -1022). Result becomes a denormal or 0.
- **NaN** (2022–23 card): either operand is a NaN ($E = 255$, $F \neq 0$), or the operation is $(+\infty) + (-\infty)$ (invalid). Result: NaN.

- **Infinity** (2022–23 card): an operand is $\pm\infty$ ($E = 255$, $F = 0$) and the other is finite or the same-signed $\infty \Rightarrow$ result is that ∞; also produced by overflow.

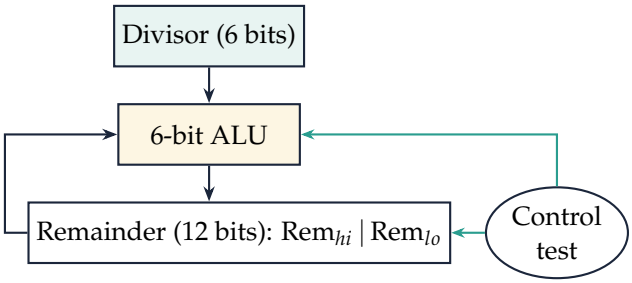
2022–2023 • Q1(b) CSE 209 • 13/01/2025 • AR04 **MARKS 10**

Suppose you have a 6-bit ALU and you are required to perform a division operation, where the dividend is `1011 1100 1111` and the divisor is `110101`. Determine the result of the division process; show the step-by-step calculation along with the value of each relevant register at each step.

SOLUTION

Hardware: the Optimized Divider (slides)

With a 6-bit ALU, $n = 6$: the **Divisor** register is 6 bits and the **Remainder** register is $2n = 12$ bits. There is no separate Quotient register. The ALU works on the left half Rem_{hi} and the Divisor, and the quotient bits are shifted in from the right end of the Remainder.



Algorithm (slides). 0. Remainder ← dividend, then shift Remainder left 1. Repeat $n = 6$ times: **1.** $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Divisor}$; **2a.** if $\text{Rem}_{hi} \geq 0$: shift Remainder left, new rightmost bit = 1; **2b.** if $\text{Rem}_{hi} < 0$: restore ($\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} + \text{Divisor}$), shift Remainder left, new rightmost bit = 0. **3.** Finally shift Rem_{hi} right 1. Then Rem_{hi} = remainder and Rem_{lo} = quotient.

Why shift first? The first quotient bit has weight 2^{n-1}: it asks whether Divisor$\times 2^{n-1}$ fits, i.e. it compares the Divisor with the dividend's top $n+1$ bits, which is what the left half holds after one left shift. Subtracting from the unshifted left half would test Divisor$\times 2^n$, a quotient bit an n-bit quotient cannot have (it always fails for a valid division). The extra left shift is undone by the final right shift of Rem_{hi}.

NOTE The dividend's leading bit is 1, so the first left shift pushes a bit out of the 6-bit left half. The ALU must keep that bit (as its carry/7th bit), otherwise the dividend would be corrupted. In the table Rem_{hi} is therefore shown with this extra leading bit (7 bits). Negative differences are in 2's complement. Values that change are in coral.

Trace: $101111001111_2 \div 110101_2$ (3023 ÷ 53)

Iter.	Step	Divisor	Remainder (Rem$_{hi}$ | Rem$_{lo}$)	sign
0	Initial values	110101	0101111 | 001111	
	Shift Rem left 1	110101	1011110 | 011110	
1	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	110101	0101001 | 011110	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	110101	1010010 | 111101	
2	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	110101	0011101 | 111101	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	110101	0111011 | 111011	
3	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	110101	0000110 | 111011	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	110101	0001101 | 110111	

Iter.	Step	Divisor	Remainder (Rem_{hi} | Rem_{lo})	sign *(cont.)*
4	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	110101	1011000 | 110111	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	110101	0011011 | 101110	
5	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	110101	1100110 | 101110	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	110101	0110111 | 011100	
6	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	110101	0000010 | 011100	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	110101	0000100 | 111001	
	3: Shift Rem_{hi} right 1	110101	0000010 | 111001	

Answer. Quotient = $Rem_{lo} = 111001_2 = 57$, Remainder = $Rem_{hi} = 000010_2 = 2$. Check: $53 \times 57 + 2 = 3023$ ✓

REPEATED ×6 Same question in 6 papers — stacked below, solved once.

2022–2023 • Q4 CSE 209 • 13/01/2025 • AR05 **MARKS 15**

Draw the hardware block diagram and the flowchart of the sequential version of the multiplication algorithm. Also, draw the hardware block diagram and the flowchart of the refined version of the algorithm.

2019–2020 • Q4(b) CSE 305 • 18/09/2021 • AR05 **MARKS 12**

See the diagram (Figure 4b) of the following multiplication hardware.

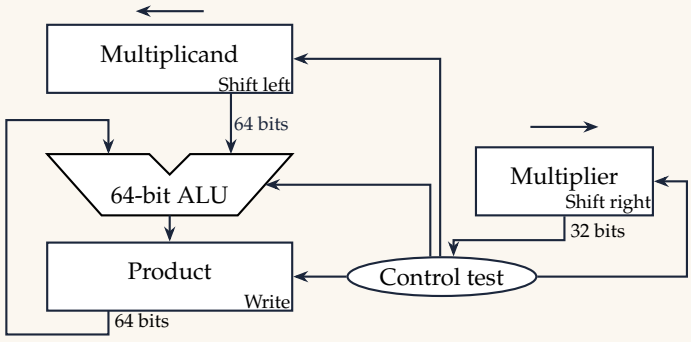


Figure 4b: Multiplication Hardware

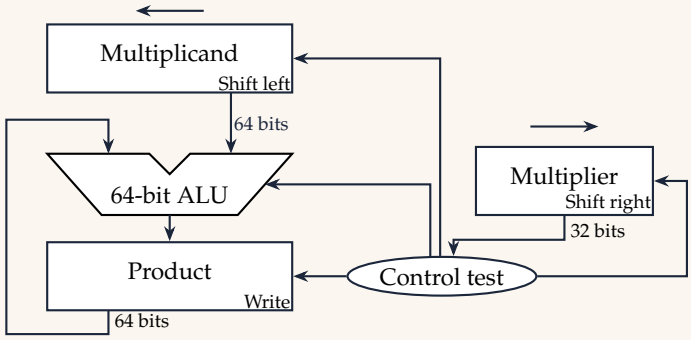
Briefly describe the reasons why this hardware is inefficient for a multiplication operation. What changes are required to make this hardware efficient? Draw the diagram of the efficient hardware.

2018–2019 • Q5(a) CSE 305 • 29/10/2019 • AR05 **MARKS 12**

Draw the hardware block diagram and the flowchart of the sequential version of the multiplication algorithm. Also, draw the flowchart of the refined version of the algorithm.

2014–2015 • Q8(b) CSE 305 • 26/01/2016 • AR05 **MARKS 8 + 5**

In the following figure, a multiplication hardware is given. Now draw the flowchart of a the sequential version of multiplication algorithm for this hardware. Please note that, this simple hardware is not register-optimized, therefore, draw a register-optimized refined version of this hardware.



2013–2014 • Q3(c) CSE 305 • 05/07/2015 • AR05 **MARKS 15**

Draw a block diagram of hardware organization and corresponding flowchart for the multiplication of two 32-bit binary unsigned integer numbers, where there is no separate multiplier register.

2011–2012 • Q6(a) CSE 305 • 27/07/2013 • AR05 **MARKS 15**

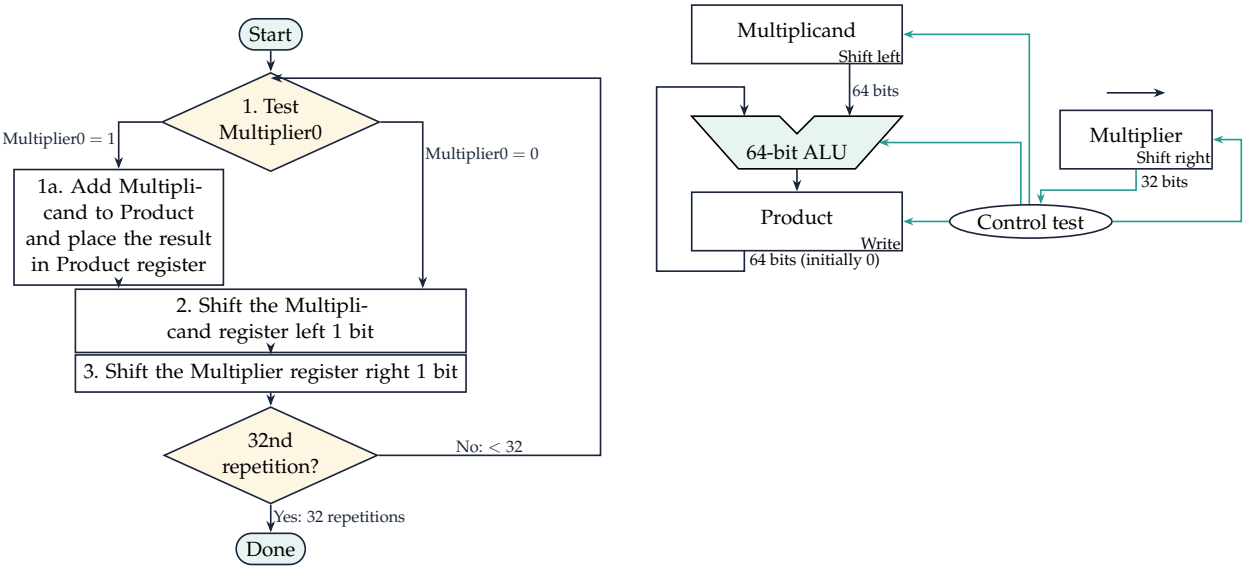
Draw diagrams of the first and refined versions of the multiplication hardware. Discuss the improvement of the refined version over the first version.

SOLUTION

1. First (sequential) version • hardware + flowchart (Figure 4b is exactly this hardware)

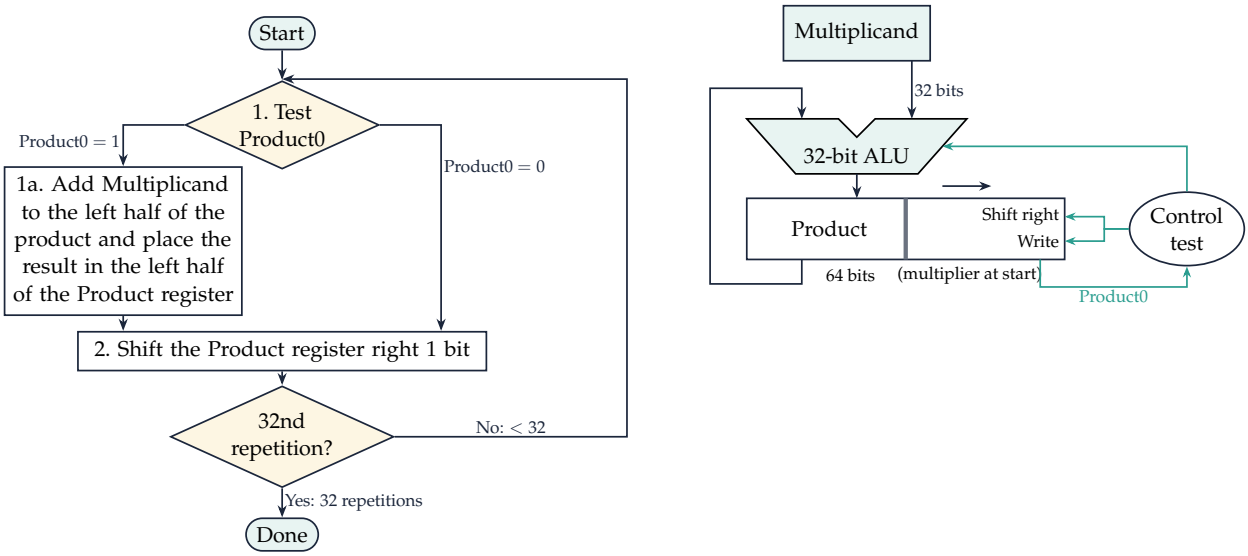
This copies pencil-and-paper multiplication. The multiplicand shifts left one place per step. The multiplier shifts right, so the bit to test is always Multiplier0, and a partial product is added only when that bit is 1. For 32×32 bits the hardware is:

- a 64-bit Multiplicand register (right half initially holds the multiplicand, left half 0);
- a 64-bit ALU;
- a 64-bit Product register (initially 0);
- a 32-bit Multiplier register.



2. Refined (register-optimized) version • hardware + flowchart

The multiplicand stays still and the *product shifts right* instead. Only 32 bits are added in each step, so the ALU and the Multiplicand register shrink to 32 bits. The multiplier starts in the right half of the 64-bit Product register, so **there is no separate Multiplier register**; this is the design asked for in 2013–14 Q3(c). The bit tested is Product0. The ALU writes the left half, and its carry-out enters the MSB on the shift. Add and shift happen in the same clock cycle, so each iteration takes one cycle.



3. Why the first version is inefficient, and the improvement (2019–20 Q4(b), 2011–12 Q6(a), 2014–15 Q8(b))

Inefficiencies of the first version (Figure 4b)	Changes in the refined hardware
At any moment half of the 64-bit Multiplicand register holds only 0s.	The Multiplicand register is 32 bits and is never shifted.
The 64-bit ALU is twice as wide as needed. Each addition really involves only 32 significant bits, because the low product bits are already final.	The product is shifted right instead, so a **32-bit ALU** adds into the left half only.
A separate 32-bit Multiplier register is used, while the right half of the Product register is empty at the start.	The multiplier is loaded into the **right half of the Product register**. It is consumed bit by bit as the product shifts in, so one register is saved.
Each iteration takes up to 3 steps (add, shift left, shift right), about $3 \times 32 \approx 96$ cycles in all.	Add and shift are done **in parallel**: one cycle per iteration, 32 cycles in total.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q8(a) CSE 305 • 25/04/2024 • AR06 **MARKS 7**

Write down equivalent efficient sequence of MIPS instructions for the following C code fragment. Assume appropriate registers for each of the variables.

```
float a, b, c;
double w, x, y, z;
if (a+b>c)
    x=(y*z)/ w;
```

2020–2021 • Q13(c) CSE 305 • 24/09/2022 • AR06 **MARKS 7**

Write the equivalent efficient sequence of MIPS instructions for the following C code fragment. Assume appropriate registers for each of the variables.

```
float a, b, c;
double w, x, y, z;
if (a+b>c)
    x=(y*z)/w;
```

SOLUTION

Rules used (slides: FP instructions in MIPS)

FP values live in coprocessor-1 registers `$f0`–`$f31`; a *double* uses an even/odd pair (`$f4`/`$f5`, ...), so doubles get even registers. Single ops end in `.s`, double ops in `.d`. A comparison `c.xx.s`/`c.xx.d` sets the FP condition flag, tested by `bc1t`/`bc1f`.

Code

Assume (singles) $a \rightarrow$`$f0`, $b \rightarrow$`$f1`, $c \rightarrow$`$f2`; (doubles) $w \rightarrow$`$f4`, $x \rightarrow$`$f6`, $y \rightarrow$`$f8`, $z \rightarrow$`$f10`. $a + b$ is a *float* expression compared with a float, and $x = (y * z)/w$ is entirely double, so no conversion (`cvt`) is needed.

```
add.s  $f12, $f0, $f1      # $f12 = a + b          (single)
c.lt.s $f2, $f12           # flag = (c < a+b)  i.e. (a+b > c)
bc1f   Skip                # condition false -> skip
mul.d  $f14, $f8, $f10     # $f14/$f15 = y * z     (double)
```

```
        div.d   $f6, $f14, $f4      # x = (y*z) / w
Skip:
```

5 instructions. (MIPS has no `c.gt`, so $a + b > c$ is written as $c < a + b$; with `bc1f` an unordered/NaN comparison also skips, matching C.)

2022–2023 • Q8(b) CSE 305 • 25/04/2024 • AR07 **MARKS 2 + 3 + 3**

Consider a 32-bit system where 19 bits are reserved for fraction, one bit for sign, and the rest for exponent. Answer the following questions:

(i) What is the precision range of this system?

(ii) What are the smallest and the largest absolute values that can be stored in this system?

(iii) What is the smallest denormalized number that can be stored using this system?

SOLUTION

Format and rules (IEEE-754 style)

$32 - 1 - 19 = 12$ exponent bits. Bias $= 2^{12-1} - 1 = 2047$; exponent fields all-0 (zero/denormals) and all-1 (∞/NaN) are reserved, so normal numbers use $E = 1 \ldots 4094$, i.e. true exponent $-2046 \ldots + 2047$.



(i) Precision: 20 significant bits (hidden 1 + 19) $\Rightarrow$ relative precision $2^{-19} \approx 1.9 \times 10^{-6}$, i.e. about $19 \times \log_{10} 2 \approx 5.7$ (5–6) decimal digits. Representable magnitudes span roughly 10^{-622} (denormal) / 10^{-616} (normal) to 10^{616} (parts ii, iii).

(ii) Smallest $= 1.0 \times 2^{-2046} \approx 1.24 \times 10^{-616}$; Largest $= (2 - 2^{-19}) \times 2^{2047} \approx 3.23 \times 10^{616}$.

(iii) Denormal (exponent field $= 0$, hidden bit 0), slides' formula: $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$. The smallest non-zero fraction is $0.00 \ldots 01_2 = 2^{-19}$, so smallest denormal $= 2^{-19} \times 2^{-2047} = 2^{-2066} \approx 1.18 \times 10^{-622}$.

> **NOTE** Real IEEE-754 hardware scales denormals by $2^{1-\text{Bias}}$ (so they join smoothly with the smallest normal 2^{-2046}), which would give $2^{-19} \times 2^{-2046} = 2^{-2065}$. The answer above follows the course slides, which use $2^{-\text{Bias}}$.

2022–2023 • Q8(c) CSE 305 • 25/04/2024 • AR08 **MARKS 10 + 10**

Consider two normalized floating point numbers f1 and f2 that are represented as IEEE single precision floats. This defines two 32-bit strings.

(i) Suppose f1 is less than f2, that is, f1 $<$ f2. If the same two 32-bit strings were treated as unsigned integers (call them i1 and i2, respectively), can we conclude that i1 $<$ i2?

(ii) Similar to (i), if the two 32-bit strings were treated as signed integers j1 and j2, can we conclude that j1 $<$ j2?

SOLUTION

Key fact

An IEEE float is **sign–magnitude**: bit 31 is the sign, and bits 30–0 (biased exponent, then fraction) form the magnitude. Because the exponent is biased (unsigned) and placed before the fraction, for two *non-negative* floats a larger value always has a larger 31-bit pattern. Integers, however, read bit 31 differently: unsigned as weight $+2^{31}$, signed (2's complement) as -2^{31}, and 2's complement does *not* reverse the order of the remaining bits like sign–magnitude does.

Case analysis for f1 $<$ f2

Case	Example (f1, f2)	i1 $<$ i2 ?	j1 $<$ j2 ?
$0 \le$ f1 $<$ f2	$1.0 =$ `0x3F800000`, $2.0 =$ `0x40000000`	✓	✓
f1 $< 0 \le$ f2	$-1.0 =$ `0xBF800000`, $1.0 =$ `0x3F800000`	× (3212836864 $>$ 1065353216)	✓ ($-1082130432 <$ 1065353216)
f1 $<$ f2 < 0	$-2.0 =$ `0xC0000000`, $-1.0 =$ `0xBF800000`	× (3221225472 $>$ 3212836864)	× ($-1073741824 > -1082130432$)

(i) Unsigned: No. It holds only when both numbers are non-negative. If f1 is negative its sign bit makes i1 $\ge 2^{31}$, larger than any non-negative f2's pattern; and if both are negative, the one with the larger magnitude (the smaller value f1) has the larger pattern, so i1 $>$ i2.

(ii) Signed: No (not in general). It works when both are non-negative (same as unsigned) and when f1 $< 0 \le$ f2 (j1 is a negative integer, j2 non-negative). But when *both are negative*, both integers are negative and the more negative float (larger magnitude) has the larger bit pattern, hence the *larger* signed integer: j1 $>$ j2. The order is reversed.

Answer. (i) No — i1 $<$ i2 is guaranteed only if both floats are non-negative. (ii) No — j1 $<$ j2 holds when both are non-negative or f1 is negative and f2 non-negative, but fails when both are negative (order reverses).

Remark: integer hardware can still sort floats if each pattern is first mapped as "if sign $= 1$ invert all bits, else flip only the sign bit", and the results are compared as unsigned integers — the reason IEEE put the sign first and used a biased exponent.

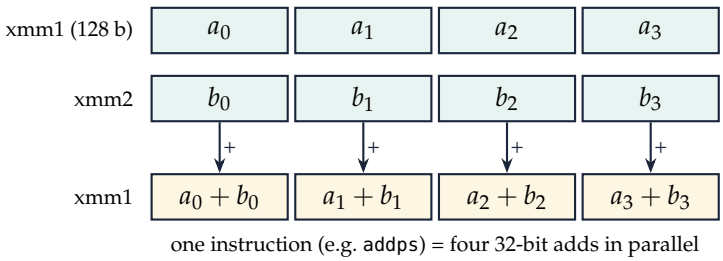
2021–2022 • Q5(c) CSE 305 • 06/04/2023 • AR09 MARKS 10

Discuss how can subword parallelism be used to accelerate common operations, such as matrix multiplication or image processing?

SOLUTION

Idea

Subword parallelism (data-level parallelism / SIMD, e.g. x86 SSE/AVX) partitions one wide register and one wide adder/multiplier into independent lanes. A single instruction then operates on all lanes at once: a 128-bit adder can do sixteen 8-bit, eight 16-bit, four 32-bit or two 64-bit additions (carries are simply cut at lane boundaries).



one instruction (e.g. `addps`) = four 32-bit adds in parallel

Matrix multiplication (slides: DGEMM with AVX)

- In $C = C + A \times B$ the same multiply-add is done on many independent elements. With 256-bit AVX registers, four doubles of a column of C are kept in one register: `_mm256_load_pd` loads 4 elements of A, `_mm256_broadcast_sd` makes 4 copies of $B[k][j]$, then `_mm256_mul_pd` and `_mm256_add_pd` (`vmulpd`, `vaddpd`) update 4 results per instruction.
- The loop over i advances by 4, so the number of FP instructions (and loop overhead) drops about 4×; P&H reports ≈ 3.8× speed-up for DGEMM from this change alone.

Image / media processing

- Pixels are 8-bit (or 16-bit) values, so a 128-bit register holds 16 pixels. Brightness change, blending/alpha compositing, filtering and colour conversion apply the same operation to every pixel: one SIMD instruction processes 16 pixels, giving up to 16× fewer instructions.
- Media SIMD adds **saturating** arithmetic (e.g. $250 + 10 \rightarrow 255$, not wrap-around to 4), packed min/max/average, and pack/unpack/shuffle to rearrange subwords — all common in graphics and audio.

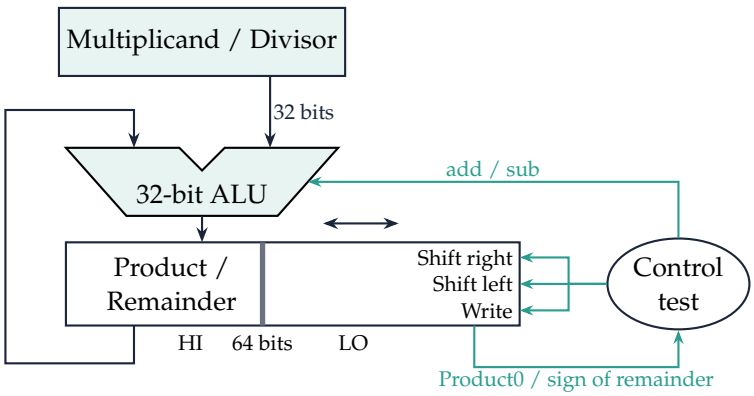
Benefits: no extra datapath width is needed (the existing wide adder is reused), fewer instructions fetched/decoded, better use of memory bandwidth because whole cache blocks of data are consumed per instruction. Requirement: data must be laid out contiguously and aligned, and the work must be the same for all lanes.

2021–2022 • Q6(c) CSE 305 • 06/04/2023 • AR10 **MARKS 10**

Draw the block diagram of 32-bit combined multiplication and division hardware which uses 32-bit ALU, and briefly mention functions of different components of this hardware.

SOLUTION

The refined (optimized) multiplier and the optimized divider have the same structure: a 32-bit register, a 32-bit ALU and a 64-bit shift register. The slides note that the same hardware can be used for both. Only the ALU operation (add or subtract) and the shift direction differ.



- **Multiplicand/Divisor register (32 bits)**: holds the multiplicand or the divisor. It never shifts.
- **32-bit ALU**: when multiplying, it adds the multiplicand to the left half (HI). When dividing, it subtracts the divisor from HI, and adds it back to restore if the result is negative.
- **64-bit Product/Remainder register**:

  - Multiply: starts as HI $= 0$, LO $=$ multiplier. It *shifts right* each step and ends holding the 64-bit product.
  - Divide: starts with the dividend in LO and *shifts left*; each new quotient bit enters at the right end. At the end HI $=$ remainder and LO $=$ quotient, which are the MIPS HI/LO registers. A final right shift of the left half corrects the remainder.
- **Control test**: looks at Product0 (multiply) or the sign of the new remainder (divide). From that it chooses add or subtract, write, and the shift direction. It also counts the 32 repetitions.

2021–2022 • Q8(b) CSE 305 • 06/04/2023 • AR11 **MARKS 14**

Consider representing the set of consecutive positive integers $\{1, 2, 3, 4, \ldots, n\}$ using IEEE single precision floating point (8-bit exponent and 23-bit mantissa). Find the largest n such that every number in the above set can be represented.

SOLUTION

A normalized single has a **24-bit significand** (1 hidden bit + 23 fraction bits): $x = 1.f_1 f_2 \ldots f_{23} \times 2^e$.

- An integer k with $2^m \le k < 2^{m+1}$ is $1.b_{m-1} \ldots b_0 \times 2^m$: it needs m fraction bits. So every integer with at most 24 significant bits ($k < 2^{24}$, i.e. $m \le 23$) is exact. The exponent range ($e \le 127$) is no limit here.
- $2^{24} = 1.0 \times 2^{24}$ is also exact (fraction all 0, $E = 24 + 127 = 151$).
- $2^{24} + 1 = 1.\underbrace{00\ldots0}_{23}1_2 \times 2^{24}$ needs **24** fraction bits; only 23 exist, so it cannot be represented (it rounds to 2^{24}; above 2^{24} the spacing of floats is 2).

Integer	Single-precision pattern	exact?
$2^{24} - 1 = 16\,777\,215$	`0 10010110 11111111111111111111111` $=$ `0x4B7FFFFF`	✓
$2^{24} = 16\,777\,216$	`0 10010111 00000000000000000000000` $=$ `0x4B800000`	✓
$2^{24} + 1 = 16\,777\,217$	would need fraction `0...01` (24 bits)	× (rounds to 2^{24})
$2^{24} + 2 = 16\,777\,218$	`0 10010111 00000000000000000000001` $=$ `0x4B800001`	✓

Answer. $n = 2^{24} = 16\,777\,216$. All of $1, \ldots, 2^{24}$ are exactly representable; $2^{24} + 1$ is the first integer that is not.

2020–2021 • Q10(c) CSE 305 • 24/09/2022 • AR12 **MARKS 5**

Briefly describe how to perform a signed multiplication operation.

SOLUTION

Sign–magnitude method (P&H)

1. Record the signs and convert both operands to positive (2's-complement negate a negative operand).
2. Multiply the magnitudes with the unsigned shift-and-add multiplier, using $n - 1$ iterations for the magnitude bits (31 for MIPS) — the sign bits are left out.
3. Product sign $=$ sign(multiplicand) $\oplus$ sign(multiplier); if it is 1, negate the $2n$-bit product.

Example (4-bit): $(-3) \times 5$: $|-3| = 0011$, $0011 \times 0101 = 0000\,1111$ (15); signs differ $\Rightarrow$ negate: $1111\,0001 = -15$.

Alternatives

Keep the operands in 2's complement and *sign-extend* the partial products (shifting the product right arithmetically), subtracting at the last step for a negative multiplier; **Booth's algorithm** does this efficiently by examining bit pairs (q_0, q_{-1}): $10 \Rightarrow$ subtract multiplicand, $01 \Rightarrow$ add, $00/11 \Rightarrow$ nothing, then arithmetic shift right. In MIPS, `mult` gives the signed 64-bit product in HI/LO (`multu` for unsigned).

2020–2021 • Q12(c) CSE 305 • 24/09/2022 • AR13 **MARKS 9**

Briefly explain the importance of guard digit, round digit, and sticky bit during rounding an intermediate floating point number.

SOLUTION

During alignment and normalization, bits are shifted out to the right of the significand. If they were simply dropped the result could be off by more than half an ulp. IEEE 754 therefore keeps extra bits in the intermediate result (slides: "extra bits of precision (guard, round, sticky)"):

- **Guard bit**: the first bit beyond the kept significand. It is needed because after a subtraction or a carry the result may be shifted *left* one place during normalization; the guard bit then becomes the new LSB instead of a lost 0.
- **Round bit**: the second extra bit. After a possible normalization shift it still leaves one bit to decide the rounding direction (above/below half).
- **Sticky bit**: the OR of *all* bits shifted past the round bit. It records whether anything non-zero was lost, which distinguishes an exact half-way case from "slightly more than half". This makes round-to-nearest-even exact.

Effect — decimal example (3 significant digits)

$2.56 \times 10^0 + 2.34 \times 10^2$: aligned, 0.0256×10^2. *With* guard (5) and round (6): $2.3400 + 0.0256 = 2.3656 \rightarrow \mathbf{2.37} \times 10^2$ (correct). *Without* them: $2.34 + 0.02 = 2.36 \times 10^2$ — off by 1 in the last digit.

Round-to-nearest-even decision (G R S after the LSB)

`0xx`	below half $\Rightarrow$ truncate
`100`	exactly half $\Rightarrow$ round to make the LSB even
`101`, `11x`	above half $\Rightarrow$ round up (add 1 ulp)

Binary example: $1.0100\,|$ `G=1 R=0 S=1` is above half, so it rounds up to 1.0101. Without the sticky bit it would look like an exact tie (`10`) and round-to-even would give 1.0100 — wrong.

Answer. Guard and round bits keep two extra bits so that normalization and rounding do not lose accuracy; the sticky bit remembers whether any non-zero bit was lost beyond them, so ties are detected correctly. Together they let the hardware round as if the result had been computed exactly.

2020–2021 • Q13(a) CSE 305 • 24/09/2022 • AR14 **MARKS 4×4**

Consider a 32-bit system where 24 bits are reserved for fraction, one bit for sign, and the rest for exponent. Answer the following questions.

(i) Why is bias used during the floating point representation?

(ii) What should be the value of the bias for this system? Why?

(iii) What are the smallest and the largest absolute values that can be stored in this system?

(iv) What is the smallest denormalized number that can be stored using this system?

SOLUTION

Format

$32 - 1 - 24 = 7$ exponent bits.



(i) Why bias:

- The exponent must hold negative and positive values. Storing it as *(true exponent + bias)* makes the field an **unsigned** number, so floats can be compared/sorted with simple integer hardware (sign, then exponent, then fraction) — no 2's-complement exponent handling.
- The smallest exponent field becomes all 0s (used for 0 and denormals) and the largest all 1s (∞, NaN).

(ii) **Bias** $= 2^{7-1} - 1 = 63$**.** With fields 0 and 127 reserved, $E = 1 \ldots 126$ gives true exponents $-62 \ldots +63$: the middle of the field, splitting the range almost symmetrically so that $2^{e_{\max}} \approx 1/2^{e_{\min}}$ (the reciprocal of the smallest normal number does not overflow), following the IEEE rule bias $= 2^{k-1} - 1$.

(iii) Smallest normal $= 1.0 \times 2^{-62} \approx 2.17 \times 10^{-19}$; Largest $= (2 - 2^{-24}) \times 2^{63} \approx 1.84 \times 10^{19}$.

(iv) Denormal (exponent field $= 0$), slides' formula $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$: smallest denormal $= 0.00 \ldots 01_2 \times 2^{-63} = 2^{-24} \times 2^{-63} = 2^{-87} \approx 6.46 \times 10^{-27}$.

> **NOTE** Real IEEE-754 hardware uses $2^{1-\text{Bias}}$ for denormals, which would give $2^{-24} \times 2^{-62} = 2^{-86}$. The answer follows the course slides ($2^{-\text{Bias}}$).

2019–2020 • Q4(a) CSE 305 • 18/09/2021 • AR15 **MARKS 6 + 5 + 4**

Suppose, you are working in a system where 16 bits are used to represent a floating-point value. The largest absolute floating-point value that can be represented by the system is $1.11 \ldots 1_{two} \times 2^{63}$ while the smallest absolute floating point value that can be represented by the system is $1.00 \ldots 0_{two} \times 2^{-62}$.

(i) Show the floating-point representation format (bit distribution among sign, exponent and fraction portion) of this system.

(ii) Show the floating-point representation for the number $-110.11_{two} \times 2^1$ in this system assuming an appropriate bias.

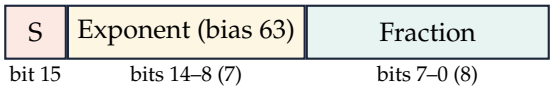
(iii) What is the smallest absolute denormalized floating-point value that can be stored in this system?

SOLUTION

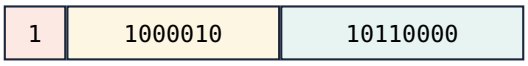
Rules (IEEE-754 style, k exponent bits)

Bias $= 2^{k-1} - 1$; fields all-0 (zero/denormals) and all-1 (∞/NaN) reserved, so normal exponents run from $1 -$ bias to $+$bias. Denormal (slides): $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$.

(i) True exponents run from -62 to $+63$: 126 values, plus the two reserved codes $= 128 = 2^7$ $\Rightarrow$ **7** exponent bits with bias 63 ($E = 1 \ldots 126$). Remaining: $16 - 1 - 7 =$ **8** fraction bits.



(ii) $-110.11_2 \times 2^1 = -1.1011_2 \times 2^3$ $(= -13.5)$. $S = 1$, $E = 3 + 63 = 66 =$ `1000010`, $F =$ `10110000`.



Answer. $-110.11_2 \times 2^1 \rightarrow$ `1 1000010 10110000` $=$ `0xC2B0`

(iii) Smallest denormal $= 0.00000001_2 \times 2^{-63} = 2^{-8} \times 2^{-63} = 2^{-71} \approx 4.24 \times 10^{-22}$.

> **NOTE** Real IEEE-754 hardware uses $2^{1-\text{Bias}}$ for denormals, which would give $2^{-8} \times 2^{-62} = 2^{-70}$. The answer follows the course slides ($2^{-\text{Bias}}$).

2019–2020 • Q4(c) CSE 305 • 18/09/2021 • AR16 **MARKS 8**

Consider the following C code fragment.

```
if(a>b)
   c = c-(c*e)/d;
else
   c = c+d;
```

If a, b, c, d and e are variables of type *double* and their values are already loaded in registers `$f2`, `$f4`, $6, `$f8` and `$f10` respectively, then write down the equivalent MIPS assembly code for it. Use the least number of MIPS instructions possible.

SOLUTION

Rules used (slides: FP instructions in MIPS)

All variables are double, so every operation uses the `.d` forms (`mul.d`, `div.d`, `sub.d`, `add.d`); the comparison `c.lt.d` sets the FP condition flag, tested by `bc1t/bc1f`. There is no "greater than" compare, so $a > b$ is tested as $b < a$.

```
       c.lt.d $f4, $f2          # flag = (b < a)  i.e. (a > b)
       bc1f   Else              # not (a > b) -> else part
       mul.d  $f12, $f6, $f10   # $f12 = c * e
       div.d  $f12, $f12, $f8   # $f12 = (c*e) / d
       sub.d  $f6, $f6, $f12    # c = c - (c*e)/d
       j      Exit
Else:  add.d  $f6, $f6, $f8     # c = c + d
Exit:
```

7 instructions (minimum: one compare, one conditional branch, one jump to skip the else, plus four arithmetic ops). `$f12` (pair `$f12/$f13`) is a temporary.

NOTE The scan prints the register of c as "$6"; since c is a double it must be the FP register `$f6` (an integer register cannot hold a double) — treated as a misprint.

2018–2019 • Q5(b) CSE 305 • 29/10/2019 • AR17 **MARKS 8**

Consider a 16-bit floating point representation system, where the exponent is 4 bits long. With this system, represent the floating point numbers $(-1.25)_{10}$ and $(0.375)_{10}$ into a bit string of length 16 using appropriate bias.

SOLUTION

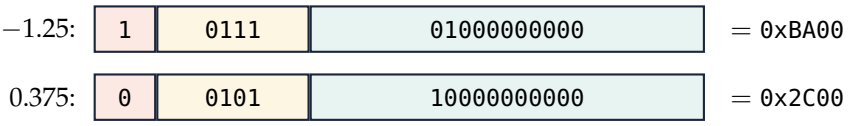
Format (IEEE-754 style)

16 bits = 1 sign + 4 exponent + 11 fraction. Bias $= 2^{4-1} - 1 = 7$ ($E = 1 \ldots 14$ for normal numbers, true exponent $-6 \ldots +7$; 0000 and 1111 reserved). Value $= (-1)^S \times 1.F \times 2^{E-7}$.



Encoding

- $-1.25 = -1.01_2 \times 2^0$: $S = 1$, $E = 0 + 7 = 7 =$ `0111`, $F =$ `01000000000`.
- $0.375 = 0.011_2 = 1.1_2 \times 2^{-2}$: $S = 0$, $E = -2 + 7 = 5 =$ `0101`, $F =$ `10000000000`.



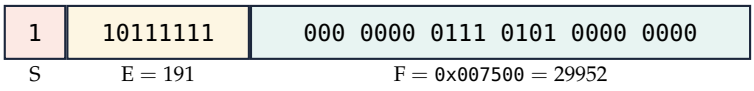
Answer. $-1.25 \to$ `1 0111 01000000000` (`0xBA00`); $0.375 \to$ `0 0101 10000000000` (`0x2C00`).

2018–2019 • Q5(c) CSE 305 • 29/10/2019 • AR18 **MARKS 5**

Determine the decimal value of the floating point number represented by the hexadecimal value $($`DF80 7500`$)_{16}$ according to IEEE 754 single precision floating point format.

SOLUTION

Split the 32 bits into S (1) | E (8) | F (23); for $1 \le E \le 254$: value $= (-1)^S (1 + F/2^{23})\, 2^{E-127}$.



`DF807500` = `1101 1111 1000 0000 0111 0101 0000 0000`. $S = 1$ (negative); $E =$ `10111111` $= 191 \Rightarrow$ exponent $191 - 127 = 64$; $1.F = 1.00000000111010100000000_2 = 1 + 2^{-9} + 2^{-10} + 2^{-11} + 2^{-13} + 2^{-15} = 1.003570556640625$.

$$x = -1.003570556640625 \times 2^{64} = -18\,512\,609\,218\,259\,845\,120$$

Answer. $($`DF807500`$)_{16} = -1.003570556640625 \times 2^{64} \approx -1.8513 \times 10^{19}$.

2018–2019 • Q6(b) CSE 305 • 29/10/2019 • AR19 **MARKS 15**

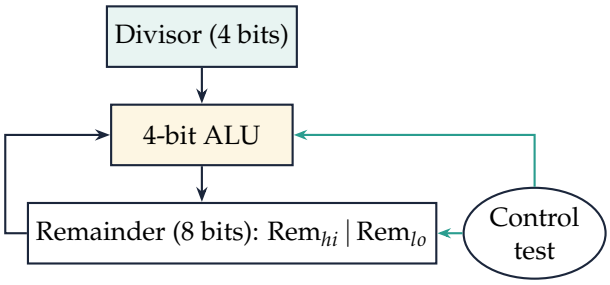
Suppose you have a 4-bit ALU and you are required to perform a division operation, where the dividend is `11001001` and the divisor is `1011`. Show the division process step by step along with the value of each relevant register at each step.

SOLUTION

Hardware and algorithm: the Optimized Divider (slides)

With a 4-bit ALU, $n = 4$: a 4-bit **Divisor** register and an 8-bit **Remainder** register (left half Rem_{hi}, right half Rem_{lo}). There is no Quotient register; quotient bits enter at the right end of the Remainder. **0.** Remainder ← dividend, shift Remainder left 1. Repeat n times: **1.** $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Divisor}$; **2a.** if ≥ 0: shift Remainder left, new rightmost bit $= 1$; **2b.** if < 0: restore ($\text{Rem}_{hi} + \text{Divisor}$), shift Remainder left, new rightmost bit $= 0$. **3.** Shift Rem_{hi} right 1. Result: Rem_{lo} = quotient, Rem_{hi} = remainder.

Why shift first? The first quotient bit has weight 2^{n-1}: it asks whether Divisor$\times 2^{n-1}$ fits, i.e. it compares the Divisor with the dividend's top $n+1$ bits, which is what the left half holds after one left shift. Subtracting from the unshifted left half would test Divisor$\times 2^n$, a quotient bit an n-bit quotient cannot have; if that test succeeds the division overflows (as happens here). The extra left shift is undone by the final right shift of Rem_{hi}.



Divide-overflow check

The quotient can have only $n = 4$ bits (it lives in Rem_{lo}). That works only if the dividend's left half is smaller than the divisor. Here $\text{Rem}_{hi} = \texttt{1100} = 12 \geq \texttt{1011} = 11$, so the true quotient $201 \div 11 = 18 = \texttt{10010}_2$ needs 5 bits: this is a **divide overflow**. Real hardware checks $\text{Rem}_{hi} \geq$ Divisor before starting and signals overflow. Below the 4-bit trace is shown as asked, and then the same algorithm on a 5-bit ALU, which gives the correct result.

(i) Strict 4-bit trace: $\texttt{11001001}_2 \div \texttt{1011}_2$

The first left shift pushes the dividend's leading 1 out of the 8-bit register, so the hardware is already working on the wrong number. Changed values are in coral; a negative difference is signalled by the ALU's borrow.

Iter.	Step	Divisor	Remainder ($\text{Rem}_{hi} \mid \text{Rem}_{lo}$)	sign
0	Initial values	`1011`	`1100` | `1001`	
	Shift Rem left 1 (MSB 1 lost)	`1011`	`1001` | `0010`	
1	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	`1011`	`1110` | `0010`	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	`1011`	`0010` | `0100`	
2	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	`1011`	`0111` | `0100`	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	`1011`	`0100` | `1000`	

Iter.	Step	Divisor	Remainder (Rem_{hi} | Rem_{lo})	sign *(cont.)*
3	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	1011	1001 | 1000	< 0
	2b: Rem<0: +Div, sll Rem, Rem_0=0	1011	1001 | 0000	
4	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	1011	1110 | 0000	< 0
	2b: Rem<0: +Div, sll Rem, Rem_0=0	1011	0010 | 0000	
	3: Shift Rem_{hi} right 1	1011	0001 | 0000	

The 4-bit hardware ends with quotient `0000` and remainder `0001`, which is **wrong** ($11 \times 0 + 1 \neq 201$). This is the divide overflow: an n-bit divider cannot produce an $(n+1)$-bit quotient.

(ii) Same algorithm on a 5-bit ALU (correct result)

With $n = 5$ the dividend is `00 1100 1001` (10-bit Remainder) and the Divisor is `01011`; now Rem_{hi} = `00110` $<$ `01011`, so no overflow.

Iter.	Step	Divisor	Remainder (Rem_{hi} | Rem_{lo})	sign
0	Initial values	01011	00110 | 01001	
	Shift Rem left 1	01011	01100 | 10010	
1	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	01011	00001 | 10010	≥ 0
	2a: Rem≥0: sll Rem, Rem_0=1	01011	00011 | 00101	
2	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	01011	11000 | 00101	< 0
	2b: Rem<0: +Div, sll Rem, Rem_0=0	01011	00110 | 01010	
3	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	01011	11011 | 01010	< 0
	2b: Rem<0: +Div, sll Rem, Rem_0=0	01011	01100 | 10100	

Iter.	Step	Divisor	Remainder (Rem_{hi} | Rem_{lo})	sign *(cont.)*
4	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	01011	00001 | 10100	≥ 0
	2a: Rem≥0: sll Rem, Rem_0=1	01011	00011 | 01001	
5	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	01011	11000 | 01001	< 0
	2b: Rem<0: +Div, sll Rem, Rem_0=0	01011	00110 | 10010	
	3: Shift Rem_{hi} right 1	01011	00011 | 10010	

Answer. The 4-bit divider overflows (quotient needs 5 bits) and gives a wrong result. With a 5-bit ALU: Quotient $= 10010_2 = 18$, Remainder $= 00011_2 = 3$. Check: $11 \times 18 + 3 = 201$ ✓

2017–2018 • Q5(b) CSE 305 • 23/09/2018 • AR20 **MARKS 15**

Consider a half precision floating point representation system where each floating point is represented by 16 bits in which the exponent is 4 bits long. Now convert the two floating points $(0.75)_{10}$ and $(1.25)_{10}$ into the mentioned representation system and then demonstrate the addition of them step by step along with the values of exponent and fraction fields at every step.

SOLUTION

Format (IEEE-754 style)

16 bits = 1 sign + 4 exponent + 11 fraction. Bias $= 2^{4-1} - 1 = 7$ ($E = 1 \ldots 14$ for normal numbers, true exponent $-6 \ldots +7$; 0000 and 1111 reserved). Value $= (-1)^S \times 1.F \times 2^{E-7}$.



Conversion

$0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$: $E = -1+7 = 6 \Rightarrow$ `0 0110 10000000000` $=$ `0x3400`. $1.25 = 1.01_2 \times 2^0$: $E = 7 \Rightarrow$ `0 0111 01000000000` $=$ `0x3A00`.

Addition (align, add, normalize, round)

Step	Exponent field	Significand (hidden bit shown)	Fraction field
0.75	`0110` (-1)	`1.10000000000`	`10000000000`
1.25	`0111` (0)	`1.01000000000`	`01000000000`
1. Align: $7-6=1$, shift 0.75 right 1	`0111`	`0.11000000000`	`11000000000`
2. Add significands	`0111`	`0.11000000000` $+$ `1.01000000000` $=$ `10.00000000000`	—
3. Normalize: shift right 1, $E+1$	`1000` $(+1)$	`1.00000000000`	`00000000000`
over/underflow?	$1 \le E = 8 \le 14 \Rightarrow$ none		
4. Round (bit shifted out $= 0$)	`1000`	`1.00000000000`	`00000000000`

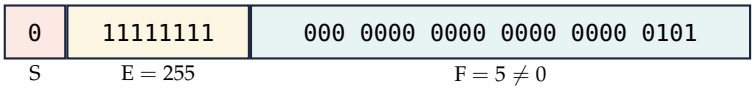
Answer. $0.75 + 1.25 = 1.0_2 \times 2^1 = 2.0$, stored as `0 1000 00000000000` $=$ `0x4000`.

2017–2018 • Q5(c) CSE 305 • 23/09/2018 • AR21 **MARKS 5**

Determine the decimal value of the floating point represented by the hexadecimal value $(\texttt{7F80 0005})_{hex}$ according to IEEE 754 single precision floating point format.

SOLUTION

Split the 32 bits into S (1) | E (8) | F (23). If $1 \le E \le 254$ the value is $(-1)^S(1 + F/2^{23})\,2^{E-127}$; $E = 255$ is reserved: $F = 0 \Rightarrow \pm\infty$, $F \neq 0 \Rightarrow$ NaN.



`7F800005` $=$ `0111 1111 1000 0000 0000 0000 0000 0101`: $S = 0$, $E =$ `11111111` $= 255$ (reserved), $F = 5 \neq 0$.

Answer. $(\texttt{7F800005})_{16}$ is **NaN** (Not-a-Number) — it has no decimal value. (With the fraction MSB $= 0$ it is a *signalling* NaN.)

2017–2018 • Q8(a) CSE 305 • 23/09/2018 • AR22 **MARKS 15**

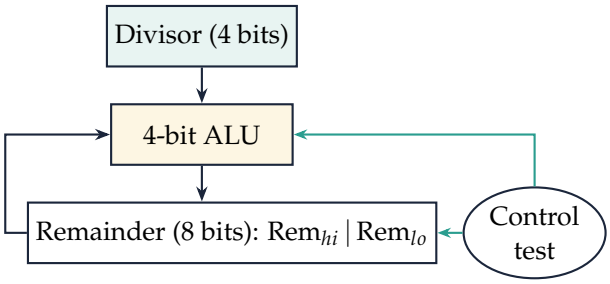
Suppose you have a 4-bit ALU and you are required to perform a division operation, where the dividend is `11010101` and the divisor is `1011`. Show the division process step by step along with value of each relevant register at every step.

SOLUTION

Hardware and algorithm: the Optimized Divider (slides)

With a 4-bit ALU, $n = 4$: a 4-bit **Divisor** register and an 8-bit **Remainder** register (left half Rem_{hi}, right half Rem_{lo}). There is no Quotient register; quotient bits enter at the right end of the Remainder. **0.** Remainder $\leftarrow$ dividend, shift Remainder left 1. Repeat n times: **1.** $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Divisor}$; **2a.** if ≥ 0: shift Remainder left, new rightmost bit $= 1$; **2b.** if < 0: restore ($\text{Rem}_{hi} + \text{Divisor}$), shift Remainder left, new rightmost bit $= 0$. **3.** Shift Rem_{hi} right 1. Result: Rem_{lo} = quotient, Rem_{hi} = remainder.

Why shift first? The first quotient bit has weight 2^{n-1}: it asks whether Divisor$\times 2^{n-1}$ fits, i.e. it compares the Divisor with the dividend's top $n+1$ bits, which is what the left half holds after one left shift. Subtracting from the unshifted left half would test Divisor$\times 2^n$, a quotient bit an n-bit quotient cannot have; if that test succeeds the division overflows (as happens here). The extra left shift is undone by the final right shift of Rem_{hi}.



Divide-overflow check

The quotient can have only $n = 4$ bits (it lives in Rem_{lo}). That works only if the dividend's left half is smaller than the divisor. Here $\text{Rem}_{hi} = \texttt{1101} = 13 \geq \texttt{1011} = 11$, so the true quotient $213 \div 11 = 19 = \texttt{10011}_2$ needs 5 bits: this is a **divide overflow**. Real hardware checks $\text{Rem}_{hi} \geq$ Divisor before starting and signals overflow. Below the 4-bit trace is shown as asked, and then the same algorithm on a 5-bit ALU, which gives the correct result.

(i) Strict 4-bit trace: $\texttt{11010101}_2 \div \texttt{1011}_2$

The first left shift pushes the dividend's leading 1 out of the 8-bit register, so the hardware is already working on the wrong number. Changed values are in **coral**; a negative difference is signalled by the ALU's borrow.

Iter.	Step	Divisor	Remainder ($\text{Rem}_{hi} \mid \text{Rem}_{lo}$)	sign
0	Initial values	1011	1101 | 0101	
	Shift Rem left 1 (MSB 1 lost)	1011	1010 | 1010	
1	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	1011	1111 | 1010	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	1011	0101 | 0100	
2	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	1011	1010 | 0100	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	1011	1010 | 1000	

Iter.	Step	Divisor	Remainder ($\text{Rem}_{hi} \mid \text{Rem}_{lo}$)	sign *(cont.)*
3	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	1011	1111 | 1000	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	1011	0101 | 0000	
4	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	1011	1010 | 0000	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	1011	1010 | 0000	
	3: Shift Rem_{hi} right 1	1011	0101 | 0000	

The 4-bit hardware ends with quotient `0000` and remainder `0101`, which is **wrong** ($11 \times 0 + 5 \neq 213$). This is the divide overflow: an n-bit divider cannot produce an $(n+1)$-bit quotient.

(ii) Same algorithm on a 5-bit ALU (correct result)

With $n = 5$ the dividend is `00 1101 0101` (10-bit Remainder) and the Divisor is `01011`; now $\text{Rem}_{hi} =$ `00110` $<$ `01011`, so no overflow.

Iter.	Step	Divisor	Remainder ($\text{Rem}_{hi} \mid \text{Rem}_{lo}$)	sign
0	Initial values	01011	00110 | 10101	
	Shift Rem left 1	01011	01101 | 01010	
1	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	01011	00010 | 01010	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	01011	00100 | 10101	
2	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	01011	11001 | 10101	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	01011	01001 | 01010	
3	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	01011	11110 | 01010	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	01011	10010 | 10100	

Iter.	Step	Divisor	Remainder ($\text{Rem}_{hi} \mid \text{Rem}_{lo}$)	sign *(cont.)*
4	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	01011	00111 | 10100	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	01011	01111 | 01001	
5	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	01011	00100 | 01001	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	01011	01000 | 10011	
	3: Shift Rem_{hi} right 1	01011	00100 | 10011	

Answer. The 4-bit divider overflows (quotient needs 5 bits) and gives a wrong result. With a 5-bit ALU: Quotient $= 10011_2 = 19$, Remainder $= 00100_2 = 4$. Check: $11 \times 19 + 4 = 213$ ✓

2017–2018 • Q8(b) CSE 305 • 23/09/2018 • AR23 **MARKS 10**

Write down the equivalent MIPS assembly code for the following function, written in C programming language. Assume appropriate registers for the variables.

```
float convert (float cels)
{
    float ferh = (9.0 * cels) / 5.0;
    ferh = ferh + 32.0;
    return ferh;
}
```

SOLUTION

Rules used (slides: FP instructions in MIPS, `f2c` example)

FP registers `$f0`–`$f31` (doubles in even/odd pairs); single ops `.s`, double ops `.d`; loads `lwc1` (single) / `ldc1` (double); conversions `cvt.d.s` / `cvt.s.d`. Convention: argument `cels` in `$f12`, result in `$f0`, constants in the global data area (offsets from `$gp`).

Exact C semantics

In C the literals `9.0`, `5.0`, `32.0` are *double*, so `cels` is promoted, the arithmetic is double, and the value is converted back to float at each assignment to `ferh`:

```
convert:
```

```
        ldc1    $f16, const9($gp)   # $f16 = 9.0  (double constant)
        cvt.d.s $f18, $f12          # $f18 = (double) cels
        mul.d   $f16, $f16, $f18    # 9.0 * cels
        ldc1    $f18, const5($gp)   # $f18 = 5.0
        div.d   $f16, $f16, $f18    # (9.0*cels) / 5.0
        cvt.s.d $f0, $f16           # ferh = (float) result
        cvt.d.s $f16, $f0           # promote ferh for ferh + 32.0
        ldc1    $f18, const32($gp)  # $f18 = 32.0
        add.d   $f16, $f16, $f18    # ferh + 32.0 (double)
        cvt.s.d $f0, $f16           # ferh = (float) ..., return in $f0
        jr      $ra
```

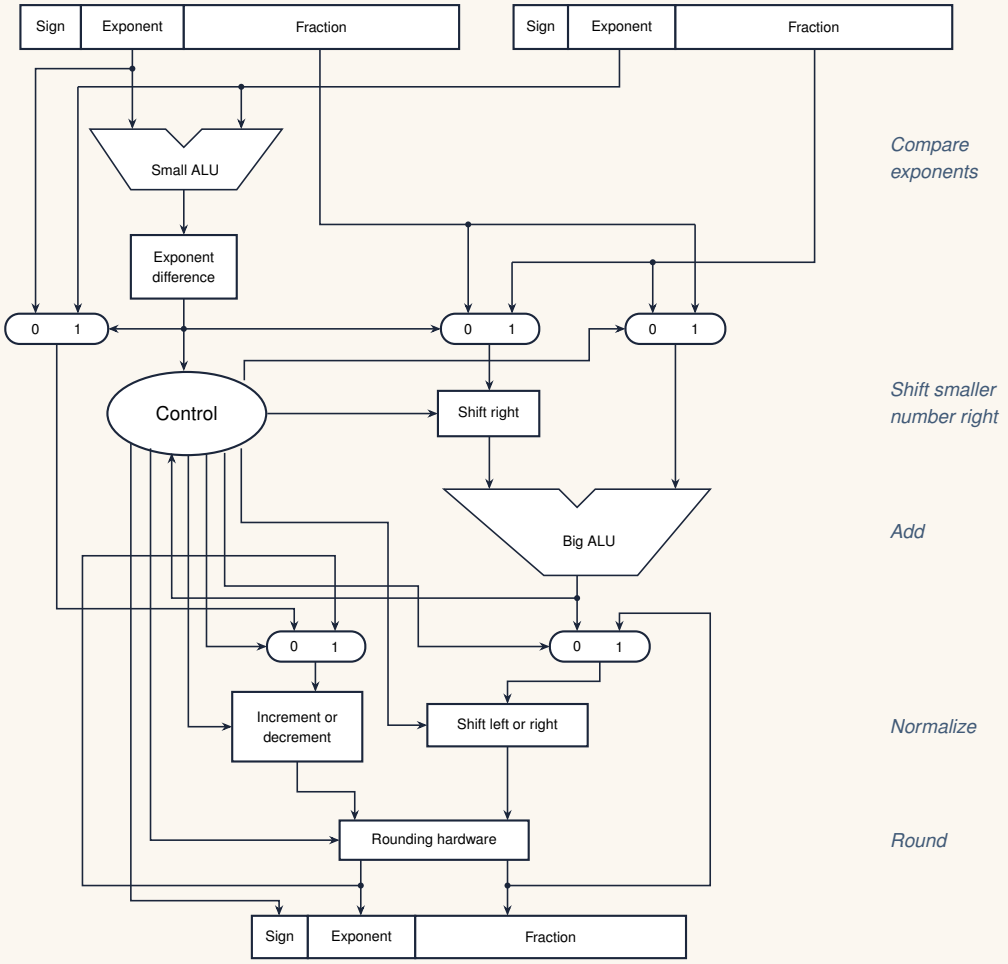
All-single version

If the constants are stored as single-precision floats (as in the slides' `f2c` example, i.e. `9.0f` etc.), everything stays single and no conversions are needed:

```
convert:
        lwc1  $f16, const9($gp)     # 9.0f
        mul.s $f16, $f16, $f12      # 9.0 * cels
        lwc1  $f18, const5($gp)     # 5.0f
        div.s $f16, $f16, $f18      # ferh = (9.0*cels)/5.0
        lwc1  $f18, const32($gp)    # 32.0f
        add.s $f0, $f16, $f18       # ferh = ferh + 32.0 -> $f0
        jr    $ra
```

2016–2017 • Q7(b) CSE 305 • 02/08/2017 • AR24 **MARKS 15**

Consider the following block diagram of a floating point addition hardware.



Block diagram of the floating-point addition hardware

Using the given hardware, simulate the floating addition algorithm by filling up the following tables. The two numbers for addition are, $-(0.25)_{10}$ and $(0.625)_{10}$.

	$-(0.25)_{10}$		$(0.625)_{10}$
Binary			
Step1: Exponent Matching			
Step2: Significand Addition			
Step3: Normalization		=	
Step4: Rounding			

SOLUTION

Method

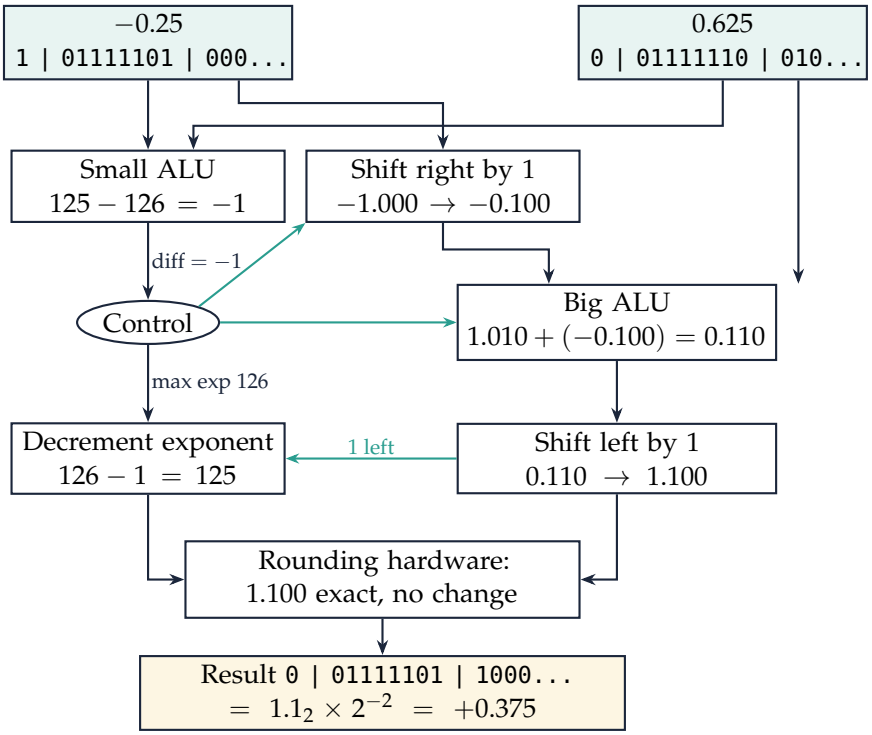
The hardware does the four steps of FP addition: (1) the small ALU subtracts the exponents and the control shifts the significand of the number with the smaller exponent right by the difference; (2) the big ALU adds the signed significands; (3) the result is normalized (shift left/right with exponent decrement/increment) and checked for over/underflow; (4) the rounding hardware rounds (and renormalizes if needed). 4-bit significands are used, as in the slides.

Filled tables

$-0.25 = -0.01_2 = -1.000_2 \times 2^{-2}$ (fields `1 | 01111101 | 000...`) and $0.625 = 0.101_2 = 1.010_2 \times 2^{-1}$ (fields `0 | 01111110 | 010...`).

	$-(0.25)_{10}$	$(0.625)_{10}$
Binary	-1.000×2^{-2}	1.010×2^{-1}
Step1: Exponent matching	-0.100×2^{-1}	1.010×2^{-1}
Step2: Significand addition	$1.010 \times 2^{-1} + (-0.100 \times 2^{-1}) = 0.110 \times 2^{-1}$	
Step3: Normalization	0.110×2^{-1} =	1.100×2^{-2}
Step4: Rounding	1.100×2^{-2}	(already fits; $= 0.375$)

How the hardware does it



Exponent after normalization $= -2$ (biased 125), inside $[-126, 127]$: no overflow/underflow, and rounding leaves it normalized.

Answer. $-0.25 + 0.625 = 1.100_2 \times 2^{-2} = +0.375$; single precision `0 01111101 10000000000000000000000` = `0x3EC00000`.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q8(a) CSE 305 • 02/08/2017 • AR25 **MARKS 5 + 5**

The following table illustrates the various combinations of two operands for binary addition-subtraction. Please identify the overflow conditions in that table. Then, formulate the overflow boolean variable as a logical function of various input variables (You don't need to simplify the function).

O	S_A	S_B	S_R	V
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Here,

Operation, $O = \begin{cases} 0, & A + B \\ 1, & A - B \end{cases}$

Sign bit of Operand A, $S_A = \begin{cases} 0, & A \geq 0 \\ 1, & A < 0 \end{cases}$

Sign bit of Operand B, $S_B = \begin{cases} 0, & B \geq 0 \\ 1, & B < 0 \end{cases}$

Sign bit of Result R, $S_R = \begin{cases} 0, & R \geq 0 \\ 1, & R < 0 \end{cases}$

Overflow, $V = \begin{cases} 1, & \text{Overflow occurred} \\ 0, & \text{No overflow possible} \end{cases}$

2014–2015 • Q8(a) CSE 305 • 26/01/2016 • AR25 **MARKS 8 + 4**

The following table illustrates the various combinations of two operands for binary addition-subtraction. Please identify the overflow conditions in that table. Then, formulate the overflow boolean variable as a logical function of various input variables (You don't need to simplify the function).

O	S_A	S_B	S_R	V
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Here,

Operation, $O = \begin{cases} 0, & A + B \\ 1, & A - B \end{cases}$

Sign bit if Operand A, $S_A = \begin{cases} 0, & A \geq 0 \\ 1, & A < 0 \end{cases}$

Sign bit of Operand B, $S_B = \begin{cases} 0, & B \geq 0 \\ 1, & B < 0 \end{cases}$

Sign bit of Result R, $S_R = \begin{cases} 0, & R \geq 0 \\ 1, & R < 0 \end{cases}$

Overflow, $V = \begin{cases} 0, & \text{Overflow occured} \\ 1, & \text{No overflow possible} \end{cases}$

SOLUTION

Identifying overflow

Overflow is possible only when the *effective* operands have the same sign: $A + B$ with $S_A = S_B$, or $A - B$ (i.e. $A + (-B)$) with $S_A \neq S_B$. It then occurs when the result sign S_R differs from S_A (slides: $+\text{ve} + +\text{ve} \rightarrow$ sign 1, $-\text{ve} + -\text{ve} \rightarrow$ sign 0, $+\text{ve} - -\text{ve} \rightarrow$ sign 1, $-\text{ve} - +\text{ve} \rightarrow$ sign 0). For the other rows overflow is impossible whatever S_R is.

O	S_A	S_B	S_R	V	Meaning
0	0	0	**1**	**1**	$(+)+(+)$ gives $-$: overflow
0	0	1	x	0	$(+)+(-)$: no overflow possible
0	1	0	x	0	$(-)+(+)$: no overflow possible
0	1	1	**0**	**1**	$(-)+(-)$ gives $+$: overflow
1	0	0	x	0	$(+)-(+)$: no overflow possible
1	0	1	**1**	**1**	$(+)-(-)$ gives $-$: overflow
1	1	0	**0**	**1**	$(-)-(+)$ gives $+$: overflow
1	1	1	x	0	$(-)-(-)$: no overflow possible

(x = either value; with the opposite S_R in the four overflow rows, $V = 0$.)

Overflow function (sum of the four minterms in O, S_A, S_B, S_R)

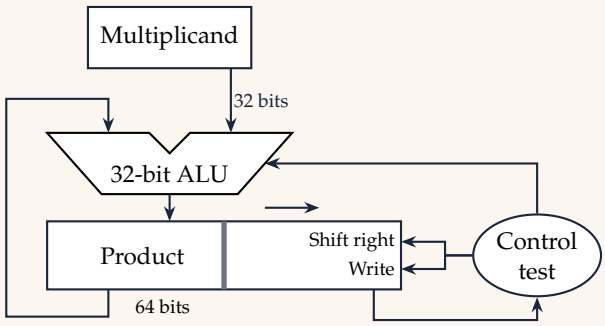
$$V = \overline{O}\,\overline{S_A}\,\overline{S_B}\,S_R \;+\; \overline{O}\,S_A\,S_B\,\overline{S_R} \;+\; O\,\overline{S_A}\,S_B\,S_R \;+\; O\,S_A\,\overline{S_B}\,\overline{S_R}$$

(Optionally simplified: $V = \overline{S_A \oplus S_B \oplus O} \cdot (S_A \oplus S_R)$ — the effective signs agree and the result sign differs.)

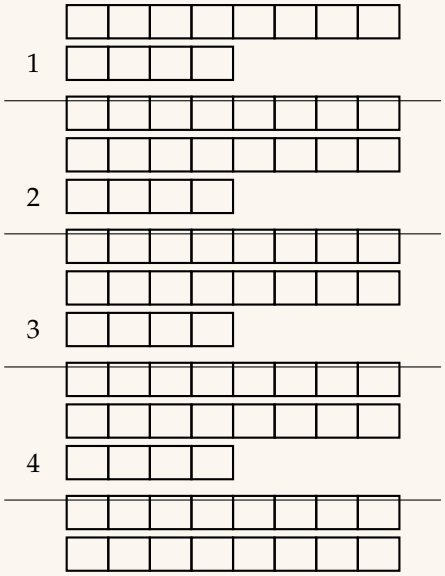
NOTE **2014–2015 card:** the scan defines $V = 0$ for "overflow occurred" and $V = 1$ for "no overflow possible" — the reverse of the 2016–17 paper and of the usual convention (probably a misprint). Under the definition exactly as printed, the V column above is simply complemented (1 in the four "no overflow" rows, 0 in the overflow rows), and $V_{\text{printed}} = \overline{\overline{O}\,\overline{S_A}\,\overline{S_B}\,S_R + \overline{O}\,S_A\,S_B\,\overline{S_R} + O\,\overline{S_A}\,S_B\,S_R + O\,S_A\,\overline{S_B}\,\overline{S_R}}$.

2016–2017 • Q8(c) CSE 305 • 02/08/2017 • AR26 MARKS 15

Consider the following multiplication hardware.



Using the given hardware, simulate the unsigned multiplication algorithm by filling up the following tables for Multiplicand, 0110 and Multiplier, 0110.



SOLUTION

Algorithm for the given (refined) hardware

The 64-bit Product register starts with 0 in its left half and the multiplier in its right half. Each of the n iterations (here $n = 4$) does two things:

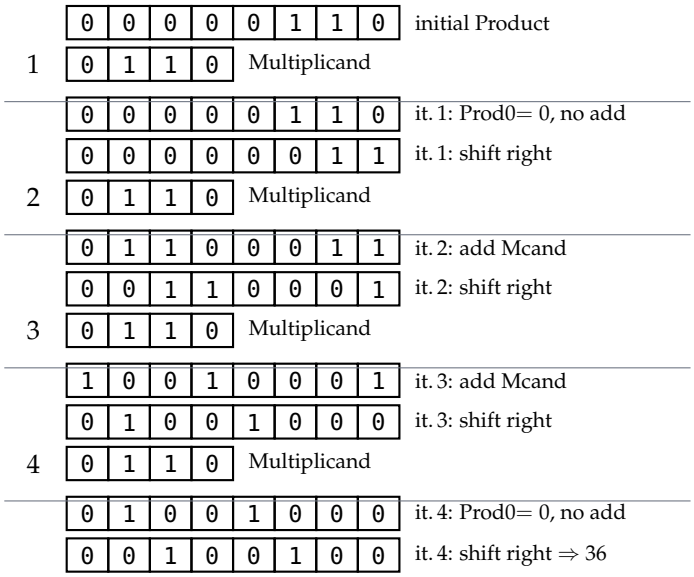
(i) If Product0 $= 1$, add the Multiplicand to the left half of the Product and write the result back into the left half.

(ii) Shift the Product register right 1 bit; the ALU carry-out enters at the MSB.

Expected result: $6 \times 6 = 36 = 0010\,0100_2$.

Iteration	Step	Multiplicand	Product
0	Initial values	0110	0000 0110
1	1: Prod0 $= 0 \Rightarrow$ no operation	0110	0000 011**0**
	2: Shift right Product	0110	0000 0011
2	1a: Prod0 $= 1 \Rightarrow 0000 + 0110$	0110	0110 001**1**
	2: Shift right Product	0110	0011 0001
3	1a: Prod0 $= 1 \Rightarrow 0011 + 0110$	0110	1001 000**1**
	2: Shift right Product	0110	0100 1000
4	1: Prod0 $= 0 \Rightarrow$ no operation	0110	0100 100**0**
	2: Shift right Product	0110	**0010 0100**

The same values written into the answer form. The 8-box rows are the Product register; the 4-box rows are the Multiplicand used in iteration k:



Answer. $0110 \times 0110 = 0010\,0100_2 = 36_{10}$.

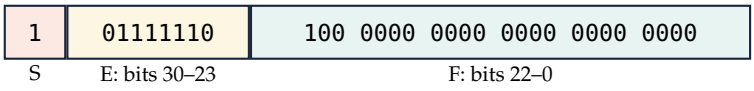
NOTE The answer form in the scan has no row labels. We read it as: the initial Product, then for each iteration k the Multiplicand box "k", followed by the Product after step 1 (add or no-op) and after the right shift.

2013–2014 • Q3(b) CSE 305 • 05/07/2015 • AR27 **MARKS 10**

Show the IEEE 754 binary representation of the number -0.75_{ten} in single precision.

SOLUTION

1. Binary: $0.75 = 0.5 + 0.25 = 0.11_2$, so $-0.75 = -0.11_2$.
2. Normalize: $-0.11_2 = (-1)^1 \times 1.1_2 \times 2^{-1}$.
3. Sign $S = 1$; Fraction $F =$ `1000...0` (the leading 1 is hidden); Exponent $E = -1 + 127 = 126 =$ `01111110`$_2$.

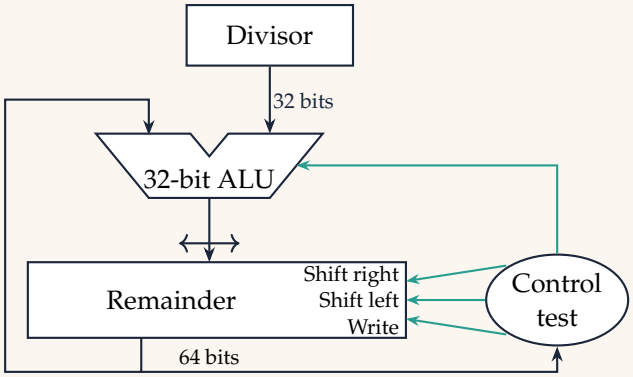


Check: $(-1)^1 \times (1 + 0.5) \times 2^{126-127} = -1.5 \times 0.5 = -0.75$ ✓

Answer. $-0.75 =$ `1 01111110 10000000000000000000000`$_2 =$ `0xBF400000`.

2012–2013 • Q8(b) CSE 305 • 10/05/2014 • AR28 **MARKS 12**

Draw the flowchart showing the execution steps of the division operation for the division hardware organization given below:

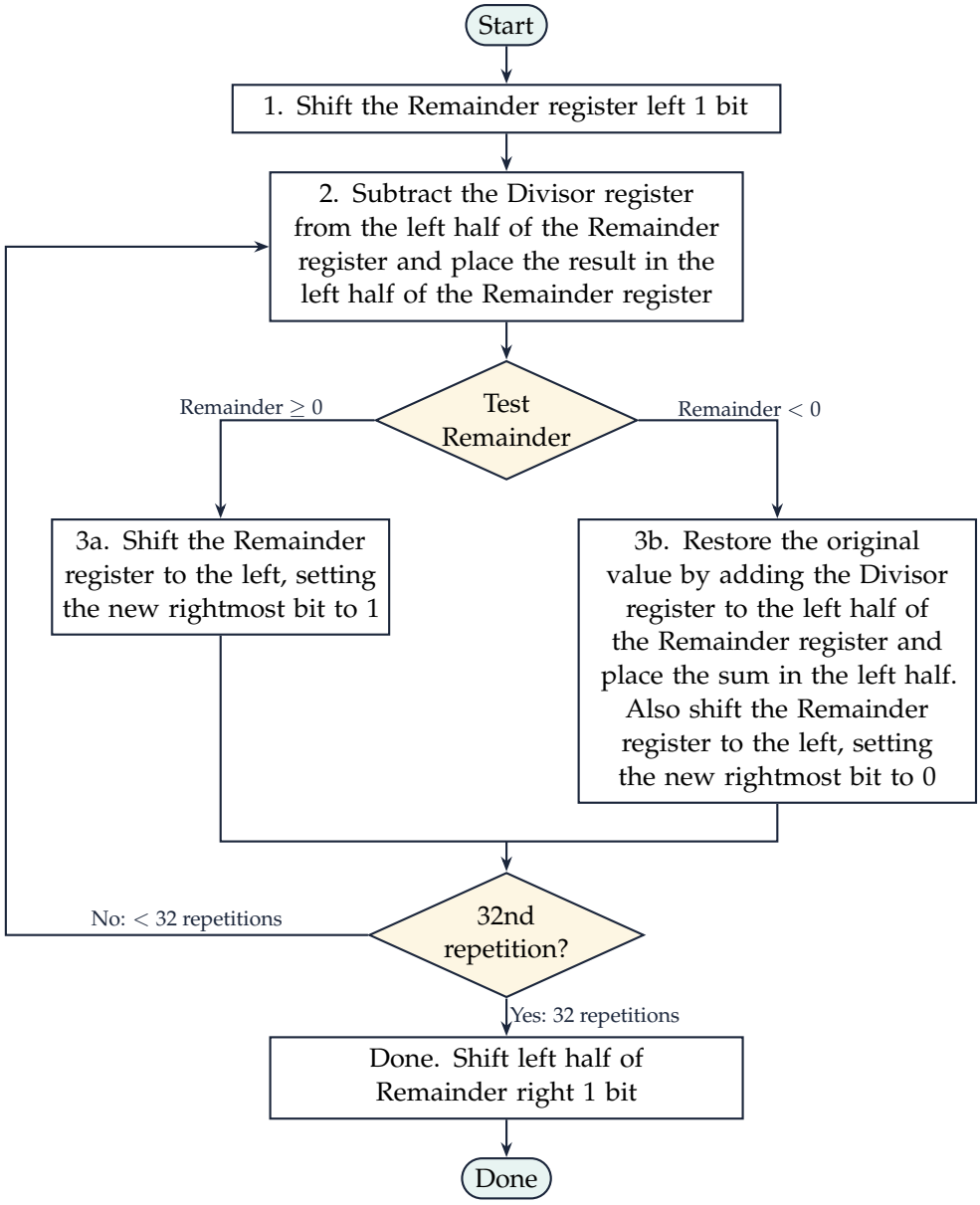


SOLUTION

The hardware

This is the refined (optimized) divider: there is no separate Quotient register and the divisor does not move. The 64-bit Remainder register initially holds the dividend (in its right half); its left half feeds the 32-bit ALU together with the Divisor; quotient bits are shifted in from the right; the control tests the sign of the result to decide whether to restore.

Flowchart (restoring division, 32 repetitions)



At the end the **left half** of the Remainder register holds the remainder and the **right half** holds the quotient (the final right shift of the left half undoes the one extra left shift). Quick check with 4 bits, $7 \div 2$: `0000 0111` $\rightarrow$ shift `0000 1110` $\rightarrow$ (restore) `0001 1100` $\rightarrow$ (restore) `0011 1000` $\rightarrow$ (subtract ok) `0011 0001` $\rightarrow$ (subtract ok) `0010 0011` $\rightarrow$ final `0001 0011`: remainder 1, quotient 3 ✓

05

The Processor: Datapath and Control

2023–2024 • Q5(a) CSE 209 • 29/04/2026 • DP01 **MARKS 20**

Consider a modified MIPS instruction set, called MIPS-v22, where each instruction is represented in 32-bits. Figure for Question 5(a) shows the format of R-type and I-type instruction of MIPS-v22.

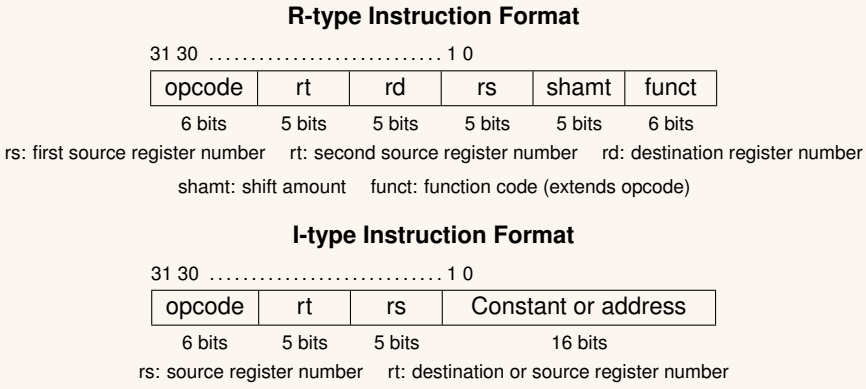


Figure for Question 5(a): R-type and I-type Instruction Formats for MIPS-v22

(i) Develop a block diagram of the partial single cycle datapath for R-type and I-type instructions of MIPS-v22 containing only an Instruction Memory, a Register file, and one or more MUXs (as required). Note that the output of the Instruction memory is a 32 bit MIPS-v22 instruction. Your task is to devise only connections from the Instruction Memory to the following 3 inputs of the Register File: Read register 1 number, Read register 2 number, Write register number. You may use as many MUXs as required. Clearly show the bit size of the connections.

(ii) Consider the numbers of the registers `$t0`, `$t1`, and `$t2` are 2, 3, 4 in decimal respectively. Translate the following 3 instructions in the binary format following the given MIPS-v22 instruction format. Assume shamt $= 0_{10}$, and funct $= 0_{10}$.

- `add $t0, $t1, $t2` Here `$t0`: destination, `$t1`: source 1, `$t2`: source 2, opcode $= 0_{10}$
- `addi $t2, $t0, 1` Here `$t2`: destination, `$t0`: source 1, opcode $= 6_{10}$
- `lw $t1, 4($t0)` Here `$t1`: destination, `$t0`: base address, opcode $= 5_{10}$

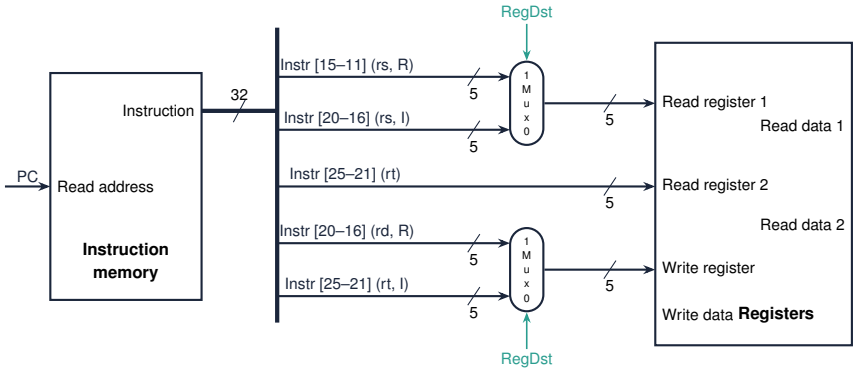
SOLUTION

(i) Where each register number sits

RF input	R-type field	I-type field	Hardware
Read register 1 (first source, rs)	[15–11]	[20–16]	2-to-1 mux
Read register 2 (second source, rt)	[25–21]	[25–21] (rt, for `sw`/`beq`)	direct wire
Write register (destination)	[20–16] (rd)	[25–21] (rt)	2-to-1 mux

Two 5-bit 2-to-1 muxes are needed. Both can share one control line, **RegDst** (1 for R-type, 0 for I-type). Read register 2 is always Instr[25–21], so it needs no mux. Every connection from the 32-bit

instruction is a 5-bit field.



Opcode-based control: for R-type (opcode 0) RegDst = 1, so the muxes pass rs = [15–11] and rd = [20–16]. For I-type RegDst = 0, so they pass rs = [20–16] and rt = [25–21].

(ii) Encodings (`$t0=2, $t1=3, $t2=4`)

	opcode	rt	rd	rs	shamt	funct
`add $t0,$t1,$t2`	`000000`	`00100 ($t2)`	`00010 ($t0)`	`00011 ($t1)`	`00000`	`000000`

	opcode	rt	rs	constant / address
`addi $t2,$t0,1`	`000110`	`00100 ($t2)`	`00010 ($t0)`	`0000 0000 0000 0001`
`lw $t1,4($t0)`	`000101`	`00011 ($t1)`	`00010 ($t0)`	`0000 0000 0000 0100`

Answer. add: `000000 00100 00010 00011 00000 000000` = `0x00821800`
addi: `000110 00100 00010 0000000000000001` = `0x18820001`
lw: `000101 00011 00010 0000000000000100` = `0x14620004`

2023–2024 • Q8(a) CSE 209 • 29/04/2026 • DP02 **MARKS 20**

A single cycle datapath implementation of a subset of 32-bit MIPS instruction set has been shown in Figure for Question 8(a). Answer the following:

(i) Why is PC incremented by 4?

(ii) How is the length of the clock cycle determined in a single cycle execution?

(iii) Why is the upper 4 bits of PC + 4 concatenated in jump address calculation?

(iv) "A single physical memory unit can be used to implement both instruction memory and data memory in a hardware based implementation of a single cycle datapath" – agree or disagree? Justify your answer.

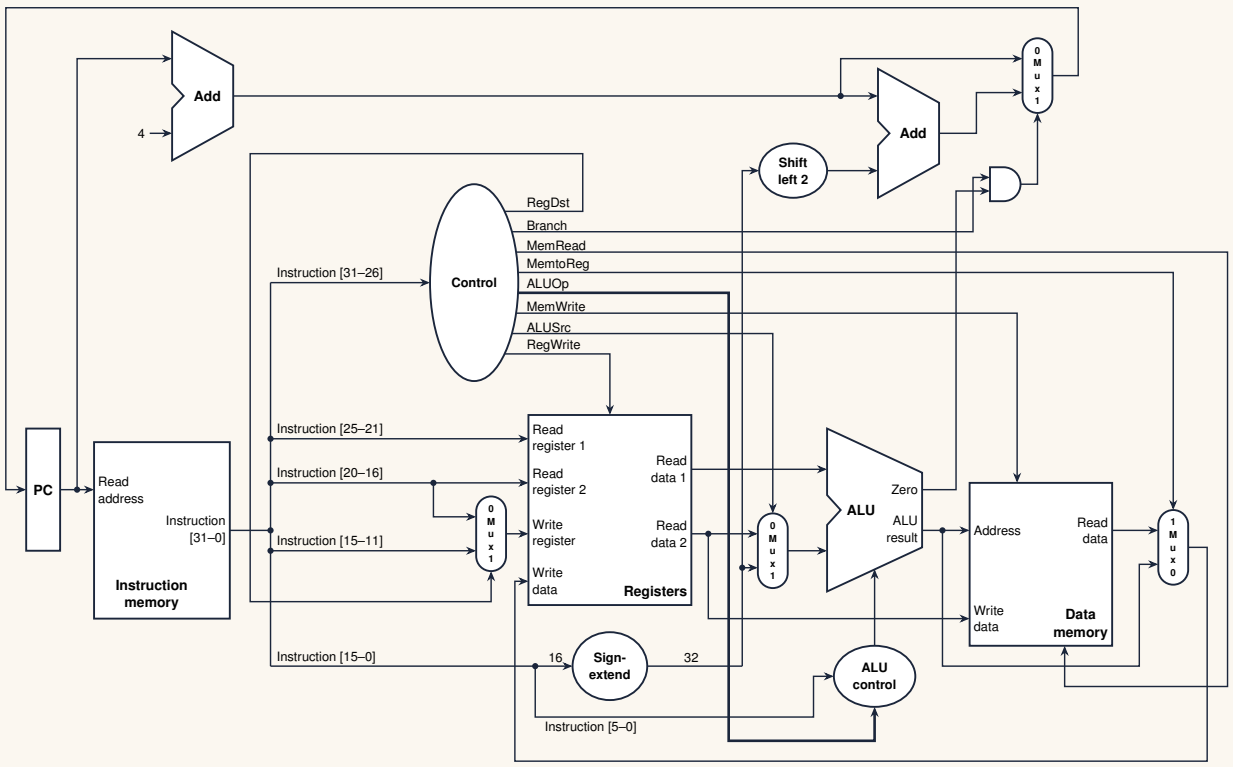


Figure for Question 8(a): Single cycle datapath with control signals

SOLUTION

(i) PC + 4. MIPS memory is *byte addressed*, and every instruction is exactly 32 bits = 4 bytes, stored word-aligned. The next sequential instruction therefore starts 4 byte-addresses later, so the PC adder adds 4. Branch offsets are also counted from PC+4.

(ii) Clock cycle length. Each instruction completes in one cycle, so the period must cover the *longest* (critical) combinational path from one state element to the next, plus register setup and clock-to-Q times. That path belongs to `lw`:

$$T_{clk} \geq t_{\text{I-mem}} + t_{\text{reg read}} + t_{\text{ALU}} + t_{\text{D-mem}} + t_{\text{mux}} + t_{\text{reg setup}}.$$

For example, with 200/100/200/200/100 ps stages this gives 800 ps. Every instruction, even a fast `add` or `j`, then takes 800 ps.

(iii) Upper 4 bits of PC+4 in jumps. The J-format has only 26 bits for the target. Since instructions are word aligned, the two low bits are always 00, so Instr[25–0]$\ll$ 2 gives 28 bits. The missing 4 high bits are taken from PC+4 (pseudo-direct addressing): JTA = $\{(\text{PC}+4)[31{:}28], \text{Instr}[25{:}0], 00\}$. This gives a full 32-bit address, and the jump stays inside the current 256 MB region. PC+4 is used rather than PC because the incremented value is already available, and the region is defined relative to the next instruction.

(iv) Disagree. In a single-cycle datapath every functional unit can be used only *once per clock cycle*. A `lw` or `sw` must, in the same cycle, (1) fetch its own instruction and (2) read or write a data word. A single-ported memory cannot do both at once, so there is a structural conflict. That is why the single-cycle design has *separate* instruction and data memories, just as it has separate adders for PC+4 and the branch target. A single unified memory works in a *multicycle* design, where fetch and data access happen in different cycles. A true dual-ported memory would really be two access paths, i.e. logically two memories.

2023–2024 • Q8(b) CSE 209 • 29/04/2026 • DP03 **MARKS 10**

Identify the component(s) required to be added to implement the jump instruction in the Figure for Question 8(a). Draw the diagram in your answer script implementing jump instruction assuming 32 bit addressing. [Include only the relevant components and control lines to implement jump instruction in your drawing.]

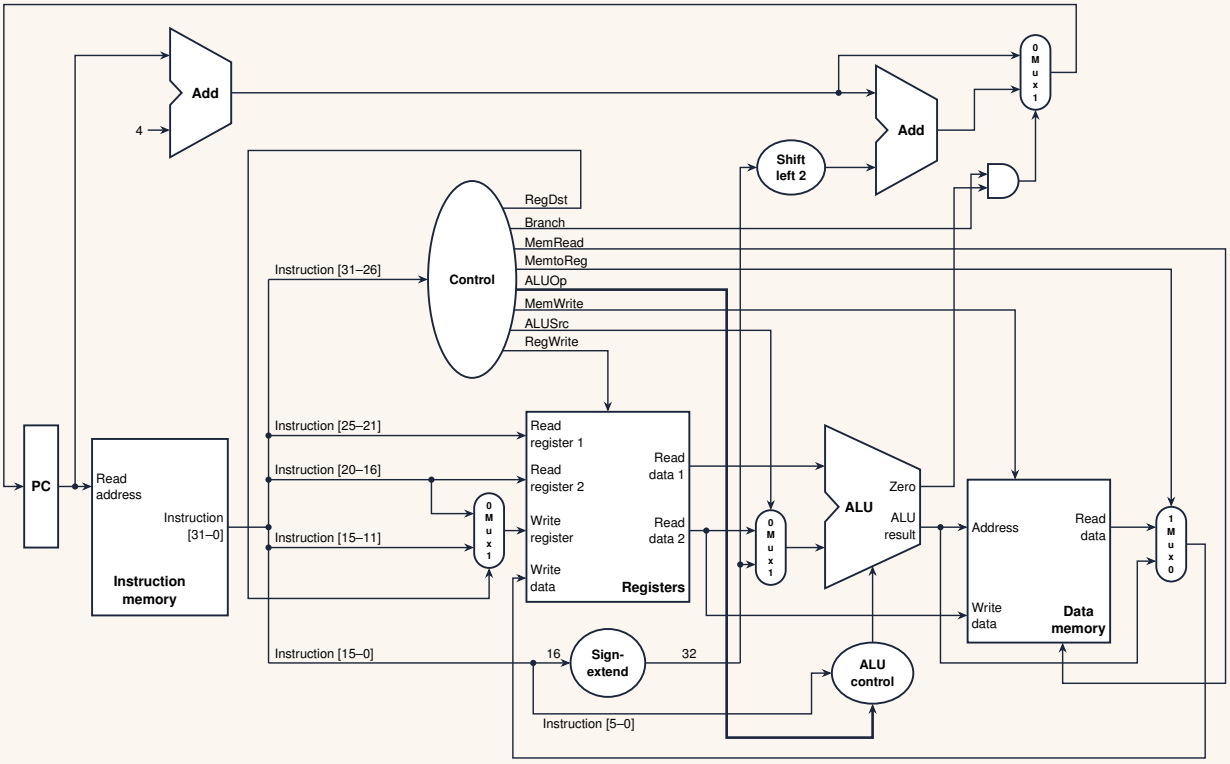


Figure for Question 8(a): Single cycle datapath with control signals

SOLUTION

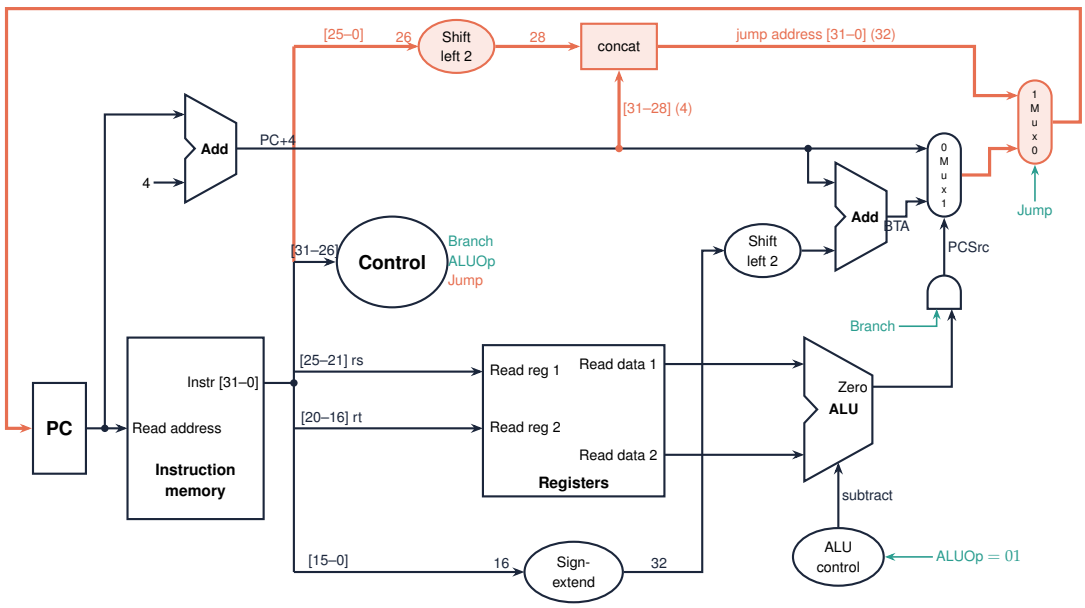
Jump target (pseudo-direct addressing): JTA $= \{(\text{PC}+4)[31{:}28],\ \text{Instr}[25{:}0],\ 00\}$, i.e. a 26-bit word address shifted left by 2 and topped up with the 4 high bits of PC+4 to make 32 bits.
Components to add:

(i) a *shift-left-2* on Instr[25–0] (26 → 28 bits; it is only wiring);

(ii) a *concatenation* of PC+4[31–28] with those 28 bits, giving the 32-bit jump address;

(iii) one more *2-to-1 mux* in front of the PC, choosing between the existing branch-mux output (PC+4 or BTA) and the jump address;

(iv) a new main-control output **Jump**, which is 1 only for opcode 000010 (for `j`, RegWrite = MemWrite = 0).



Coral = the added jump hardware. Black = existing relevant parts (fetch, PC+4, branch mux).

2022–2023 • Q3 CSE 209 • 13/01/2025 • DP04 **MARKS 15**

In the given incomplete MIPS single cycle data path shown in the Figure for Question 3, add new connections (no need to draw the control unit; if you need control signal(s) just naming them will suffice) to support `beq`, `bneq`, `j` and `jr` instructions using minimum numbers of MUX. You can only use 2×1 MUXs.

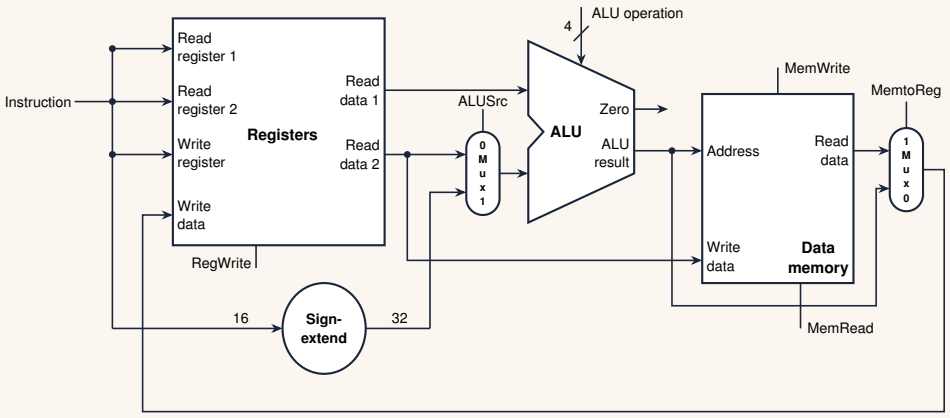


Figure for Question 3

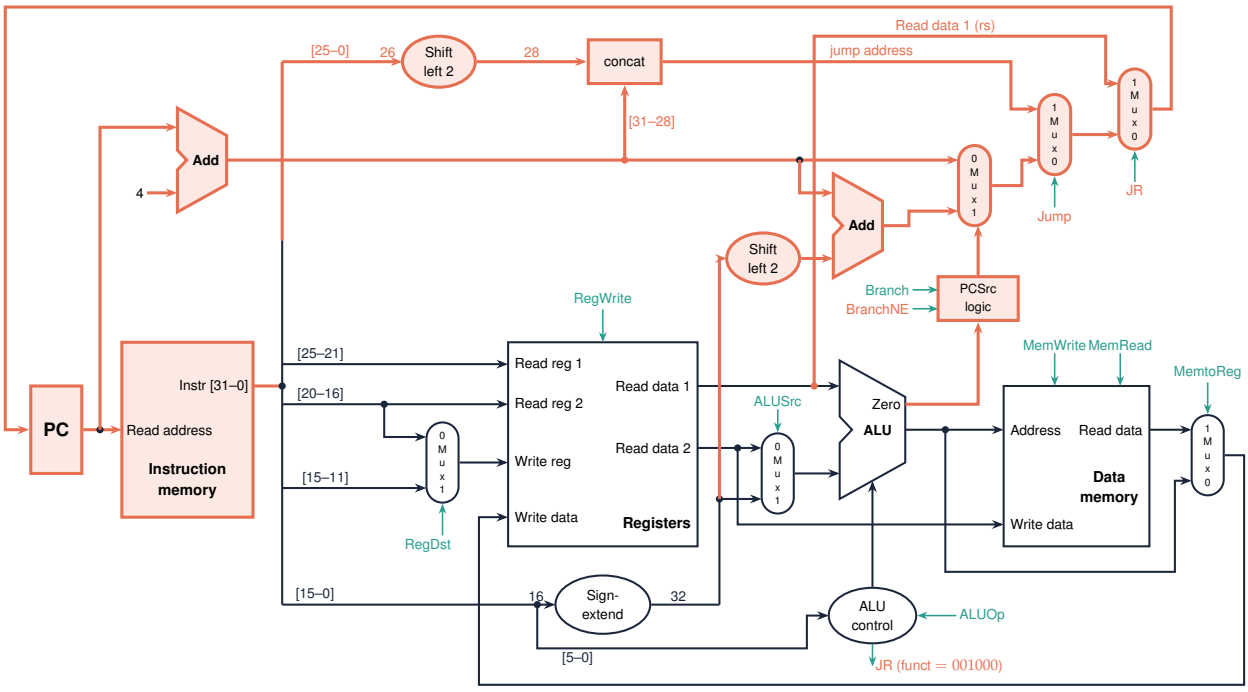
SOLUTION

What the four instructions need

- `beq rs,rt,L`: PC ← BTA = PC+4+SignExt(imm)×4 if rs = rt. The ALU subtracts and gives Zero.
- `bneq rs,rt,L`: the same target, taken if rs ≠ rt. It shares the BTA mux; only the select logic differs.
- `j L`: PC ← {PC+4[31:28], Instr[25–0], 00}.

- `jr rs`: PC ← Reg[rs] = Read data 1.

So the PC has *four* possible sources: PC+4, BTA, jump address and Reg[rs]. Choosing among n sources with 2:1 muxes needs at least $n-1$ of them, so the minimum is **3 muxes**.



Black = the given partial datapath. Coral = added hardware: PC, instruction memory, PC+4 adder, shift-left-2 + branch adder, PCSrc logic, jump shift + concatenation, and the three 2:1 muxes (branch, Jump, JR).

PCSrc logic (two AND gates, one inverter, one OR gate):

$$\text{PCSrc} = \text{Branch} \cdot \text{Zero} \;+\; \text{BranchNE} \cdot \overline{\text{Zero}}.$$

The three muxes: (1) branch mux: PC+4 / BTA (PCSrc); (2) jump mux: previous / jump address (Jump); (3) JR mux: previous / Read data 1 (JR).

Instr.	Branch	BranchNE	Jump	JR	RegWrite	MemWrite	ALU op
`beq`	1	0	0	0	0	0	subtract
`bneq`	0	1	0	0	0	0	subtract
`j`	0	0	1	0	0	0	X
`jr`	0	0	X	1	0	0	X

`jr` is R-type (op = 0, funct = 8), so JR is decoded from the funct field and must force RegWrite = 0.

2022–2023 • Q4(a) CSE 305 • 25/04/2024 • DP05 **MARKS 16**

Consider the following branch type instruction format given in Figure for Question 4(a) assuming 32 bit addressing. Construct the formula to calculate branch target address. Draw the single cycle datapath for branch-on-equal instruction assuming 32 bit addressing. [Include only the relevant components and control lines in your design. For example, you can exclude data memory from your design.]

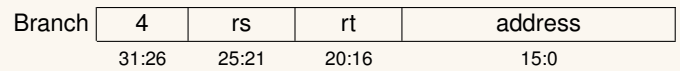


Figure for Question 4(a): Branch Instruction Format in MIPS

SOLUTION

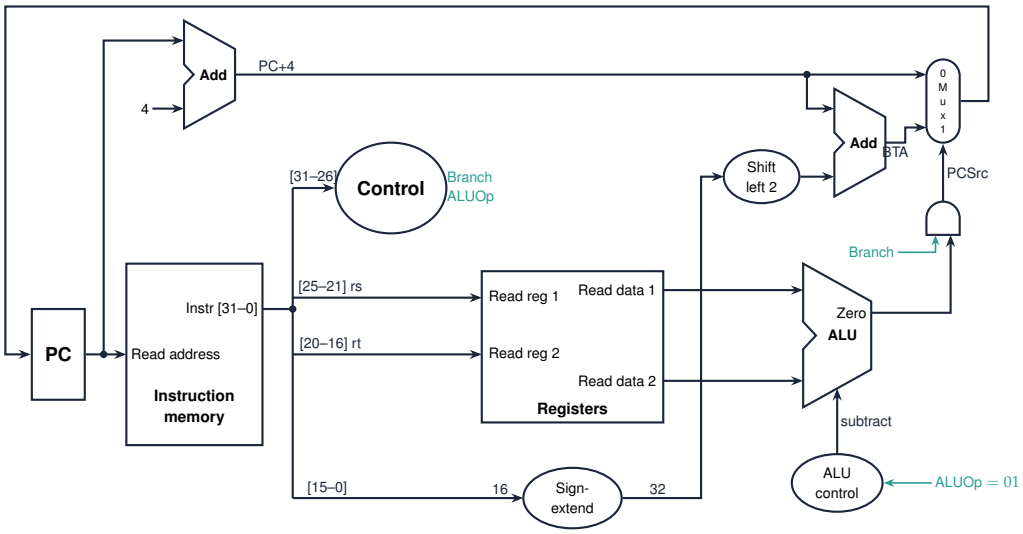
Branch target address

The 16-bit *address* field is a signed *word* offset relative to the next instruction:

$$\text{BTA} = (\text{PC} + 4) + \left(\text{SignExt}_{32}(\text{Instr}[15{:}0]) \ll 2\right) = \text{PC} + 4 + 4 \times \text{offset}.$$

The reach is $-2^{15} \ldots 2^{15} - 1$ words from PC+4.

Single-cycle datapath for beq (relevant parts only)



- **Fetch:** PC addresses the instruction memory, and an adder forms PC+4.
- **Compare:** rs and rt are read. The ALU subtracts them (Control: ALUOp $= 01 \rightarrow$ ALU control $=$ subtract), and *Zero* $= 1$ if they are equal.
- **Target:** Instr[15–0] is sign-extended to 32 bits, shifted left 2, and added to PC+4 by a separate adder (BTA).
- **Select:** PCSrc $=$ Branch $\cdot$ Zero picks BTA, otherwise PC+4. For `beq`: Branch $= 1$, RegWrite $=$ MemWrite $=$ MemRead $= 0$, ALUSrc $= 0$.

2021–2022 • Q1(a) CSE 305 • 06/04/2023 • DP06 **MARKS 10**

Complete the provided incomplete diagram of the MIPS single cycle datapath for the branch if equal (`beq`) and jump (`j`) instructions.

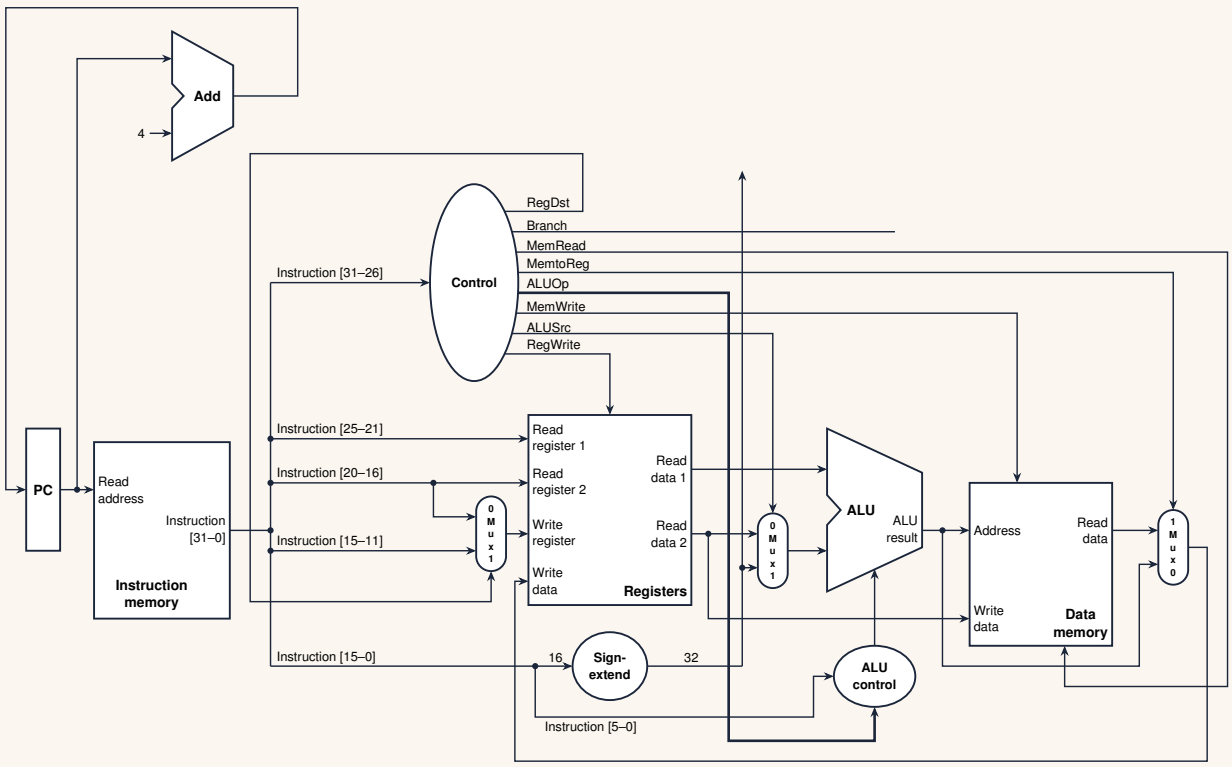
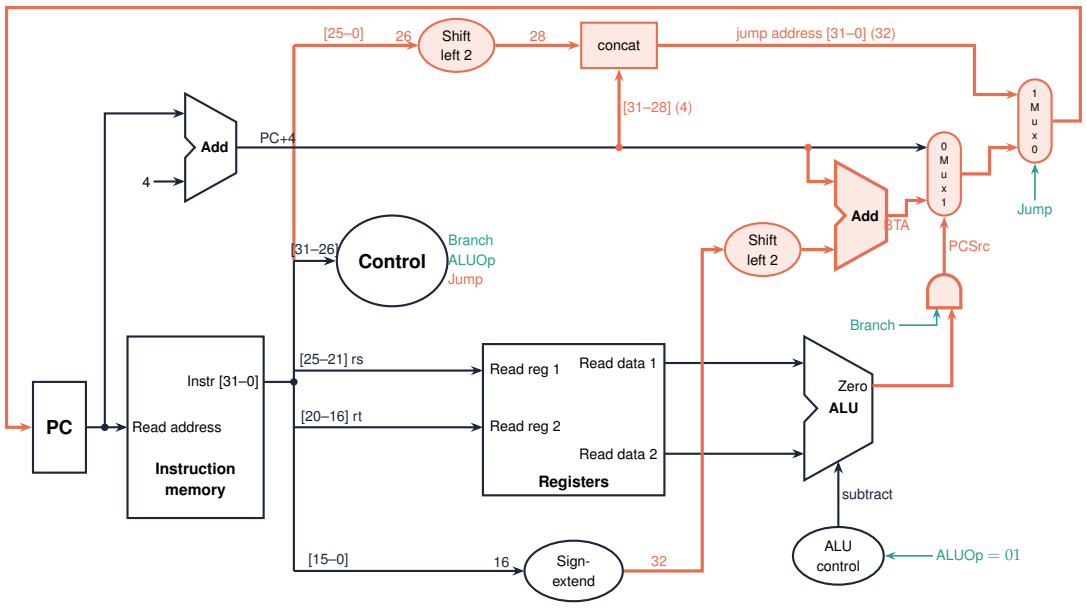


Figure for Question 1(a)

SOLUTION

Target addresses. BTA $= (\text{PC} + 4) + (\text{SignExt}(\text{Instr}[15{:}0]) \ll 2)$ and JTA $= \{(\text{PC} + 4)[31{:}28], \text{Instr}[25{:}0], 00\}$.

The completed PC-update part is drawn below (added hardware in coral). The rest of the given figure (data memory, write-back mux, RegDst and ALUSrc muxes) is unchanged and omitted for clarity.



- **beq**: add a shift-left-2 on the sign-extended offset, an adder (PC+4 + offset×4), an AND gate (Branch · Zero = PCSrc) and a 2:1 mux (PC+4 / BTA). The ALU subtracts (ALUOp = 01) to

produce Zero.

- **j**: add a shift-left-2 on Instr[25–0], a concatenation with PC+4[31–28], and a second 2:1 mux driven by a new control output *Jump*.

Instr.	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	Jump
`beq`	X	0	X	0	0	0	1	01	0
`j`	X	X	X	0	0	0	0	XX	1

2020–2021 • Q3 CSE 305 • 24/09/2022 • DP07 **MARKS 15**

A full datapath with a control unit is given in Figure 1. Highlight the datapath elements that will be active for the `ori $t1, 100($t2)` instruction. Also, mention which control bits will be set. [You may highlight the given figure and attach it to your answer script.]

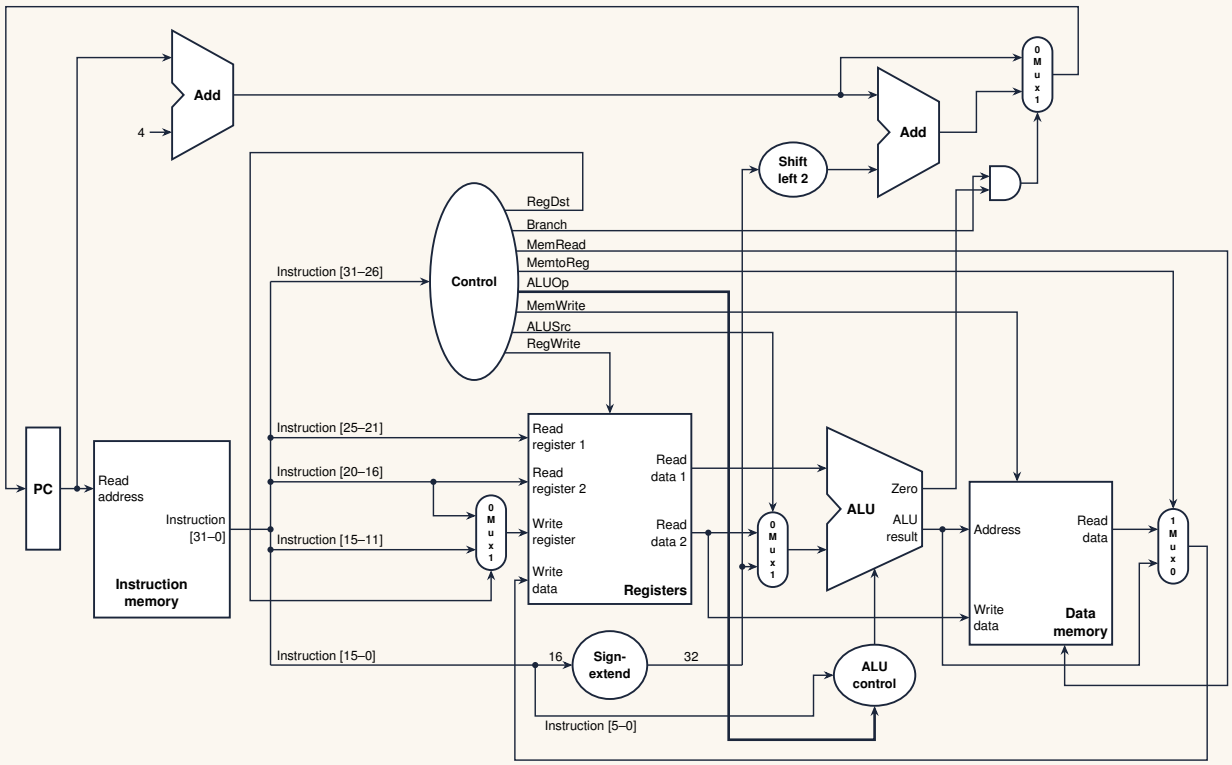
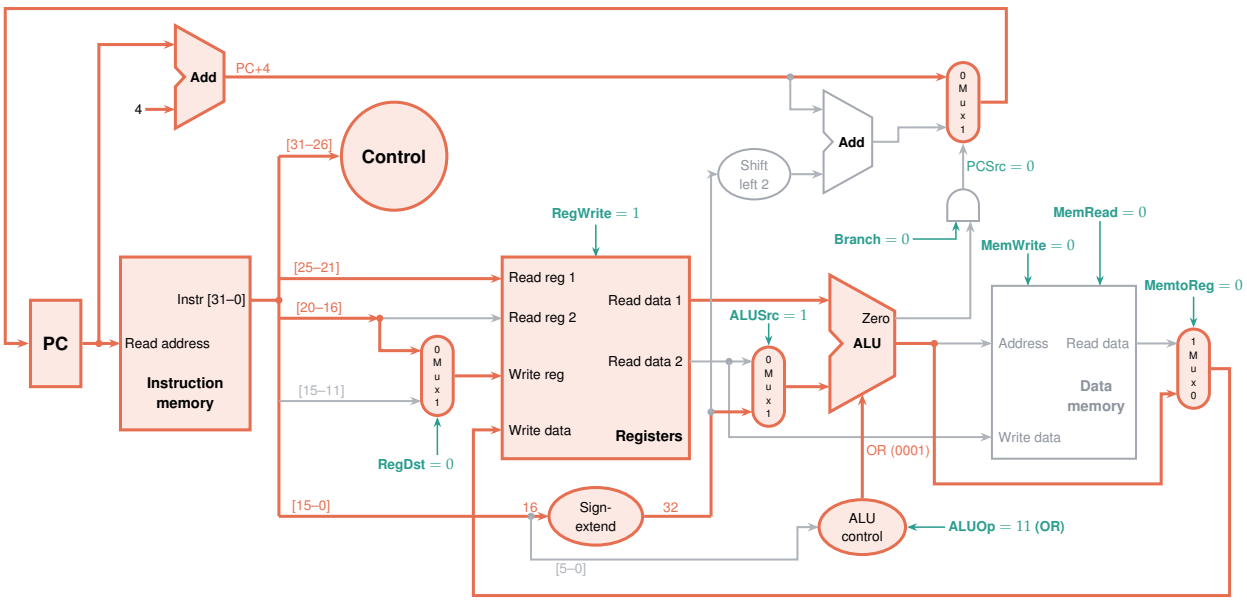


Figure 1 (missing from the scan; the standard single-cycle datapath with control unit is shown)

SOLUTION

NOTE Figure 1 is not in the scan. It is taken to be the standard single-cycle MIPS datapath with control (as in the slides), redrawn below. The operand syntax `100($t2)` is unusual for `ori`, which is a logical-immediate I-type instruction. It is read as `ori $t1, $t2, 100`, i.e. `$t1` ← `$t2` OR 100: rs = `$t2`, rt = `$t1`, immediate = 100.

What `ori` needs. Fetch and PC+4. Read rs (`$t2`). Feed the immediate into the ALU's second input (ALUSrc = 1). The ALU does *OR*. Write the ALU result (not memory data) into rt (`$t1`) (RegDst = 0, MemtoReg = 0, RegWrite = 1). Data memory and the branch path are *not* used.



Coral = active elements and paths for `ori`. Grey = idle (the data memory, the branch adder/shift path, the rt read port's data and the [15–11] field). Teal = control values.

Control bits

RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	ALU control
0	1	0	**1**	0	0	0	11	0001 (OR)

Answer. Set (=1): **ALUSrc** and **RegWrite**. All the others are 0 (RegDst, MemtoReg, MemRead, MemWrite, Branch/PCSrc). The ALU must perform OR. The basic 2-bit ALUOp codes (00 add, 01 sub, 10 funct) cannot ask for OR without a funct field, so a new code, ALUOp = 11 → OR, is assumed.

NOTE Strictly, `ori` *zero*-extends its immediate. With the standard sign-extend unit the result is still correct here, because 100 is positive (bit 15 = 0). A full implementation adds a zero-extend option, chosen by an *ExtOp* control bit.

2019–2020 • Q8(a) CSE 305 • 18/09/2021 • DP08 **MARKS 5×3 = 15**

Figure 8a shows a single cycle datapath implementation of a subset of MIPS instruction set. For each of the following three instructions: **add**, **sw**, **beq**, determine the values (0/1) of the following control signals shown in Figure 8a.

RegDst, Branch, MemRead, MemtoReg, MemWrite, ALUSrc, RegWrite

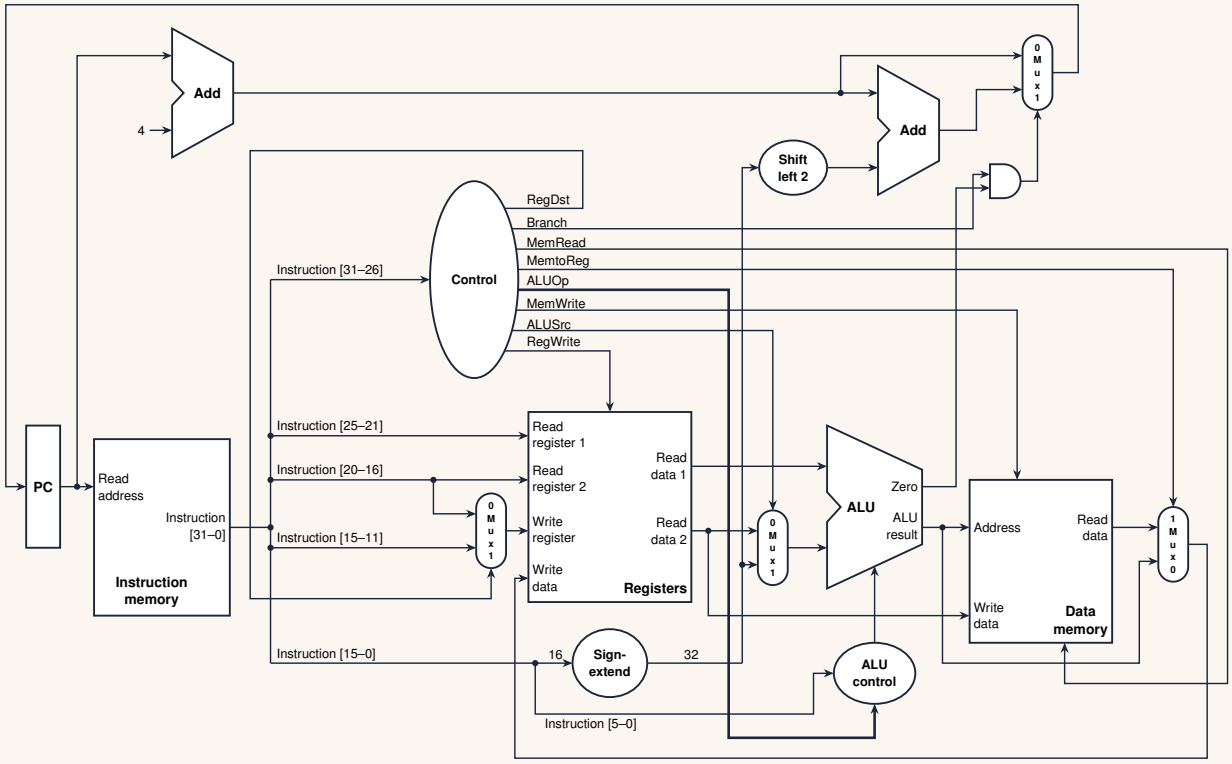


Figure 8a: Single cycle datapath with control signals

SOLUTION

- `add` (R-type): writes rd (RegDst = 1, RegWrite = 1). The second operand is Read data 2 (ALUSrc = 0), the result comes from the ALU (MemtoReg = 0), and there is no memory access and no branch.
- `sw`: the address is rs + offset (ALUSrc = 1). It writes memory (MemWrite = 1) and *no* register (RegWrite = 0), so RegDst and MemtoReg are don't-care.
- `beq`: compares rs and rt by subtraction (ALUSrc = 0, ALUOp = 01) with Branch = 1, and writes neither memory nor a register.

Instr.	RegDst	Branch	MemRead	MemtoReg	MemWrite	ALUSrc	RegWrite
`add`	1	0	0	0	0	0	1
`sw`	X (0)	0	0	X (0)	1	1	0
`beq`	X (0)	1	0	X (0)	0	0	0

X = don't care: the instruction writes no register, so the value is irrelevant. Write 0 if a strict 0/1 answer is required. For completeness, ALUOp = 10 (add), 00 (sw), 01 (beq).

2018–2019 • Q1(a–b) CSE 305 • 29/10/2019 • DP09 **MARKS 30 + 5**

(a) Consider the single cycle datapath shown in Figure 1.A. Update the figure to implement the following MIPS instructions: `bne, j, jr, jal, slt, lui`. Briefly explain your updates. You can make the changes in Figure 1.A and have that page stapled with your script. The question page containing Figure 1.A must then be signed by the invigilator. (30)
(b) Why are single cycle implementations not very efficient? (5)

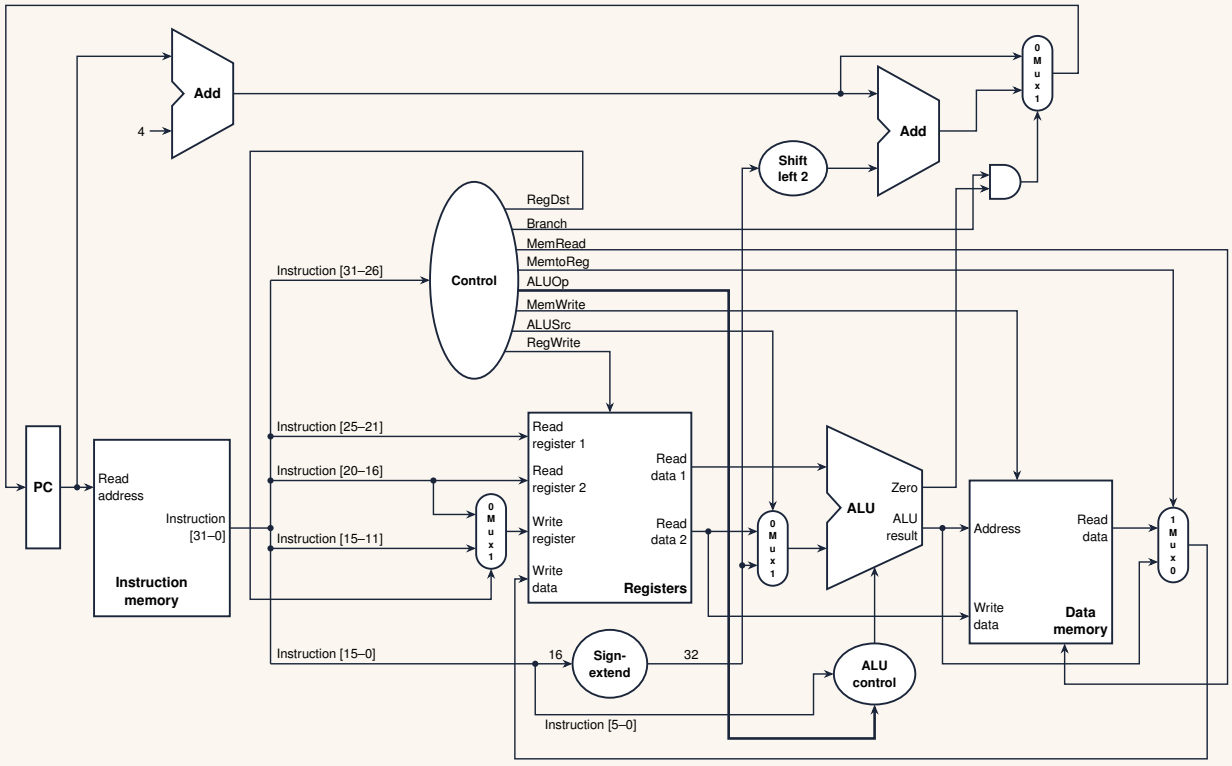


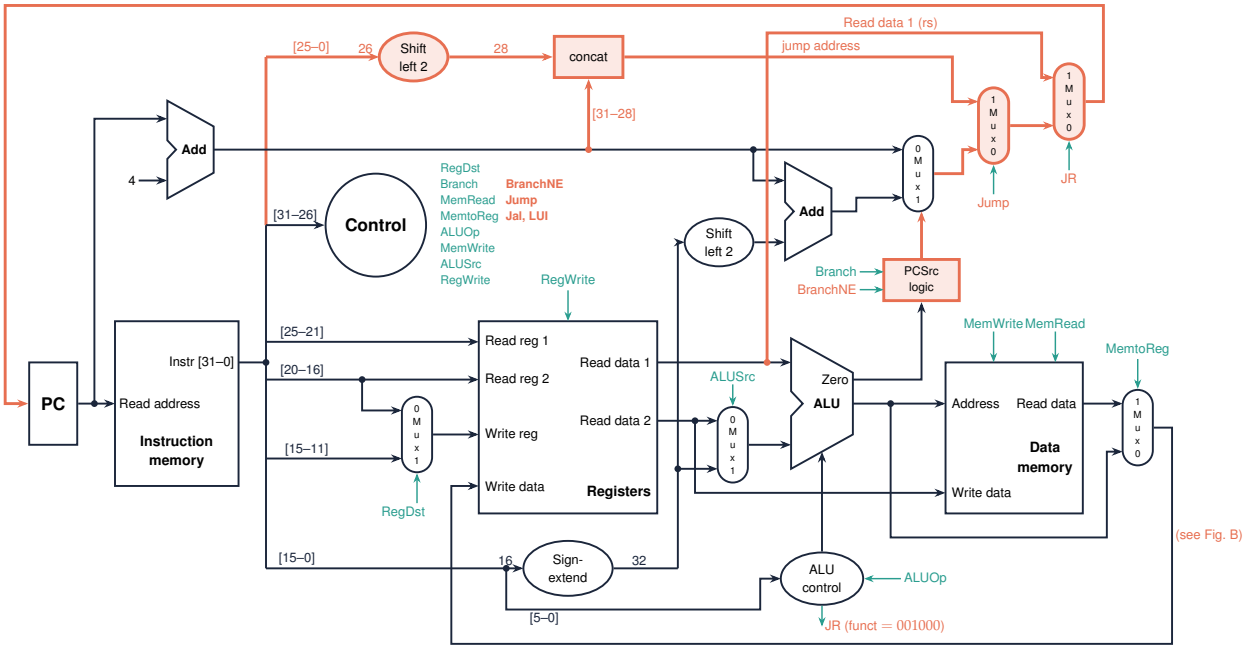
Figure 1.A

SOLUTION

(a) Updates, one per instruction

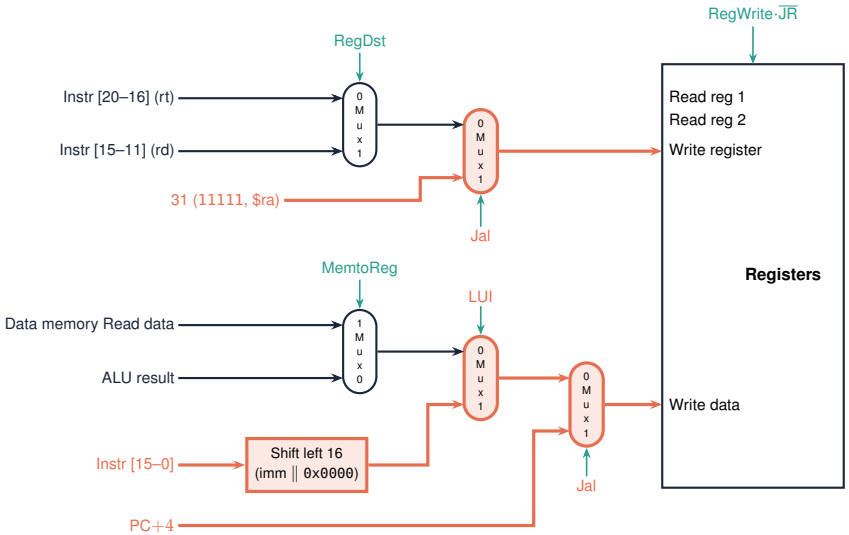
Instr.	Required behaviour ⇒ hardware added (coral in the figures)
`bne`	PC ← BTA if rs ≠ rt. Reuse the ALU subtract and the branch adder/mux. Add control **BranchNE** and gates: PCSrc = Branch·Zero + BranchNE·$\overline{\text{Zero}}$.
`j`	PC ← {PC+4[31:28], Instr[25–0], 00}. Add shift-left-2 on [25–0], a concatenation, a 2:1 *jump mux*, and control **Jump**.
`jr`	PC ← Reg[rs]. Add a wire from Read data 1 to a 2:1 *JR mux* in front of the PC, and control **JR**. `jr` is R-type (op = 0, funct = 8), so JR is decoded from funct, and RegWrite is gated: $\text{RegWrite}_{\text{eff}}$ = RegWrite·$\overline{\text{JR}}$.
`jal`	$ra(31) ← PC+4; PC ← jump address. Reuse the jump path (Jump = 1). Add a 2:1 mux that forces Write register = 31, a 2:1 mux that routes PC+4 to Write data, and control **Jal**.
`slt`	R-type with funct 101010. The ALU ("set on less than", ALU control 0111) and the R-type path already support it, so *no hardware change* is needed.
`lui`	rt ← imm ≪ 16. Add a "shift-left-16" unit (imm ‖ 16 zeros), a 2:1 mux into Write data, and control **LUI**. RegDst = 0 (rt).

Figure A — PC-update side (bne, j, jr, and the jump half of jal)



PCSrc logic = two AND gates and an OR gate: PCSrc = Branch·Zero + BranchNE·$\overline{\text{Zero}}$ (one inverter on Zero). The PC now has four sources (PC+4, BTA, jump address, Reg[rs]), so three 2:1 muxes are needed.

Figure B — write-back side (jal, lui)



Control values

Instr.	RegDst	Jal	ALUSrc	MemtoReg	LUI	RegWrite	MemRead	MemWrite	Branch	BranchNE	Jump	JR	ALUOp
`bne`	X	0	0	X	X	0	0	0	0	1	0	0	01
`j`	X	0	X	X	X	0	0	0	0	0	1	0	XX
`jr`	X	0	X	X	X	0*	0	0	0	0	X	1	XX
`jal`	X	1	X	X	X	1	0	0	0	0	1	0	XX
`slt`	1	0	0	0	0	1	0	0	0	0	0	0	10
`lui`	0	0	X	X	1	1	0	0	0	0	0	0	XX

*The main control gives R-type values for `jr` (op = 0). JR is decoded from funct = 001000, and RegWrite·$\overline{\text{JR}}$ = 0. The new lines (Jal, LUI, BranchNE, Jump, JR) are 0 for all the original instructions.

(b) Why single-cycle is inefficient

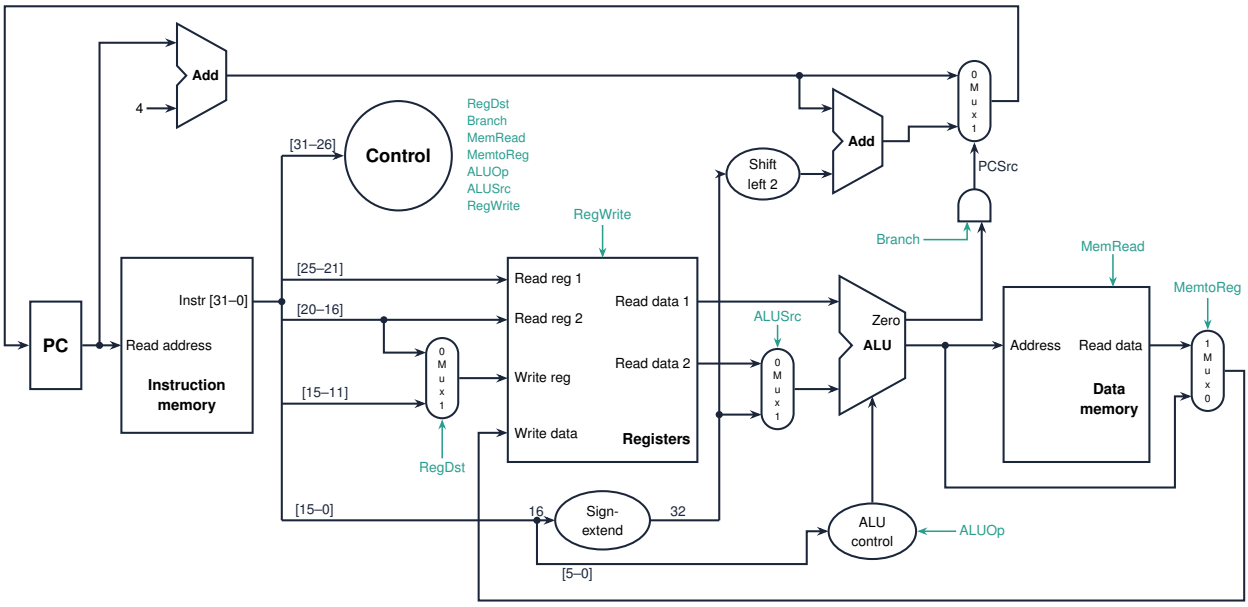
- The clock period is fixed by the *slowest* instruction (`lw`: I-mem → reg → ALU → D-mem → reg). Fast instructions (R-type, `beq`, `j`) waste most of the cycle, which violates "make the common case fast".
- Hardware cannot be reused within an instruction, so it is duplicated: separate instruction and data memories, and extra adders.
- Slow operations (floating-point divide, complex addressing) would stretch the cycle even more. Multicycle and pipelined designs solve this by using a short cycle and overlapping work.

2017–2018 • Q1(b) CSE 305 • 23/09/2018 • DP10 **MARKS 15**

Consider the following set of instructions: `add, lw, beq`
Draw the complete single cycle data path (not pipelined) and control that can execute the above set of instructions.

SOLUTION

The PC addresses the instruction memory, and a dedicated adder forms PC+4. Instruction fields go to the register file, the sign-extender and the Control unit. The ALU does the R-type operation, the `lw` address calculation and the `beq` comparison. The data memory is only *read* (`lw`), so there is no Write-data port or MemWrite. A separate adder computes the branch target. Control signals are teal.



Control (main control + ALU control)

Instr.	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	Branch	ALUOp1	ALUOp0
`add` (R)	1	0	0	1	0	0	1	0
`lw`	0	1	1	1	1	0	0	0
`beq`	X	0	X	0	0	1	0	1

ALU control: ALUOp = 00 → add (0010) for `lw`; 01 → subtract (0110) for `beq`; 10 with funct 100000 → add (0010). Branch logic: PCSrc = Branch · Zero.

- The RegDst mux is needed because `add` writes rd (Instr[15–11]) while `lw` writes rt (Instr[20–16]).
- The MemtoReg mux is needed because `add` writes the ALU result while `lw` writes memory data.

2016–2017 • Q2(a–e) CSE 305 • 02/08/2017 • DP11 **MARKS 5 + 10 + 10 + 5 + 5**

Answer the following questions based on MIPS architecture shown in Figure 1.
(a) Write the functions of the control signals ALUSrc, ALUOp, RegDst, MemWrite and RegWrite. (5)
(b) Explain how the ALU control bits are set depending on the ALUOp control bits and the function codes for the R-type instruction. (10)
(c) Describe the datapaths for R-type instructions. (10)
(d) How is the content of the PC updated for the branch (beq) instruction? (5)
(e) What are the two inputs of the MUX at the input of the ALU? (5)

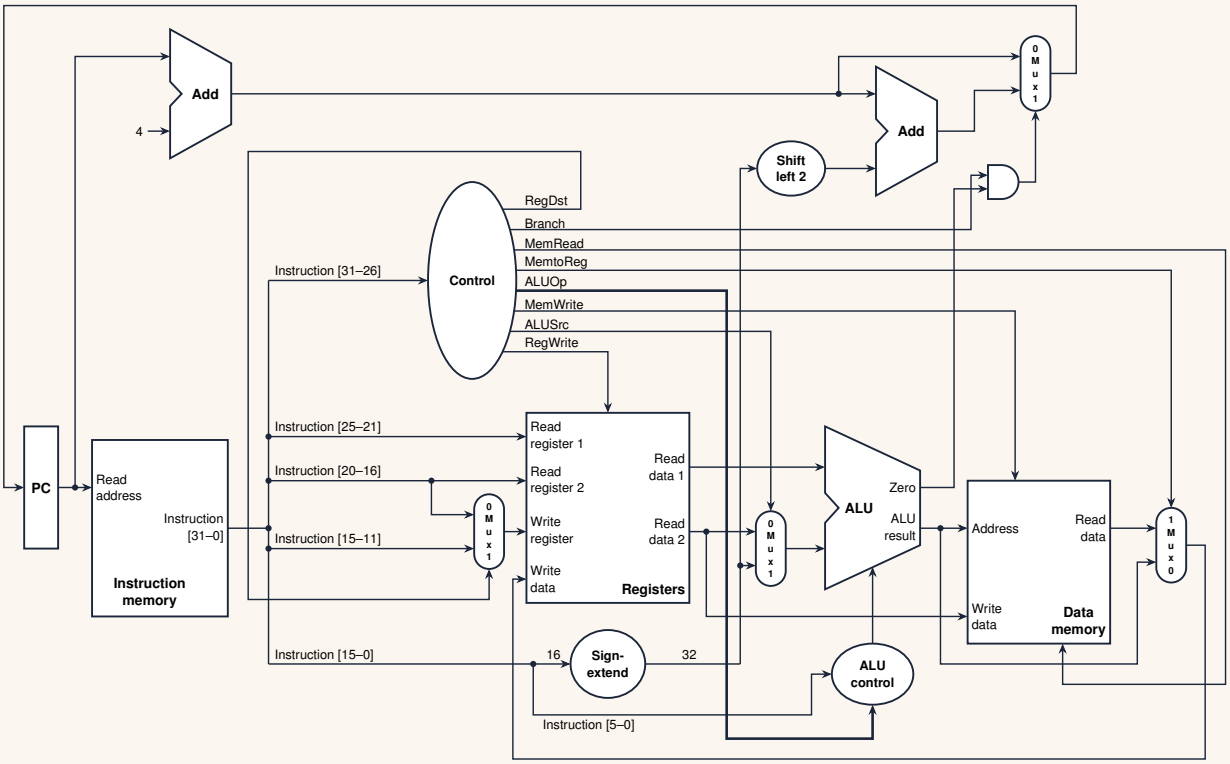


Figure 1: Figure for Question 2.

SOLUTION

(a) Functions of the control signals

Signal	Function
ALUSrc	Selects the ALU's second operand: 0 = Read data 2 (register rt), 1 = sign-extended Instr[15–0].
ALUOp (2 bits)	Tells ALU control what class of operation is needed: 00 = add (lw/sw), 01 = subtract (beq), 10 = decided by the funct field (R-type).
RegDst	Selects the destination register number: 0 = rt (Instr[20–16], for lw), 1 = rd (Instr[15–11], for R-type).
MemWrite	When 1, the data memory at the given address is written with Write data (sw).
RegWrite	When 1, the register named by Write register is written with Write data on the clock edge (R-type, lw).

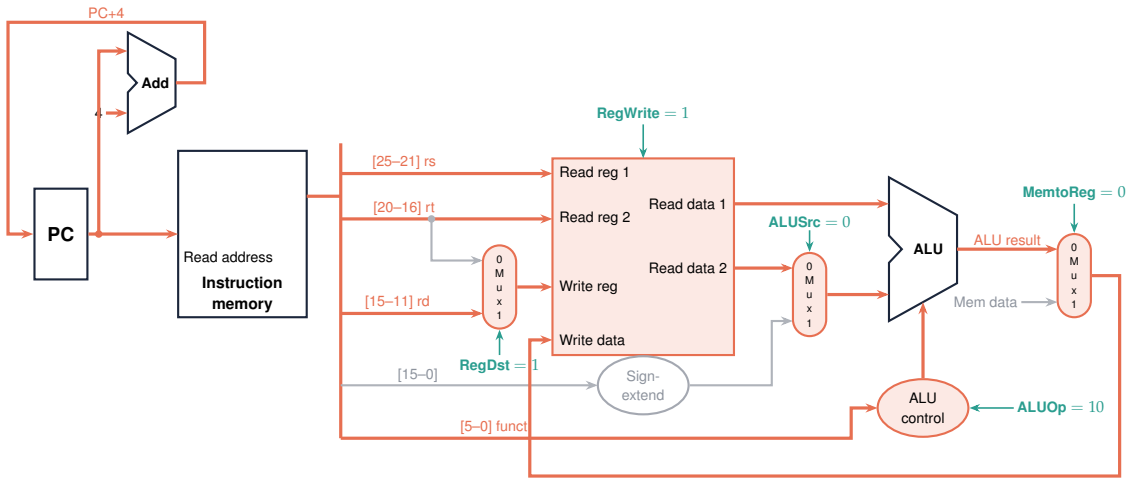
(b) ALU control

ALU control is a small combinational block. Its inputs are ALUOp (from the main control, derived from the opcode) and the 6-bit funct field Instr[5–0]. Its output is the 4-bit ALU control code. Funct is consulted *only* when ALUOp = 10 (R-type).

Opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	0010
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	slt	0111

The 4-bit code is (Ainvert, Bnegate, Operation_1, Operation_0). For R-type it is decoded from funct: e.g. funct = 100010 (sub) → Bnegate = 1, Operation = 10 (adder) → 0110. Two-level decoding keeps the main control small, since it only needs to know the instruction class.

(c) Datapath for R-type (e.g. `add rd, rs, rt`) — active path in coral



Steps for `add rd, rs, rt` in one cycle:

(i) Fetch the instruction at PC, and compute PC+4 (the new PC, since Branch = 0).

(ii) Read rs (Instr[25–21]) and rt (Instr[20–16]) from the register file.

(iii) The ALU operates on Read data 1 and Read data 2 (ALUSrc = 0). The operation is set by ALU control from funct (ALUOp = 10).

(iv) The ALU result goes through the MemtoReg mux (0) to Write data. It is written into rd (RegDst = 1, Instr[15–11]) because RegWrite = 1. MemRead = MemWrite = 0.

(d) PC update for beq

The ALU computes rs − rt, and its *Zero* output is ANDed with the *Branch* control signal:

$$\text{PC} \leftarrow \begin{cases} (\text{PC}+4) + (\text{SignExt}(\text{Instr}[15{:}0]) \ll 2) & \text{if Branch} \cdot \text{Zero} = 1 \ (\text{rs} = \text{rt}) \\ \text{PC}+4 & \text{otherwise.} \end{cases}$$

The target is formed by the separate branch adder from PC+4 and the shifted offset. PCSrc = Branch · Zero selects it in the PC mux.

(e) Inputs of the ALUSrc mux

- Input 0: **Read data 2** of the register file (rt). Used by R-type and `beq`.
- Input 1: the **sign-extended 16-bit immediate** Instr[15–0] (32 bits). Used by `lw`/`sw` (and immediates) as the offset.

2014–2015 • Q8(c) CSE 305 • 26/01/2016 • DP12 **MARKS 10**

A simple MIPS datapath and its respective control signals are illustrated below. Please design the control bits corresponding to the instruction "`beq rs, rt, address`" for this datapath.

Control Signal	0	1
RegDst	Write register address = rt	Write register address = rd
RegWrite	–	Write register
ALUSrc	ALU Second Operand = Read data 2	ALU Second Operand = lower 16-bit of instruction
MemRead	–	Read data from memory
MemWrite	–	Write data into memory
MemtoReg	Register Write Data from ALU	Register Write Data from data memory
PCSrc	PC=PC+4	PC=branch target
ALUOP1	not dependent on function	dependent on function
ALUOP0	addition	subtraction

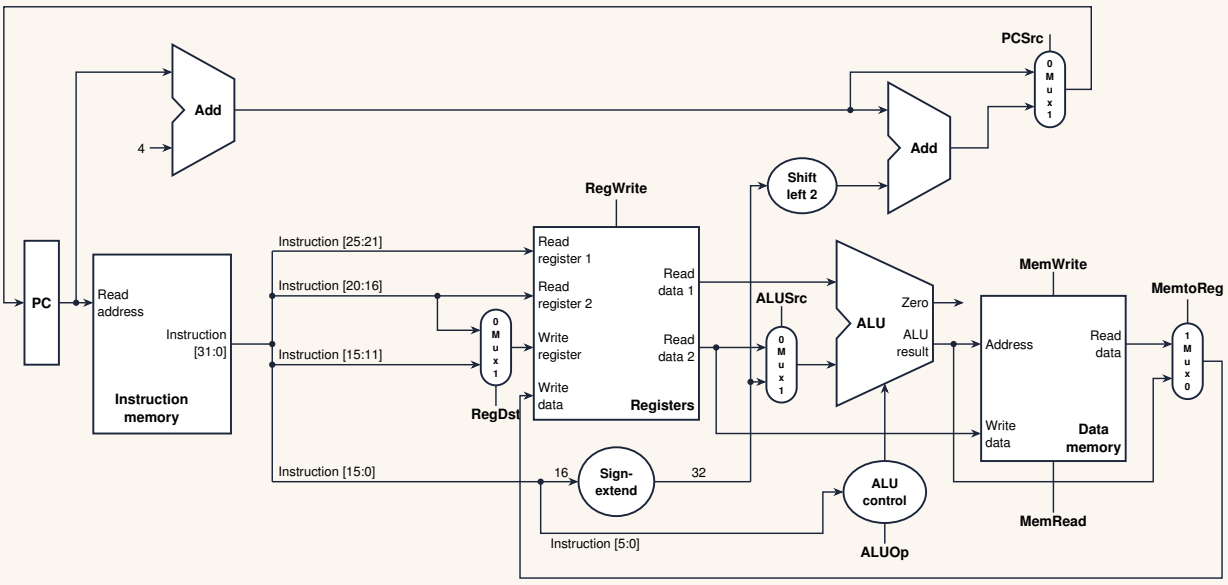


Figure for Question 8(c): a simple MIPS datapath with its control signals

SOLUTION

NOTE In the scan, the "rt"/"rd" entries of the RegDst row and the signal names on the figure are handwritten. They match the standard meanings and are used as such.

	RegDst	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	PCSrc	ALUOP1	ALUOP0
`beq`	X	0	0	0	0	X	Zero	0	1

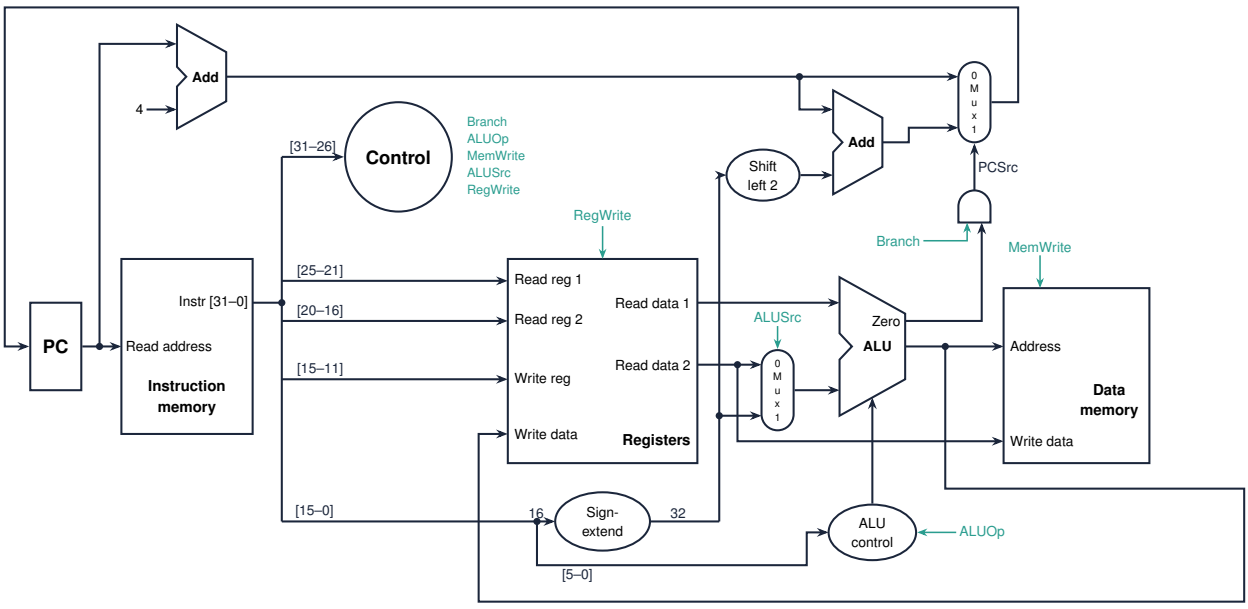
- ALUSrc = 0: the ALU must compare Read data 1 (rs) with Read data 2 (rt), not with the immediate.
- ALUOP1 = 0 (operation does not depend on the funct field) and ALUOP0 = 1 (subtraction), so the ALU computes rs − rt and asserts *Zero* if they are equal.
- PCSrc = 1 only when the branch is taken: PCSrc = Branch · Zero, i.e. PCSrc = Zero for `beq`. If rs = rt, PC ← PC+4+SignExt(address)×4; otherwise PC ← PC+4.
- `beq` writes neither register nor memory: RegWrite = MemRead = MemWrite = 0. RegDst and MemtoReg are don't-cares.

2013–2014 • Q7(c) CSE 305 • 05/07/2015 • DP13 **MARKS 14**

Consider the following set of instruction: `add, sw, beq`
Draw the complete single cycle datapath (not pipelined) and control that can execute the above set of instructions.

SOLUTION

Only `add` writes a register. It always writes the ALU result into rd, so the RegDst and MemtoReg muxes are not needed: Write register = Instr[15–11] and Write data = ALU result. The data memory is only *written* (`sw`: address = rs + offset, data = rt), so it has no Read-data output or MemRead. `beq` uses the ALU subtraction (Zero) and the separate branch adder.



Control (main control + ALU control)

Instr.	ALUSrc	RegWrite	MemWrite	Branch	ALUOp1	ALUOp0
`add` (R)	0	1	0	0	1	0
`sw`	1	0	1	0	0	0
`beq`	0	0	0	1	0	1

ALU control: ALUOp = 00 → add (0010) for the `sw` address; 01 → subtract (0110) for `beq`; 10 with funct 100000 → add. PCSrc = Branch · Zero.

2012–2013 • Q2 CSE 305 • 10/05/2014 • DP14 **MARKS 35**

Consider the single cycle datapath in Figure 2 which is constructed for *add, sub, and, or, lw, sw, beq*. Now, implement the following instructions in the datapath: *bne, j, jr, jal*. You need to clearly explain your implementation and mention the values of the control lines (including any additional control lines you have introduced) for the new instructions.
[You may modify Figure 2 (separate sheet) and attach the update figure with your answer script after having it signed by the respected invigilator.]

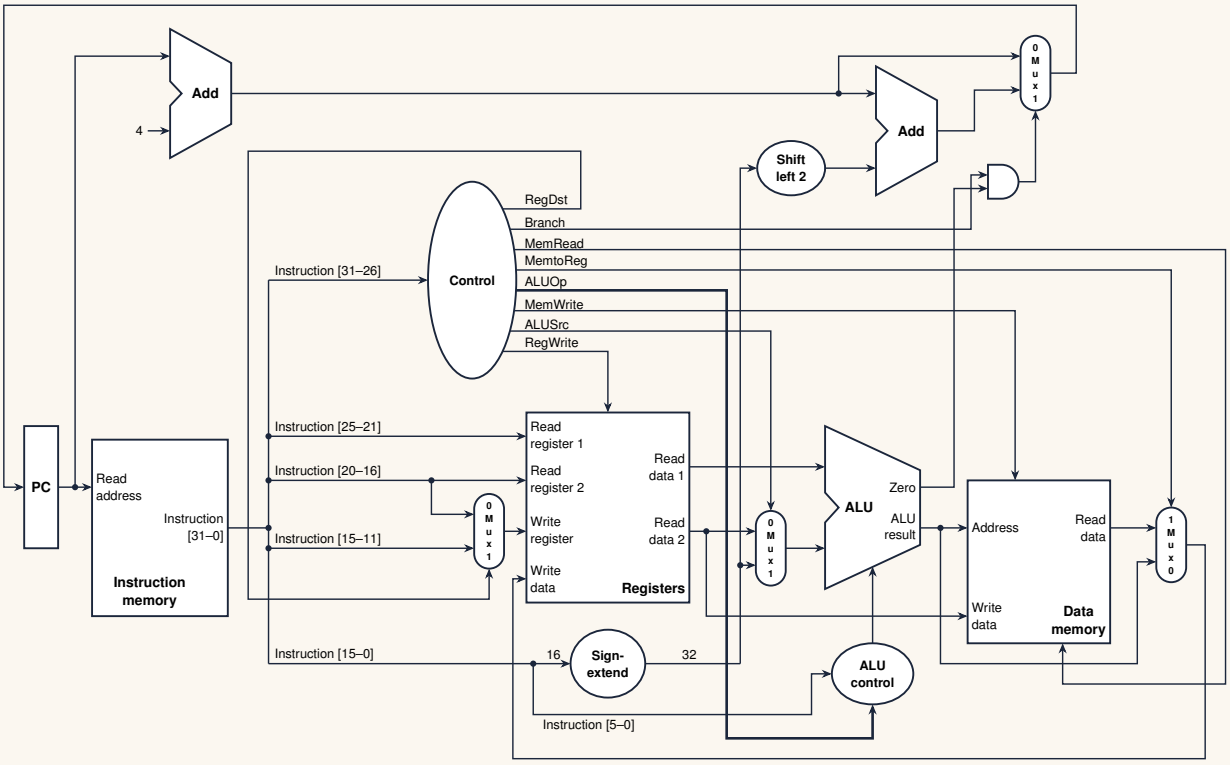


Figure 2

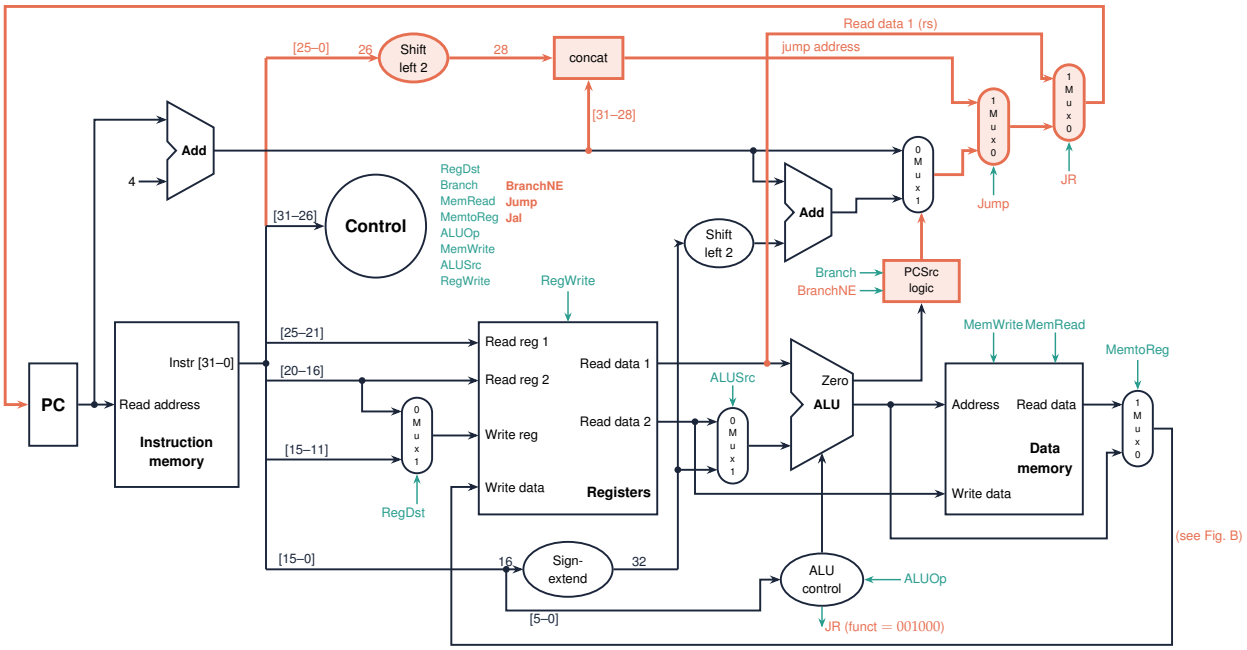
SOLUTION

Implementation

- **bne** (PC ← BTA if rs ≠ rt): reuse the ALU subtraction, the branch adder and the branch mux. Add a control line **BranchNE** and gate logic so that PCSrc = Branch·Zero + BranchNE·$\overline{\text{Zero}}$.
- **j** (PC ← {PC+4[31:28], Instr[25–0], 00}): add a shift-left-2 on Instr[25–0] (26 → 28 bits), concatenate it with PC+4[31–28], and add a 2:1 *jump mux* before the PC with control **Jump**.

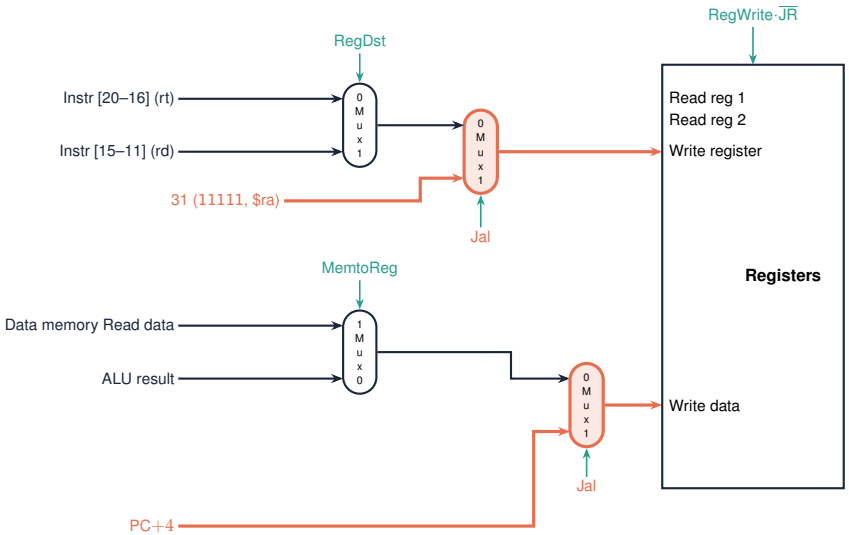
- **jr** (PC ← Reg[rs]): connect Read data 1 to a new 2:1 *JR mux*, the last mux before the PC. `jr` is R-type (op = 0, funct = 001000), so **JR** is decoded from funct next to ALU control. It also gates RegWrite ($\text{RegWrite}_{\text{eff}} = \text{RegWrite}\cdot\overline{\text{JR}}$) so that no register is written.
- **jal** ($ra ← PC+4, PC ← jump address): reuse the jump path (Jump = 1). Add a 2:1 mux after the RegDst mux that forces Write register = 31, and a 2:1 mux after the MemtoReg mux that selects PC+4 as Write data. Both are driven by the new control **Jal**.

Figure A — PC-update side (added hardware in coral)



PCSrc logic: $\text{PCSrc} = \text{Branch}\cdot\text{Zero} + \text{BranchNE}\cdot\overline{\text{Zero}}$ (2 AND, 1 NOT, 1 OR).

Figure B — write-back side for jal



Control-line values for the new instructions

Instr.	RegDst	Jal	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	BranchNE	Jump	JR	ALUOp
`bne`	X	0	0	X	0	0	0	0	1	0	0	01
`j`	X	0	X	X	0	0	0	0	0	1	0	XX
`jr`	X	0	X	X	0*	0	0	0	0	X	1	XX
`jal`	X	1	X	X	1	0	0	0	0	1	0	XX

*Effective value after gating with $\overline{\text{JR}}$ (the main control alone gives R-type values for op $= 0$). For the original instructions `add, sub, and, or, lw, sw, beq`, the new lines Jal, BranchNE, Jump and JR are all 0.

2011–2012 • Q3(a–b) CSE 305 • 27/07/2013 • DP15 **MARKS 15 + 6**

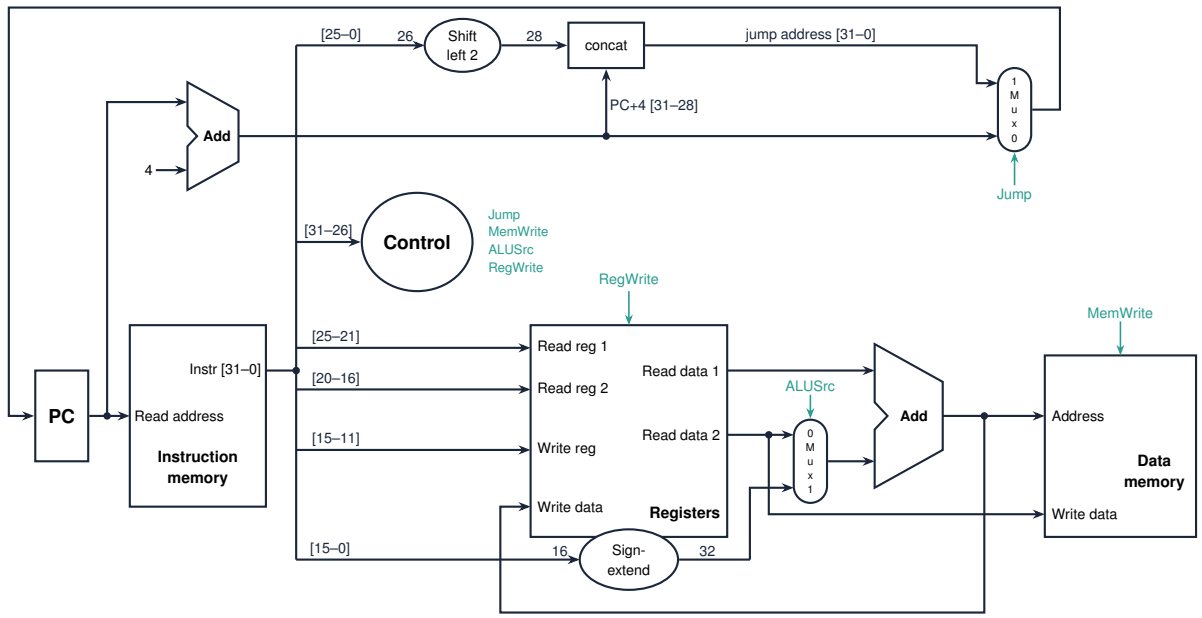
(a) Consider the following set of instructions: `ADD, SW, JUMP`. Draw the single cycle datapath and control that can execute the above set of instructions. Minimize the number of hardware used as much as possible, i.e., there should be no redundant hardware in your design. (Note that explanation is not required.) (15)

(b) Write the purpose of using ALU unit for R-type, load/store, and branch instructions. (6)

SOLUTION

(a) Minimal datapath for ADD, SW, JUMP

`ADD` and `SW` both need only an *addition*, so the ALU is a plain adder and there is no ALU control. Nothing branches, so there is no Zero output, branch adder or shift of the offset. Only `ADD` writes a register (always rd, always the adder output), so there is no RegDst or MemtoReg mux. The memory is write-only. Jump adds a 26-bit field shift, a concatenation with PC+4[31–28], and one mux.



	RegWrite	ALUSrc	MemWrite	Jump
`ADD`	1	0	0	0
`SW`	0	1	1	0
`JUMP`	0	X	0	1

(b) Purpose of the ALU

- **R-type**: performs the actual operation (add, sub, AND, OR, slt) on the two register operands. The operation is chosen by the funct field (ALUOp $= 10$).
- **Load/store**: computes the memory *effective address* $=$ Reg[rs] $+$ sign-extended 16-bit offset (ALU does *add*, ALUOp $= 00$).
- **Branch (beq)**: compares the two registers by *subtracting* them (ALUOp $= 01$). The *Zero* output tells whether they are equal, and that decides PCSrc. The target address itself comes from the separate branch adder.

06

The Processor: Pipelining, Hazards and ILP

CT-3 • Q1 CSE 209 • Class Test • PL01 **MARKS 20**

You are given the following MIPS code:

```
1. lw   $t0, 0($s0)
2. sub  $t1, $t0, $s1
3. beq  $t1, $zero, L1
4. add  $t2, $t1, $s2
5. sw   $t2, 4($s0)
6. j    L2
L1:
7. add  $t2, $t0, $s3
8. sw   $t2, 8($s0)
L2:
9. or   $t4, $t1, $t2
```

Assumptions:

- 5 stage MIPS: IF – ID – EX – MEM – WB
- Code rescheduling is not available
- All stage forwarding available
- Branch resolved in ID
- Taken branch penalty = 1 cycle
- Load-use hazard = 1 stall
- Unconditional jump penalty = 1 cycle

Here, L1 and L2 are labels used for simplification; they are not instructions.

Determine the following:

a) List all data hazards.

b) Which hazards cause stalls?

c) Identify all control hazards.

d) Calculate total cycles if: (i) Branch at instruction 3 is not taken; (ii) Branch at instruction 3 is taken.

SOLUTION

Rules used (course slides, Chapter 4 Part 2)

- **Load-use:** an instruction that needs a `lw` result in EX right after the load waits 1 cycle, then gets it by forwarding from MEM/WB.
- **ALU result into ALU / store data:** forwarded from EX/MEM or MEM/WB, so no stall.
- **Branch resolved in ID:** the comparator works in ID, so its operands are needed one stage *earlier* than an ALU instruction needs them. If a compared register is written by the *immediately preceding ALU* instruction, the branch waits **1 stall** and then takes the value forwarded from EX/MEM into the comparator.
- **Control:** predict not taken. A taken branch or a `j` flushes the one instruction fetched behind it (1 cycle).
- **Cycle count:** total $= 5 + (n - 1) +$ stalls $+$ flushes, where n is the number of instructions actually executed. A stall cell means the instruction repeats its previous stage while a bubble moves ahead.

NOTE The assumptions list only the load-use stall, but they also say "branch resolved in ID" with forwarding. By the slides' rule ("Data hazards for branches"), `beq $t1` right after `sub $t1` costs 1 extra stall, and it is counted below.

a) Data hazards (RAW).

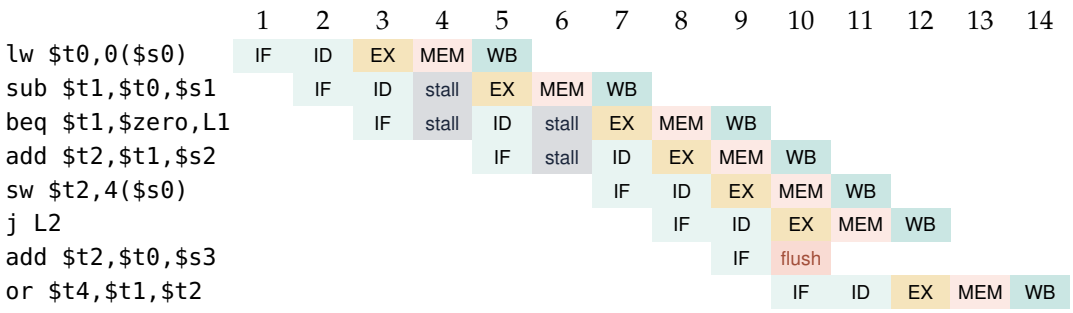
Producer → consumer	Reg	Kind	Resolution
1 `lw` → 2 `sub`	`$t0`	load-use	1 stall + MEM/WB→EX forward
2 `sub` → 3 `beq`	`$t1`	ALU → branch in ID	1 stall + EX/MEM→ID forward
2 `sub` → 4 `add`	`$t1`	ALU → ALU (distance 2)	forwarding, no stall
4 `add` → 5 `sw`	`$t2`	ALU → store data	EX/MEM forward, no stall
7 `add` → 8 `sw`	`$t2`	ALU → store data	EX/MEM forward, no stall
4 or 7 `add` → 9 `or`	`$t2`	ALU → ALU	MEM/WB forward / register file
2 `sub` → 9 `or`; 1 `lw` → 7 `add`	`$t1`, `$t0`	far apart	register file, no action

The `$t2` writes in 4 and 7 lie on different paths. WAR and WAW cases are not hazards in the in-order 5-stage pipeline.

b) Hazards that stall: `lw`→`sub` (`$t0`, 1 stall) and `sub`→`beq` (`$t1`, 1 stall), giving **2 data stalls** on either path.

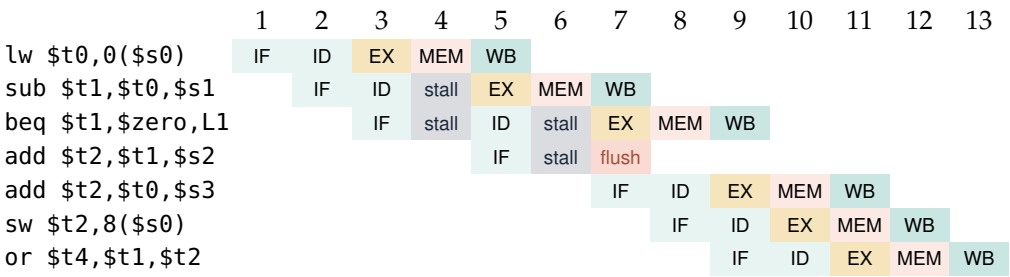
c) Control hazards: instruction 3 `beq` costs 1 cycle only when taken (it flushes instruction 4). Instruction 6 `j` always costs 1 cycle, because the target is known only in ID, so the instruction fetched behind it (7) is flushed. `j` runs only on the not-taken path.

d)(i) Branch not taken: executes 1,2,3,4,5,6,9, so $n = 7$, with 2 stalls + 1 jump flush.



Answer. Not taken: $5 + (7 - 1) + 2 + 1 = \mathbf{14}$ cycles.

d)(ii) Branch taken: executes 1,2,3,7,8,9, so $n = 6$, with 2 stalls + 1 taken-branch flush.



Answer. Taken: $5 + (6 - 1) + 2 + 1 = \mathbf{13}$ cycles.

REPEATED ×5 Same question in 5 papers — stacked below, solved once.

2023–2024 • Q1(a) CSE 209 • 29/04/2026 • PL02 **MARKS 10**

Design a complete forwarding unit in the provided diagram in Figure for Question 1(a) with the necessary equations for the MIPS pipeline, considering double data hazard.
(The figure is not in the available scan. It is assumed to be the same simplified pipelined datapath as in the 2021–2022 paper, drawn below.)

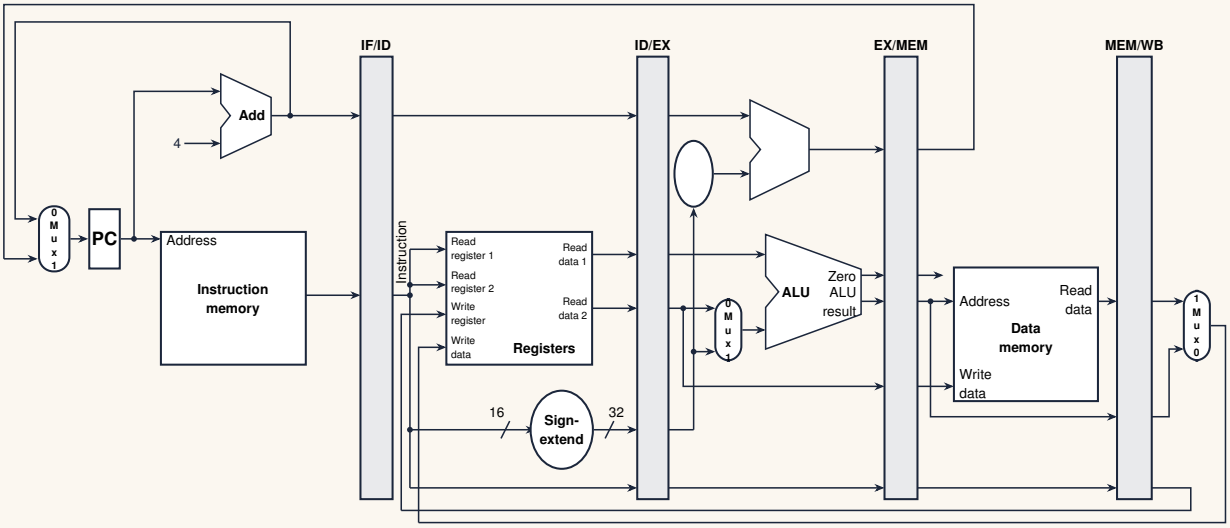


Figure for Question 1(a)

2021–2022 • Q2(a) CSE 305 • 06/04/2023 • PL02 **MARKS 15**

What do you mean by forwarding for data hazard? Design a complete forwarding unit in the provided diagram with necessary equations for the MIPS pipeline considering double data hazard.

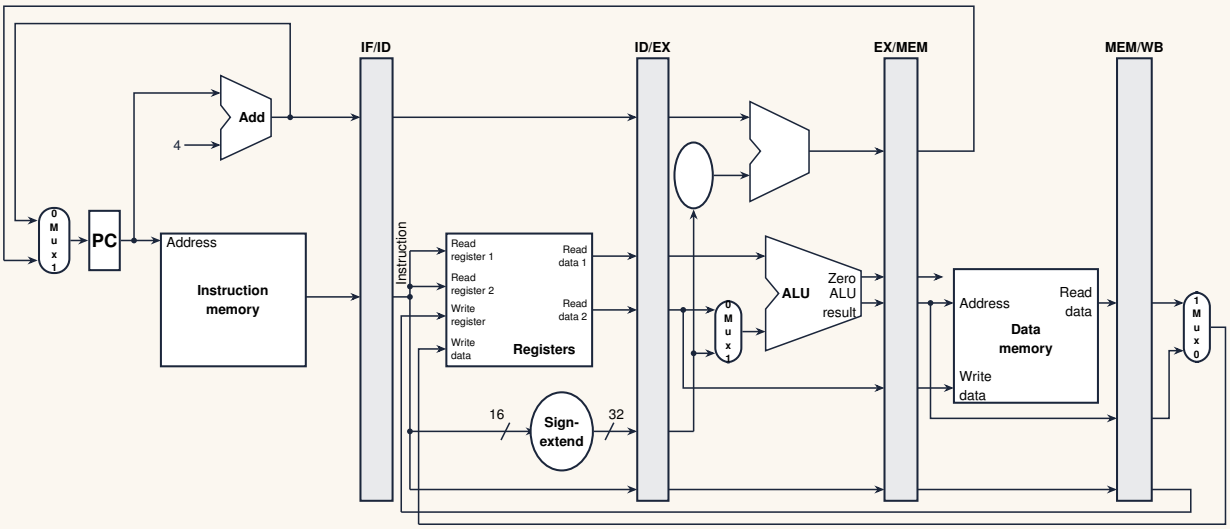


Figure for Question 2 (a)

2017–2018 • Q2(a–b) CSE 305 • 23/09/2018 • PL02 **MARKS 10 + 15**

(a) Briefly explain double data hazard with example. How MIPS detect hazard due to the use of load instruction and how MIPS stall the pipeline? (10)
(b) Design a complete forwarding unit with necessary equations for MIPS considering double data hazard. Draw the complete forwarding unit using the provided figure below. (15)

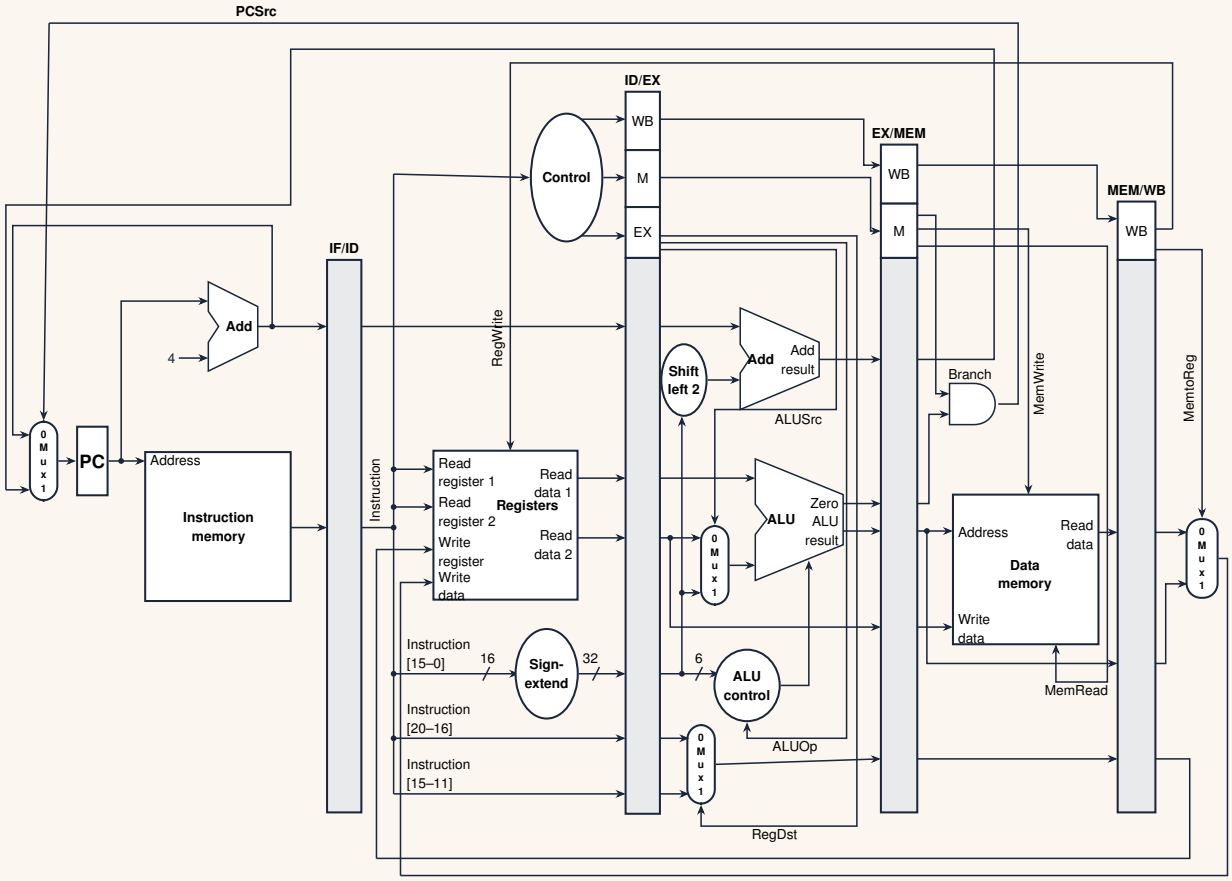


Figure for Question 2(b)

2015–2016 • Q3(a) CSE 305 • 18/01/2017 • PL02 **MARKS 10**

What do you mean by double data hazard? Write down the equations to detect forwarding in MIPS considering double data hazard? How MIPS detect hazard due to the use of load instruction?

2012–2013 • Q4(a) CSE 305 • 10/05/2014 • PL02 **MARKS 17**

Discuss how you can detect data hazards and solve those with the help of a forwarding unit.

SOLUTION

Forwarding (bypassing)

A data hazard occurs when an instruction needs a register value that an earlier instruction has not yet written back. With *forwarding*, the result is used as soon as it has been computed: it is taken from the EX/MEM or MEM/WB pipeline register and routed through extra paths and

muxes straight to the ALU inputs, instead of waiting for WB. Example: in `sub $2,$1,$3; and $12,$2,$5`, the `sub` result sits in EX/MEM when `and` is in EX, so it is forwarded and no stall is needed.

Detecting the need to forward

Register numbers travel down the pipeline. The ALU operands in EX are named by `ID/EX.RegisterRs` and `ID/EX.RegisterRt`. A hazard exists when

1a.	EX/MEM.RegisterRd = ID/EX.RegisterRs	2a.	MEM/WB.RegisterRd = ID/EX.RegisterRs
1b.	EX/MEM.RegisterRd = ID/EX.RegisterRt	2b.	MEM/WB.RegisterRd = ID/EX.RegisterRt

Cases 1a/1b forward from EX/MEM, and cases 2a/2b forward from MEM/WB. A forward happens only if the earlier instruction really writes a register (`RegWrite`) and its destination is not `$zero`. Here "RegisterRd" is the destination after the RegDst mux, so it is `rt` for `lw` and `addi`.

Double data hazard

```
add $1,$1,$2      # $1 = 10+30 = 40
add $1,$1,$3      # $1 = 40+20 = 60
add $1,$1,$4      # needs 60, result 64
```

(The values follow the slides: initially `$1`=10, `$2`=30, `$3`=20, `$4`=4.) When the third `add` is in EX, EX/MEM holds 60 from the 2nd `add` and MEM/WB holds 40 from the 1st. *Both* conditions 1a and 2a are true. The most recent value (60) must be used. If the MEM condition were allowed to override, 40 would be forwarded, which is wrong. The fix is to **forward from MEM/WB only if the EX hazard condition is not true**.

Complete forwarding equations (with the revised MEM hazard)

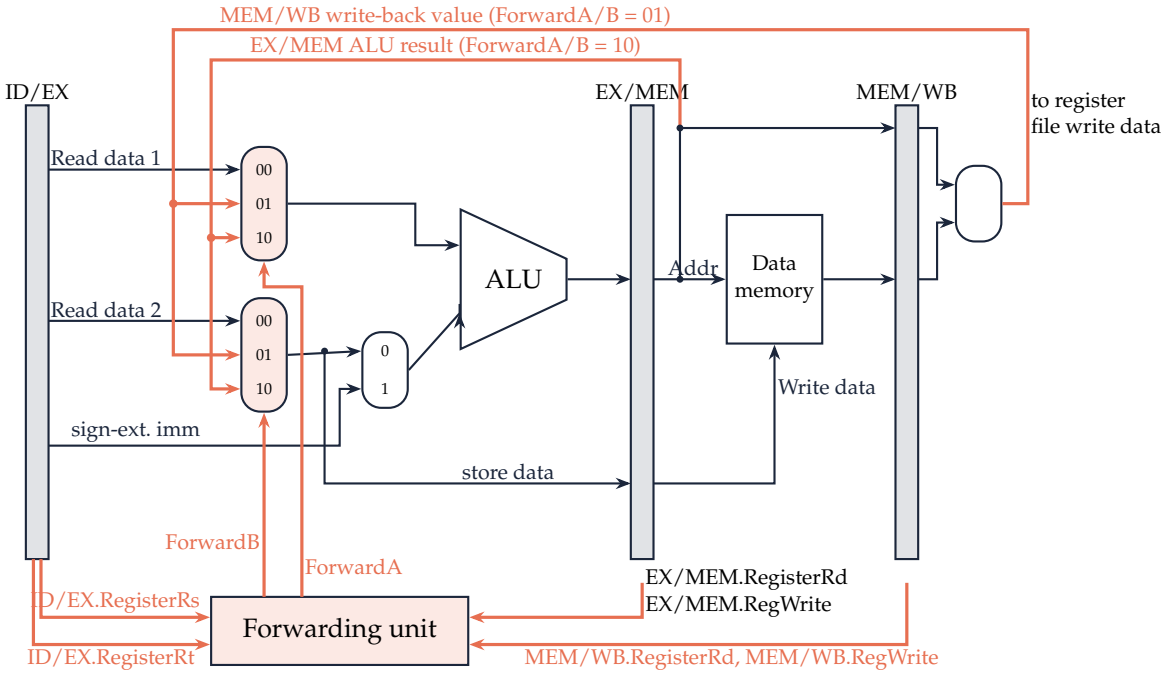
EX hazard

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
   and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
   and (EX/MEM.RegisterRd = ID/EX.RegisterRt)) ForwardB = 10
```

MEM hazard (revised for double data hazard)

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
   and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
   and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01
otherwise ForwardA = 00, ForwardB = 00
```

Forwarding unit added to the datapath (new hardware in coral)



Mux control	Source	Meaning
ForwardA/B = 00	ID/EX	operand from the register file (no hazard)
ForwardA/B = 10	EX/MEM	forwarded result of the instruction 1 ahead (EX hazard)
ForwardA/B = 01	MEM/WB	ALU or load result of the instruction 2 ahead (MEM hazard)

Forwarding into ID is not needed. The register file is written in the first half of a cycle and read in the second half, so an instruction 3 ahead is handled by the register file itself.

Hazard that forwarding cannot remove: load-use (2017–18 (a), 2015–16)

A `lw` has its data only at the end of MEM, but the very next instruction needs it at the start of its EX, one cycle too early ("can't forward backward in time"). The hazard detection unit in ID checks

```
ID/EX.MemRead and ((ID/EX.RegisterRt = IF/ID.RegisterRs)
                or (ID/EX.RegisterRt = IF/ID.RegisterRt))
```

How the pipeline is stalled: (1) force the control fields written into ID/EX to 0, so a bubble (nop) goes through EX, MEM and WB; (2) set PCWrite = 0 and IF/IDWrite = 0, so the using instruction is decoded again and the following instruction is fetched again. After the 1-cycle stall the loaded value is in MEM/WB and is forwarded to EX (ForwardA/B = 01).

	1	2	3	4	5	6	7	8
`lw $2,20($1)`	IF	ID	EX	MEM	WB			
`and $4,$2,$5`		IF	ID	stall	EX	MEM	WB	
`or $8,$2,$6`			IF	stall	ID	EX	MEM	WB

Answer. Forwarding unit: ForwardA/ForwardB select 10 on an EX hazard and 01 on a MEM hazard only when no EX hazard exists for the same operand. Both cases need RegWrite = 1 and Rd $\neq 0$. A load-use hazard is detected in ID and costs one stall (bubble + PC/IF–ID hold).

2023–2024 • Q1(b) CSE 209 • 29/04/2026 • PL03 **MARKS 15**

You are given the following MIPS code:

```
      lw   $t0, 0($s0)
      add  $t1, $t0, $s1
      beq  $t1, $zero, L1
      add  $t2, $t1, $s2
      bne  $t2, $zero, L2
      sub  $t3, $t2, $t1
L1:   and  $t4, $t1, $t2
L2:   or   $t5, $t3, $t4
```

You can assume the following:

- 5 stage MIPS: IF – ID – EX – MEM – WB
- Code rescheduling is not available
- All stage forwarding available
- Branch resolved in ID
- Taken branch penalty = 1 cycle
- Load-use hazard = 1 stall

Now, identify all the data and control hazards and evaluate the following:

(i) Total stall cycles

(ii) Total cycles if both branches are not taken

(iii) Total cycles if the first branch is taken

(iv) Total cycles if the first branch is not taken, but the second branch is taken

SOLUTION

Rules used (course slides)

Load-use costs 1 stall, then MEM/WB→EX forwarding. ALU→ALU dependences are forwarded with no stall. The branch is resolved in ID, so if it compares a register written by the *immediately preceding ALU* instruction it waits **1 stall**, then gets the value forwarded from EX/MEM into the comparator (slides: "Data hazards for branches"). Predict not taken: a taken branch flushes 1 instruction. Total $= 5 + (n - 1) +$ stalls $+$ flushes, with n = instructions executed. A stall cell means the instruction repeats its previous stage.

> **NOTE** The assumptions list only the load-use stall. With the branch resolved in ID, the slides' rule adds 1 stall for each branch that follows the ALU instruction producing its operand (`add $t1`→`beq`, `add $t2`→`bne`). These stalls are counted below.

Data hazards: `lw`→`add $t1` (`$t0`, load-use, **1 stall**); `add $t1`→`beq` (`$t1`, ALU→branch in ID, **1 stall**); `add $t2`→`bne` (`$t2`, ALU→branch in ID, **1 stall**). These pairs are covered by forwarding with no stall: `$t1` into `add $t2`, `sub`, `and`; `$t2` into `sub`, `and`; `$t3` from `sub` into `or` (MEM/WB); `$t4` from `and` into `or` (EX/MEM).

Control hazards: `beq` and `bne`, each costing a 1-cycle flush when taken.

(i) Total stall cycles.

Path	data stalls	taken-branch flushes	total stall cycles
both not taken	3	0	3
first taken (bne never executed)	2	1	3
first not taken, second taken	3	1	4

If every instruction executes, there are 3 data-hazard stalls.

(ii) Both not taken: $n = 8$, 3 stalls.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB										
`add $t1,$t0,$s1`		IF	ID	stall	EX	MEM	WB								
`beq $t1,$zero,L1`			IF	stall	ID	stall	EX	MEM	WB						
`add $t2,$t1,$s2`					IF	stall	ID	EX	MEM	WB					
`bne $t2,$zero,L2`							IF	ID	stall	EX	MEM	WB			
`sub $t3,$t2,$t1`								IF	stall	ID	EX	MEM	WB		
`and $t4,$t1,$t2`										IF	ID	EX	MEM	WB	
`or $t5,$t3,$t4`											IF	ID	EX	MEM	WB

Answer. $5 + (8 - 1) + 3 = \mathbf{15}$ cycles.

(iii) First branch taken: executes `lw, add, beq, and, or`, so $n = 5$, with 2 stalls + 1 flush.

	1	2	3	4	5	6	7	8	9	10	11	12
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB							
`add $t1,$t0,$s1`		IF	ID	stall	EX	MEM	WB					
`beq $t1,$zero,L1`			IF	stall	ID	stall	EX	MEM	WB			
`add $t2,$t1,$s2`					IF	stall	flush					
`and $t4,$t1,$t2`							IF	ID	EX	MEM	WB	
`or $t5,$t3,$t4`								IF	ID	EX	MEM	WB

Answer. $5 + (5 - 1) + 2 + 1 = \mathbf{12}$ cycles.

(iv) First not taken, second taken: executes `lw, add, beq, add, bne, or`, so $n = 6$, with 3 stalls + 1 flush.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB									
`add $t1,$t0,$s1`		IF	ID	stall	EX	MEM	WB							
`beq $t1,$zero,L1`			IF	stall	ID	stall	EX	MEM	WB					
`add $t2,$t1,$s2`					IF	stall	ID	EX	MEM	WB				
`bne $t2,$zero,L2`							IF	ID	stall	EX	MEM	WB		
`sub $t3,$t2,$t1`								IF	stall	flush				
`or $t5,$t3,$t4`										IF	ID	EX	MEM	WB

Answer. $5 + (6 - 1) + 3 + 1 = \mathbf{14}$ cycles.

REPEATED ×6 Same question in 6 papers — stacked below, solved once.

2023–2024 • Q1(c) CSE 209 • 29/04/2026 • PL04 **MARKS 10**

Consider the following code:

```
outer:
```

```
        // some code
inner:
        // some code
beq $t0, $zero, inner
// some code
beq $t1, $zero, outer
```

Explain the problem with the above code if we use 1 bit branch predictor? How can you solve that?
"Even with a perfect branch predictor, 1-cycle penalty for a taken branch" – why does this penalty occur, and how can it be eliminated?

2021–2022 • Q1(d) CSE 305 • 06/04/2023 • PL04 **MARKS 5**

Consider the following code:

```
outer:
        // some code
inner:
        // some code
        beq $t0, $zero, inner
        // some code
        beq $t1, $zero, outer
```

What is the problem with the above code if we use 1 bit branch predictor? How can you solve that?

2017–2018 • Q1(d) CSE 305 • 23/09/2018 • PL04 **MARKS 5**

"Even with a perfect branch predictor, 1-cycle penalty for a taken branch". Why is that? How to solve this?

2017–2018 • Q2(d) CSE 305 • 23/09/2018 • PL04 **MARKS 5**

Consider the following code:

```
outer:
        // some code
inner:
        // some code
        beq $t0, $zero, inner
        // some code
        beq $t1, $zero, outer
```

What is the problem with the above code if we use 1 bit branch predictor? How can you solve that?

2015–2016 • Q2(e) CSE 305 • 18/01/2017 • PL04 **MARKS 5**

"Even with a perfect branch predictor, 1-cycle penalty for a taken branch". Why is that? How to solve this?

2013–2014 • Q8(d) CSE 305 • 05/07/2015 • PL04 **MARKS 4**

Consider the following code:

```
outer:
        // some code
inner:
        // some code
beq $t0, $zero, inner
// some code
beq $t1, $zero, outer
```

What is the problem with the above code if we use 1 bit branch predictor? How you can solve that? Explain

SOLUTION

Part 1 — the 1-bit predictor problem (2023–24, 2021–22, 2017–18 Q2(d), 2013–14)

A 1-bit predictor (branch history table) stores only the *last* outcome of each branch and predicts the same outcome next time. On a misprediction it flushes and flips the bit.

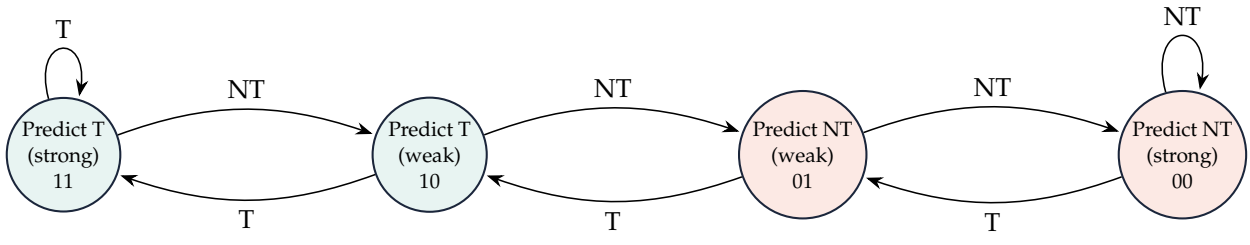
The inner-loop branch `beq $t0,$zero,inner` is taken on every iteration except the last. Say the inner loop runs N times per outer iteration: the branch goes T,T,…,T,NT.

- **Last inner iteration:** the bit says T (from the previous iteration), but the branch falls through. That is misprediction 1, and the bit becomes NT.
- The outer loop then branches back and the inner loop starts again. **First inner iteration:** the bit still says NT, but the branch is taken. That is misprediction 2.

So the inner-loop branch is **mispredicted twice per pass** even though it is "almost always taken". Accuracy is $(N-2)/N$; for example, $N = 10$ gives only 80%.

Inner iteration	1	2	…	$N-1$	N	(next pass) 1
Actual	T	T	…	T	NT	T
1-bit prediction	**NT**	T	…	T	**T**	**NT**
Correct?	×	✓	…	✓	×	×

Solution: 2-bit predictor. The prediction changes only after *two successive* mispredictions. The single NT at loop exit moves the state from "strongly taken" to "weakly taken", and the prediction is still T. So the first iteration of the next pass is predicted correctly: only **1 misprediction per pass**, giving accuracy $(N-1)/N$ (90% for $N = 10$).



The outer branch behaves the same way, but it runs far less often. (Further improvements are correlating / tournament predictors.)

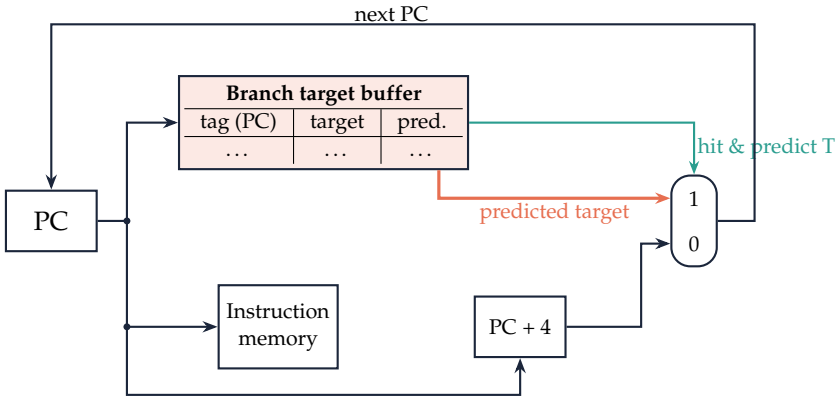
Part 2 — why a perfect predictor still costs 1 cycle on a taken branch (2023–24, 2017–18 Q1(d), 2015–16)

A predictor only answers *whether* the branch is taken. To fetch the target, the processor also needs *where* it is: PC + 4 + (sign-ext(imm) ≪ 2). In the MIPS pipeline that target is computed by the

branch adder in the **ID** stage, once the instruction has been fetched and decoded. During the cycle in which the branch itself is fetched (IF), nobody yet knows it is a branch or what its offset is. So the next fetch can only go to PC + 4, and that slot is lost when the branch is taken:

	1	2	3	4	5	6
`beq` (predicted T, correct)	IF	ID	EX	MEM	WB	
`PC+4` instruction		IF	flush			
target instruction			IF	ID	EX	MEM

Elimination — branch target buffer (BTB): a small cache of target addresses, indexed by the PC *at fetch time*. In IF, the PC is looked up in the BTB in parallel with the instruction-memory access. On a hit for a branch that is predicted taken, the stored target becomes the next PC immediately, so the target is fetched in cycle 2 and the penalty is 0. The real target and outcome are still checked in ID; on a mismatch the pipeline flushes and the BTB entry is updated. (A compiler-filled *delayed branch* slot is the other way to hide this cycle.)



Answer. 1-bit predictor: the inner-loop branch is mispredicted twice per outer iteration (at loop exit and at loop re-entry). A 2-bit (hysteresis) predictor cuts this to one. Perfect predictor: the target address is known only after decode (ID), so one fetch slot is lost on every taken branch. A branch target buffer looked up in IF supplies the target at once and removes that penalty.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q4(b) CSE 209 • 29/04/2026 • PL05 **MARKS 15**

Consider the following program:

```
Loop: lw   $t0, 0($s1)
      addu $t0, $t0, $s2
      sw   $t0, 0($s1)
      addi $s1, $s1, -4
      bne  $s1, $zero, Loop
```

Schedule the above program for a dual-issue MIPS with the following configuration: Two-issue packets, one ALU/branch instruction, one load/store instruction, ALU/branch then load/store. What will be the IPC for this scheduling? Which technique can further increase the IPC, and How?

2021–2022 • Q2(c) CSE 305 • 06/04/2023 • PL05 **MARKS 10**

Consider the following program:

```
Loop: lw   $t0, 0($s1)
      addu $t0, $t0, $s2
      sw   $t0, 0($s1)
      addi $s1, $s1, -4
      bne  $s1, $zero, Loop
```

Schedule the above program for a dual-issue MIPS with the following configuration: Two-issue packets, one ALU/branch instruction, one load/store instruction, ALU/branch then load/store. What will be the Instructions per Cycle (IPC) for this scheduling? Which technique can increase the IPC even more?

SOLUTION

Scheduling rules of the static dual-issue MIPS (slides)

- A packet = one ALU/branch slot + one load/store slot; an empty slot holds a `nop`.
- No true dependence inside a packet: an ALU result can be used by a load/store only in a *later* packet (EX data hazard). A register may be read and overwritten in the same packet (the read happens in ID, the write in WB).
- Load-use latency is still one cycle: a load in cycle n feeds an ALU instruction in cycle $n+2$.
- If `addi` moves above a load/store that uses the same base register, adjust the offset (e.g. `sw $t0,0($s1)` becomes `sw $t0,4($s1)`).

Schedule (one ALU/branch + one load/store slot per packet)

	ALU/branch	Load/store	cycle
Loop:	`nop`	`lw $t0, 0($s1)`	1
	`addi $s1, $s1, -4`	`nop`	2
	`addu $t0, $t0, $s2`	`nop`	3
	`bne $s1, $zero, Loop`	`sw $t0, 4($s1)`	4

`addu` has to wait for the load (cycle 3). `addi` is moved up into the free cycle 2, so the store now uses offset 4. The store and the branch share the last packet.

Answer. 5 instructions in 4 cycles: IPC $= 5/4 =$ **1.25** (the peak is 2).

How to increase the IPC: loop unrolling with register renaming

Copy the loop body 4 times, so there is one `addi`/`bne` pair for every 4 elements (less loop-control overhead). Give each copy its own temporary register (`$t0`–`$t3`). This is register renaming: it removes the name (anti-)dependences on `$t0`, so the copies can be interleaved freely. (Assume the element count is a multiple of 4.)

	ALU/branch	Load/store	cycle
Loop:	`addi $s1, $s1, -16`	`lw $t0, 0($s1)`	1
	`nop`	`lw $t1, 12($s1)`	2
	`addu $t0, $t0, $s2`	`lw $t2, 8($s1)`	3
	`addu $t1, $t1, $s2`	`lw $t3, 4($s1)`	4
	`addu $t2, $t2, $s2`	`sw $t0, 16($s1)`	5
	`addu $t3, $t3, $s2`	`sw $t1, 12($s1)`	6
	`nop`	`sw $t2, 8($s1)`	7
	`bne $s1, $zero, Loop`	`sw $t3, 4($s1)`	8

Answer. Unrolled: 14 instructions in 8 cycles, IPC $= 14/8 = \mathbf{1.75}$. This is closer to 2, but it costs more registers and larger code. Dynamic multiple issue (out-of-order superscalar) can also raise IPC.

NOTE The slides print the 6th packet as `addu $t3, $t4, $s2`. That is a typo; it should be `addu $t3, $t3, $s2`, as used above.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2022–2023 • Q1(c) CSE 209 • 13/01/2025 • PL06 **MARKS 10**

Consider the following code

$$A = B + C; \qquad D = A + E;$$

All the above are memory instructions (i.e. the values of the variable are stored in a memory location). The base address is in register `$t0` and the offsets are 0, 4, 8, 12, 16 for A, B, C, D, and E, respectively. Write the equivalent MIPS assembly code to determine the number of cycles that are required to execute the code.
Can you reduce the number of cycles computed above using code scheduling? If yes, then rearrange the MIPS assembly code and recount the number of cycles required to execute the rearranged code.

2017–2018 • Q1(a) CSE 305 • 23/09/2018 • PL06 **MARKS 10**

Consider the following C code:

$$A = B + C;\ D = A + E;$$

All the above are memory instructions. The base address is in register t0 and the offsets are A (0), B (4), C (8), D (12), E (16). How many cycles are required to execute the above? Can you reduce the number of cycles using code scheduling? If yes, what will be resulting number of cycles? Show each step.

2015–2016 • Q2(d) CSE 305 • 18/01/2017 • PL06 **MARKS 10**

Consider the following C code

$$A = B + C; \quad D = A + E;$$

All the above are memory instructions. The base address is in register t0 and the offset are A(0), B(4), C(8), D(12), E(16). How many cycles are required to execute the above? Can you reduce the number of cycles using code scheduling? If yes, what will be the resulting number of cycles? Show each step.

SOLUTION

Method

The pipeline is the 5-stage MIPS pipeline with full forwarding (as in the slides). ALU→ALU and ALU→`sw` values are forwarded with no stall. The only stall is the **load-use hazard**: an instruction that uses a `lw` result right after the load waits **1 cycle**, then gets it from MEM/WB. Total

$= 5 + (n - 1) +$ stalls. stall = the instruction repeats its previous stage.

Step 1: straightforward code

```
lw   $t1, 4($t0)     # B
lw   $t2, 8($t0)     # C
add  $t3, $t1, $t2   # A=B+C  (stall)
sw   $t3, 0($t0)     # store A
lw   $t4, 16($t0)    # E
add  $t5, $t3, $t4   # D=A+E  (stall)
sw   $t5, 12($t0)    # store D
```

Step 2: scheduled code

```
lw   $t1, 4($t0)     # B
lw   $t2, 8($t0)     # C
lw   $t4, 16($t0)    # E  (moved up)
add  $t3, $t1, $t2   # A
sw   $t3, 0($t0)
add  $t5, $t3, $t4   # D
sw   $t5, 12($t0)
```

Step 1 count. `add $t3` uses `$t2` right after its load, which costs 1 stall. `add $t5` uses `$t4` right after its load, another stall. Both `sw`s get their data by forwarding, and A is kept in `$t3`, so it is not re-loaded.

	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw $t1,4($t0)`	IF	ID	EX	MEM	WB								
`lw $t2,8($t0)`		IF	ID	EX	MEM	WB							
`add $t3,$t1,$t2`			IF	ID	stall	EX	MEM	WB					
`sw $t3,0($t0)`				IF	stall	ID	EX	MEM	WB				
`lw $t4,16($t0)`						IF	ID	EX	MEM	WB			
`add $t5,$t3,$t4`							IF	ID	stall	EX	MEM	WB	
`sw $t5,12($t0)`								IF	stall	ID	EX	MEM	WB

Total $= 5 + (7 - 1) + 2 = 13$ cycles.

Step 2 count. The load of E depends on nothing above it, so it moves into the slot after `lw $t2`. Now `add $t3` is 2 instructions after `lw $t2` (MEM/WB forward), and `add $t5` is 3 after `lw $t4` (register file). There are no stalls.

	1	2	3	4	5	6	7	8	9	10	11
`lw $t1,4($t0)`	IF	ID	EX	MEM	WB						
`lw $t2,8($t0)`		IF	ID	EX	MEM	WB					
`lw $t4,16($t0)`			IF	ID	EX	MEM	WB				
`add $t3,$t1,$t2`				IF	ID	EX	MEM	WB			
`sw $t3,0($t0)`					IF	ID	EX	MEM	WB		
`add $t5,$t3,$t4`						IF	ID	EX	MEM	WB	
`sw $t5,12($t0)`							IF	ID	EX	MEM	WB

Answer. Original: 2 load-use stalls, $5 + 6 + 2 =$ **13 cycles**. Yes, scheduling helps: move `lw` of E above the first add, giving 0 stalls and $5 + 6 + 0 =$ **11 cycles**.

2022–2023 • Q2 CSE 209 • 13/01/2025 • PL07 MARKS 15

Consider the following code snippet:

```
lw    $t0, 55($t1)
addi  $t1, $t0, 1
lw    $t0, 57($t1)
sw    $t0, 32($t4)
```

Identify the hazard(s) present in the given code snippet. Write the detection conditions for each identified hazard. Complete the *Hazard Detection Unit* by including all the necessary input and output signals in the provided reference figure (Figure for Question 2) to ensure that hazard(s) can be detected and stalls can be applied when necessary. Draw the given partial diagram in your script and complete only the Hazard Detection Unit.

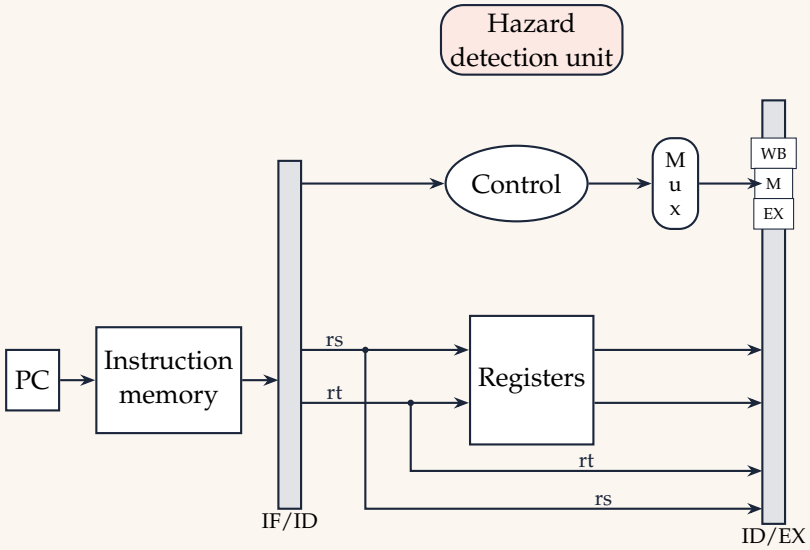


Figure for Question 2 (redrawn)

SOLUTION

Hazards in the snippet

Pair	Register	Type and handling
`lw $t0,55($t1)` → `addi $t1,$t0,1`	`$t0` (rs)	**load-use** RAW: **1 stall** by the hazard detection unit, then MEM/WB→EX forward
`addi $t1,$t0,1` → `lw $t0,57($t1)`	`$t1` (rs = base)	ALU→address RAW: EX/MEM→EX forward (ForwardA = 10), no stall
`lw $t0,57($t1)` → `sw $t0,32($t4)`	`$t0` (rt = store data)	**load→store** RAW: caught by the load-use check on rt, so 1 stall, then forward
`lw`/`lw` on `$t0`; `lw`/`addi` on `$t1`	—	WAW / WAR: not hazards in the in-order 5-stage pipeline

Detection conditions

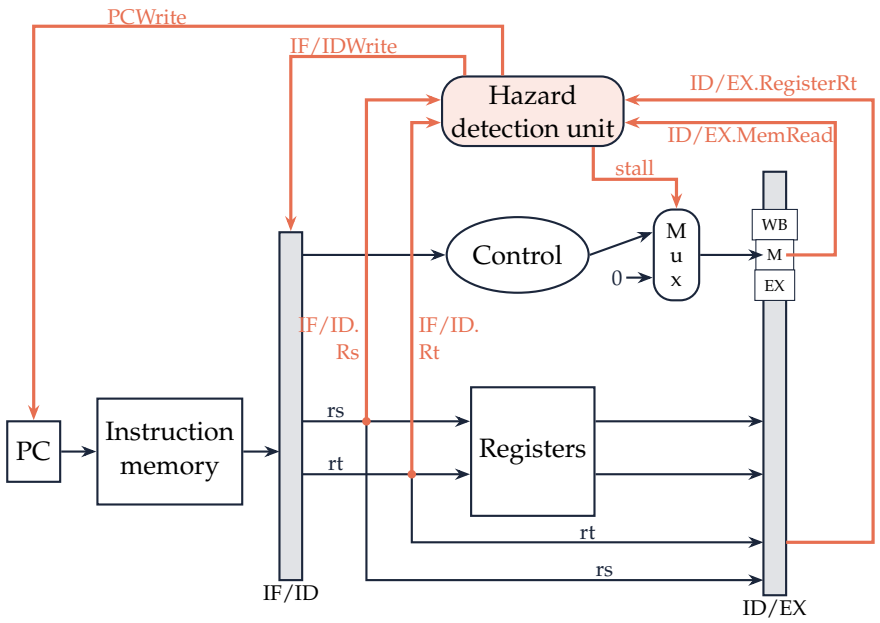
```
(1) load-use (lw→addi), checked when addi is in ID:
    ID/EX.MemRead and ((ID/EX.RegisterRt = IF/ID.RegisterRs)
                    or (ID/EX.RegisterRt = IF/ID.RegisterRt))
    here ID/EX.RegisterRt = $t0 = IF/ID.RegisterRs  ⇒ stall

(2) EX hazard (addi→lw), checked when lw is in EX:
    EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs)  ⇒ ForwardA = 10
    (RegisterRd = RegDst-selected destination = rt = $t1 for addi)

(3) load→store (lw→sw), checked when sw is in ID: same as (1),
    matching ID/EX.RegisterRt = $t0 = IF/ID.RegisterRt  ⇒ stall
```

NOTE The `sw` needs `$t0` only in its MEM stage, so this stall could be avoided by an extra MEM/WB→data-memory forward ("Load → Store" slide). The standard hazard detection unit of the slides compares rt as well, so it stalls one cycle and then forwards MEM/WB→EX. The timing below uses the standard unit.

Completed hazard detection unit (added signals in coral)



Signal	Equation
`stall`	ID/EX.MemRead ∧ [(ID/EX.Rt = IF/ID.Rs) ∨ (ID/EX.Rt = IF/ID.Rt)]
`PCWrite`	$\overline{\texttt{stall}}$ (PC holds; the next instruction is fetched again)
`IF/IDWrite`	$\overline{\texttt{stall}}$ (the using instruction stays in ID, decoded again)
control-mux select	`stall` (EX/M/WB control bits into ID/EX forced to 0 = bubble)

Resulting timing (standard unit, full forwarding)

	1	2	3	4	5	6	7	8	9	10
`lw $t0,55($t1)`	IF	ID	EX	MEM	WB					
`addi $t1,$t0,1`		IF	ID	stall	EX	MEM	WB			
`lw $t0,57($t1)`			IF	stall	ID	EX	MEM	WB		
`sw $t0,32($t4)`					IF	ID	stall	EX	MEM	WB

Answer. Hazards: load-use on `$t0` (stall + forward), ALU→base on `$t1` (forward 10, no stall), load→store on `$t0` (caught by the rt comparison, 1 stall). The hazard detection unit takes IF/ID.Rs, IF/ID.Rt, ID/EX.Rt and ID/EX.MemRead, and drives PCWrite, IF/IDWrite and the control-zeroing mux. Total $5 + 3 + 2 = 10$ cycles.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q5 CSE 209 • 13/01/2025 • PL08 **MARKS 15**

Consider the following code snippet:

```
beq $1, $2, TARGET   # branch is/will be taken for this one
lw  $3, 40($4)
add $2, $3, $4
sw  $2, 40($4)

TARGET:
or $1,$1,$2
```

Assume that branch decision is checked using the ALU zero signal (i.e., no separate hardware is

placed to decide early). Now can an attempt to flush and stall the pipeline occur simultaneously? If yes, explain with the help of an illustration.
Will there be an issue if flush and stall signals occur simultaneously? If there is an issue, you need to provide a solution. If there is no issue, you need to clearly explain why not.

2018–2019 • Q2(a) CSE 305 • 29/10/2019 • PL08 **MARKS 10**

Consider the following code snippet:

```
        beq $1, $2, TARGET   # branch is/will be taken for this one
        lw  $3, 40($4)
        add $2, $3, $4
        sw  $2, 40($4)
TARGET: or  $1, $1, $2
```

Assume that branch decision is checked using the ALU zero signal (i.e., no separate hardware is placed to decide early). Now can an attempt to flush and stall the pipeline occur simultaneously? If yes, then will there be an issue? Explain with the help of a single cycle illustration. If there is an issue, you need to provide a solution. If there is no issue, you need to clearly explain why not.

SOLUTION

Setting

There is no early-branch hardware, so the pipeline is the standard one of the slides ("Branch Hazards: if branch outcome determined in MEM"). The ALU produces `Zero` in EX, it is latched in EX/MEM, and `PCSrc` = `Branch` ∧ `Zero` is formed in **MEM**. When the branch is taken, the three younger instructions (in IF, ID, EX) are flushed by zeroing their control values. Predict-not-taken: `lw`, `add`, `sw` are fetched behind the `beq`.

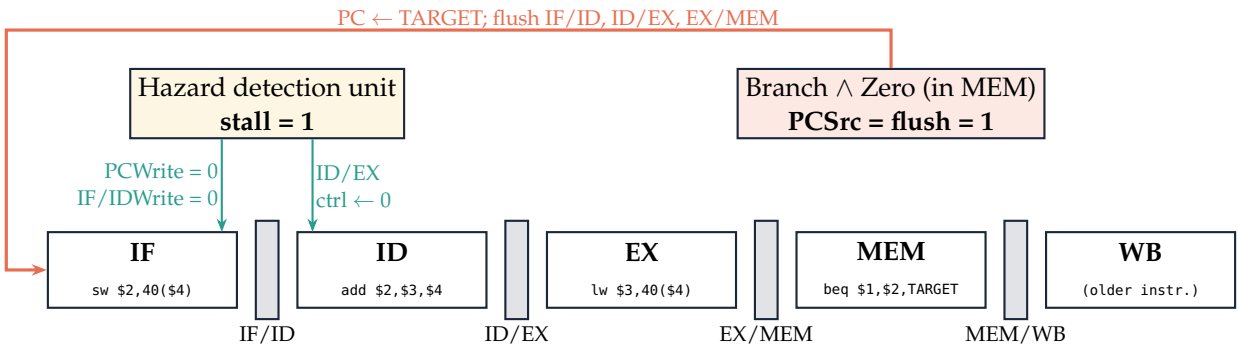
Yes — both happen in clock cycle 4

	1	2	3	4	5	6
`beq $1,$2,TARGET`	IF	ID	EX	MEM	WB	
`lw $3,40($4)`		IF	ID	EX		
`add $2,$3,$4`			IF	ID		
`sw $2,40($4)`				IF		

In cycle 4:

- `lw $3` is in EX (ID/EX.MemRead = 1, ID/EX.Rt = `$3`) and `add $2,$3,$4` is in ID (IF/ID.Rs = `$3`). The **hazard detection unit** sees a load-use hazard, so it asserts **stall**: PCWrite = 0, IF/IDWrite = 0, and a bubble goes into ID/EX.
- `beq` is in MEM with Zero = 1, so PCSrc = 1. The **branch logic** asserts **flush**: load PC ← TARGET, and zero IF/ID, the ID/EX controls and the EX/MEM controls (kill `sw`, `add`, `lw`).

Single-cycle illustration (cycle 4):



conflict: stall says "hold PC and IF/ID", flush says "load PC with TARGET and clear IF/ID"

Is there an issue? Yes

If the stall wins (PCWrite = 0, IF/IDWrite = 0), then:

- the PC is *not* loaded with TARGET. The `beq` leaves MEM in this cycle, so the taken branch is lost for good;
- IF/ID is held, not cleared, so the wrong-path `add` (and then `sw`) is decoded again and executes after the bubble. It writes `$2` and memory, which the program never intended.

The result is incorrect execution: `or` at TARGET is reached late or not at all, and wrong-path instructions change the state.

Solution: give the flush (the older instruction) priority over the stall

The stall exists only to help `add`, which is being squashed anyway. So the hazard detection unit's stall is suppressed whenever a taken branch is flushing:

$$\texttt{stall}_{\text{eff}} = \texttt{stall} \wedge \overline{\texttt{PCSrc}}, \qquad \texttt{PCWrite} = \texttt{IF/IDWrite} = \overline{\texttt{stall}_{\text{eff}}} = \overline{\texttt{stall}} \vee \texttt{PCSrc},$$
$$\texttt{IF.Flush} = \texttt{ID.Flush} = \texttt{EX.Flush} = \texttt{PCSrc}.$$

In hardware this is one AND gate (with PCSrc inverted) on the hazard unit's output, or equivalently a priority rule in the control: flush > stall. Zeroing the ID/EX controls is needed by both actions, so there is no conflict there.

	1	2	3	4	5	6	7	8	9
`beq $1,$2,TARGET`	IF	ID	EX	MEM	WB				
`lw $3,40($4)`		IF	ID	EX	flush				
`add $2,$3,$4`			IF	ID	flush				
`sw $2,40($4)`				IF	flush				
`or $1,$1,$2`					IF	ID	EX	MEM	WB

Answer. Yes. In cycle 4 the load-use hazard (`lw $3`→`add`) asks for a stall while the taken `beq` in MEM asks for a flush. If the stall wins, the branch target is never loaded and wrong-path instructions execute. Fix: let the flush override the stall ($\texttt{stall} \wedge \overline{\texttt{PCSrc}}$), so the PC loads TARGET and the three younger instructions are squashed. `or` is fetched in cycle 5 (3-cycle branch penalty).

2022–2023 • Q7 CSE 209 • 13/01/2025 • PL09 **MARKS 15**

Read the following conversation between three friends.
Chandler: In the pipelined version of MIPS implementation, every jump instruction would need a 1-cycle stall.
Joey: If we just move the control unit in IF stage, we won't need any stall for jump instruction.

Ross: That would cause other problems. I have a better solution to avoid stalls (most of the time) which wouldn't cause any side effect problem.
Now answer the following:

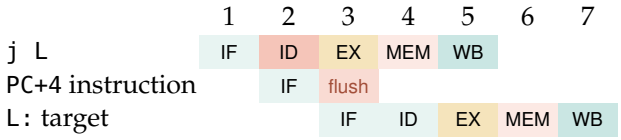
(i) Do you agree with Chandler's statement? Justify your answer.

(ii) If you implement Joey's solution what would be the problem? Explain your answer.

(iii) Can you rediscover Ross's solution, which avoids the problem present in Joey's solution? Provide a brief explanation of your solution.

SOLUTION

(i) Chandler — yes, for the standard 5-stage pipeline. A `j` is recognised only when the control unit decodes it in **ID**. Its target $\{\text{PC+4}[31{:}28],\ \text{address} \times 4\}$ is also formed there. While `j` was being fetched, IF had no way of knowing that it was a jump, so it fetched PC+4. That instruction has to be flushed (IF.Flush), which loses one cycle on *every* jump, since a jump is always taken. (Strictly it is a flush/bubble, not a data-hazard stall, but the cost is the same: 1 cycle.) The only exception is a delayed-branch ISA, where a useful instruction placed in the delay slot hides the cycle.

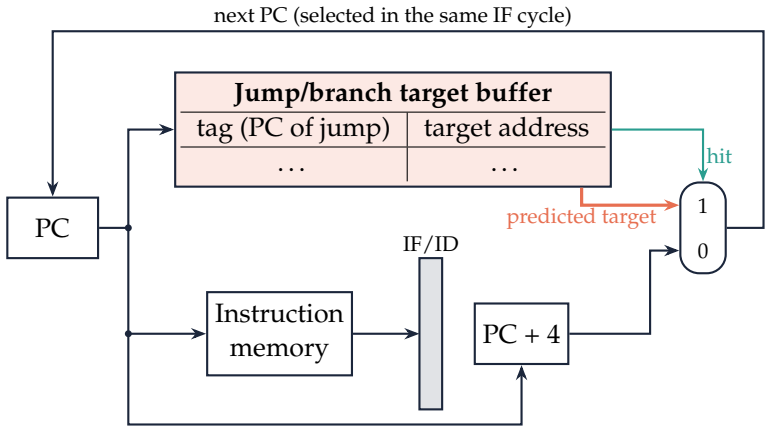


(ii) Joey's idea — decode in IF. The instruction word only comes out of instruction memory at the *end* of IF. Decoding it, forming the jump target and steering the PC mux in the same cycle puts IM access + control decode + target mux + PC set-up all on one combinational path:

- IF becomes by far the longest stage. The clock period is set by the slowest stage, so **every instruction** (not just the 2–3% that are jumps) runs slower, and the pipeline is no longer balanced. The loss is much larger than the one cycle saved per jump.
- The control signals produced in IF must now also be carried in IF/ID (a wider pipeline register). The hazard logic that zeroes control on a stall or flush (which lives in ID) has to reach back into IF, which complicates stalling: a stalled or flushed instruction may already have changed the PC.

(iii) Ross's solution — predict the jump target in IF with a Branch/Jump Target Buffer.

- Add a small cache (BTB) indexed by the *PC of the instruction being fetched*. Each entry holds a tag (the PC of a jump/taken branch) and its target address.
- In IF, look up the BTB *in parallel* with the instruction-memory access. This needs no decoding and does not lengthen the IF path much. On a hit, the next PC is the stored target, so the target is fetched in the very next cycle with **0 penalty**.
- The first time a jump is seen (or after it is evicted) the BTB misses. The jump then costs the normal 1-cycle flush and its target is written into the BTB. This is why it avoids the stall "most of the time".
- No side effect: the clock period is unchanged, and the decode in ID still checks the target. A wrong BTB prediction (e.g. an aliased entry) is simply flushed like a misprediction.



Answer. (i) Agree: a jump is identified and its target formed only in ID, so the fetch behind it is always flushed (1 cycle per jump, unless a delay slot is used). (ii) Decoding in IF puts IM access + decode + target selection in one stage, which lengthens the clock for every instruction and complicates stalling and flushing. (iii) A jump (branch) target buffer looked up with the PC in IF supplies the target in the same cycle: 0 penalty on a hit, the normal 1 cycle on a miss, and no change to the cycle time.

2022–2023 • Q1(a–c) CSE 305 • 25/04/2024 • PL10 **MARKS 12 + 16 + 7**

(a) Assume pipelined execution for the following sequence of instructions.

```
ins 1: sub $2, $1, $3
ins 2: and $2, $2, $5
ins 3: or  $6, $2, $7
```

Assume that **no data forwarding mechanism** is available to you. Construct the execution pipeline for the given sequence along with the stalls required to produce correct output. Evaluate the total number of clock cycles required to execute all three instructions. (12)

(b) Adapt **data forwarding mechanism** for the instruction set in Question 1(a) to enhance parallelism. Reconstruct the execution pipeline for the given sequence showing all pipelined registers and the forwarding lines. Revise the evaluation of the total number of clock cycles required to execute all three instructions. Explain double-data hazard with the help of your constructed pipeline. (16)

(c) Formulate the data forwarding condition between ins 1 and ins 2 in your constructed pipeline for 1 (b). (7)

SOLUTION

Dependences

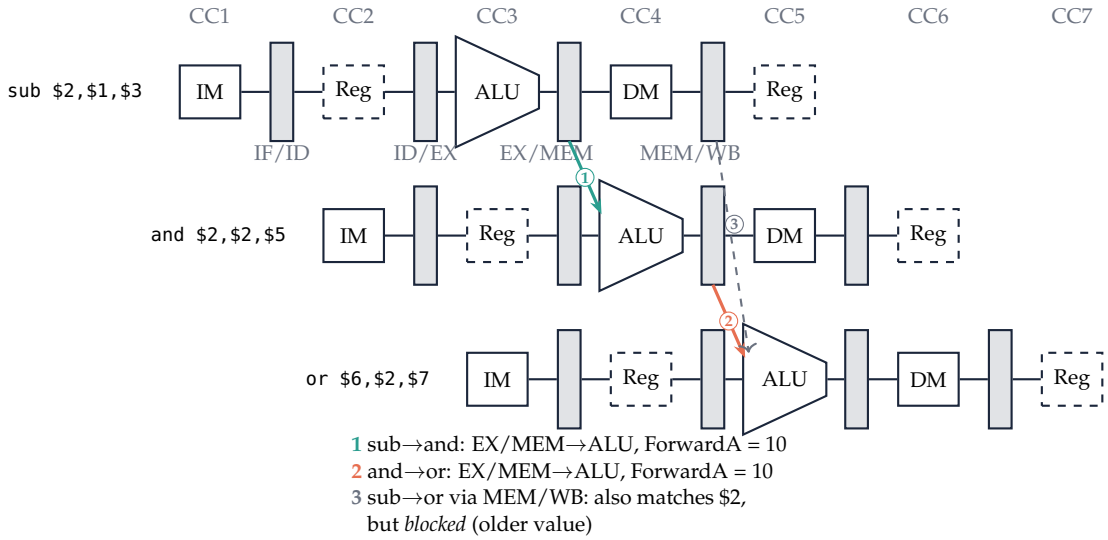
`and` reads `$2` written by `sub` (RAW, distance 1). `or` reads `$2`, and the value it must get is the one written by `and` (the most recent writer). `sub` also writes `$2`, two instructions earlier. Cycle count: total $= 5 + (n - 1) +$ stalls.

(a) No forwarding. A value can be used only after it has been written to the register file. Writes happen in the first half of WB and reads in the second half of ID, so a consumer's (final) ID may coincide with the producer's WB. `and` must wait until `sub`'s WB in cycle 5, which costs 2 stalls. `or` must wait for `and`'s WB in cycle 8, another 2 stalls. (stall = the instruction repeats its previous stage.)

	1	2	3	4	5	6	7	8	9	10	11
`sub $2,$1,$3`	IF	ID	EX	MEM	WB						
`and $2,$2,$5`		IF	ID	stall	stall	EX	MEM	WB			
`or $6,$2,$7`			IF	stall	stall	ID	stall	stall	EX	MEM	WB

Answer. No forwarding: 4 stall cycles; total $= 5 + 2 + 4 = \mathbf{11}$ clock cycles.

(b) With forwarding. Each result is taken from the pipeline register as soon as the ALU has produced it, so no stalls are needed:



- **CC4** (`and` in EX): EX/MEM holds `sub`'s result, and EX/MEM.Rd = `$2` = ID/EX.Rs, so ForwardA = 10 (path 1).
- **CC5** (`or` in EX): EX/MEM holds `and`'s result for `$2`, and MEM/WB holds `sub`'s result, also for `$2`. That is the **double data hazard**: both the EX-hazard and the MEM-hazard conditions are true for ID/EX.Rs = `$2`. The correct value is the most recent one (`and`), so ForwardA = 10 (path 2). The revised MEM-hazard condition ("forward from MEM/WB only if the EX hazard condition is not true") suppresses path 3. Forwarding `sub`'s value would give `or` the wrong `$2`.

	1	2	3	4	5	6	7
`sub $2,$1,$3`	IF	ID	EX	MEM	WB		
`and $2,$2,$5`		IF	ID	EX	MEM	WB	
`or $6,$2,$7`			IF	ID	EX	MEM	WB

Answer. With forwarding: 0 stalls; total $= 5 + 2 + 0 = \mathbf{7}$ clock cycles (down from 11).

(c) Forwarding condition ins 1 → ins 2 (EX hazard on the first ALU operand, evaluated in CC4):

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10
```

Here EX/MEM.RegWrite = 1 (`sub` is R-type), EX/MEM.RegisterRd = $2 \neq 0$, and ID/EX.RegisterRs = 2. So ForwardA = 10 and the ALU's upper input takes `sub`'s result from EX/MEM. ID/EX.RegisterRt = $5 \neq 2$, so ForwardB = 00.

Answer. ForwardA = 10 when EX/MEM.RegWrite $\wedge$ (EX/MEM.Rd $\neq 0$) $\wedge$ (EX/MEM.Rd = ID/EX.Rs), true here with Rd = Rs = `$2`.

2022–2023 • Q4(b) CSE 305 • 25/04/2024 • PL11 **MARKS 12**

Four hardware components have been added in the ID stage of the pipelined datapath design as shown in Figure for Question 4(b) to reduce branch stalls. Write down the names of these four newly added components. Explain how the branch decision and the target address becomes available by the end of the ID stage of the *beq* instruction.

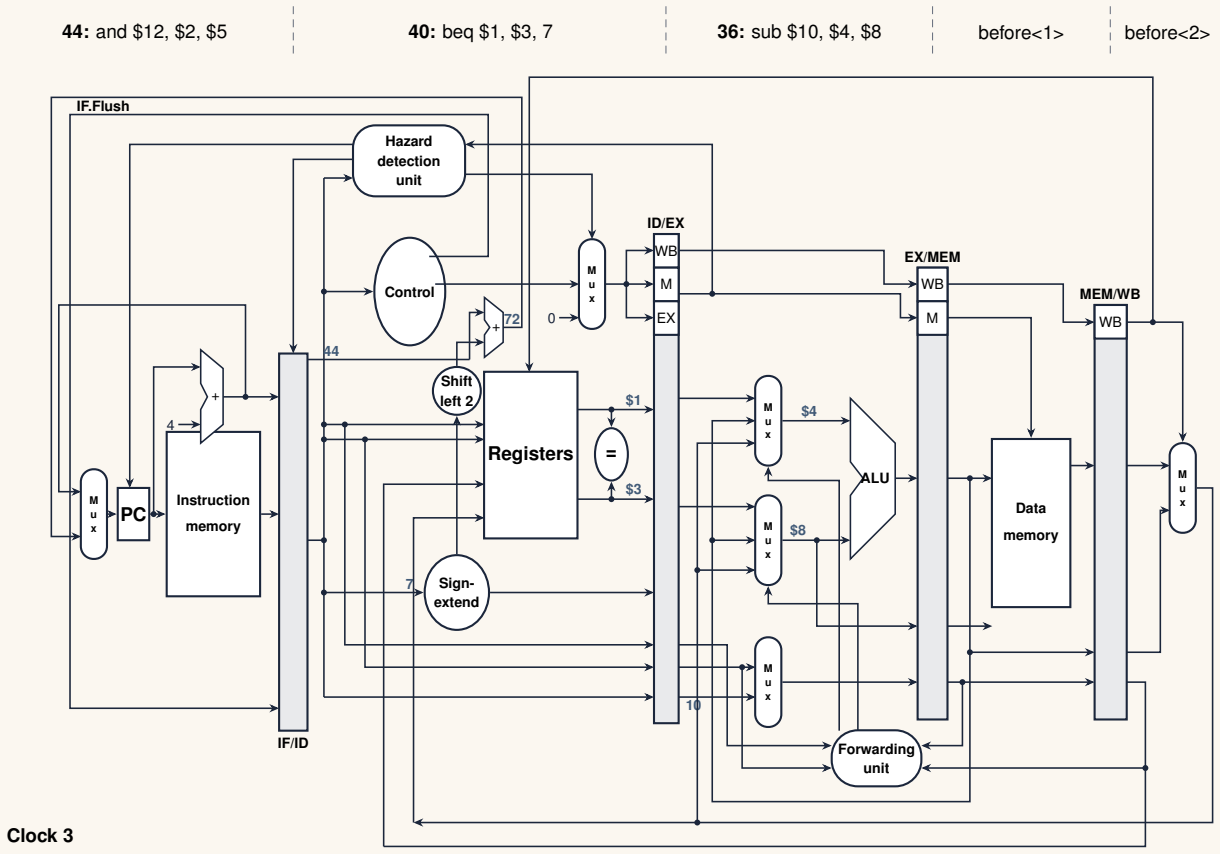


Figure for Question 4(b): Branch Stall Reduction Mechanism

SOLUTION

NOTE The figure is too blurry to read every label. It shows the slides' "Reducing Branch Delay" example (36: `sub`, 40: `beq $1,$3,7`, 44: `and`, ..., 72: `lw $4,50($7)`), i.e. P&H Fig. 4.62. The four components named below are those of that standard figure.

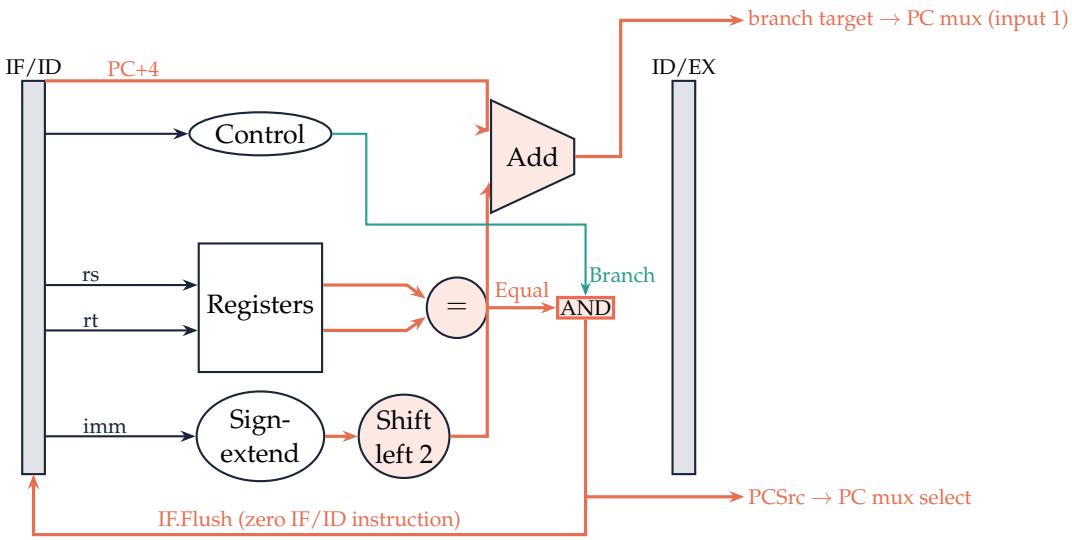
The four components added in ID

(i) Shift-left-2 unit for the sign-extended branch offset, moved from EX into ID.

(ii) Branch target adder, moved into ID: it adds IF/ID.PC+4 and (offset $\ll$ 2).

(iii) Register equality comparator ("=", a bank of XORs feeding a NOR) on the two register-file outputs. It replaces the ALU `Zero` test done in EX.

(iv) IF.Flush control: a control line from the branch logic that zeroes the instruction field of IF/ID (turning the wrongly fetched instruction into a nop), together with the AND gate that forms PCSrc = Branch $\wedge$ Equal and drives the PC mux.



How the decision and the target are ready at the end of ID (beq $1,$3,7 at address 40, clock cycle 3)

- **Target:** IF/ID carries PC+4 = 44. The 16-bit offset 7 is sign-extended and shifted left 2 ($7 \times 4 = 28$). The ID-stage adder gives $44 + 28 = \mathbf{72}$ within cycle 3.
- **Decision:** the register file reads `$1` and `$3` in the first part of ID, and the equality comparator compares them directly. No subtraction in the ALU is needed, which is why it fits in ID. Control decodes `beq` (Branch = 1), and the AND gate produces PCSrc = Branch $\wedge$ Equal.
- **End of cycle 3 (taken):** the PC mux loads 72 instead of 48. IF.Flush zeroes IF/ID, so the `and` at 44 already fetched becomes a nop. In cycle 4 `lw $4,50($7)` at 72 is fetched. A taken branch costs only **1 cycle** instead of 3 (decision in MEM).
- The comparator needs its operands earlier, so forwarding paths into ID (from EX/MEM, MEM/WB) and stalls for a just-computed or just-loaded operand are required. These are the "data hazards for branches" cases.

Answer. Added in ID: shift-left-2, branch-target adder, register equality comparator, and the IF.Flush control (with the Branch $\wedge$ Equal gate driving the PC mux). The target PC+4+(imm$\ll$2) and the equality test both finish in ID, so the PC is redirected at the end of ID and only the one instruction in IF is flushed (1-cycle penalty).

2021–2022 • Q1(b) CSE 305 • 06/04/2023 • PL12 **MARKS 10**

Consider the following MIPS code:

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
add  $t3, $t1, $t2
sw   $t3, 12($t0)
lw   $t4, 8($t0)
add  $t5, $t1, $t4
sw   $t5, 16($t0)
```

How many cycles are required to execute the above code? Can you reduce the number of cycles using code scheduling? If yes, what will be resulting number of cycles?

SOLUTION

Method

The pipeline is the 5-stage MIPS pipeline with full forwarding. ALU→ALU and ALU→`sw` dependences are forwarded. Only a **load-use hazard** (a `lw` result needed by the very next instruction) costs **1 stall**. Total $= 5 + (n-1) +$ stalls. This is the slides' example $A = B + E;\ C = B + F$.

Original code. `lw $t2`→`add $t3` is a load-use hazard (1 stall), and so is `lw $t4`→`add $t5` (1 stall). `add`→`sw` values are forwarded from EX/MEM.

	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB								
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB							
`add $t3,$t1,$t2`			IF	ID	stall	EX	MEM	WB					
`sw $t3,12($t0)`				IF	stall	ID	EX	MEM	WB				
`lw $t4,8($t0)`						IF	ID	EX	MEM	WB			
`add $t5,$t1,$t4`							IF	ID	stall	EX	MEM	WB	
`sw $t5,16($t0)`								IF	stall	ID	EX	MEM	WB

Total $= 5 + 6 + 2 = 13$ cycles.

Scheduled code. `lw $t4` is independent of the instructions above it, so it moves up to third place. Every load is then at least 2 instructions ahead of its use:

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
lw   $t4, 8($t0)      # moved up
add  $t3, $t1, $t2    # $t2 forwarded from MEM/WB
sw   $t3, 12($t0)     # $t3 forwarded from EX/MEM
add  $t5, $t1, $t4    # $t4 via register file
sw   $t5, 16($t0)
```

	1	2	3	4	5	6	7	8	9	10	11
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB						
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB					
`lw $t4,8($t0)`			IF	ID	EX	MEM	WB				
`add $t3,$t1,$t2`				IF	ID	EX	MEM	WB			
`sw $t3,12($t0)`					IF	ID	EX	MEM	WB		
`add $t5,$t1,$t4`						IF	ID	EX	MEM	WB	
`sw $t5,16($t0)`							IF	ID	EX	MEM	WB

Answer. 13 cycles (2 load-use stalls). Yes: moving `lw $t4,8($t0)` up removes both stalls, giving $5 + 6 + 0 =$ **11 cycles**.

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2021–2022 • Q1(c) CSE 305 • 06/04/2023 • PL13 **MARKS 10**

What do you mean by data hazard for branches? Give MIPS code examples with explanations for each of the following cases of data hazard for branches:

(i) Needs no stall to resolve

(ii) Needs one cycle stall to resolve

(iii) Needs two cycle stalls to resolve

2017–2018 • Q2(c) CSE 305 • 23/09/2018 • PL13 **MARKS 5**

Give the MIPS codes for each of the following cases of data hazard for branches:

(i) Needs one cycle stall to resolve

(ii) Needs two cycle stalls to resolve

2013–2014 • Q8(c) CSE 305 • 05/07/2015 • PL13 **MARKS 9**

What do you mean by data hazard for branches? Give examples with MIPS code for each of the following cases of data hazards for branches:

(i) Needs no stall to resolve

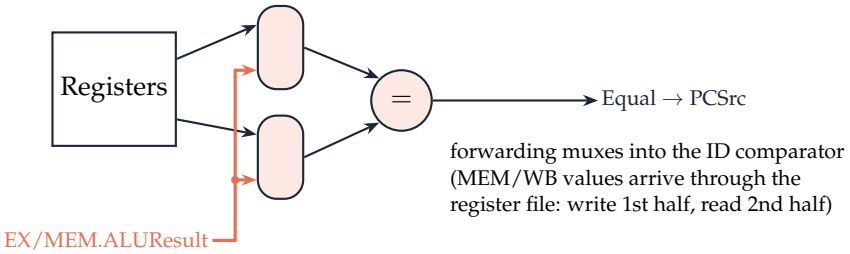
(ii) Needs one cycle stall to resolve

(iii) Needs two cycle stalls to resolve

SOLUTION

Data hazard for branches

To cut the branch penalty to 1 cycle, MIPS moves the branch decision into **ID**: a register comparator there tests `rs` = `rt`. The comparator therefore needs its operands at ID, one stage *earlier* than an ALU instruction needs them (EX). If a compared register is the destination of a recent instruction whose result is not yet available, that is a **data hazard for branches**. It is handled by forwarding into the ID comparator (from EX/MEM or MEM/WB) and, when the value does not exist yet, by stalling the branch in ID.



forwarding muxes into the ID comparator (MEM/WB values arrive through the register file: write 1st half, read 2nd half)

(stall cells below mean the `beq` repeats ID while bubbles go ahead.)

(i) No stall — the producer is the 2nd or 3rd preceding ALU instruction

```
add $4, $5, $6
add $1, $2, $3
sub $7, $8, $9       # independent instruction
beq $1, $4, target
```

When `beq` is in ID (cycle 5), `add $1` is in MEM, so `$1` is forwarded from EX/MEM into the comparator. `add $4` is in WB, so `$4` is written in the first half and read in the second. No stall.

	1	2	3	4	5	6	7	8
`add $4,$5,$6`	IF	ID	EX	MEM	WB			
`add $1,$2,$3`		IF	ID	EX	MEM	WB		
`sub $7,$8,$9`			IF	ID	EX	MEM	WB	
`beq $1,$4,target`				IF	ID	EX	MEM	WB

(ii) One stall — the producer is the immediately preceding ALU instruction, or a load 2 instructions back

```
lw  $1, addr
add $4, $5, $6
beq $1, $4, target   # $4 from the add just before; $1 from the lw 2 back
```

In cycle 4 `beq` is in ID, but `add $4` is still in EX and `lw $1` is in MEM, so neither value exists at the start of ID. After 1 stall (cycle 5), `$4` is forwarded from EX/MEM and `$1` comes through the register file (lw's WB).

	1	2	3	4	5	6	7	8
`lw $1,addr`	IF	ID	EX	MEM	WB			
`add $4,$5,$6`		IF	ID	EX	MEM	WB		
`beq $1,$4,target`			IF	ID	stall	EX	MEM	WB

(iii) Two stalls — the producer is the immediately preceding load

```
lw  $1, addr
beq $1, $0, target
```

The loaded value exists only at the end of `lw`'s MEM (cycle 4). `beq` (ID in cycle 3) must wait until cycle 5, when `$1` is written in the first half of WB and read by the comparator. That is 2 stalls.

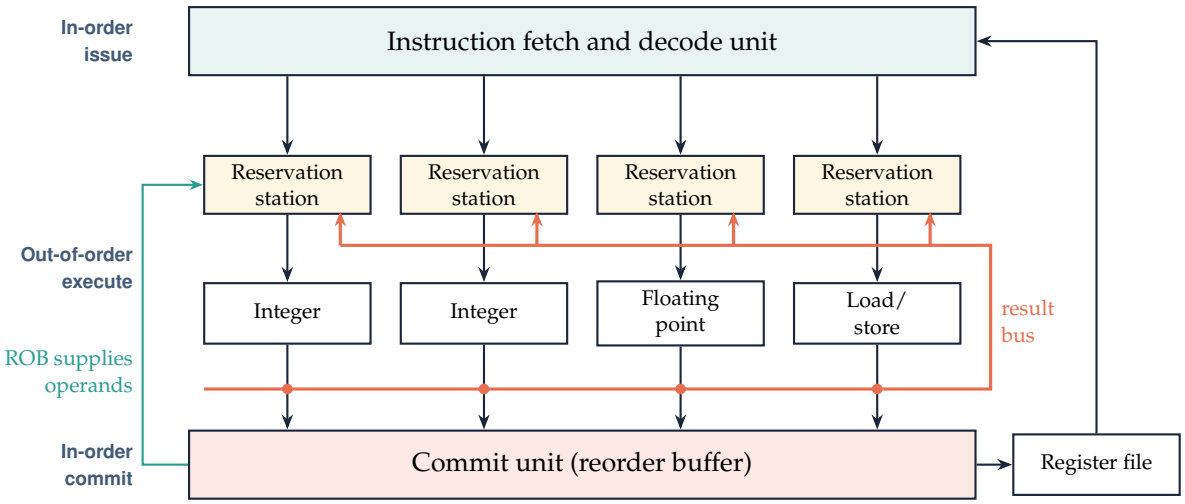
	1	2	3	4	5	6	7	8
`lw $1,addr`	IF	ID	EX	MEM	WB			
`beq $1,$0,target`		IF	ID	stall	stall	EX	MEM	WB

Answer. A branch compared in ID needs its registers one stage early. 0 stalls if the producer is an ALU instruction 2–3 instructions back (forwarding / register file). 1 stall if it is the immediately preceding ALU instruction or a load 2 back. 2 stalls if it is the immediately preceding load.

2021–2022 • Q2(b) CSE 305 • 06/04/2023 • PL14 **MARKS 10**

Draw the block diagram of a dynamically scheduled multiple-issue super-scalar CPU.

SOLUTION



- **Fetch & decode unit** fetches several instructions per cycle. It decodes them, renames their registers, and *issues* them in program order to the reservation station of the right unit. Each instruction also gets a reorder-buffer entry.

- **Reservation stations** hold an issued instruction and its operands. They keep the pending operands (or the tag of the unit that will produce them). An instruction is dispatched as soon as its operands are ready, so independent instructions run *out of order* around a stalled one. Example: `sub $s4,$s4,$t3` runs while `addu $t1,$t0,$t2` waits for `lw $t0,20($s2)`.
- **Functional units** (integer, FP, load/store) execute. Each result is broadcast to every reservation station waiting for it (forwarding) and to the commit unit.
- **Commit unit / reorder buffer** buffers the results and writes them to the register file and memory *in program order*. It also supplies operands of instructions that are done but not yet committed, which preserves dependences. On a mispredicted (speculated) branch, the buffered results after the branch are discarded.

2020–2021 • Q4 CSE 305 • 24/09/2022 • PL15 **MARKS 15**

Suppose you are maintaining a 5-stage pipeline in a MIPS datapath. You are given the following code snippet to execute. Identify all the data hazards that can occur using a multi-cycle pipeline diagram and mark where there is a need for forwarding. Also write down the forwarding conditions to mitigate the hazards. Can all of the hazards in this snippet be addressed using forwarding? Defend your position.

```
lw   $t0, 0($t1)
add  $t0, $t0, $t1
add  $t0, $t2, $t0
add  $t0, $t0, $t3
sub  $t1, $t2, $t3
and  $t4, $t5, $t6
sw   $t0, 0($t6)
```

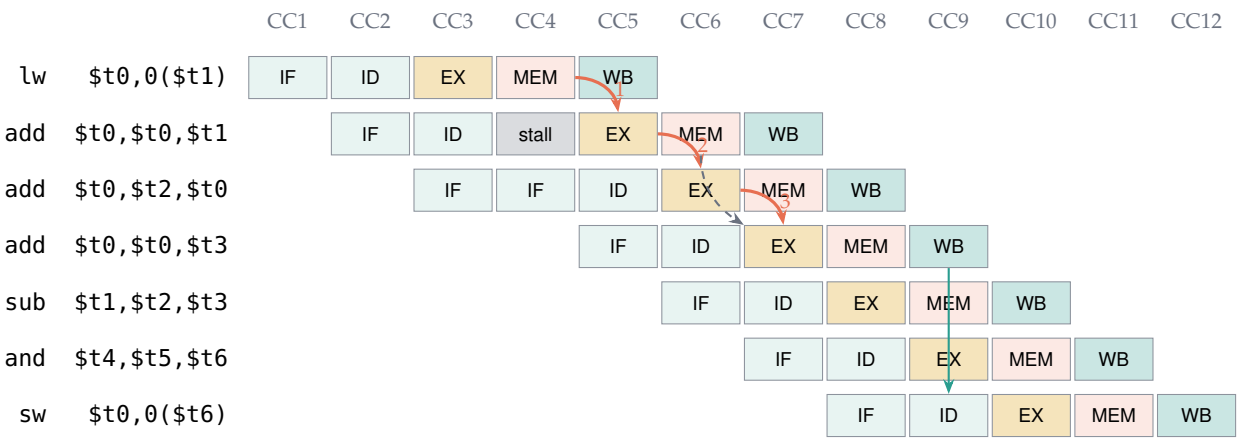
SOLUTION

Common rules

A **RAW** dependence is a hazard when the consumer is fewer than 3 instructions after the producer. The register file writes in the first half of a cycle and reads in the second half, so a distance of 3 is safe. WAR and WAW dependences cannot cause hazards in the in-order 5-stage MIPS pipeline.

- Distance 1 (ALU producer): forward from **EX/MEM** (Forward = 10).
- Distance 2: forward from **MEM/WB** (Forward = 01).
- Load followed directly by a use: **load-use hazard**. Forwarding cannot go backward in time, so the hazard detection unit inserts 1 stall, and then the value is forwarded from MEM/WB.

Dependences on `$t0`: I1→I2 (load-use), I2→I3, I3→I4 (I2→I4 as well, a *double data hazard*), and I4→I7 (distance 3, so no hazard). I5 (`sub`) overwrites `$t1`, which I1/I2 read earlier; that is WAR, so no hazard. I6 is independent.



Coral arrows are the forwarding needed. Dashed: the older MEM/WB value of `$t0` (from I2), which must *not* be used. Teal: I4 writes `$t0` in CC9 and I7 reads it in CC9, through the register file. The whole snippet takes 12 cycles (7 + 4 + 1 stall).

#	Hazard	Resolution	Forwarding condition (checked in EX)
1	I1→I2, `$t0` (load-use)	stall 1 + fwd	detect: `ID/EX.MemRead` ∧ `ID/EX.Rt = IF/ID.Rs` then `MEM/WB.Rd = ID/EX.Rs` ⇒ ForwardA = 01
2	I2→I3, `$t0` as Rt	EX/MEM fwd	`EX/MEM.Rd = ID/EX.Rt` ⇒ ForwardB = 10
3	I3→I4, `$t0` as Rs	EX/MEM fwd	`EX/MEM.Rd = ID/EX.Rs` ⇒ ForwardA = 10
	(I2→I4 double hazard)	MEM/WB blocked	MEM/WB fwd only if the EX-hazard test is false

The general conditions, including the RegWrite and $\neq$ \$0 checks and the double-data-hazard rule:

```
EX : if (EX/MEM.RegWrite and EX/MEM.Rd != 0 and EX/MEM.Rd = ID/EX.Rs) ForwardA = 10
     if (EX/MEM.RegWrite and EX/MEM.Rd != 0 and EX/MEM.Rd = ID/EX.Rt) ForwardB = 10
MEM: if (MEM/WB.RegWrite and MEM/WB.Rd != 0
         and not (EX/MEM.RegWrite and EX/MEM.Rd != 0 and EX/MEM.Rd = ID/EX.Rs)
         and MEM/WB.Rd = ID/EX.Rs) ForwardA = 01
     (same with Rt for ForwardB = 01)
```

Answer. **No**, forwarding alone cannot remove every hazard. In the load-use pair I1→I2, `$t0` exists only at the end of I1's MEM stage, but I2 needs it at the start of its own EX, in the same cycle. Data cannot be forwarded backward in time, so one stall is unavoidable unless the code is rescheduled. All other hazards are removed by forwarding. Scheduling fix: move the independent `and $t4,$t5,$t6` between I1 and I2. (`sub $t1,...` cannot be moved there, because I2 reads `$t1`.) The stall disappears: `$t0` reaches the first `add` by MEM/WB forwarding, and `sw` (now 2 behind I4) also gets it from MEM/WB. The snippet then runs in 11 cycles.

2020–2021 • Q5 CSE 305 • 24/09/2022 • PL16 **MARKS 15**

You are given the following code snippet and have been tasked to run multiple issues in parallel for faster execution. Refactor the code so that IPC is maximized. Also compute the effective IPC. [Hint: you do not have to limit the code to two issues.]

```
Loop: lw   $t0, 0($s0)
      lw   $t1, 0($s1)
      addu $t0, $t0, $t1
      sw   $t0, 0($s1)
      addi $s0, $s0, -4
      addi $s1, $s1, -4
      bne  $s1, $zero, Loop
```

SOLUTION

Scheduling rules of the static dual-issue MIPS (slides)

- A packet = one ALU/branch slot + one load/store slot; an empty slot holds a `nop`.
- No true dependence inside a packet: an ALU result can be used by a load/store only in a *later* packet (EX data hazard). A register may be read and overwritten in the same packet (the read happens in ID, the write in WB).
- Load-use latency is still one cycle: a load in cycle n feeds an ALU instruction in cycle $n+2$.
- If `addi` moves above a load/store that uses the same base register, adjust the offset (e.g. `sw $t0,0($s1)` becomes `sw $t0,4($s1)`).

Step 1 – plain dual issue (baseline).

	ALU/branch	Load/store	cycle
`Loop:`	`addi $s0, $s0, -4`	`lw $t0, 0($s0)`	1
	`addi $s1, $s1, -4`	`lw $t1, 0($s1)`	2
	`nop`	`nop`	3
	`addu $t0, $t0, $t1`	`nop`	4
	`bne $s1, $zero, Loop`	`sw $t0, 4($s1)`	5

The 7 instructions take 5 cycles, so IPC $= 1.4$. Unrolling alone does not help much here. Unrolled $\times 4$ the loop has 19 instructions, and 12 of them (8 `lw` + 4 `sw`) need the single load/store slot. That forces ≥ 12 cycles, so IPC $\leq 19/12 \approx 1.58$. As the hint says, we must issue more than two instructions per cycle.

Step 2 – unroll $\times 4$, rename, and use a 4-issue static machine (2 ALU/branch + 2 load/store slots, same latencies as the dual-issue MIPS). Element k of an iteration uses `$t2`k (from `$s0`) and `$t2`$k{+}1$ (from `$s1`). Both pointers are decremented by 16 in cycle 1, so later accesses use offsets $+16, +12, +8, +4$.

ALU/br 1	ALU/br 2	Ld/St 1	Ld/St 2	cyc
`addi $s0,$s0,-16`	`addi $s1,$s1,-16`	`lw $t0,0($s0)`	`lw $t1,0($s1)`	1
`nop`	`nop`	`lw $t2,12($s0)`	`lw $t3,12($s1)`	2
`addu $t0,$t0,$t1`	`nop`	`lw $t4,8($s0)`	`lw $t5,8($s1)`	3
`addu $t2,$t2,$t3`	`nop`	`lw $t6,4($s0)`	`lw $t7,4($s1)`	4
`addu $t4,$t4,$t5`	`nop`	`sw $t0,16($s1)`	`sw $t2,12($s1)`	5
`addu $t6,$t6,$t7`	`nop`	`sw $t4,8($s1)`	`nop`	6
`bne $s1,$zero,Loop`	`nop`	`sw $t6,4($s1)`	`nop`	7

Why 7 cycles is the minimum here: the 8 loads need at least 4 cycles in 2 slots. The last load (cycle 4) gives its `addu` in cycle 6 (load-use), and its `sw` then goes in cycle 7.

Answer. Refactored loop (unroll $\times 4$, renamed `$t0`–`$t7`, 4-wide issue): 19 instructions in 7 cycles per 4 elements. $\text{IPC}_{\text{eff}} = 19/7 \approx \mathbf{2.71}$, compared with 1.4 for the plain dual-issue schedule. Per element, time falls from 5 to $7/4 = 1.75$ cycles.

NOTE The 2020–21 question does not fix the issue width. We assumed a 4-issue static machine (2 ALU/branch + 2 load/store slots), an element count that is a multiple of 4, and arrays at `$s0` and `$s1` that do not overlap.

2019–2020 • Q5(a–c) CSE 305 • 18/09/2021 • PL17 **MARKS 15 + 5 + 15**

(a) Assume pipelined execution for the following sequence of instructions. The corresponding C code is $A = B + C$; $D = A + D$.

```
lw  $t1, 0($t0)      // load B
lw  $t2, 4($t0)      // load C
add $t3, $t1, $t2    // add B & C
sw  $t3, 12($t0)     // store A
lw  $t4, 8($t0)      // load D
add $t4, $t3, $t4    // add A & D
sw  $t4, 8($t0)      // store D
```

Draw the execution pipeline for the given sequence of instructions and clearly show the stalls and the forwarding paths. Note that data forwarding is available if needed. (15)

(b) How many stalls and clock cycles are required in question 5(a)? (5)

(c) Is it possible to rearrange the code in question 5(a) to reduce the required number of stalls? If not, then explain why it is not possible. If yes, then write down the rearranged code block and indicate the number of stalls and the total number of clock cycles required after the rearrangement. (15)

SOLUTION

Method

The pipeline is the 5-stage MIPS pipeline with full forwarding. ALU→ALU and ALU→`sw` values are forwarded with no stall. A **load-use hazard** (a `lw` result used by the next instruction) costs **1 stall**, then MEM/WB→EX forwarding. The register file is written in the first half of a cycle and read in the second. Total $= 5 + (n - 1) +$ stalls. stall = the instruction repeats its previous stage.

(a) Pipeline with stalls and forwarding paths

	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB								
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB							
`add $t3,$t1,$t2`			IF	ID	stall	EX	MEM	WB					
`sw $t3,12($t0)`				IF	stall	ID	EX	MEM	WB				
`lw $t4,8($t0)`						IF	ID	EX	MEM	WB			
`add $t4,$t3,$t4`							IF	ID	stall	EX	MEM	WB	
`sw $t4,8($t0)`								IF	stall	ID	EX	MEM	WB

From → to	Reg	Path (cycle)	Note
`lw $t1` → `add $t3`	`$t1`	register file (WB c5, read 2nd half)	read in the repeated ID (c5)
`lw $t2` → `add $t3`	`$t2`	MEM/WB → ALU in B (c6)	after 1 load-use stall
`add $t3` → `sw $t3`	`$t3`	EX/MEM → ALU in B / store data (c7)	no stall
`add $t3` → `add $t4`	`$t3`	register file (WB c8, read in ID c8)	no forwarding needed
`lw $t4` → `add $t4`	`$t4`	MEM/WB → ALU in B (c10)	after 1 load-use stall
`add $t4` → `sw $t4`	`$t4`	EX/MEM → ALU in B / store data (c11)	no stall

(b)

Answer. 2 stalls (both load-use: `lw $t2`→`add $t3` and `lw $t4`→`add $t4`); clock cycles $= 5 + (7 - 1) + 2 =$ **13**.

(c) Rearrangement — yes. `lw $t4,8($t0)` (load D) depends on nothing before it. Address 8 is not written by `sw $t3,12($t0)`, so it can move up behind `lw $t2`:

```
lw  $t1, 0($t0)      # B
lw  $t2, 4($t0)      # C
```

```
lw  $t4, 8($t0)      # D  (moved up)
add $t3, $t1, $t2    # A = B + C   ($t2 via MEM/WB)
sw  $t3, 12($t0)     # store A     ($t3 via EX/MEM)
add $t4, $t3, $t4    # D = A + D   ($t3 via MEM/WB)
sw  $t4, 8($t0)      # store D     ($t4 via EX/MEM)
```

	1	2	3	4	5	6	7	8	9	10	11
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB						
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB					
`lw $t4,8($t0)`			IF	ID	EX	MEM	WB				
`add $t3,$t1,$t2`				IF	ID	EX	MEM	WB			
`sw $t3,12($t0)`					IF	ID	EX	MEM	WB		
`add $t4,$t3,$t4`						IF	ID	EX	MEM	WB	
`sw $t4,8($t0)`							IF	ID	EX	MEM	WB

Answer. After rearrangement: **0 stalls**, $5 + 6 + 0 = \mathbf{11}$ clock cycles.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q8(b) CSE 305 • 18/09/2021 • PL18 **MARKS 8**

Draw the finite state machine for a 2-bit dynamic branch prediction scheme.

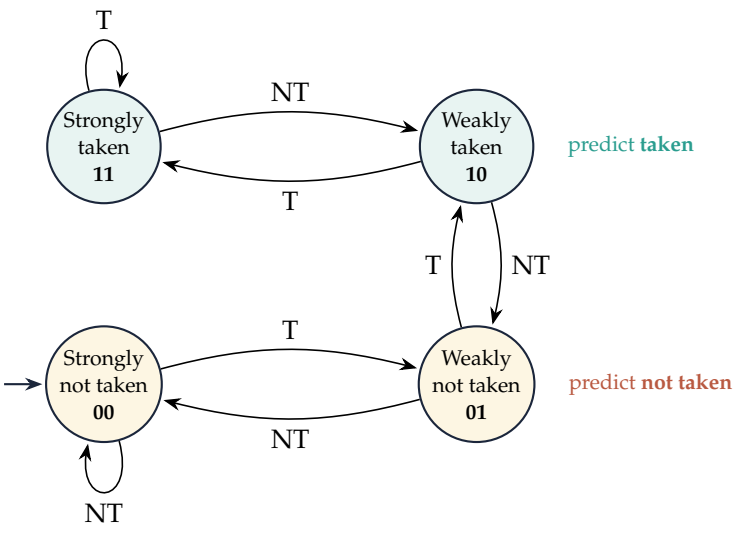
2014–2015 • Q3(b) CSE 305 • 26/01/2016 • PL18 **MARKS 10**

Briefly describe different dynamic branch prediction techniques for dealing with control hazards.

SOLUTION

2-bit predictor FSM (2019–2020 Q8(b))

Each branch-history-table entry is a 2-bit saturating counter. The prediction changes only after **two successive mispredictions**. A loop branch is therefore mispredicted only once per loop (at the exit), not twice as with a 1-bit predictor.



The top two states predict taken, the bottom two predict not taken. T = actual outcome taken

(counter +1), NT = not taken (counter −1), saturating at 11 and 00.

2014–2015 Q3(b): dynamic branch prediction techniques

- **Branch prediction buffer / branch history table (1-bit)**: a small memory indexed by the low bits of the branch address. It stores the last outcome; fetch follows that outcome, and on a misprediction the pipeline is flushed and the bit flipped. Weakness: a loop branch is mispredicted twice per loop (last iteration, and the first iteration the next time).
- **2-bit saturating counter**: the FSM above. The prediction flips only after two consecutive mispredictions, so a loop branch is mispredicted once per loop.
- **Branch target buffer**: a cache of target addresses indexed by the PC at fetch time. On a hit for a branch predicted taken, the target is fetched in the next cycle. This removes the 1-cycle penalty of computing the target, which remains even with a perfect predictor.
- **Correlating (2-level) predictors**: use the outcomes of the last few branches (global history) together with the branch address to pick a counter, so they capture branches that depend on each other.
- **Tournament predictors**: keep a local and a global predictor for each branch plus a selector counter that chooses the one that has been more accurate.

On a wrong guess the CPU stalls while it re-fetches, flushes the wrongly fetched instructions, and updates the history.

2019–2020 • Q8(c) CSE 305 • 18/09/2021 • PL19 MARKS 6×2 = 12

Figure 8c shows a pipelined execution of three instructions **sub**, **and**, and **or**. The pipeline registers are denoted respectively as IF/ID, ID/EX, EX/MEM, and MEM/WB. The data forwarding path between **sub** and **and** has been shown in *green* color and the data forwarding path between **sub** and **or** has been shown in *yellow* color. For each of these two data forwarding cases, write down the data forwarding conditions.

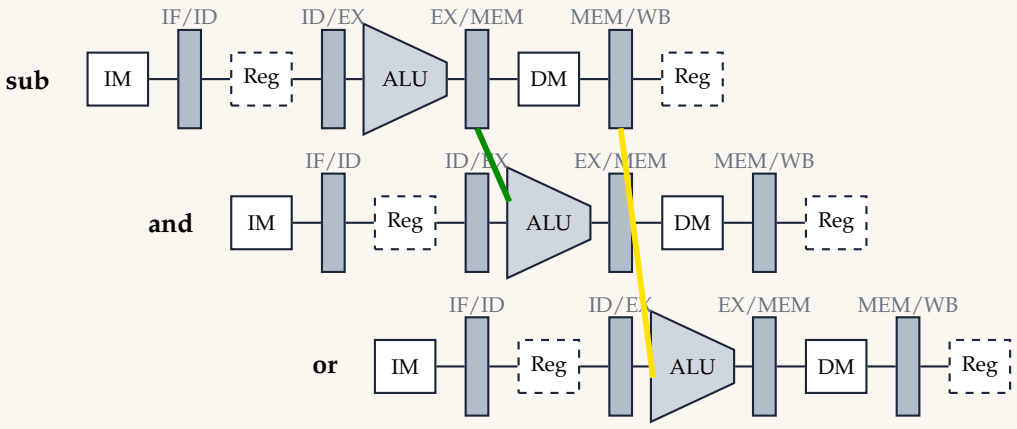


Figure 8c (redrawn): pipelined execution with data forwarding — green: sub→and, yellow: sub→or.

SOLUTION

NOTE Figure 8c does not print the operands. They are taken as in the course slides (P&H): `sub $2,$1,$3`, and `$12,$2,$5`, `or $13,$6,$2`. So `$2` is the *first* source (rs) of `and` and the *second* source (rt) of `or`. The general conditions below hold whichever operand is used: replace Rs/ForwardA by Rt/ForwardB as needed.

Green path: sub → and (EX hazard, forward from EX/MEM)

When `and` is in EX (CC4), `sub` is in MEM and its ALU result sits in EX/MEM.

```
if (EX/MEM.RegWrite                                          [sub writes a register]
   and (EX/MEM.RegisterRd ≠ 0)                          [destination is not $zero]
   and (EX/MEM.RegisterRd = ID/EX.RegisterRs))                        [$2 = rs of and]
      ForwardA = 10                                [ALU input A ← EX/MEM.ALUResult]
```

Yellow path: sub → or (MEM hazard, forward from MEM/WB)

When `or` is in EX (CC5), `sub` is in WB and its result sits in MEM/WB. By the revised (double-data-hazard-safe) condition:

```
if (MEM/WB.RegWrite                                          [sub writes a register]
   and (MEM/WB.RegisterRd ≠ 0)                                          [not $zero]
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRt))   [and is not a newer writer]
   and (MEM/WB.RegisterRd = ID/EX.RegisterRt))                        [$2 = rt of or]
      ForwardB = 01                              [ALU input B ← MEM/WB result]
```

Here `and` writes `$12` ≠ `$2`, so the "not EX hazard" term is true and the yellow forward happens.

Path	Cycle	Source register	Condition checks	Mux setting
green (sub→and)	CC4	EX/MEM	RegWrite, Rd≠0, EX/MEM.Rd = ID/EX.Rs	ForwardA = 10
yellow (sub→or)	CC5	MEM/WB	RegWrite, Rd≠0, no EX hazard, MEM/WB.Rd = ID/EX.Rt	ForwardB = 01

Answer. Green: EX/MEM.RegWrite ∧ EX/MEM.Rd≠ 0 ∧ EX/MEM.Rd = ID/EX.Rs ⇒ ForwardA = 10. Yellow: MEM/WB.RegWrite ∧ MEM/WB.Rd≠ 0 ∧ ¬(EX hazard on the same operand) ∧ MEM/WB.Rd = ID/EX.Rt ⇒ ForwardB = 01.

2018–2019 • Q2(b) CSE 305 • 29/10/2019 • PL20 **MARKS 10**

Discuss what are the issues and how can they be handled in the following code snippet. Assume that using extra hardware branch decision is checked as early as possible. Use multicycle illustrations to explain. How many cycles will be needed to complete the code snippet?

```
lw  $1, 100($2)
add $1, $1, $2
beq $1, $2, TARGET
```

SOLUTION

Setting

The branch is decided as early as possible, i.e. in **ID** (comparator + target adder in ID), with full forwarding. A branch compared in ID needs its operands one stage earlier than an ALU instruction. Cycle count: total $= 5 + (n - 1) +$ stalls. A stall cell means the instruction repeats its previous stage.

Issues

(i) Load-use hazard `lw $1`→`add $1,$1,$2`: the loaded `$1` exists only after MEM. The hazard detection unit (ID/EX.MemRead and ID/EX.Rt = IF/ID.Rs) inserts **1 stall** (bubble, PC and

IF/ID held). Then `$1` is forwarded MEM/WB→EX (ForwardA = 01).

(ii) **Data hazard for branch** `add $1`→`beq $1,$2`: `beq` compares in ID, but `add` (the immediately preceding ALU instruction) produces `$1` only at the end of its EX. That costs **1 more stall**. Then `$1` is forwarded EX/MEM→ID comparator. The `lw` is also 2 instructions back, but `$1` is overwritten by `add`, so only the newest producer matters.

(iii) **Control hazard** of `beq`: the outcome is known at the end of its ID. The instruction fetched behind it (predict not taken) is flushed if the branch is taken, which costs 1 cycle before TARGET is fetched.

(iv) `$1` is written by both `lw` and `add` (WAW) and read/written by `add` (WAR). These are not hazards in the in-order 5-stage pipeline.

Multicycle illustration

	1	2	3	4	5	6	7	8	9	10
`lw $1,100($2)`	IF	ID	EX	MEM	WB					
`add $1,$1,$2`		IF	ID	stall	EX	MEM	WB			
`beq $1,$2,TARGET`			IF	stall	ID	stall	EX	MEM	WB	
next instr. (PC+4)					IF	stall	flush			
TARGET (if taken)							IF	ID	EX	MEM

- Cycle 3: `add` in ID, `lw` in EX ⇒ load-use detected, bubble in cycle 4.
- Cycle 5: `add` in EX gets `$1` from MEM/WB. `beq` in ID needs `$1` now, but it is only being computed ⇒ stall.
- Cycle 6: `add` in MEM, and its result in EX/MEM is forwarded to the ID comparator. `beq` resolves and the target address is ready at the end of cycle 6. The instruction fetched behind `beq` (cycle 5, held in 6) is flushed if the branch is taken, and TARGET is fetched in cycle 7.

Answer. 2 stall cycles (1 load-use + 1 branch-in-ID data hazard). The snippet completes in $5 + (3 - 1) + 2 = \mathbf{9}$ cycles. If the branch is taken, the one instruction fetched behind `beq` is flushed, so TARGET is fetched in cycle 7 (1-cycle penalty).

2017–2018 • Q1(c) CSE 305 • 23/09/2018 • PL21 **MARKS 5**

Consider the following information for load word (lw) instruction:

Instr.	Instruction Fetch	Register read	ALU op	Memory access	Register write
lw	200 ps	100 ps	100 ps	200 ps	100 ps

Consider the following program:

```
lw $1, 100($0)
lw $2, 200($0)
lw $3, 300($0)
```

What is the speedup for the pipelined datapath over the single cycle datapath for this program? What will be the speedup if we add 3,000,000 more load word (lw) instructions?

SOLUTION

Method (as in the slides)

- Single-cycle: the clock must fit the slowest instruction (`lw`), so $T_c = \sum(\text{all stage times})$, and each instruction takes one such cycle.
- Pipelined: $T_c = \max(\text{stage time}) = 200\,\text{ps}$. n instructions need $(n+4)$ cycles (4 cycles to fill the 5-stage pipe, then one completes every cycle).

	1	2	3	4	5	6	7
`lw $1,100($0)`	IF	ID	EX	MEM	WB		
`lw $2,200($0)`		IF	ID	EX	MEM	WB	
`lw $3,300($0)`			IF	ID	EX	MEM	WB

The 3 loads take $3+4=7$ pipeline cycles of 200 ps each, so $T_{\text{pipe}} = 1400\,\text{ps}$.

Speedup

$T_c^{\text{single}} = 200+100+100+200+100 = 700\,\text{ps}.$

$$\text{3 lw: } \frac{3\times 700}{7\times 200} = \frac{2100}{1400} = 1.5, \qquad \text{+3,000,000: } \frac{3{,}000{,}003\times 700}{3{,}000{,}007\times 200} = \frac{2{,}100{,}002{,}100}{600{,}001{,}400} \approx 3.5$$

Answer. Speedup $=$ **1.5** for the 3 loads, and $\approx$ **3.5** $(= 700/200)$ with 3,000,000 more loads. The stages are unbalanced, so the speedup stays below 5.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q4(a) CSE 305 • 02/08/2017 • PL22 **MARKS 10**

What are pipeline hazards? Explain a structural pipeline hazard with a time-space diagram. What type of pipeline hazards occurs in the following code segment? How can it be overcome?

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
add  $t3, $t1, $t2
sw   $t3, 12($t0)
lw   $t4, 8($t0)
add  $t5, $t1, $t4
sw   $t5, 16($t0)
```

2014–2015 • Q3(a) CSE 305 • 26/01/2016 • PL22 **MARKS 10**

What are pipeline hazards? Explain a structural pipeline hazard with a time-space diagram. What type of pipeline hazards occurs in the following code segment? How can it be overcome?

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
add  $t3, $t1, $t2
sw   $t3, 12($t0)
lw   $t4, 8($t0)
add  $t5, $t1, $t4
sw   $t5, 16($t0)
```

SOLUTION

Pipeline hazards

Hazards are situations that prevent the next instruction from starting in the next clock cycle:

- **Structural hazard:** a required hardware resource is busy.
- **Data hazard:** an instruction needs data that an earlier instruction has not yet produced.
- **Control hazard:** the next fetch depends on the outcome of a branch not yet decided.

Structural hazard — time-space diagram

Suppose the pipeline has a *single* memory for instructions and data. In cycle 4 the `lw` is in MEM (data access) while instruction 4 wants to be fetched (instruction access) from the same memory. The fetch must wait one cycle, which puts a bubble in the pipeline:

	1	2	3	4	5	6	7	8	9
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB				
instr 2		IF	ID	EX	MEM	WB			
instr 3			IF	ID	EX	MEM	WB		
instr 4				stall	IF	ID	EX	MEM	WB

Cure: separate instruction and data memories (split I-cache/D-cache), as in the MIPS pipeline.

Hazards in the given code

- **Data (RAW) hazards:** `lw $t2`→`add $t3` (`$t2`) and `lw $t4`→`add $t5` (`$t4`) are **load-use** hazards, 1 stall each even with forwarding. `lw $t1`→`add $t3` (distance 2), `add $t3`→`sw $t3` and `add $t5`→`sw $t5` are removed by forwarding.
- **Structural:** only with a single memory, where every `lw`/`sw` in MEM collides with a later instruction fetch.
- **Control:** none (there are no branches).

	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB								
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB							
`add $t3,$t1,$t2`			IF	ID	stall	EX	MEM	WB					
`sw $t3,12($t0)`				IF	stall	ID	EX	MEM	WB				
`lw $t4,8($t0)`						IF	ID	EX	MEM	WB			
`add $t5,$t1,$t4`							IF	ID	stall	EX	MEM	WB	
`sw $t5,16($t0)`								IF	stall	ID	EX	MEM	WB

How to overcome

Use **forwarding** for the ALU/store dependences, and **code scheduling** for the load-use stalls: move the independent `lw $t4,8($t0)` up behind `lw $t2`.

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
lw   $t4, 8($t0)      # moved up: no load-use stall left
add  $t3, $t1, $t2
sw   $t3, 12($t0)
add  $t5, $t1, $t4
sw   $t5, 16($t0)
```

Answer. The code has data hazards (two load-use hazards plus forwardable ALU→store dependences), no control hazard, and a structural hazard only with a single memory. With forwarding it takes 13 cycles (2 stalls). Reordering removes the stalls, giving 11 cycles. Separate I/D memories remove the structural hazard.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q4(b) CSE 305 • 02/08/2017 • PL23 MARKS 15

Perform the necessary modification of the single-cycle data path in Figure 2 for pipeline implementation by inserting Pipeline Registers, Forwarding Unit and Hazard Detection Unit. Describe the modification step by step and draw the final modified datapath with all necessary control signals.

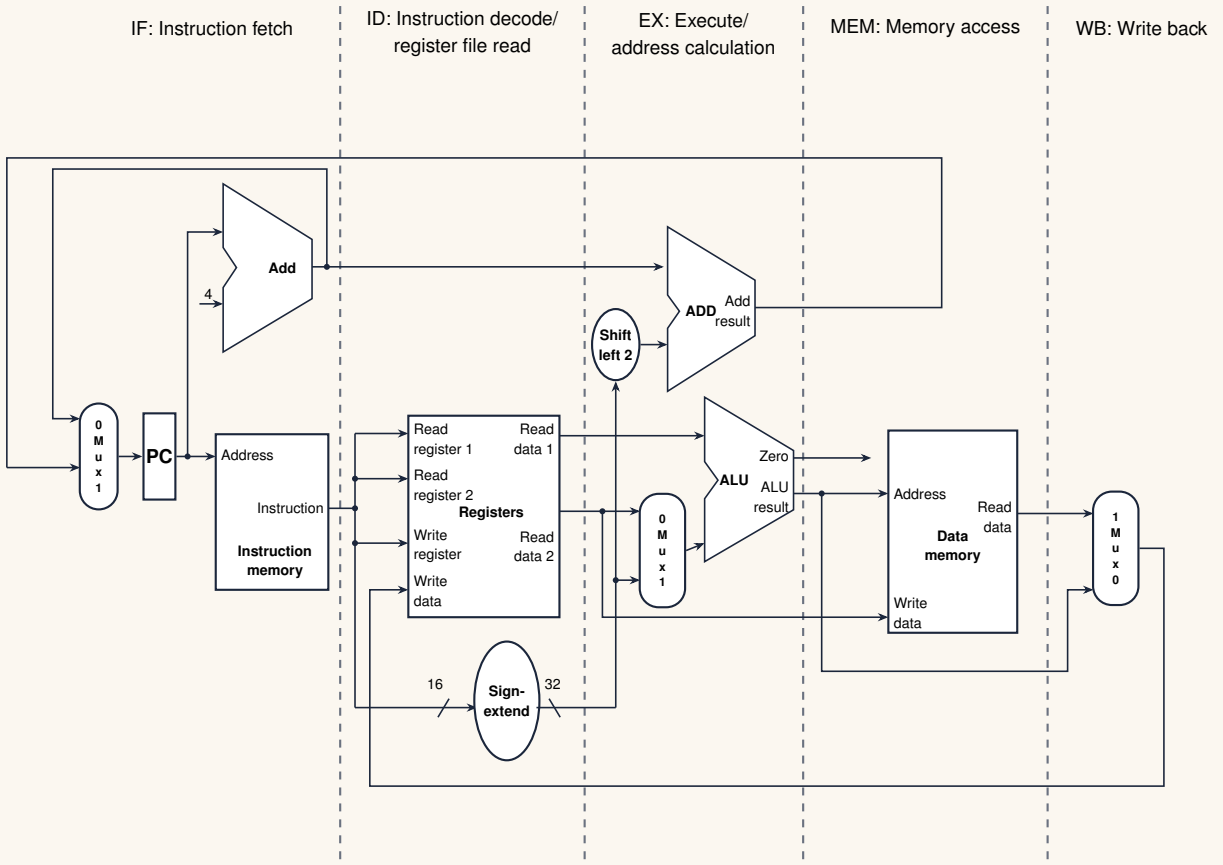


Figure 2: Figure for Question 4(b)

2014–2015 • Q3(c) CSE 305 • 26/01/2016 • PL23 MARKS 15

Perform the necessary modification of the single-cycle data path in Figure 1 for pipeline implementation by inserting pipeline registers and a forwarding unit. Describe the modification step by step and draw the final modified datapath with all necessary control signals.

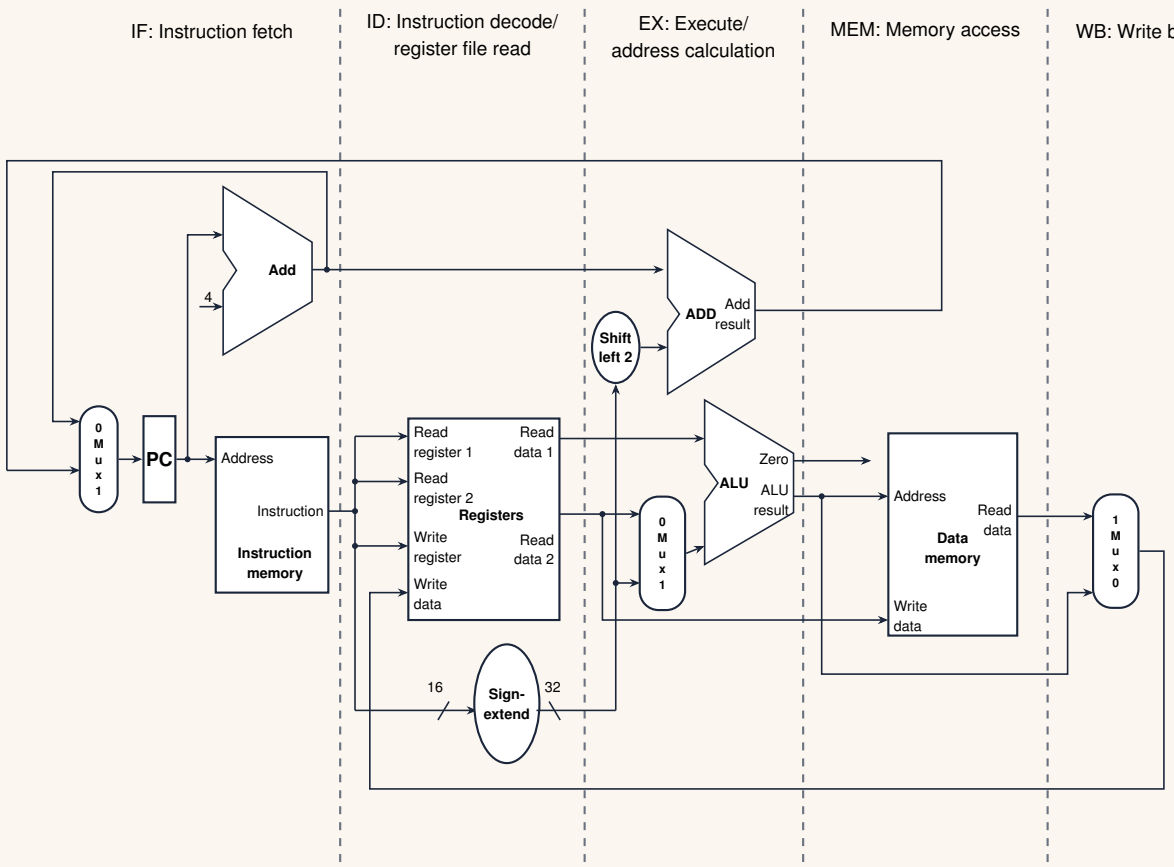


Figure 1: Figure for Question 3(c)

SOLUTION

Step-by-step modification

(i) **Insert pipeline registers** IF/ID, ID/EX, EX/MEM, MEM/WB at the four stage boundaries. Every value produced in a stage and needed later is latched there each cycle: IF/ID = instruction + PC+4 (64 bits). ID/EX = PC+4, Read data 1, Read data 2, sign-extended immediate (128 bits) + Rs, Rt, Rd. EX/MEM = branch target, Zero, ALU result, Read data 2 + destination register (97+5 bits). MEM/WB = memory data + ALU result + destination register.

(ii) **Carry the destination register number** (output of the RegDst mux) through EX/MEM and MEM/WB back to *Write register*. Otherwise the write would use the register field of a later instruction (the "corrected datapath for load").

(iii) **Pipeline the control**: the main control unit still decodes the opcode in ID. Its outputs are stored in ID/EX as three groups: **EX** (RegDst, ALUOp, ALUSrc), **M** (Branch, MemRead, MemWrite) and **WB** (RegWrite, MemtoReg). Each group is used in its own stage and dropped; the M and WB groups move on to EX/MEM, and the WB group to MEM/WB. PCSrc = Branch $\wedge$ Zero is formed in MEM. IF and ID need no special control.

(iv) **Forwarding unit** (EX stage): add 3-input muxes (ForwardA, ForwardB) in front of the ALU inputs. Inputs: register file (00), EX/MEM ALU result (10), MEM/WB write-back value (01). The unit compares `ID/EX.Rs/Rt` with `EX/MEM.Rd` and `MEM/WB.Rd`:

- `if (EX/MEM.RegWrite and EX/MEM.Rd≠0 and EX/MEM.Rd = ID/EX.Rs) ForwardA =`

`10` (same with Rt for B)

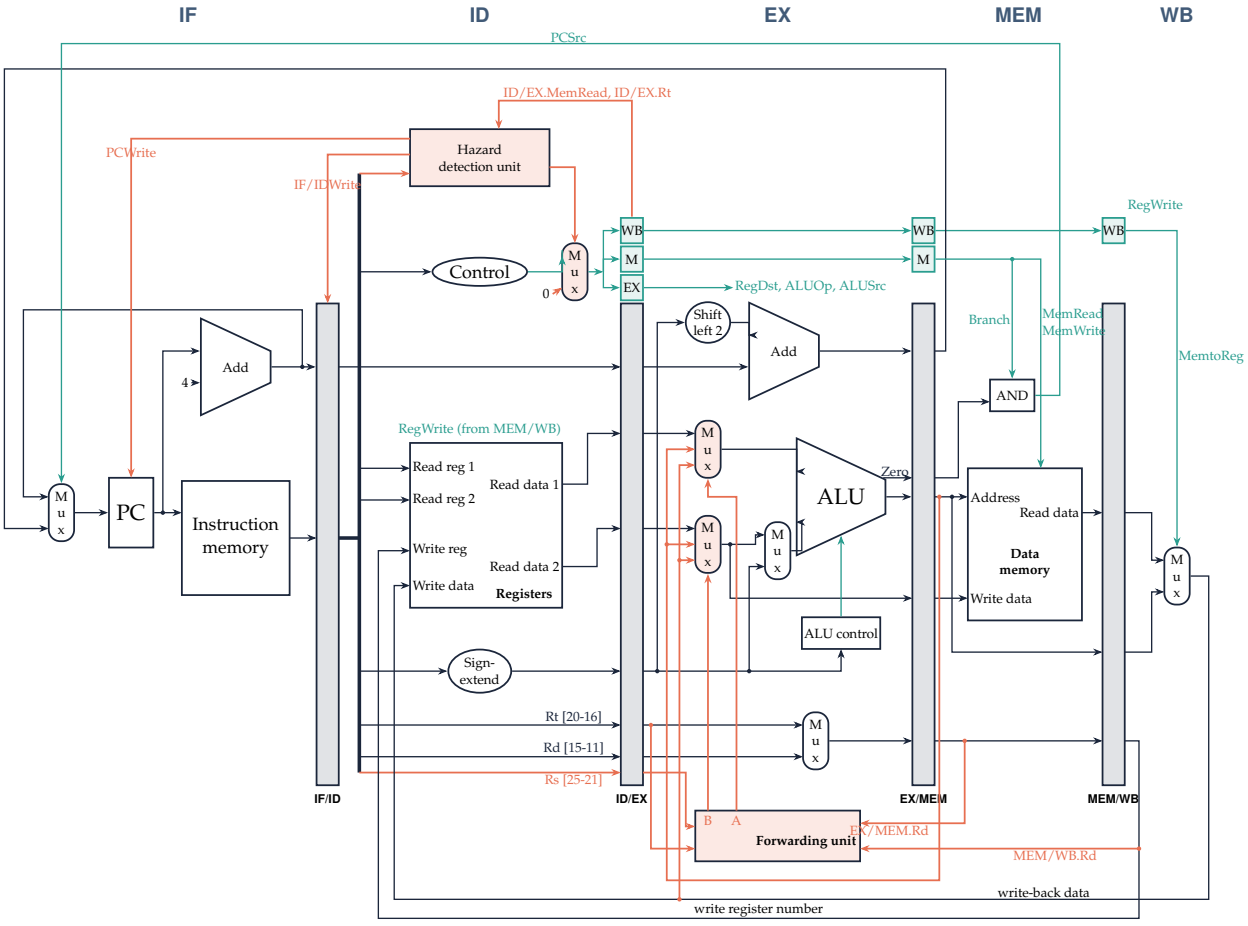
- `if (MEM/WB.RegWrite and MEM/WB.Rd≠0 and not(EX hazard on Rs) and MEM/WB.Rd = ID/EX.Rs) ForwardA = 01` (same with Rt for B)

The "not EX hazard" term handles the double data hazard: the most recent value wins.

(v) **Hazard detection unit** (ID stage, *2016–2017 only*): it detects a load-use hazard with `if (ID/EX.MemRead and (ID/EX.Rt = IF/ID.Rs or ID/EX.Rt = IF/ID.Rt)) stall`. On a stall it (a) sets `PCWrite`= 0 and `IF/IDWrite`= 0, so the same instructions are fetched and decoded again; (b) switches a mux so that all-zero control goes into ID/EX (a bubble).

Final pipelined datapath with control, forwarding and hazard detection

Coral = hardware added by the forwarding unit and the hazard detection unit. For 2014–2015 Q3(c), leave out the hazard detection unit, its control mux, and the PCWrite/IF/IDWrite lines.



Mux inputs of ForwardA/ForwardB, top to bottom: 00 = register file, 10 = EX/MEM, 01 = MEM/WB. The EX/MEM and MEM/WB RegWrite bits also go into the forwarding unit (not drawn).

2015–2016 • Q2(b) CSE 305 • 18/01/2017 • PL24 **MARKS 5**

Consider the following information for load word (lw) instruction:

Instr.	Instruction Fetch	Register read	ALU op	Memory access	Register write
lw	200 ps	100 ps	200 ps	200 ps	100 ps

Consider the following program:

```
lw $1, 100($0)
lw $2, 200($0)
lw $3, 300($0)
```

What is the speedup for the pipelined dadapath over the single cycle datapath for this program? What will be the speedup if we add 2,000,000 more load word (lw) instructions?

SOLUTION

Method (as in the slides)

- Single-cycle: the clock must fit the slowest instruction (`lw`), so $T_c = \sum(\text{all stage times})$, and each instruction takes one such cycle.
- Pipelined: $T_c = \max(\text{stage time}) = 200\,\text{ps}$. n instructions need $(n+4)$ cycles (4 cycles to fill the 5-stage pipe, then one completes every cycle).

	1	2	3	4	5	6	7
`lw $1,100($0)`	IF	ID	EX	MEM	WB		
`lw $2,200($0)`		IF	ID	EX	MEM	WB	
`lw $3,300($0)`			IF	ID	EX	MEM	WB

The 3 loads take $3+4=7$ pipeline cycles of 200 ps each, so $T_{\text{pipe}} = 1400\,\text{ps}$.

Speedup

$T_c^{\text{single}} = 200+100+200+200+100 = 800\,\text{ps}$.

$$\text{3 lw: } \frac{3\times 800}{7\times 200} = \frac{2400}{1400} = 1.714$$

$$\text{+2,000,000: } \frac{2{,}000{,}003\times 800}{2{,}000{,}007\times 200} = \frac{1{,}600{,}002{,}400}{400{,}001{,}400} \approx 3.99999 \approx 4$$

Answer. Speedup $= \mathbf{1.714}$ for the 3 loads, and $\approx \mathbf{4}$ $(= 800/200)$ after adding 2,000,000 loads. For long programs the fill time does not matter, so the speedup approaches $T_c^{\text{single}}/T_c^{\text{pipe}}$. Throughput improves; the latency of each instruction does not.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2015–2016 • Q2(c) CSE 305 • 18/01/2017 • PL25 **MARKS 10**

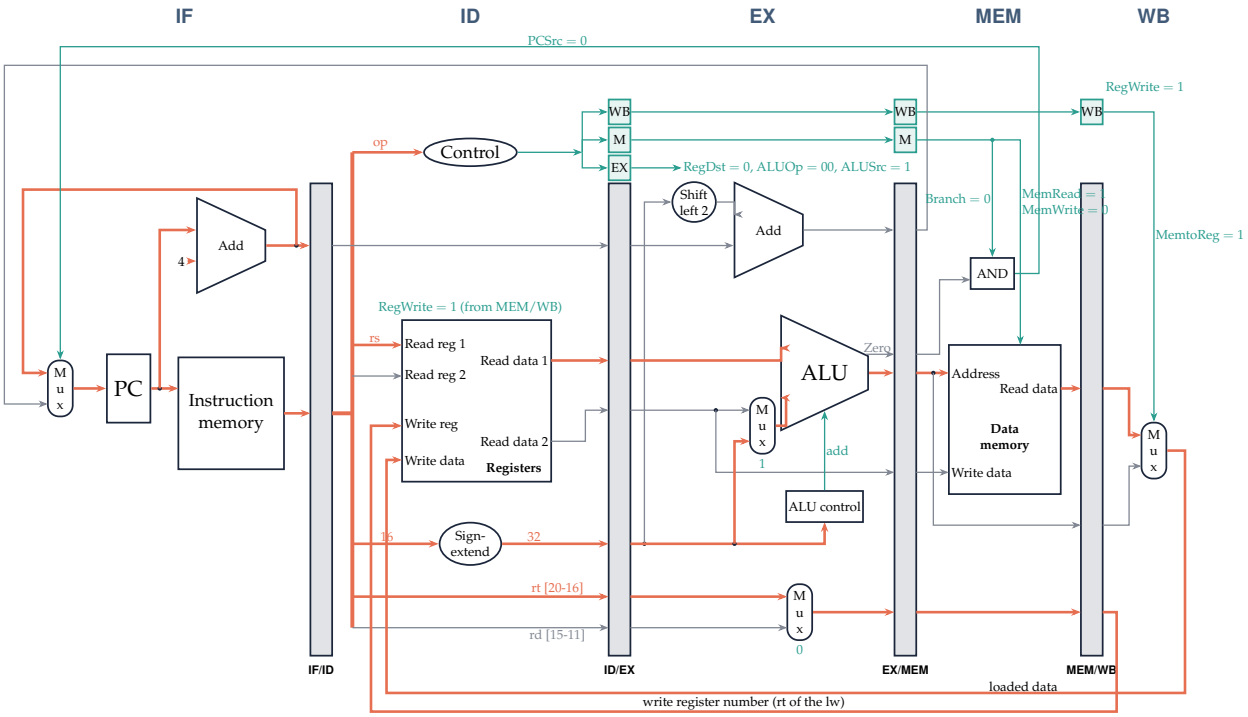
Draw the complete MIPS pipelined datapath and control for load (lw) instruction with pipelined registers.

2013–2014 • Q8(a) CSE 305 • 05/07/2015 • PL25 **MARKS 15**

Draw the complete MIPS pipelined datapath and control for load word (lw) instruction with pipelined registers.

SOLUTION

Pipelined datapath with control; path used by `lw $rt, offset($rs)` in coral



What `lw` does in each stage (and what the pipeline registers carry)

Stage	Action	Latched into
IF	read instruction memory at PC; PC ← PC+4 (PCSrc = 0)	IF/ID: instruction, PC+4
ID	decode; read `$rs`; sign-extend the 16-bit offset; control unit generates EX/M/WB fields	ID/EX: PC+4, Read data 1, Read data 2, imm, rt, rd, control
EX	ALU: `$rs` + sign-ext(offset). ALUSrc = 1, ALUOp = 00 (add), RegDst = 0 (dest = rt)	EX/MEM: address, Read data 2, dest reg, M+WB control
MEM	read data memory at the address (MemRead = 1, MemWrite = 0, Branch = 0)	MEM/WB: loaded word, ALU result, dest reg, WB control
WB	MemtoReg = 1 selects the loaded word; RegWrite = 1 writes it into register rt	—

NOTE The destination register number (rt) must travel with the instruction through ID/EX, EX/MEM and MEM/WB. If Write register were taken from IF/ID, the write would go to the rt field of whatever instruction is in ID at that time (three instructions later). This is the slides' "corrected datapath for load".

2014–2015 • Q1(d) CSE 305 • 26/01/2016 • PL26 **MARKS 13**

Show the time-space diagram for pipelining in a superscalar processor. Briefly describe the architecture of a typical superscalar RISC processor with necessary block diagrams. Mention the differences between a superscalar processor and a VLIW processor. (3+8+2)

SOLUTION

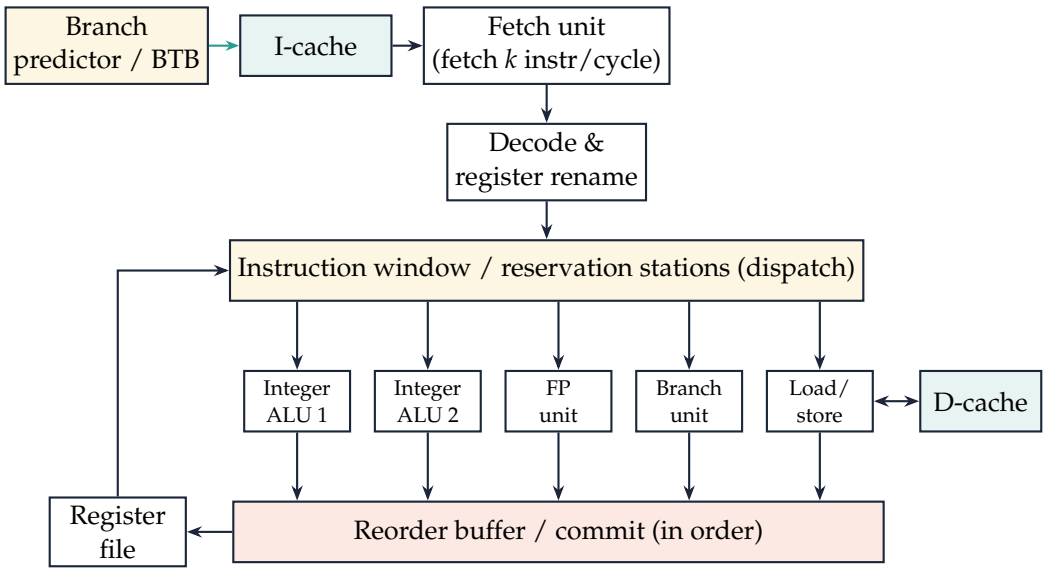
Time-space diagram (2-way superscalar, 5-stage pipeline)

Two instructions enter every stage in every clock cycle, so in the ideal case n instructions take

$n/2 + 4$ cycles. Peak IPC $= 2$ (CPI $= 0.5$). A scalar pipeline would need $n + 4$ cycles.

	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2	IF	ID	EX	MEM	WB			
I3		IF	ID	EX	MEM	WB		
I4		IF	ID	EX	MEM	WB		
I5			IF	ID	EX	MEM	WB	
I6			IF	ID	EX	MEM	WB	
I7				IF	ID	EX	MEM	WB
I8				IF	ID	EX	MEM	WB

Architecture of a typical superscalar RISC processor



- **Fetch unit** reads several instructions per cycle from the I-cache. The branch predictor/BTB steers the fetch so the pipeline is not starved at branches.
- **Decode & rename** decodes several instructions in parallel and renames registers, which removes WAR/WAW (name) dependences.
- **Instruction window / reservation stations**: hardware checks dependences and *dispatches* up to k ready instructions per cycle to free functional units. The order can differ from program order (dynamic scheduling).
- **Multiple functional units** (several integer ALUs, FP, branch, load/store) run in parallel. Results are forwarded to waiting instructions.
- **Reorder buffer / commit** retires the results to the register file *in program order*. This allows speculation and precise exceptions.

Superscalar vs VLIW

	Superscalar	VLIW
Scheduling	dynamic: hardware decides at run time what to issue (0, 1, 2, … per cycle)	static: the compiler packs independent operations into one long instruction (issue packet)
Hazard handling	hardware dependence check, renaming, out-of-order execution	compiler removes hazards; pads with `nop`
Hardware	complex, power hungry	simple decode/issue logic
Code	ordinary sequential code; binary compatible across implementations	code tied to the machine width/latencies; larger code (nops)

2013–2014 • Q7(d) CSE 305 • 05/07/2015 • PL27 **MARKS 9**

What are the five states of MIPS pipeline? Consider the following information for load word (lw) instruction

Instruction	Instruction Fetch	Register read	ALU op	Memory access	Register write
lw	200 ps	100 ps	200 ps	200 ps	100 ps

Consider the following program:

```
lw $1, 100($0)
lw $2, 200($0)
lw $3, 300($0)
```

What is the speedup for the pipelined datapath over the single cycle datapath for this program? What will be the speedup if we add 1,000,000 more load word (lw) instructions?

SOLUTION

Five stages of the MIPS pipeline

IF – fetch the instruction from instruction memory, PC $\leftarrow$ PC $+ 4$; **ID** – decode and read the registers; **EX** – execute the operation or compute the address; **MEM** – access the data-memory operand; **WB** – write the result back to the register file.

Method (as in the slides)

- Single-cycle: the clock must fit the slowest instruction (`lw`), so $T_c = \sum(\text{all stage times})$, and each instruction takes one such cycle.
- Pipelined: $T_c = \max(\text{stage time}) = 200\,\text{ps}$. n instructions need $(n + 4)$ cycles (4 cycles to fill the 5-stage pipe, then one completes every cycle).

	1	2	3	4	5	6	7
`lw $1,100($0)`	IF	ID	EX	MEM	WB		
`lw $2,200($0)`		IF	ID	EX	MEM	WB	
`lw $3,300($0)`			IF	ID	EX	MEM	WB

The 3 loads take $3 + 4 = 7$ pipeline cycles of 200 ps each, so $T_{\text{pipe}} = 1400\,\text{ps}$.

Speedup

$T_c^{\text{single}} = 200 + 100 + 200 + 200 + 100 = 800\,\text{ps}$.

$$3\,\text{lw}: \quad \frac{3 \times 800}{7 \times 200} = \frac{2400}{1400} = 1.714$$

$$+1{,}000{,}000: \quad \frac{1{,}000{,}003 \times 800}{1{,}000{,}007 \times 200} = \frac{800{,}002{,}400}{200{,}001{,}400} \approx 3.99998 \approx 4$$

Answer. Speedup $= \mathbf{1.714}$ for the 3 loads, and $\approx \mathbf{4}$ $(= 800/200)$ after adding 1,000,000 loads. For long programs the fill time does not matter, so the speedup approaches $T_c^{\text{single}} / T_c^{\text{pipe}}$. Throughput improves; the latency of each instruction does not.

2013–2014 • Q8(b) CSE 305 • 05/07/2015 • PL28 **MARKS 7**

Consider the following MIPS code:

```
add  $s0, $t0, $t1
sub  $t2, $s0, $t3
lw   $s1, 20($t1)
sub  $t4, $s1, $t2
```

How many data hazards are there? Describe the techniques to eliminate data hazards in the above code.

SOLUTION

Common rules

A **RAW** dependence is a hazard when the consumer is fewer than 3 instructions after the producer. The register file writes in the first half of a cycle and reads in the second half, so a distance of 3 is safe. WAR and WAW dependences cannot cause hazards in the in-order 5-stage MIPS pipeline.

- Distance 1 (ALU producer): forward from **EX/MEM** (Forward = 10).
- Distance 2: forward from **MEM/WB** (Forward = 01).
- Load followed directly by a use: **load-use hazard**. Forwarding cannot go backward in time, so the hazard detection unit inserts 1 stall, and then the value is forwarded from MEM/WB.

Dependence	Register	Distance	Elimination
`add`→`sub` (I1→I2)	`$s0`	1	forward EX/MEM→ALU (ForwardA = 10)
`lw`→`sub` (I3→I4)	`$s1`	1 (load-use)	1 stall + forward MEM/WB, or reschedule
`sub`→`sub` (I2→I4)	`$t2`	2	forward MEM/WB→ALU (ForwardB = 01)

Answer. There are **3 data (RAW) hazards**: `$s0`, `$s1` and `$t2`.

Techniques:

- **Forwarding (bypassing)**: send the ALU result from the EX/MEM or MEM/WB register straight to the ALU inputs, without waiting for WB. This removes the `$s0` and `$t2` hazards.
- **Stall / bubble**: the hazard detection unit sees `ID/EX.MemRead` with `ID/EX.Rt = IF/ID.Rs` (`$s1`). It sets the ID/EX control to 0 and freezes the PC and IF/ID for one cycle. (After the stall, `$t2` arrives via the register file.)
- **Code scheduling** by the compiler: put the load first. Then no stall is needed:

```
lw   $s1, 20($t1)     # independent of add/sub -> move up
add  $s0, $t0, $t1
sub  $t2, $s0, $t3    # $s0 forwarded from EX/MEM
sub  $t4, $s1, $t2    # $t2 from EX/MEM, $s1 via register file (distance 3)
```

This takes 8 cycles instead of 9.

2012–2013 • Q4(b–c) CSE 305 • 10/05/2014 • PL29 **MARKS 13 + 5**

(b) Present a scenario when you cannot solve a data hazard with the help of a forwarding unit. Suggest what to do in this case. (13)

(c) What is a branch hazard? (5)

SOLUTION

(b) Scenario: the load-use data hazard

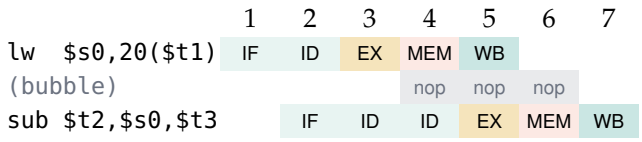
```
lw   $s0, 20($t1)     # $s0 is ready only at the END of MEM (cycle 4)
sub  $t2, $s0, $t3    # needs $s0 at the START of EX (cycle 4)
```

The loaded word leaves data memory at the end of cycle 4, but `sub` needs it as an ALU input at the beginning of cycle 4. A forwarding unit can only pass a value that already exists in a pipeline register (EX/MEM or MEM/WB). It cannot send data *backward in time*, so forwarding alone fails. (For R-type producers the result is ready at the end of EX, which is why forwarding works there.)

What to do

(i) **Stall one cycle (interlock).** A hazard detection unit in ID checks `ID/EX.MemRead and (ID/EX.Rt = IF/ID.Rs or ID/EX.Rt = IF/ID.Rt)`. If true, it (a) forces the ID/EX control fields to 0, so a bubble (`nop`) goes down EX, MEM and WB, and (b) holds the PC and IF/ID (PCWrite = IF/IDWrite = 0), so `sub` is decoded again and the next instruction is fetched again. After the 1-cycle stall, the value is forwarded from MEM/WB to the ALU.

(ii) **Code scheduling** by the compiler: move an independent instruction between the load and its use, so no stall is needed (e.g. the slides' $A = B + E;\ C = B + F$ example drops from 13 to 11 cycles).



`sub` repeats ID in cycle 4. In cycle 5 the loaded value is forwarded from MEM/WB into EX.

(c) Branch (control) hazard

A **branch hazard** happens when the pipeline must fetch the next instruction before it knows the outcome or target of a branch. In the basic MIPS pipeline `beq` is resolved in MEM, so the instructions fetched after it may be the wrong ones.

- Penalty: wrongly fetched instructions must be *flushed* (3 cycles if resolved in MEM; 1 cycle if the comparator and target adder are moved to ID).
- Remedies: stall on branch; predict not taken / static or dynamic (1-bit, 2-bit) prediction with a branch target buffer; delayed branch (always execute the next instruction).

2011–2012 • Q3(d) CSE 305 • 27/07/2013 • PL30 **MARKS 8**

Consider the following MIPS instructions:

(i) `LW $6, 36($1)`

(ii) `ADD $5, $5, $5`

What do these instructions perform in EX and MEM stages. Assume that the pipelined datapath has 5 stages: IF (Instruction Fetch), ID (Instruction decode and register file read), EX (Execution

op address calculation), MEM (Data memory access), and WB (Write Back).

SOLUTION

Stage	`LW $6, 36($1)`	`ADD $5, $5, $5`
EX	Address calculation: ALU computes `$1` + sign-ext(36). ALUSrc = 1 (immediate), ALUOp = 00 (add), RegDst = 0 (destination = rt = `$6`). Address, `$6` number and M/WB control go into EX/MEM.	Execution: ALU computes `$5` + `$5` (i.e. 2×`$5`). ALUSrc = 0, ALUOp = 10 (funct = add), RegDst = 1 (destination = rd = `$5`). Result and `$5` number go into EX/MEM.
MEM	Data-memory read: Mem[`$1`+36] is read (MemRead = 1, MemWrite = 0). The loaded word goes into MEM/WB.	Nothing is done in memory (MemRead = MemWrite = 0, Branch = 0). The ALU result is just passed from EX/MEM to MEM/WB.
WB (for reference)	MemtoReg = 1, RegWrite = 1: `$6` ← loaded word	MemtoReg = 0, RegWrite = 1: `$5` ← ALU result

Answer. In EX, `LW` computes the effective address $\$1 + 36$, while `ADD` does the real arithmetic $\$5 + \$5$. In MEM, `LW` reads data memory at that address. `ADD` is idle in MEM: it only carries its result forward, because every instruction must pass through all 5 stages.

2011–2012 • Q4(a–d) CSE 305 • 27/07/2013 • PL31 **MARKS 6 + 8 + 3 + 4**

(a) What are the values stored in IF/ID and ID/EX pipeline registers? (6)
(b) What is a pipeline hazard? What are the different types of pipeline hazards? (2+6)
(c) Give an example of double data hazard. (3)
(d) Consider the following sequence of instructions:

```
LW  $1, 40($6)
ADD $6, $2, $2
SW  $6, 50($1)
```

Assume that there is no forwarding in this pipelined processor. Estimate data hazards with "nop" instruction. (4)

SOLUTION

NOTE The scan prints "IF/D"; a handwritten correction changes it to IF/ID, which is what is meant. In (d), "Estimate" is circled and corrected by hand to "Eliminate". Both are answered: the hazards are identified and then removed with `nop`s.

(a) Contents of IF/ID and ID/EX

Register	Fields	Size
IF/ID	fetched instruction (32) + PC+4 (32)	64 bits
ID/EX	PC+4, Read data 1, Read data 2, sign-extended immediate (4 × 32 = 128)	128 bits
	+ register numbers Rt, Rd (+ Rs when forwarding is used)	+ 10/15 bits
	+ control: **EX** (RegDst, ALUOp[2], ALUSrc), **M** (Branch, MemRead, MemWrite), **WB** (RegWrite, MemtoReg)	+ 9 bits

(b) Pipeline hazards

A **pipeline hazard** is a situation that prevents the next instruction from starting in the next clock cycle.

- **Structural hazard**: a required resource is busy. Example: with a single memory, a load's data access and a later instruction fetch clash. Fix: separate instruction/data memories (caches).
- **Data hazard**: an instruction needs a result that an earlier instruction has not yet written. Example: `add $s0,$t0,$t1` then `sub $t2,$s0,$t3`. Fix: forwarding, stalls, code scheduling.
- **Control (branch) hazard**: the fetch depends on a branch outcome that is not yet known. Fix: stall, branch prediction, resolving the branch in ID, delayed branch.

(c) Double data hazard

```
add $1, $1, $2
add $1, $1, $3
add $1, $1, $4     # $1 needed: EX/MEM has the newest value, MEM/WB an older one
```

When the third `add` is in EX, both EX/MEM and MEM/WB hold a new value of `$1`. The newest one (from EX/MEM) must be used. So the MEM-hazard forwarding condition is applied only if the EX-hazard condition is false.

(d) No forwarding: hazards and nops

RAW hazards: `SW` reads `$6` written by `ADD` (distance 1) and `$1` written by `LW` (distance 2). `ADD` writing `$6`, which `LW` reads earlier, is WAR, so it is not a hazard. Without forwarding, a consumer's ID (register read, second half of the cycle) must not come before the producer's WB (write, first half). `ADD` writes in CC6, so `SW` may decode at CC6 at the earliest. That needs **2 nops**, and these also cover `$1` (written in CC5).

```
LW  $1, 40($6)
ADD $6, $2, $2
nop
nop
SW  $6, 50($1)
```

	1	2	3	4	5	6	7	8	9
LW $1,40($6)	IF	ID	EX	MEM	WB				
ADD $6,$2,$2		IF	ID	EX	MEM	WB			
nop			nop	nop	nop	nop	nop		
nop				nop	nop	nop	nop	nop	
SW $6,50($1)					IF	ID	EX	MEM	WB

Answer. 2 data hazards ($6: ADD→SW, $1: LW→SW). Inserting 2 nops after ADD removes both; the sequence takes 9 cycles.

2011–2012 • Q4(e) CSE 305 • 27/07/2013 • PL32 **MARKS 7**

Consider the following repeating pattern of branch outcome: T, T, NT, T. What is the accuracy of the two-bit predictor for this sequence of branch outcomes? What is the accuracy of the two-bit predictor in steady state if this pattern is repeated forever? Assume that the 2-bit predictor starts off in the bottom left state (predict not taken) of the figure given below.

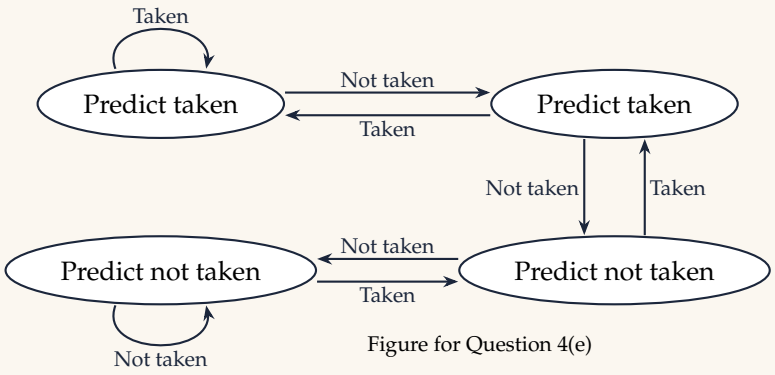
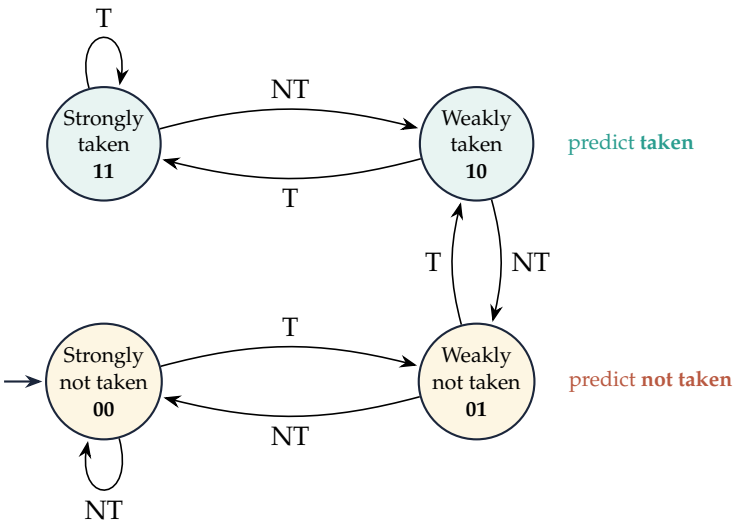


Figure for Question 4(e)

SOLUTION

The 2-bit predictor of the figure (saturating counter)

Each branch-history-table entry is a 2-bit saturating counter. The prediction changes only after **two successive mispredictions**. A loop branch is therefore mispredicted only once per loop (at the exit), not twice as with a 1-bit predictor.



The top two states predict taken, the bottom two predict not taken. T = actual outcome taken (counter $+1$), NT = not taken (counter -1), saturating at 11 and 00.

Simulation of T, T, NT, T

Start in the bottom-left state (strongly not taken, 00).

Pass	Outcome	State before	Prediction	Correct?	State after
1	T	00 (SNT)	NT	×	01 (WNT)
1	T	01 (WNT)	NT	×	10 (WT)
1	NT	10 (WT)	T	×	01 (WNT)
1	T	01 (WNT)	NT	×	10 (WT)
2	T	10 (WT)	T	✓	11 (ST)
2	T	11 (ST)	T	✓	11 (ST)
2	NT	11 (ST)	T	×	10 (WT)
2	T	10 (WT)	T	✓	11 (ST)
3, 4, …	T, T, NT, T	11	T, T, T, T	✓ ✓ × ✓	11

Answer. First pass of the pattern: $0/4$ correct, so the accuracy is **0%**. Steady state: the predictor alternates between 11 and 10 and always predicts taken. Only the NT is mispredicted, so the accuracy is $3/4 = 75\%$.

07

Memory Hierarchy: Caches, Virtual Memory and Storage

CT-4 • Q1(a–c) CSE 209 • Class Test • MH01 **MARKS 12**

A processor has a 32-bit address, a cache of 64 KB, a block size of 64 bytes, and a 4-way set-associative organization.

(a) Determine the number of cache blocks and sets. (4)

(b) Determine the number of bits for the tag, set index, and block offset. (4)

(c) For address `0xABCDEF12`, determine the tag, set index, and block offset. (4)

SOLUTION

(a) Blocks and sets

$$\#\text{blocks} = \frac{64\,\text{KB}}{64\,\text{B}} = \frac{2^{16}}{2^{6}} = 2^{10} = 1024, \qquad \#\text{sets} = \frac{1024}{4\ \text{ways}} = 256 = 2^{8}.$$

Answer. 1024 blocks, organised as 256 sets of 4 blocks each.

(b) Address fields

Block offset $= \log_2 64 = 6$ bits (byte within the block); set index $= \log_2 256 = 8$ bits; tag $= 32 - 8 - 6 = 18$ bits.



Answer. Tag = 18 bits, set index = 8 bits, block offset = 6 bits.

(c) Address `0xABCDEF12`

$$\texttt{0xABCDEF12} = \underbrace{1010\,1011\,1100\,1101\,11}_{\text{tag (31–14)}}\ \underbrace{10\,1111\,00}_{\text{index (13–6)}}\ \underbrace{01\,0010}_{\text{offset (5–0)}}$$



Answer. Tag = `0x2AF37` (175927), set index = `0xBC` = 188, block offset = `0x12` = 18 (byte 18 of the block).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

CT-4 • Q2 CSE 209 • Class Test • MH02 **MARKS 8**

What are the advantages and disadvantages of using a large block? How can the disadvantages be addressed? (4+4)

2023–2024 • Q2(c) CSE 209 • 29/04/2026 • MH02 **MARKS 10**

What are the advantages and disadvantages of larger cache blocks? What do you mean by early restart and critical-word-first? What are the uses of the write buffer in the write-through and write-back schemes?

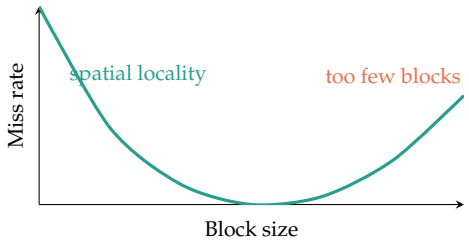
SOLUTION

Advantages of larger blocks

- **Exploit spatial locality:** one miss brings in many neighbouring words, so later accesses to them hit ⇒ lower miss rate (fewer compulsory misses).
- **Less tag overhead:** for the same data capacity there are fewer blocks, so fewer tag/valid bits per byte of data.
- **Efficient transfers:** DRAM delivers a burst of consecutive words cheaply once the row is open.

Disadvantages

- **Fewer blocks** in a fixed-size cache ⇒ more competition for entries; beyond a point the miss rate *rises* again.
- **Pollution:** words that are never used are brought in and displace useful data.
- **Larger miss penalty:** more words must be transferred on each miss; this can cancel the gain from the lower miss rate.



Addressing the disadvantages (CT-4)

- **Early restart:** resume execution as soon as the *requested* word of the block arrives, instead of waiting for the whole block.
- **Critical-word-first (requested word first):** memory sends the requested word first, then the rest of the block (wrapping around); the CPU restarts immediately.
- Reduce the transfer time: wider memory/bus, interleaved banks, DRAM burst mode.
- Keep the number of blocks reasonable: use a larger cache or higher associativity together with the larger block size.

Write buffer (2023–2024 Q2(c))

- **Write-through:** every store must also update memory. The write buffer holds data waiting to be written to memory, so the CPU continues immediately; it stalls only if the buffer is full.
- **Write-back:** when a *dirty* block is replaced it must be written back. The victim block is copied into the write buffer, so the new block can be read from memory first; the buffer writes the old block back in the background.

2023–2024 • Q2(a) CSE 209 • 29/04/2026 • MH03 **MARKS 10**

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache, and a clock rate of 4 GHz. The main memory access time is 200 ns, including all misshandling overhead. The miss rate per instruction in the primary cache is 5%.

(i) How much faster will the processor be if we add a second-level cache with an access time of 10 ns (for both hits and misses), assuming it reduces the miss rate to main memory to 2%?

(ii) How much faster will the processor be if we further add a third-level cache with an access time of 40 ns (for both hits and misses), assuming it reduces the miss rate to main memory to 1%?

SOLUTION

Method (slides' multilevel-cache example)

Penalty in cycles = time / cycle time. Every L1 miss pays the L2 access; only references that also miss L2 (global miss rate) pay the next level: $\text{CPI} = \text{CPI}_{\text{base}} + m_{L1}t_{L2} + m_{L2,\text{global}}t_{L3} + m_{\text{to mem}}t_{\text{mem}}$; speed-up $= \text{CPI}_{\text{old}}/\text{CPI}_{\text{new}}$.

Penalties

Cycle $= 1/4\,\text{GHz} = 0.25\,\text{ns}$: memory $= 200/0.25 = 800$ cycles, L2 $= 10/0.25 = 40$ cycles, L3 $= 40/0.25 = 160$ cycles.

$$\begin{aligned} \text{L1 only:} &\quad 1 + 0.05 \times 800 = 41 \\ \text{L1+L2:} &\quad 1 + 0.05 \times 40 + 0.02 \times 800 = 1 + 2 + 16 = 19 \\ \text{L1+L2+L3:} &\quad 1 + 0.05 \times 40 + 0.02 \times 160 + 0.01 \times 800 = 1 + 2 + 3.2 + 8 = 14.2 \end{aligned}$$

Answer. (i) $41/19 = 2.16\times$ faster with L2. (ii) $19/14.2 = 1.34\times$ faster again with L3 ($41/14.2 = 2.89\times$ faster than L1 only).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q2(b) CSE 209 • 29/04/2026 • MH04 **MARKS 15**

For a direct-mapped cache design with a 32-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
31–10	9–5	4–0

Starting from power on, the following byte-addressed cache references are recorded.

0	4	16	132	232	160	1024	30	140	3100	180	2180

(i) What is the cache block size (in words)?

(ii) How many entries does the cache have?

(iii) How many blocks are replaced?

(iv) What is the hit ratio?

(v) List the final state of the cache, with each valid entry represented as ⟨index, tag, data⟩.

2015–2016 • Q4(b) CSE 305 • 18/01/2017 • MH04 **MARKS 10**

For a direct-mapped cache design with a 32-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
31–10	9–5	4–0

What is the cache block size (in words) and how many entries does the cache have? Starting from power on, the following byte-addressed cache references are recorded.

0, 4, 16, 132, 232, 160, 1024, 30, 140, 3100, 180, 2180

How many blocks are replaced and what is the hit ratio?

SOLUTION

Both cards use the same cache and the same trace; the 2023–2024 card additionally asks for the final state (v).



(i), (ii) Block size and number of entries

Offset $= 5$ bits $\Rightarrow$ block $= 2^5 = 32$ bytes $= 8$ words. Index $= 5$ bits $\Rightarrow 2^5 = 32$ entries (lines).

Answer. Block size $= 8$ words (32 bytes); the cache has 32 entries.

Trace

Block address $= \lfloor A/32 \rfloor$, index $=$ block address mod 32, tag $= \lfloor A/1024 \rfloor$.

Address	Bits 11–10 | 9–5 | 4–0	Block	Index	Tag	H/M	Action
0	00 00000 00000	0	0	0	×	load block 0 (bytes 0–31)
4	00 00000 00100	0	0	0	✓	
16	00 00000 10000	0	0	0	✓	
132	00 00100 00100	4	4	0	×	load bytes 128–159
232	00 00111 01000	7	7	0	×	load bytes 224–255
160	00 00101 00000	5	5	0	×	load bytes 160–191
1024	01 00000 00000	32	0	1	×	**replace** index 0 (tag 0 → 1)
30	00 00000 11110	0	0	0	×	**replace** index 0 (tag 1 → 0)
140	00 00100 01100	4	4	0	✓	
3100	11 00000 11100	96	0	3	×	**replace** index 0 (tag 0 → 3)
180	00 00101 10100	5	5	0	✓	
2180	10 00100 00100	68	4	2	×	**replace** index 4 (tag 0 → 2)

(iii), (iv) Replacements and hit ratio

Answer. 4 blocks are replaced (by 1024, 30, 3100 and 2180). Hits $= 4$ (4, 16, 140, 180) out of 12 references $\Rightarrow$ hit ratio $= 4/12 = 33.3\%$.

(v) Final state (2023–2024 card)

Index	Valid	Tag	Data
0	1	3	Mem[3072 … 3103]
4	1	2	Mem[2176 … 2207]
5	1	0	Mem[160 … 191]
7	1	0	Mem[224 … 255]
all other indices invalid			

Answer. $\langle 0, 3, \text{Mem}[3072..3103]\rangle$, $\langle 4, 2, \text{Mem}[2176..2207]\rangle$, $\langle 5, 0, \text{Mem}[160..191]\rangle$, $\langle 7, 0, \text{Mem}[224..255]\rangle$.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q3(a) CSE 209 • 29/04/2026 • MH05 **MARKS 10**

Although both are the TLB and data cache are types of cache memory, explain how they differ. Illustrate, with a flow diagram, the interaction between the TLB and data cache during both read and write operations.

2018–2019 • Q3(a) CSE 305 • 29/10/2019 • MH05 **MARKS 8**

With the help of a flowchart show how a real read or write request is processed in a system with a TLB interacting with an appropriate cache and virtual memory.

2017–2018 • Q4(d) CSE 305 • 23/09/2018 • MH05 **MARKS 5**

Explain (with flow chart) the interaction between TLB and data cache for only write operation.

2015–2016 • Q3(d) CSE 305 • 18/01/2017 • MH05 **MARKS 10**

Though both TLB and data cache are cache memory, how are they different from each other? Explain with flow chart the interaction between TLB and data cache for both read and write operation.

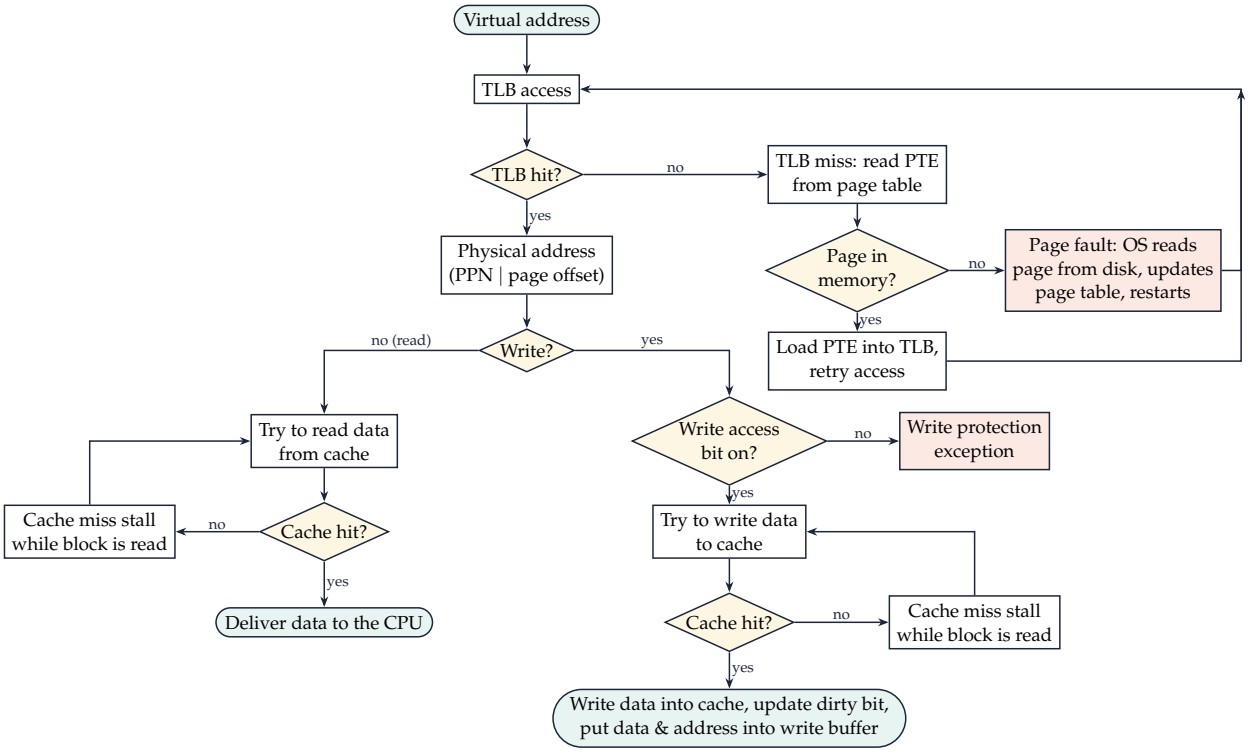
SOLUTION

TLB vs data cache (2023–2024 and 2015–2016 cards)

	TLB	Data cache
What it caches	Page-table entries: VPN → PPN translation plus valid, dirty, reference, protection bits	Copies of memory blocks (the program's data)
Looked up with	Virtual page number (tag = VPN or part of it)	Physical address (in a physically indexed/tagged cache)
Size / organisation	Very small (16–512 entries), usually fully or highly associative	Large (KB–MB), direct-mapped or set-associative
Unit	One PTE (maps a whole page, e.g. 4 KB)	One block of a few words
Miss handling	Read PTE from the page table in memory (hardware walker or OS exception); may turn into a page fault	Stall and fetch the block from L2/memory (pure hardware)
Order of use	First: produces the physical address	Second: accessed with that physical address

Flowchart of a read / write request (all four cards)

The TLB translates first; the physically addressed cache is accessed next. The right-hand branch (page table, page fault) is the virtual-memory part asked for in the 2018–2019 card; the 2017–2018 card needs only the *write* branch (right half below "Write?").



- **Read:** TLB hit gives the physical address → read the cache → on a hit deliver the word; on a miss stall, fetch the block and retry.
- **Write:** after translation the write-access (protection) bit is checked; then the cache is written. In a write-through cache the data and address also go to the write buffer; in a write-back cache only the dirty bit is set.
- **TLB miss:** the PTE is fetched from the page table; if the page is not in memory a page fault is raised and the OS brings the page from disk, then the instruction is restarted.

2023–2024 • Q3(b) CSE 209 • 29/04/2026 • MH06 **MARKS 15**

Virtual memory uses a page table to map virtual addresses to physical addresses. The following data constitutes a stream of virtual addresses as seen on a system. Assume 4 KB pages, a 4-entry fully associative TLB, and LRU replacement. If pages must be brought in from disk, increment the next largest page number (for example, the new pages will be assigned page numbers 13, 14, 15, and so on). The virtual page number is computed as the floor of the virtual address divided by the page size in bytes.

4669	2227	13916	34587	48870	12608	49225

TLB

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
0	4	9

Page Table

Valid	Physical Page or in Disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

Given the address stream shown and the initial TLB and page table states provided above,

(i) List for each reference if it is a hit in the TLB, a hit in the page table, or a page fault.

(ii) Show the final state of the page table and TLB.

SOLUTION

Page size $= 4096$ B $\Rightarrow$ VPN $= \lfloor \mathrm{VA}/4096 \rfloor$. The page-table rows are VPN 0–11 (top to bottom). In a fully associative TLB the tag is the whole VPN.

NOTE The recency order of the three initially valid TLB entries is not given. Assumed: the invalid entry is filled first, and the valid entries were last used in table order (tag 11 least recently used, then 7, then 3). VPN 12 lies beyond the 12 rows shown, so it is treated as not resident (page fault).

(i) Reference-by-reference

VA	VPN	Offset	Result	Action	TLB after (LRU → MRU)
4669	1	573	× TLB miss, **page fault**	page → PPN 13; fill invalid entry	11, 7, 3, **1**
2227	0	2227	× TLB miss, **PT hit** (PPN 5)	replace LRU tag 11	7, 3, 1, **0**
13916	3	1628	✓ **TLB hit** (PPN 6)	–	7, 1, 0, **3**
34587	8	1819	× TLB miss, **page fault**	page → PPN 14; replace tag 7	1, 0, 3, **8**
48870	11	3814	× TLB miss, **PT hit** (PPN 12)	replace tag 1	0, 3, 8, **11**
12608	3	320	✓ **TLB hit** (PPN 6)	–	0, 8, 11, **3**
49225	12	73	× TLB miss, **page fault**	page → PPN 15; replace tag 0	8, 11, 3, **12**

Answer. TLB hits: 13916, 12608. Page-table hits: 2227, 48870. Page faults: 4669, 34587, 49225.

(ii) Final state

TLB

Entry	Valid	Tag	PPN
0	1	**12**	**15**
1	1	**8**	**14**
2	1	3	6
3	1	**11**	**12**

Page table

VPN	Valid	PPN / Disk
0	1	5
1	**1**	**13**
2	0	Disk
3	1	6
4	1	9
5	1	11
6	0	Disk
7	1	4
8	**1**	**14**
9	0	Disk
10	1	3
11	1	12
12	**1**	**15**

(Entry 0 held tag 11, then tag 0, finally tag 12; entry 3 was the invalid entry, then tag 1, finally tag 11. Bold = changed.)

Answer. Final TLB: {V=1, tag 12, PPN 15}, {1, 8, 14}, {1, 3, 6}, {1, 11, 12}. Final page table: as initially, plus VPN 1 → 13, VPN 8 → 14, VPN 12 → 15 (all valid).

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q3(c) CSE 209 • 29/04/2026 • MH07 MARKS 10

What do you mean by cache coherence problem? Explain how snooping protocol solves the cache coherence problem with an example.

2021–2022 • Q3(d) CSE 305 • 06/04/2023 • MH07 MARKS 5

Explain how snooping protocol solves the cache coherence problem with an example.

2015–2016 • Q8(a) CSE 305 • 18/01/2017 • MH07 MARKS 10

What are the different methods to maintain coherence among all caches and global memory in a shared memory system? Describe briefly.

2013–2014 • Q1(c) CSE 305 • 05/07/2015 • MH07 MARKS 9

State three conditions required for a cache/memory to be coherent.

SOLUTION

The cache coherence problem

When cores share one physical address space but each has a private cache, two caches can hold different values for the same address. Example (write-through caches, $X = 0$ in memory):

Step	Event	CPU A's cache	CPU B's cache	Memory
0	–			0
1	CPU A reads X	0		0
2	CPU B reads X	0	0	0
3	CPU A writes 1 to X	1	**0 (stale)**	1

After step 3, CPU B would read the old value 0 ⇒ the memory system is *incoherent*.

Conditions for coherence (2013–2014 card)

(i) **Program order:** if P writes X and then P reads X with no intervening writes by others, the read returns the value P wrote.

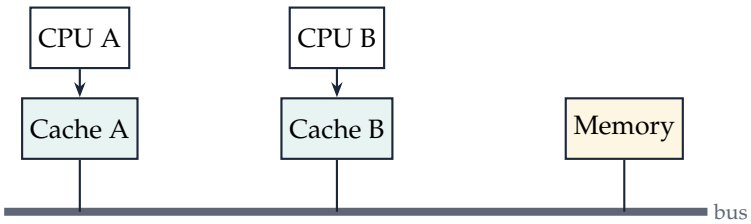
(ii) **Write propagation:** if P1 writes X and P2 reads X sufficiently later (no other writes in between), P2's read returns the value written by P1 (B reading X after step 3 must get 1).

(iii) **Write serialization:** two writes to the same location by any two processors are seen in the same order by all processors, so all end with the same final value.

Methods to maintain coherence (2015–2016 card)

- **Snooping protocols:** every cache controller monitors (snoops) the shared bus; coherence is kept by observing all bus transactions. Two variants: *write-invalidate* (a writer broadcasts an invalidate so other copies are discarded) and *write-update/broadcast* (the new value is broadcast and all copies are updated). Best for small systems with a shared bus.
- **Directory-based protocols:** a directory records, for every memory block, which caches hold a copy and in what state; a read/write consults the directory, which sends point-to-point invalidations/updates only to the sharers. Scales to large systems (no bus broadcast).
- The hardware supports *migration* (data moved to the local cache) and *replication* (read-shared copies in several caches) while keeping them coherent.

How snooping (write-invalidate) solves it — example



snoop: every controller watches every bus transaction

	CPU activity	Bus activity	A's cache	B's cache	Memory
0					0
1	CPU A reads X	Cache miss for X	0		0
2	CPU B reads X	Cache miss for X	0	0	0
3	CPU A writes 1 to X	**Invalidate for X**	1	**(invalid)**	0
4	CPU B reads X	Cache miss for X	1	1	1

- Before writing, A gets *exclusive* access: it broadcasts "invalidate X". B's controller, snooping the bus, sees the address and invalidates its copy.
- B's next read of X misses; A's cache (the owner of the dirty block) snoops this miss and supplies the up-to-date value 1, and memory is updated.

- Hence every read returns the most recently written value and writes to X are serialized on the bus.

2022–2023 • Q9(a) CSE 209 • 13/01/2025 • MH08 **MARKS 15**

Assume there are two processors, P1 and P2 where P1 has two levels of cache (L1 and L2) and P2 has three levels of cache (L1, L2 and L3). The DRAM memory access time is 100 ns, and memory accesses constitute 30% of all instructions. Additionally, the L2 hit time is twenty times the cycle times for both P1 and P2. The base CPI for both P1 and P2 is 1.

L1 Cache Data:

Processor	L1 Size	L1 Miss Rate	L1 Hit Time
P1	16 KB	4%	1.16 ns
P2	32 KB	3%	2.04 ns

L2 Cache Data:

Processor	L2 Size	L2 Miss Rate	L2 Hit Time
P1	16 MB	65%	27.5 ns
P2	8 MB	50%	30.5 ns

L3 Cache Data:

L3 Size	L3 Global Miss Rate	L3 Hit Time
64 MB	1%	50.7 ns

(i) Determine the faster processor between P1 and P2. Justify your answer by determining the effective CPI for each processor.

(ii) If P1 is faster, determine the miss rate P2 would need in its L2 cache to match P1's performance. If P2 is faster, determine the miss rate P1 would need in its L2 cache to match P2's performance.

SOLUTION

NOTE Assumptions: "memory accesses constitute 30% of instructions" is taken as 0.3 cache accesses per instruction that can miss (the base CPI of 1 already contains L1 hits). The L1/L2 miss rates in the tables are *local* rates; the L3 rate is stated to be *global* (per memory access).

Cycle times and penalties in cycles

L2 hit time = 20 cycles $\Rightarrow t_{c,P1} = 27.5/20 = 1.375\,\text{ns},\ t_{c,P2} = 30.5/20 = 1.525\,\text{ns}.$

	P1	P2
L2 access	20 cycles	20 cycles
L3 access	–	$50.7/1.525 = 33.25$ cycles
DRAM access	$100/1.375 = 72.73$ cycles	$100/1.525 = 65.57$ cycles

(i) Effective CPI

Stall cycles per memory access:

$$\text{P1: } 0.04 \times \left(20 + 0.65 \times 72.73\right) = 0.04 \times 67.27 = 2.691$$

$$\text{P2: } 0.03 \times 20 + \underbrace{0.03 \times 0.50}_{\text{L2 global}=1.5\%} \times 33.25 + 0.01 \times 65.57$$

$$= 0.600 + 0.499 + 0.656 = 1.754$$

$$\text{CPI}_{P1} = 1 + 0.3 \times 2.691 = 1.807, \qquad \text{CPI}_{P2} = 1 + 0.3 \times 1.754 = 1.526$$

Time per instruction: P1 $= 1.807 \times 1.375 = 2.485$ ns; P2 $= 1.526 \times 1.525 = 2.328$ ns.

Answer. P2 is faster: effective CPI 1.53 vs 1.81, and 2.328 ns vs 2.485 ns per instruction ($\approx 1.07\times$ faster) even though its clock is slower.

(ii) L2 miss rate P1 needs to match P2

Require $T_{P1} = T_{P2}$:

$$1.375\left[1 + 0.3 \times 0.04\,(20 + 72.73\,m)\right] = 2.328 \;\Rightarrow\; 1 + 0.012\,(20 + 72.73\,m) = 1.693$$

$$\Rightarrow\; m = \frac{0.693/0.012 - 20}{72.73} = 0.519$$

Answer. P1's L2 (local) miss rate must drop from 65% to about **51.9%**. (If only the CPIs were matched, $1 + 0.012(20 + 72.73m) = 1.526$ gives $m \approx 32.8\%$; execution time is the proper measure because the clock periods differ.)

2022–2023 • Q9(b) CSE 209 • 13/01/2025 • MH09 **MARKS 10**

Determine the total size of a 4-way set-associative cache with the following specifications. Assume the cache has tag bits, a valid bit, a dirty bit, a protection bit, and the data bits.

- Address length: 48 bits
- Cache (Data only) size: 64 KB
- Block size: 16 words
- The cache has hardware support for the instruction `lb` (load byte).

SOLUTION

Method

Block offset $= \log_2(\text{block bytes})$; sets $=$ blocks / ways; index $= \log_2(\text{sets})$; tag $=$ address $-$ index $-$ offset; total $=$ blocks $\times$ (data $+$ tag $+$ status bits).

Fields

Block $= 16 \times 4 = 64$ B $\Rightarrow$ offset $= 6$ bits (`lb` support means single bytes are addressed, so the 2-bit byte offset is kept). Blocks $= 2^{16}/2^6 = 1024$; sets $= 1024/4 = 256 \Rightarrow$ index $= 8$; tag $= 48 - 8 - 6 = 34$ bits.



$$\text{bits per line} = 34\,(\text{tag}) + 1\,(\text{V}) + 1\,(\text{D}) + 1\,(\text{prot.}) + 512\,(\text{data}) = 549$$

Answer. Total $= 1024 \times 549 = 562{,}176$ bits $= 70{,}272$ bytes ≈ 68.6 KB (of which 64 KB is data).

2022–2023 • Q10(a) CSE 209 • 13/01/2025 • MH10 **MARKS 15**

Consider a memory system where the virtual address is 38 bits long, the physical DRAM installed is 8 GB, the page size is 16 KB, and the PTE size is 4 bytes. The computer system where this memory system is used has 75 processes running. The PID of each process is 8 bits long.

(i) For a single-level page table, calculate how much physical memory is needed for storing the page tables.

(ii) For a three-level page table where each level has an equal length of bits, calculate how much physical memory is needed for storing the page tables in the best case, average case, and worst case scenario.

(iii) For an inverted page table, calculate how much physical memory is needed for storing the page table. An extra 4 bytes of information besides the required ones is stored in the PTE.

SOLUTION

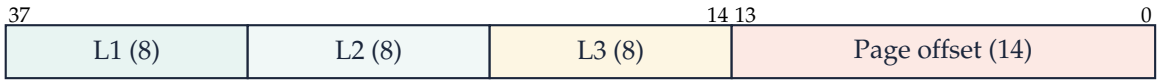
Address fields

Page offset $= \log_2 16\,\text{K} = 14$ bits $\Rightarrow$ VPN $= 38 - 14 = 24$ bits. PA $= \log_2 8\,\text{GB} = 33$ bits $\Rightarrow$ $2^{33}/2^{14} = 2^{19}$ physical pages. A forward page table has one PTE per *virtual* page of each process; an inverted table has one entry per *physical* page.

(i) Single level: $2^{24} \times 4\,\text{B} = 64\,\text{MB}$ per process; $75 \times 64 = 4800\,\text{MB}$.

Answer. 4800 MB $\approx$ 4.69 GB for the 75 page tables.

(ii) Three levels, 8 bits each:



Each table has 2^8 entries $\times 4\,\text{B} = 1\,\text{KB}$.

Case	Tables per process (L1 + L2 + L3)	Per process	75 processes
Best (one page path used)	$1+1+1=3$	3 KB	225 KB
Average (half the VA space mapped)	$1+128+32768=32{,}897$	$\approx$ 32.1 MB	$\approx$ 2.35 GB
Worst (whole VA space mapped)	$1+256+65536=65{,}793$	$\approx$ 64.25 MB	$\approx$ 4.71 GB

NOTE "Average case" is not defined in the question; it is taken here as a process that uses half of its virtual address space.

(iii) Inverted page table: one entry per physical page $= 2^{19}$ entries. Each entry must hold the VPN (24 bits) + PID (8 bits) $= 4\,\text{B}$, plus the extra $4\,\text{B} \Rightarrow 8\,\text{B}$ per entry.

Answer. $2^{19} \times 8\,\text{B} = 4\,\text{MB}$, a single table shared by all 75 processes.

2022–2023 • Q10(b) CSE 209 • 13/01/2025 • MH11 **MARKS 10**

Draw the block diagram of TLB and data cache interaction with necessary labels in a memory system with the following specifications.

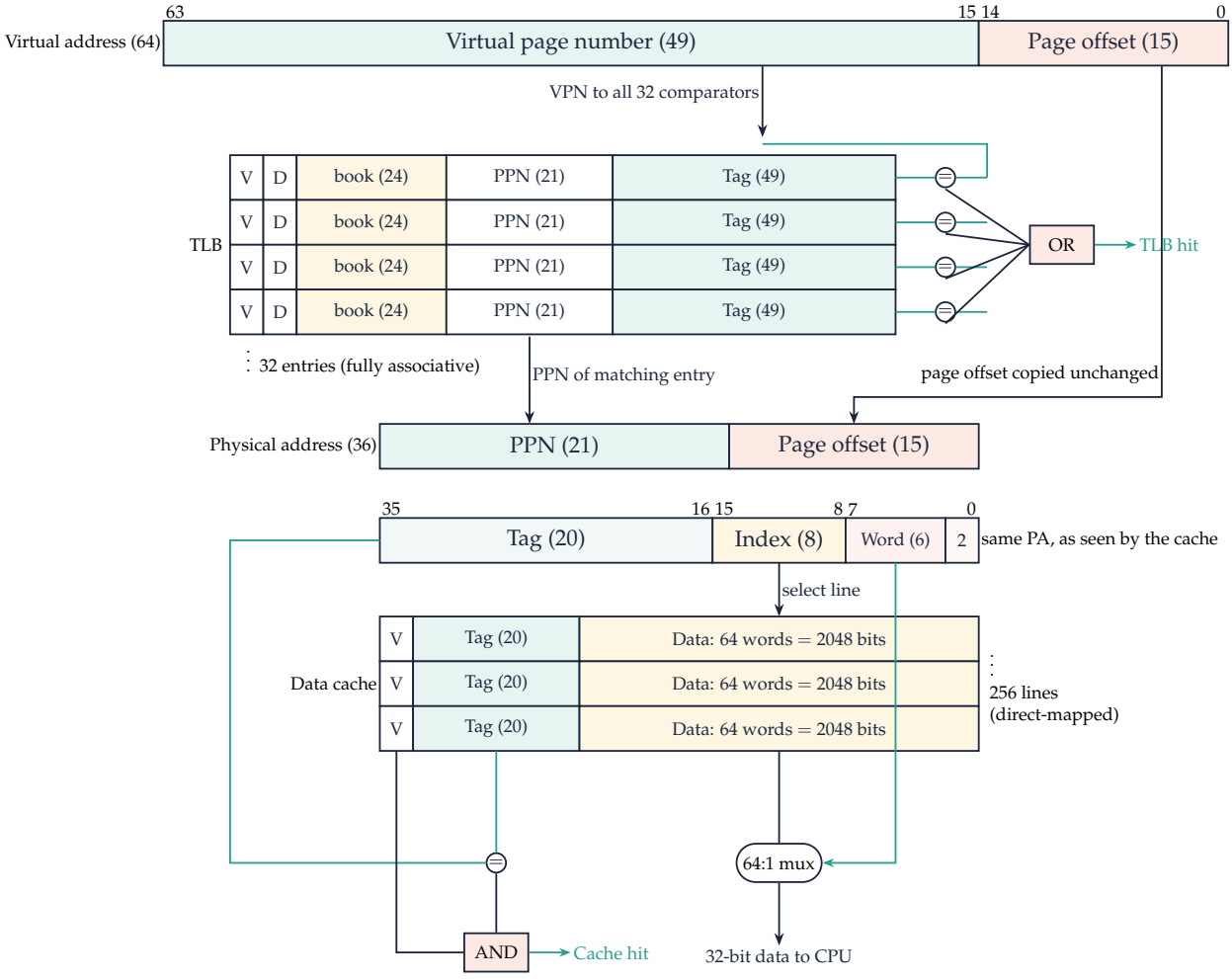
- Page size is 32 KB, and the virtual address is 64 bits long.
- The physical DRAM size is 64 GB.
- TLB cache has 32 entries and is fully associative. The cache has tag bits, a valid bit, a dirty bit, 24 bookkeeping bits, and the data bits.
- The data cache is 64 KB and direct-mapped. It has 64 words/blocks. It is a physically indexed, physically tagged cache.

SOLUTION

Field widths

- Page offset $= \log_2 32\,\text{K} = 15$ bits; VPN $= 64 - 15 = 49$ bits.
- PA $= \log_2 64\,\text{GB} = 36$ bits; PPN $= 36 - 15 = 21$ bits.
- TLB (fully associative, 32 entries): tag = whole VPN = 49 bits; each entry = 1 (V) +1 (D) +24 (bookkeeping) +49 (tag) +21 (PPN, the "data") = 96 bits $\Rightarrow 32 \times 96 = 3072$ bits; 32 parallel comparators.
- Data cache: block $= 64 \times 4 = 256$ B $\Rightarrow$ offset = 8 (6 word + 2 byte); lines $= 64\,\text{KB}/256 = 256$ $\Rightarrow$ index = 8; tag $= 36 - 8 - 8 = 20$ bits. Each line: V + 20-bit tag + 2048 data bits.

Block diagram



- The 49-bit VPN is compared simultaneously with all 32 TLB tags; on a TLB hit the matching entry supplies the 21-bit PPN (its dirty/bookkeeping bits are updated). A TLB miss is handled by reading the PTE from the page table.
- The PPN concatenated with the unchanged 15-bit page offset forms the 36-bit physical address.
- PIPT cache: PA bits 15–8 select one of 256 lines, PA bits 35–16 are compared with the stored tag (AND valid) $\Rightarrow$ cache hit; bits 7–2 drive the 64:1 multiplexer that selects the requested word.

Answer. VPN 49 / offset 15; PPN 21; TLB entry 96 bits ($32 \times 96 = 3072$ bits); cache split tag 20 | index 8 | offset 8, 256 lines of $1 + 20 + 2048$ bits (529,664 bits ≈ 64.7 KB).

2022–2023 • Q10(c) CSE 209 • 13/01/2025 • MH12 **MARKS 10**

Why must the operating system clear the TLB and data cache during a context switch? How does the operating system address these issues to avoid clearing the caches?

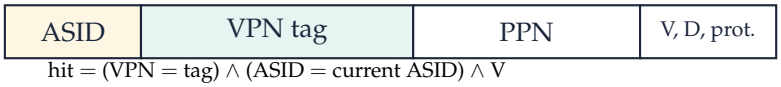
SOLUTION

Why flushing is needed

- Every process has its *own* virtual address space: the same virtual page number (e.g. VPN `0x400`) maps to different physical pages in process A and process B.
- TLB entries are tagged only with the VPN. After switching from A to B, a stale entry of A would **hit** for B's virtual address and return A's physical page ⇒ B reads/writes A's memory: wrong results and a protection violation.
- The same holds for a *virtually* addressed (virtually tagged) data cache: a block tagged with A's virtual address would hit for B's identical virtual address (homonym problem).
- Hence, without extra hardware, the OS must invalidate the TLB (and a virtual cache) on every switch — causing a burst of TLB/cache misses each time the process resumes.

How it is avoided

- **Address-space identifier (ASID / PID) in each entry:** the OS assigns each process an ASID and loads it into a CPU register on a context switch. A TLB (or virtual-cache) hit now requires *tag match AND ASID match*, so entries of several processes coexist and nothing has to be flushed.



$\text{hit} = (\text{VPN} = \text{tag}) \wedge (\text{ASID} = \text{current ASID}) \wedge \text{V}$

- **Global bit** for kernel/shared pages: such entries match for every ASID and are never flushed.
- **Physically indexed, physically tagged caches** (as in Q10(b)) need no flushing at all: physical addresses are unique across processes. Only the TLB needs the ASID.
- The OS flushes only when ASIDs run out and must be recycled, or when it changes a mapping (e.g. a page is moved to disk), in which case only the affected entries are invalidated.

2022–2023 • Q12(d) CSE 209 • 13/01/2025 • MH13 **MARKS 10**

Consider Figure 12.1 and Figure 12.2, which show the comparison of Quicksort and Radix Sort in terms of instructions and clock cycles. As expected, for large arrays, Radix Sort has an algorithmic advantage over Quicksort regarding the number of operations, as shown in Figure 12.1. Figure 12.2 shows clock cycles taken instead of instructions executed. We see that the lines start on the same trajectory as in Figure 12.1, but then the Radix Sort line diverges as the data to sort increases. Explain briefly why Figure 12.2 is not following Figure 12.1.

Figure 12.1: Instructions executed: Radix Sort vs Quicksort

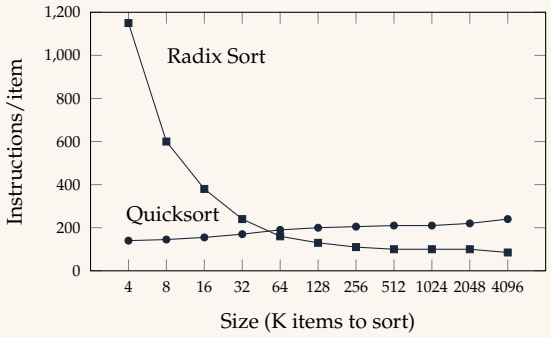
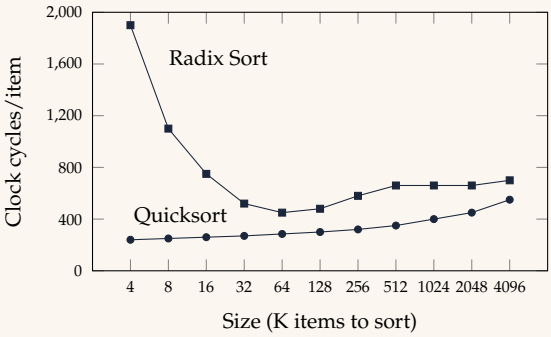


Figure 12.2: Clock Cycles: Radix Sort vs Quicksort



(plots redrawn approximately from the question paper)

SOLUTION

Reason: memory-hierarchy (cache) behaviour

- Clock cycles = instructions × CPI, and CPI = base CPI + memory-stall cycles per instruction. Figure 12.1 counts only instructions; Figure 12.2 also includes the time lost to cache misses.
- For small arrays the data fit in the cache, stalls are rare, so both figures have the same shape.
- **Radix sort has poor locality:** each pass reads the input and scatters every key into one of many buckets/counters spread over memory, so successive accesses jump to unrelated blocks. Once the array (and buckets) no longer fit in the cache, almost every access misses ⇒ cycles per item stop falling and rise (to ≈ 600–700), although instructions per item keep falling.
- **Quicksort has good locality:** partitioning sweeps sequentially through a sub-array (spatial locality) and then recursively works on ever smaller sub-arrays that fit in the cache (temporal locality), so its miss rate stays low and its cycle curve follows its instruction curve.
- Lesson (slides, "Interactions with Software"): misses depend on the memory-access pattern of the algorithm; an algorithm with fewer instructions can be slower if it ignores the memory hierarchy.

2022–2023 • Q2(a) CSE 305 • 25/04/2024 • MH14 MARKS 20

Assume that there are two small caches, each consisting of eight (8) blocks. One cache is direct-mapped, and the other is a two-way set-associative cache. For each cache organization.

(i) Construct the following table for each cache organization for the following sequence of memory block addresses: 32, 36, 35, 32, 48, 32, 51 and 35. Note that you are allowed to subdivide the column "Cache content after access" as needed. Also, when it is necessary to replace an existing cache entry on a cache miss, follow the least recently used (LRU) replacement policy.

Address of Memory Block Accessed	Cache Index	Hit or Miss	Cache content after access

Table 1: Table for Question 2(a)

(ii) Compute the hit ratio from the constructed tables in (i) for each cache organization.

SOLUTION

Method (slides: "Associativity Example")

Direct-mapped: index = block address mod 8. Two-way: 4 sets, set = block address mod 4; on a miss fill an empty way, else replace the **LRU** block of the set. Cells give the block number held in each slot after the access (✓ hit, × miss; Sk = a way of set k).

Direct-mapped

Blk	Idx	H/M	0	1	2	3	4	5	6	7
32	0	×	32							
36	4	×	32				36			
35	3	×	32			35	36			
32	0	✓	32			35	36			
48	0	×	48			35	36			
32	0	×	32			35	36			
51	3	×	32			51	36			
35	3	×	32			35	36			

misses 7, hits 1

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1	S2	S2	S3	S3
32	0	×	32							
36	0	×	32	36						
35	3	×	32	36					35	
32	0	✓	32	36					35	
48	0	×	32	48					35	
32	0	✓	32	48					35	
51	3	×	32	48					35	51
35	3	✓	32	48					35	51

misses 5, hits 3

Answer. (ii) Hit ratio: direct-mapped = $1/8 = 12.5\%$; two-way set associative = $3/8 = 37.5\%$ (48 replaces the LRU block 36, so 32 and 35 survive).

2022–2023 • Q2(b) CSE 305 • 25/04/2024 • MH15 MARKS 15

How many total bits are required for an **eight-way set associative** cache of 4K blocks, a 4-word block size (i.e., 16 bytes), assuming a 32-bit address? [Show set number and tag bit calculations.]

SOLUTION

Sets and tag

Offset = $\log_2 16 = 4$ bits. Sets = $\dfrac{4096}{8} = 512 = 2^9 \Rightarrow$ index = 9 bits. Tag = $32 - 9 - 4 = 19$ bits.



Total bits

Per block: 128 data +19 tag +1 valid = 148 bits.

$$\text{Total} = 4096 \times 148 = 606{,}208 \text{ bits}; \qquad \text{tag bits alone} = 19 \times 4096 = 77{,}824$$

Answer. 512 sets, 19-bit tag; total = 606,208 bits (≈ 74 KB, of which 64 KB is data).

2022–2023 • Q3(a) CSE 305 • 25/04/2024 • MH16 **MARKS 25**

Suppose we have a processor with a base CPI of 2.0, assuming all references hit in the primary cache, and a clock cycle length of 0.5 ns. Assume a main memory access time of 150 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 5%.

(i) Calculate the total CPI for one-level of caching. How much faster the ideal CPU is?

(ii) Now assume that we add a secondary cache that has a 15 ns access time for either a hit or a miss and is large enough to reduce the miss rate to 0.5%. The miss rate for primary cache remains 5%. Calculate the total CPI for two-level of caching. [Note that this secondary cache is accessed when there is a miss in the primary cache]

(iii) How much faster will the processor be with a two-level of caching compared to the processor with just one-level of caching?

SOLUTION

Method (slides' multilevel-cache example)

Penalty in cycles = time / cycle time. Every L1 miss pays the L2 access time; only the misses to main memory (0.5% of instructions) also pay the memory time: $\text{CPI} = \text{CPI}_{\text{base}} + m_{L1}\, t_{L2} + m_{\text{to mem}}\, t_{\text{mem}}$.

Penalties

Memory $= 150/0.5 = 300$ cycles; L2 $= 15/0.5 = 30$ cycles.

(i) One level: $\text{CPI} = 2 + 0.05 \times 300 = 2 + 15 = 17$. The ideal CPU (CPI = 2) is $17/2 = 8.5\times$ faster.

(ii) Two levels: $\text{CPI} = 2 + 0.05 \times 30 + 0.005 \times 300 = 2 + 1.5 + 1.5 = 5.0$.

(iii) Speed-up $= 17/5.0 = 3.4$.

Answer. $\text{CPI}_{\text{1-level}} = 17$ (the ideal CPU is $8.5\times$ faster); $\text{CPI}_{\text{2-level}} = 5.0$; the two-level system is $3.4\times$ faster.

2022–2023 • Q3(b) CSE 305 • 25/04/2024 • MH17 **MARKS 5**

What is difference between a virtual address and a physical address of instructions of a program? What is the purpose of the virtual addressing of instructions of a program?

SOLUTION

	Virtual address	**Physical address**
Generated by	The program / CPU (PC, load/store effective address)	The address translation hardware (TLB / page table)
Space	Private to each process, can be larger than DRAM (e.g. 2^{32} or 2^{64} bytes)	One shared space, limited to the installed DRAM
Form	Virtual page number | page offset	Physical page number | page offset (same offset)
Used by	Compiler, linker, the running program	Caches (PIPT) and main memory

Purpose of virtual addressing

- **Relocation:** a program is compiled/linked for a fixed virtual layout (code at the same virtual address every run) and can be loaded into *any* free physical pages, not necessarily contiguous.
- **Sharing and protection:** several programs share main memory, each in its own protected

address space; a program cannot touch another's pages (protection bits, supervisor-only page tables).

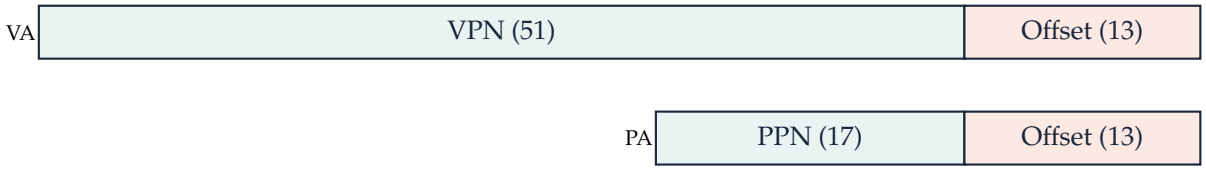
- **Larger-than-memory programs:** main memory acts as a cache of the disk; only the pages in use need to be resident (page faults bring in the rest).

2022–2023 • Q3(c) CSE 305 • 25/04/2024 • MH18 **MARKS 5**

Consider a mapping of a 64-bit virtual address and to a 30-bit physical address. How many entries are there in the page table if the page size is 8 KB.

SOLUTION

The page table has one entry per virtual page. Page offset $= \log_2 8\,\text{K} = 13$ bits $\Rightarrow$ VPN $= 64 - 13 = 51$ bits.



Answer. $2^{51} \approx 2.25 \times 10^{15}$ page-table entries (the 30-bit PA only fixes the PPN width, $30 - 13 = 17$ bits per entry).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q3(a) CSE 305 • 06/04/2023 • MH19 **MARKS 10**

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache and a clock rate of 4 GHz. Assume a main memory access time of 200 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 4%. How much faster will the processor be if we add a second level cache that has a 10 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 1%? How much faster will the processor be if we add a third level cache that has a 40 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%?

2017–2018 • Q4(a) CSE 305 • 23/09/2018 • MH19 **MARKS 10**

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache and a clock rate of 4 GHz. Assume a main memory that has an access time of 200 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 4%. How much faster will the processor be if we add a second level cache that has a 10 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 1%? How much faster will the processor be if we add a third level cache that has a 40 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%?

SOLUTION

Method (slides' multilevel-cache example)

Penalty in cycles = time / cycle time. Every L1 miss pays the L2 access; only references that also

miss L2 (global miss rate) pay the next level: $\text{CPI} = \text{CPI}_{\text{base}} + m_{L1}t_{L2} + m_{L2,\text{global}}t_{L3} + m_{\text{to mem}}t_{\text{mem}}$; speed-up $= \text{CPI}_{\text{old}}/\text{CPI}_{\text{new}}$.

Penalties

Cycle $= 0.25$ ns: memory $= 200/0.25 = 800$ cycles, L2 $= 10/0.25 = 40$ cycles, L3 $= 40/0.25 = 160$ cycles.

$$\begin{aligned} \text{L1 only:} &\quad 1 + 0.04 \times 800 = 33 \\ \text{L1+L2:} &\quad 1 + 0.04 \times 40 + 0.01 \times 800 = 1 + 1.6 + 8 = 10.6 \\ \text{L1+L2+L3:} &\quad 1 + 0.04 \times 40 + 0.01 \times 160 + 0.005 \times 800 = 1 + 1.6 + 1.6 + 4 = 8.2 \end{aligned}$$

Answer. Adding L2: $33/10.6 = 3.11\times$ faster. Adding L3 as well: $10.6/8.2 = 1.29\times$ faster than L1+L2 (i.e. $33/8.2 = 4.02\times$ faster than L1 only).

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2021–2022 • Q3(b) CSE 305 • 06/04/2023 • MH20 **MARKS 10**

Explain with necessary diagrams how the page table works with respect to the virtual address, physical address, main memory, and disk.

2017–2018 • Q3(c) CSE 305 • 23/09/2018 • MH20 **MARKS 5**

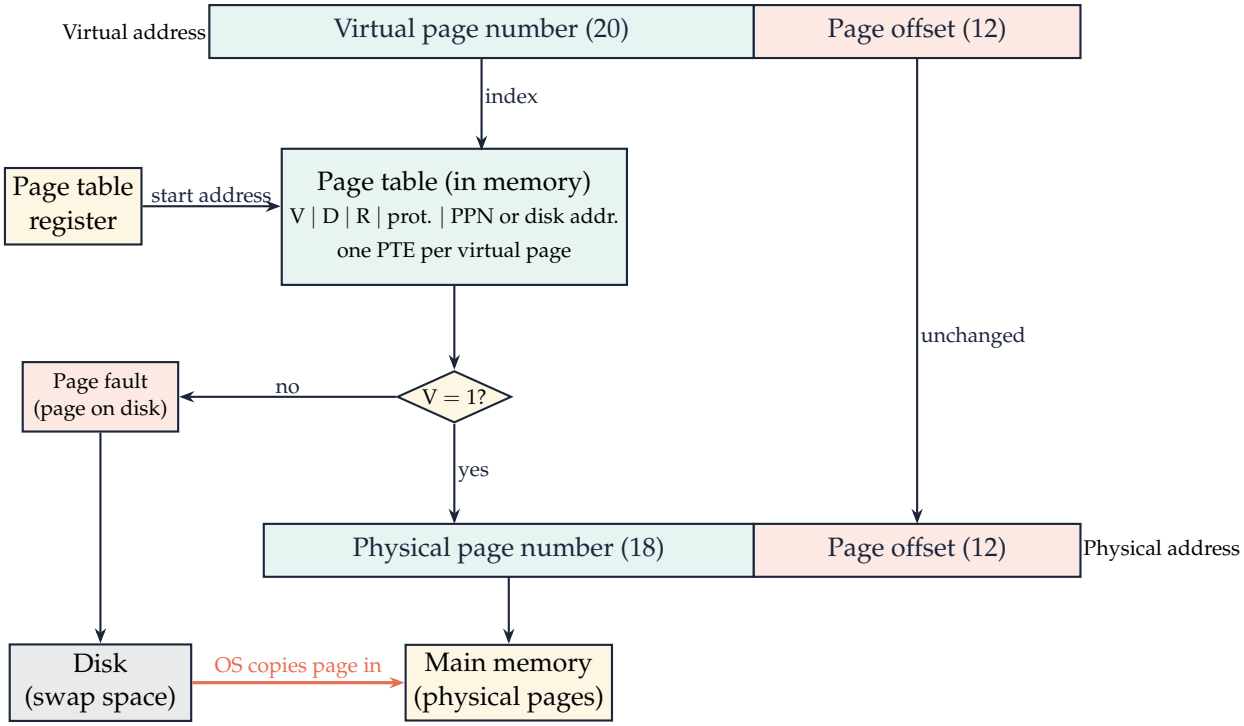
Briefly explain how page table works with respect to virtual address, physical address, main memory and disk.

2015–2016 • Q3(c) CSE 305 • 18/01/2017 • MH20 **MARKS 5**

Briefly explain how page table works with respect to virtual address, physical address, main memory and disk.

SOLUTION

Translation using the page table (32-bit VA, 4 KB pages, as in the slides)



- The **virtual address** is split into a virtual page number (VPN) and a page offset.
- The **page table register** holds the physical start address of the running process's page table; the VPN indexes it to fetch one page table entry (PTE).
- **Valid = 1:** the page is in **main memory**; the PTE's physical page number is concatenated with the unchanged offset to give the **physical address**, which accesses memory (via the cache). Reference/dirty bits are updated.
- **Valid = 0:** the page is on **disk** (the PTE or an OS structure holds its swap location) ⇒ page fault: the OS chooses a victim page (LRU approximation; written back if dirty), reads the page from disk into a free physical page, updates the PTE (valid = 1, new PPN) and restarts the instruction.
- Placement is fully associative (any virtual page in any physical page), writes are always write-back, and a TLB caches recent PTEs so most translations need no extra memory access.

2021–2022 • Q3(c) CSE 305 • 06/04/2023 • MH21 **MARKS 10**

You are given the following hit/miss status of TLB, Page Table and Cache.

Serial No.	TLB	Page Table	Cache
1	Hit	Hit	Hit
2	Hit	Miss	Hit
3	Miss	Miss	Hit
4	Miss	Miss	Miss
5	Miss	Hit	Hit

Table 3(c)

Explain which of the scenarios in Table 3(c) is possible or impossible and under what circumstance with proper reasoning.

SOLUTION

Two facts decide every row: (a) a TLB entry is only a copy of a *valid* PTE, so a TLB hit implies the page is in memory; (b) data can be in the cache only if its page is in main memory (the OS flushes a page's blocks when it moves the page to disk).

No.	TLB	PT	Cache	Verdict and reasoning
1	Hit	Hit	Hit	✓ **Possible** — the best case. (The page table is not actually consulted on a TLB hit, but its entry is valid.)
2	Hit	Miss	Hit	× **Impossible** — a translation cannot be in the TLB if the page is not present in memory.
3	Miss	Miss	Hit	× **Impossible** — data cannot be in the cache if its page is not in main memory.
4	Miss	Miss	Miss	✓ **Possible** — page fault: the OS loads the page from disk, updates the page table and TLB, the access is restarted and then misses in the cache.
5	Miss	Hit	Hit	✓ **Possible** — the PTE was evicted from the TLB (small TLB) while the page and its block are still in memory/cache; after the TLB refill, the retried access hits in the cache.

Answer. Possible: 1, 4, 5. Impossible: 2 and 3.

2021–2022 • Q4(a) CSE 305 • 06/04/2023 • MH22 **MARKS 15**

Assume there are three small caches, each consisting of four one-word blocks and uses LRU replacement policy. One cache is fully associative, a second is two-way set-associative, and the third is direct-mapped. Find the number of misses for each cache organization given the following sequence of block addresses:

1, 3, 2, 4, 5, 1, 3, 1, 3, 5, 3, 2

SOLUTION

Method (slides: "Associativity Example")

Direct-mapped: index = block mod 4. Two-way: 2 sets, set = block mod 2. Fully associative: any of 4 ways. On a miss fill an empty way, else replace the **LRU** block of the set. Cells give the block held in each slot after the access (Wk = way k; Sk = a way of set k).

Fully associative

Blk	H/M	W0	W1	W2	W3
1	×	1			
3	×	1	3		
2	×	1	3	2	
4	×	1	3	2	4
5	×	5	3	2	4
1	×	5	1	2	4
3	×	5	1	3	4
1	✓	5	1	3	4
3	✓	5	1	3	4
5	✓	5	1	3	4
3	✓	5	1	3	4
2	×	5	1	3	2

misses 8, hits 4

Two-way set associative

Blk	Set	H/M	S0		S1	
1	1	×			1	
3	1	×			1	3
2	0	×	2		1	3
4	0	×	2	4	1	3
5	1	×	2	4	5	3
1	1	×	2	4	5	1
3	1	×	2	4	3	1
1	1	✓	2	4	3	1
3	1	✓	2	4	3	1
5	1	×	2	4	3	5
3	1	✓	2	4	3	5
2	0	✓	2	4	3	5

misses 8, hits 4

Direct-mapped

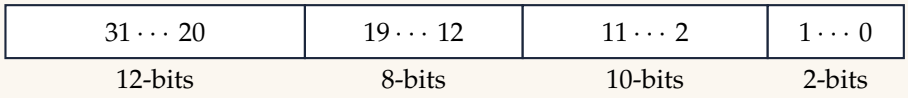
Blk	Idx	H/M	0	1	2	3
1	1	×		1		
3	3	×		1		3
2	2	×		1	2	3
4	0	×	4	1	2	3
5	1	×	4	5	2	3
1	1	×	4	1	2	3
3	3	✓	4	1	2	3
1	1	✓	4	1	2	3
3	3	✓	4	1	2	3
5	1	×	4	5	2	3
3	3	✓	4	5	2	3
2	2	✓	4	5	2	3

misses 7, hits 5

Answer. Misses: fully associative 8, two-way 8, direct-mapped 7. (Here LRU in the fully associative cache evicts exactly the blocks about to be reused, so higher associativity does not help.)

2020–2021 • Q6 CSE 305 • 24/09/2022 • MH23 **MARKS 15**

Answer the questions below for the following memory address subdivision and a 2-way set associative LRU cache:



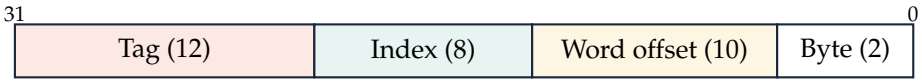
(a) How many distinct blocks of data can there be in the cache?

(b) In which cache index will the decimal address 24092022 map to?

(c) If a cache index is valid, what is the probability that one of its elements will have to be replaced in the next request? Assume each memory address has equal likelihood of being requested and each element in a cache index has equal likelihood of being filled.

SOLUTION

Reading the fields from the left as **tag**, **index (set)**, **block (word) offset**, **byte offset**:



So a block holds 2^{10} words $= 2^{12}$ bytes $= 4\,\text{KB}$, there are $2^8 = 256$ sets, and each set has 2 ways.

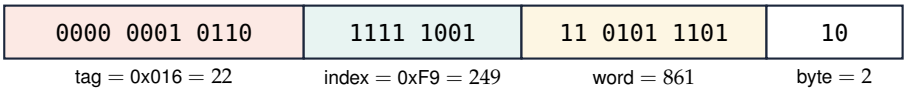
(a) Number of distinct blocks in the cache

$$\#\text{blocks} = \#\text{sets} \times \text{associativity} = 2^8 \times 2 = 512.$$

Answer. 512 blocks can be resident at once (256 sets $\times$ 2 ways; data capacity $512 \times 4\,\text{KB} = 2\,\text{MB}$).

(b) Index of address 24092022

$24092022_{10} =$ `0x016F9D76`. Splitting the 32-bit pattern:



Equivalently, index $= \lfloor 24092022/2^{12} \rfloor \bmod 2^8 = 5881 \bmod 256 = 249$.

Answer. The address maps to cache index (set) **249** (11111001_2), with tag 22.

(c) Probability that a valid set must replace an element on the next request

A replacement is needed only if both conditions hold (the set is full because it is "valid", and the two ways hold two different tags):

- the request maps to this set: $P = 1/2^8 = 1/256$ (every address equally likely, index bits uniform);
- it misses, i.e. its tag is neither of the 2 stored tags: $P = (2^{12} - 2)/2^{12} = 4094/4096$.

$$P(\text{replace}) = \frac{1}{256} \times \frac{4094}{4096} = \frac{4094}{1\,048\,576} \approx 3.90 \times 10^{-3}.$$

The victim is the LRU way of the two; since each element is equally likely to have been filled, each way is the victim with probability $\frac{1}{2}$ ($\approx 1.95 \times 10^{-3}$ each).

Answer. $P \approx 0.0039$ (about 0.39%). Given that the request falls in this set, the replacement probability is $4094/4096 \approx 0.9995$.

NOTE "Valid index" is taken to mean that both ways of the set hold valid blocks. If it means only that at least one way is filled, each way filled independently with probability $\frac{1}{2}$, then the set is full with probability $\frac{1/4}{3/4} = \frac{1}{3}$, and the answer becomes $\frac{1}{3} \times 3.90 \times 10^{-3} \approx 1.30 \times 10^{-3}$.

2020–2021 • Q7 CSE 305 • 24/09/2022 • MH24 **MARKS 15**

Suppose in a memory hierarchy there are four types of memories: Cache → RAM → SSD → HDD. Their local miss rates are 5%, 15%, 25%, and 0% respectively. Their access times are 1, 10, 100, and 1,000 cycles respectively. Now answer the following questions:

(a) What is the probability that a data will be fetched from the SSD?

(b) What is the expected time of a fetch?

(c) If the cache is separated into an I-cache and a D-cache, what will the miss rate of the I-cache be for a program that accesses data elements in 10% of its instructions if the overall miss rate remains constant (i.e., 5%) and the D-cache miss rate is 10%?

SOLUTION

A **local** miss rate is the fraction of the requests *reaching* a level that miss there. So the fraction of all requests reaching a level is the product of the local miss rates of the levels above it.



(a) Probability that the data is fetched from the SSD

The request must miss in the cache and in RAM, and then hit in the SSD:

$$P(\text{SSD supplies}) = 0.05 \times 0.15 \times (1 - 0.25) = 0.0075 \times 0.75 = 0.005625.$$

Answer. $P = 0.005625 \approx 0.56\%$. (The SSD is *accessed* on $0.05 \times 0.15 = 0.75\%$ of requests; on a quarter of those the data comes from the HDD.)

(b) Expected time of a fetch

On a miss each level is accessed in turn, so the AMAT formula is applied recursively from the bottom:

$$T_{\text{SSD}} = 100 + 0.25 \times 1000 = 350 \text{ cycles}$$
$$T_{\text{RAM}} = 10 + 0.15 \times 350 = 62.5 \text{ cycles}$$
$$T_{\text{fetch}} = 1 + 0.05 \times 62.5 = 4.125 \text{ cycles}$$

Check by summing over levels: $1 \cdot 1 + 0.05 \cdot 10 + 0.0075 \cdot 100 + 0.001875 \cdot 1000 = 1 + 0.5 + 0.75 + 1.875 = 4.125$.

Answer. Expected fetch time = 4.125 cycles.

NOTE This assumes that a miss pays the access time of every level it passes through. If instead each request pays only the access time of the level that finally supplies the data, the result is $0.95(1) + 0.0425(10) + 0.005625(100) + 0.001875(1000) = 3.8125$ cycles.

(c) I-cache miss rate after splitting

Per instruction there is 1 instruction fetch and 0.1 data access, so 1.1 cache accesses in total. To keep the overall miss rate at 5% per access:

$$\frac{1 \cdot m_I + 0.1 \cdot 0.10}{1 + 0.1} = 0.05 \;\Rightarrow\; m_I = 0.05 \times 1.1 - 0.01 = 0.045.$$

Answer. I-cache miss rate $m_I = 4.5\%$.

2020–2021 • Q8 CSE 305 • 24/09/2022 • MH25 MARKS 15

Suppose you have implemented an approximate LRU page replacement scheme using a reference bit for each page table entry. Your virtual memory has eight [pages] and page table has four entries. After every four time-steps, the reference bits are cleared. Assuming a fully set associative page table, simulate your page replacement scheme for the virtual memory reference below:

$$0,\ 0,\ 2,\ 7,\ 0,\ 6,\ 1,\ 2,\ 6,\ 7,\ 1,\ 1,\ 7,\ 1,\ 4,\ 1$$

You have to print the page table after each page table reference. You also have to mention whether or not it is a hit or miss.

SOLUTION

Scheme (LRU approximation with a reference bit)

- The table has 4 fully associative entries (frames 0–3). Each entry holds a virtual page and its reference bit R.
- Every reference sets $R = 1$ for its page. After time-steps 4, 8, 12 and 16 the OS clears all R bits to 0.
- **Hit**: the page is already in the table. **Miss**: the page goes into a free entry if there is one. Otherwise it replaces the lowest-numbered entry with $R = 0$, i.e. a page not used in the current interval, which is the LRU approximation.

Each cell shows $page^R$. The table is the state *after* each reference, and a shaded cell is the entry just loaded (the victim on a replacement).

Step	Ref	H/M	Entry 0	Entry 1	Entry 2	Entry 3	Action
1	0	×	0^1	–	–	–	compulsory, free entry
2	0	✓	0^1	–	–	–	
3	2	×	0^1	2^1	–	–	free entry
4	7	×	0^1	2^1	7^1	–	free entry
after step 4: all $R \leftarrow 0$ $\Rightarrow$ 0^0, 2^0, 7^0, –							
5	0	✓	0^1	2^0	7^0	–	
6	6	×	0^1	2^0	7^0	6^1	free entry
7	1	×	0^1	1^1	7^0	6^1	replace 2 ($R = 0$)
8	2	×	0^1	1^1	2^1	6^1	replace 7 ($R = 0$)
after step 8: all $R \leftarrow 0$ $\Rightarrow$ 0^0, 1^0, 2^0, 6^0							
9	6	✓	0^0	1^0	2^0	6^1	
10	7	×	7^1	1^0	2^0	6^1	replace 0 ($R = 0$)
11	1	✓	7^1	1^1	2^0	6^1	
12	1	✓	7^1	1^1	2^0	6^1	
after step 12: all $R \leftarrow 0$ $\Rightarrow$ 7^0, 1^0, 2^0, 6^0							
13	7	✓	7^1	1^0	2^0	6^0	
14	1	✓	7^1	1^1	2^0	6^0	
15	4	×	7^1	1^1	4^1	6^0	replace 2 ($R = 0$)
16	1	✓	7^1	1^1	4^1	6^0	(then all $R \leftarrow 0$)

Answer. 8 hits (steps 2, 5, 9, 11, 12, 13, 14, 16) and 8 misses (steps 1, 3, 4, 6 fill free entries; steps 7, 8, 10, 15 replace a page whose reference bit is 0). Hit rate $= 8/16 = 50\%$. Final table: $\{7, 1, 4, 6\}$.

NOTE The question does not say which entry to evict when several have $R = 0$. The tie-break used here is the lowest-numbered entry. It never happens here that all four R bits are 1 on a miss; if it did, entry 0 would be evicted. At step 15 this choice evicts page 2, which is also the true-LRU victim (last used at step 8, page 6 at step 9).

2019–2020 • Q6(a) CSE 305 • 18/09/2021 • MH26 **MARKS 20**

Assume there are *three* small caches, each consisting of eight (8) one-word blocks. One cache is **direct-mapped**, a second is **two-way set-associative**, and the third is **four-way set-associative**. For each cache organization, complete the following table for the following sequence of memory block addresses: **0**, **12**, **16**, **18**, and **0**. You are allowed to subdivide the third column "Contents of cache blocks after reference" as needed.
Note that when it is necessary to replace an existing cache entry on a cache miss, follow the least recently used (LRU) replacement rule to determine which cache entry to replace and show the cache content after the replacement.

Address of memory block accessed	Hit or Miss	Contents of cache blocks after reference
0		
12		
16		
18		
0		

SOLUTION

Method (slides: "Associativity Example")

DM: index $= b \bmod 8$ $(0,4,0,2,0)$. 2-way: 4 sets, set $= b \bmod 4$ $(0,0,0,2,0)$. 4-way: 2 sets, set $= b \bmod 2$ (all set 0). On a miss fill an empty way, else replace the **LRU** block of the set. Cells give the block held in each slot (Sk = a way of set k).

Direct-mapped

Blk	Idx	H/M	0	1	2	3	4	5	6	7
0	0	×	0							
12	4	×	0				12			
16	0	×	16				12			
18	2	×	16		18		12			
0	0	×	0		18		12			

misses 5, hits 0

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1	S2	S2	S3	S3
0	0	×	0							
12	0	×	0	12						
16	0	×	16	12						
18	2	×	16	12			18			
0	0	×	16	0			18			

misses 5, hits 0

Four-way set associative

Blk	Set	H/M	S0	S0	S0	S0	S1	S1	S1	S1
0	0	×	0							
12	0	×	0	12						
16	0	×	0	12	16					
18	0	×	0	12	16	18				
0	0	✓	0	12	16	18				

misses 4, hits 1

Answer. Direct-mapped: 5 misses; two-way: 5 misses (16 evicts LRU block 0); four-way: 4 misses, 1 hit (the final 0).

2019–2020 • Q6(b) CSE 305 • 18/09/2021 • MH27 MARKS 15

Assuming a cache of 4K blocks, a 4-word block size, and a 32-bit address, find the *total* number of tag bits for a cache that is **eight-way set associative**.

SOLUTION

Block $=$ 4 words $=$ 16 B $\Rightarrow$ offset $=$ 4 bits. Sets $= 4096/8 = 512 = 2^9 \Rightarrow$ index $=$ 9 bits. Tag $= 32 - 9 - 4 = 19$ bits.



Every one of the 4096 blocks stores its own tag:

Answer. Total tag bits $= 19 \times 4096 = 77{,}824$ bits $(= 76\,\text{Kbit})$.

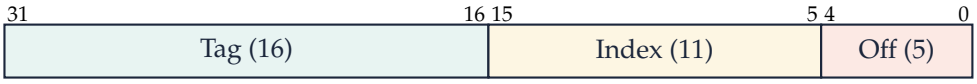
2019–2020 • Q7(a) CSE 305 • 18/09/2021 • MH28 **MARKS 20**

How many *total* bits are required for a **direct-mapped** cache with 64 KB of data and 8-word blocks (i.e., 32 bytes), assuming a 32-bit address?

SOLUTION

Fields

Offset $= \log_2 32 = 5$ bits. Blocks $= 2^{16}/2^5 = 2048 = 2^{11} \Rightarrow$ index $= 11$ bits. Tag $= 32 - 11 - 5 = 16$ bits.



Total

Each block: $8 \times 32 = 256$ data bits $+16$ tag $+1$ valid $= 273$ bits.

$$\text{Total} = 2^{11} \times 273 = 2048 \times 273 = 559{,}104 \text{ bits}$$

Answer. 559,104 bits = 546 Kbit ≈ 68.25 KB, i.e. about 1.07× the 64 KB of data.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q7(b) CSE 305 • 18/09/2021 • MH29 **MARKS 5**

The TLB acts as a cache of the page table for the entries that map to physical pages only as shown in Figure 7b. Write down the purpose of the tag bits in TLB.

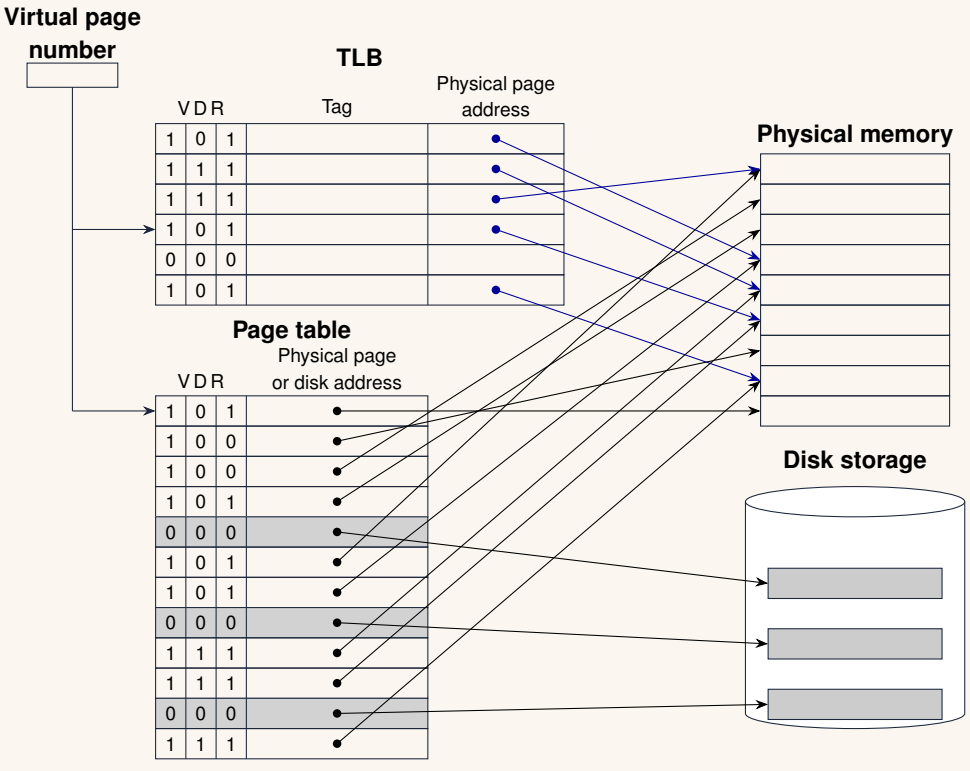


Figure 7b: TLB acts as a cache of the page table (arrows schematic, redrawn)

2012–2013 • Q6(c) CSE 305 • 10/05/2014 • MH29 **MARKS 7**

"The TLB acts as a cache of the page table for the entries that map to physical pages only" – Do you agree or disagree? Explain.

SOLUTION

2019–2020 Q7(b) • Purpose of the tag bits in the TLB

The page table has one entry for *every* virtual page, indexed directly by the VPN, so it needs no tag. The TLB is a small cache holding only a few of these entries, so many virtual pages compete for each TLB entry. The tag records *which* virtual page the cached translation belongs to:

- On a reference, the VPN from the virtual address is compared with the tag of every candidate entry (all entries in a fully associative TLB, as in Figure 7b; the entries of one set in a set-associative TLB).
- Tag match with $V = 1$: **TLB hit**. The physical page number is read out and joined to the page offset. No match: **TLB miss**, and the page table is consulted.
- Tag size: fully associative TLB, tag = the whole VPN. Set-associative TLB, tag = VPN bits above the set index.

Answer. The tag holds the virtual page number (or its upper part) of the cached page-table entry. It lets the hardware check whether the translation for the requested VPN is present in the TLB (hit) or not (miss).

2012–2013 Q6(c) • Agree or disagree?

Agree. In Figure 7b the TLB entries point only into physical memory. The page-table entries for non-resident pages ($V = 0$, shaded) point to disk, and none of them is cached in the TLB.

- A TLB entry stores only a *physical page number*, not a disk address. Its job is to speed up a translation that ends in main memory. A page on disk cannot be accessed by the CPU anyway; it must first be brought into memory.
- On a TLB miss the handler loads the PTE only if the page is present. If $V = 0$ in the page table, a *page fault* is raised instead. The OS reads the page from disk, updates the page table, and only then is the (now valid) translation loaded into the TLB.
- When the OS evicts a page to disk, it also invalidates any TLB entry for that page. So the TLB never holds a mapping to a non-resident page.

Answer. Agree: the TLB caches only valid page-table entries (pages resident in physical memory). Translations of pages that are on disk stay in the page table and are handled through a page fault, never through the TLB.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q7(c) CSE 305 • 18/09/2021 • MH30 **MARKS 10**

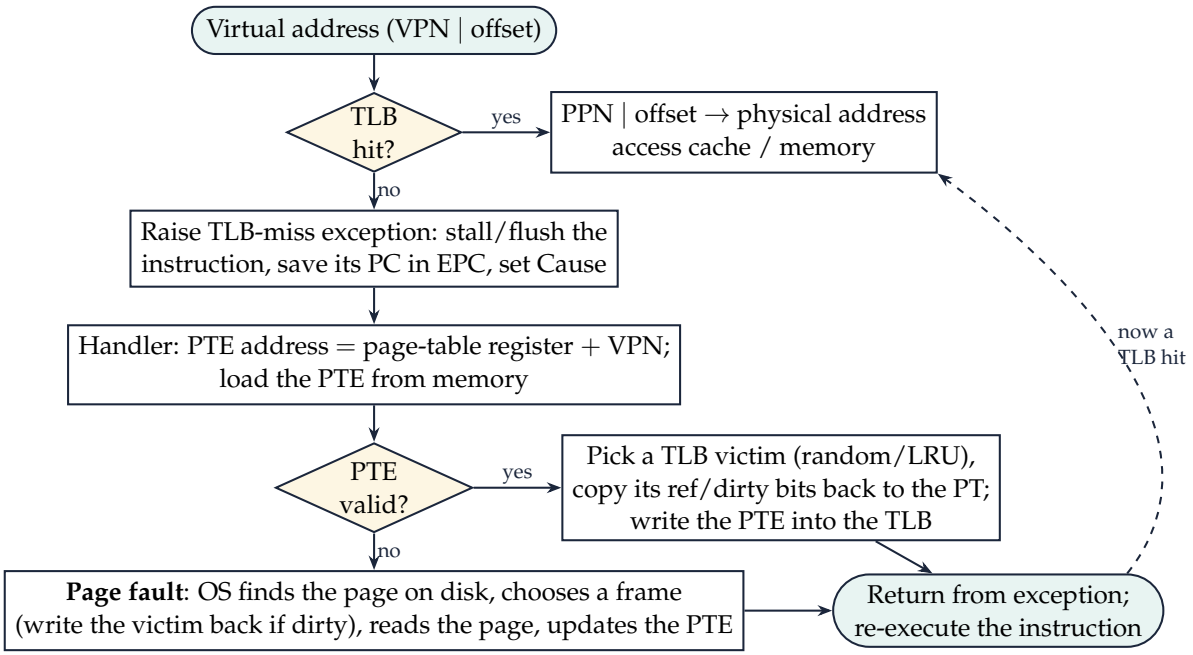
Explain how a TLB miss is handled.

2013–2014 • Q2(e) CSE 305 • 05/07/2015 • MH30 **MARKS 10**

Write down the procedure to handle a TLB page miss.

SOLUTION

A TLB miss means one of two things: the page is in memory but its PTE is not in the TLB, or the page is not in memory at all. The handler must tell the two apart.



Procedure

(i) **Detect.** The VPN matches no valid TLB tag. The access is stopped before it can change any state. In MIPS the miss can occur in IF (instruction fetch) or MEM (load/store).

(ii) **Trap.** The miss is handled either by *hardware*, which walks the page table itself (x86), or by *software*: a special exception with a short, optimized handler (MIPS). In software, the PC of the faulting instruction goes into EPC and the faulting address into BadVAddr.

(iii) **Fetch the PTE.** The page-table register plus the VPN gives the address of the PTE, which is loaded from memory (it is often found in the cache).

(iv) **Page present ($V = 1$).** Choose a TLB entry to replace and copy its reference/dirty bits back to its PTE. Write the new VPN tag, PPN and status bits into the TLB (MIPS: `tlbwr`). This takes about 10–100 cycles.

(v) **Page not present ($V = 0$).** This is a page fault, a separate exception handled by the OS. It locates the page on disk and chooses a victim frame, writing the victim to disk first if it is dirty. It reads the page in, updates the page table and the TLB, and then makes the process runnable again. This takes millions of cycles.

(vi) **Restart.** Return from the exception and re-execute the faulting instruction, which now hits in the TLB.

Answer. On a TLB miss, read the PTE from the page table (via the page-table register and the VPN). If it is valid, load it into the TLB (replacing an entry and saving that entry's ref/dirty bits) and restart the instruction. If it is invalid, raise a page fault so the OS brings the page from disk and updates the page table/TLB, then restart the instruction.

2018–2019 • Q3(b) CSE 305 • 29/10/2019 • MH31 **MARKS 19**

Consider a virtual memory system with 40-bit virtual byte address, 16KB pages and 36-bit physical byte address. All the virtual pages are in use and there are valid, protection, dirty and use bits. Disk addresses are not stored in the page table. There is a 2-way set associative TLB with a total of 256 TLB entries. Now answer the following questions: (4+3+12=19)

a. What is size of the page table?

b. What will be the size of the page table if we use the inverted page table concept?

c. With the help of an appropriate illustration show how the virtual-to-physical mapping is done. Assume a 16 KB direct-mapped cache with 16 words block.

SOLUTION

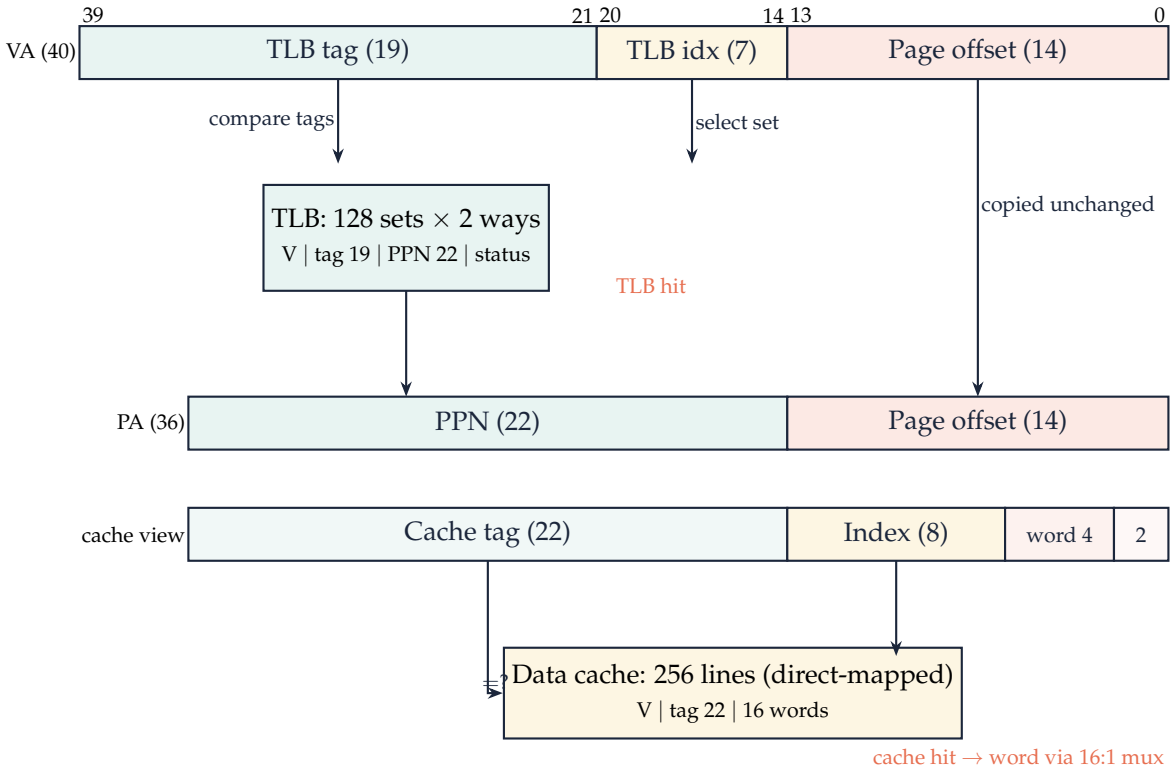
Address fields

Page offset $= \log_2 16\,\text{K} = 14$; VPN $= 40 - 14 = 26$ bits; PPN $= 36 - 14 = 22$ bits. PTE $= 22 + 4$ (valid, protection, dirty, use) $= 26$ bits.

a. One PTE per virtual page: 2^{26} entries $\times 26$ bits $= 1{,}744{,}830{,}464$ bits $= 208$ MB (256 MB if each PTE is padded to 32 bits).

b. Inverted: one entry per physical page $= 2^{22}$ entries; each stores the VPN (26) + 4 status bits $= 30$ bits $\Rightarrow 2^{22} \times 30$ bits $= 15$ MB (16 MB with 32-bit entries; a PID field would add to this).

c. TLB: 256 entries, 2-way $\Rightarrow$ 128 sets $\Rightarrow$ TLB index $= 7$ bits, TLB tag $= 26 - 7 = 19$ bits. Cache: 16 KB, 64 B blocks $\Rightarrow$ 256 lines $\Rightarrow$ index $= 8$, block offset $= 6$ (4 word + 2 byte), tag $= 36 - 8 - 6 = 22$ bits.



Steps: (1) TLB index (VA bits 20–14) selects a set, both tags are compared with VA bits 39–21; (2) on a hit the 22-bit PPN is concatenated with the 14-bit page offset $\Rightarrow$ 36-bit PA; (3) PA bits 13–6 index the cache, bits 35–14 are compared with the stored tag, bits 5–2 select the word. Note: cache index + offset $= 14$ bits $=$ page offset, so the cache can be indexed in parallel with the TLB lookup.

2018–2019 • Q3(c) CSE 305 • 29/10/2019 • MH32 **MARKS 8**

You have designed a processor with 16-entry TLB and 4 KB pages and have exhausted your budget for implementing the system. During trial you have found that programs are accessing at least 2MB of memory at a time. How would your system perform and why? Can you suggest how can you improve the system performance of the system if no more money is available? What if more money is available?

SOLUTION

Performance: the TLB is far too small (TLB reach)

$$\text{TLB reach} = 16 \times 4\,\text{KB} = 64\,\text{KB}, \qquad \text{pages in the working set} = \frac{2\,\text{MB}}{4\,\text{KB}} = 512.$$

The TLB can map only $64\,\text{KB}/2\,\text{MB} = 1/32$ of the data in use. With accesses spread over 512 pages and only 16 entries, entries are evicted before they are reused, so the TLB *thrashes*. Almost every new page touched causes a TLB miss, and each miss costs a page-table walk or miss handler (10–100 cycles) on top of the memory access.

Answer. The system will perform poorly: a TLB reach of 64 KB is much smaller than the 2 MB working set, so the TLB miss rate is high and most memory accesses pay the TLB-miss penalty.

No more money (no new hardware)

- **Larger pages**: the OS uses bigger pages if the MMU supports more than one page size (e.g. with 128 KB pages, $16 \times 128\,\text{KB} = 2\,\text{MB}$ covers the whole working set). Larger pages also shrink the page table.
- **Improve locality in software**: compiler/programmer restructuring (loop interchange, blocking/tiling, packing hot data together) so fewer pages are touched in any short interval.
- **Cheaper misses**: a fast, hand-optimized software miss handler, and page tables kept cache-friendly so a PTE fetch usually hits in the cache.

More money available

- A larger TLB, at least $2\,\text{MB}/4\,\text{KB} = 512$ entries, or a second-level TLB behind the 16-entry L1 TLB (as in the Core i7).
- Hardware support for large/variable page sizes, and a hardware page-table walker to cut the miss penalty.
- A bigger cache, so that page-table entries and data are found on-chip when a TLB miss does occur.

2018–2019 • Q4(a) CSE 305 • 29/10/2019 • MH33 **MARKS 4 + 3 + 12**

Consider three processors with different cache configurations and data as follows:

Processor/ Cache	Processor cycle time	Organization	Block size (Word)	Instruction Miss rate	Data Miss rate
1	420 ps	Direct-Mapped	1	4%	6%
2	420 ps	Direct-Mapped	4	2%	4%
3	310 ps	Two-way Set Associative	4	2%	3%

For these processors, one half of the instructions contain a data reference and cache miss penalty is 6+ block size in words. The CPI of this workload was measured on Processor 1 (i.e., with cache 1) and was found to be 2.0. Now, answer the following questions.

a. Which processor spends the most cycle on cache misses?

b. Which processor is the fastest and which one is the slowest?

c. For processor 3, show the cache organization and how the cache will be accessed with the help of a detailed illustration.

SOLUTION

Miss cycles per instruction $= (m_I + 0.5\,m_D) \times$ penalty, with penalty $= 6 +$ block size (cycles). The base CPI (no misses) is found from Processor 1, whose measured CPI of 2.0 already includes its misses:

$$\text{CPI}_{\text{base}} = 2.0 - (0.04 + 0.5 \times 0.06) \times 7 = 2.0 - 0.49 = 1.51.$$

Proc.	Penalty	Miss cycles / instr.	CPI	Cycle time	Time / instr.
1	$6+1=7$	$(0.04+0.03)\times 7 = 0.49$	2.00	420 ps	840.0 ps
2	$6+4=10$	$(0.02+0.02)\times 10 = 0.40$	1.91	420 ps	802.2 ps
3	$6+4=10$	$(0.02+0.015)\times 10 = 0.35$	1.86	310 ps	576.6 ps

a. Most cycles on cache misses

Answer. Processor 1: 0.49 miss cycles per instruction, against 0.40 for P2 and 0.35 for P3.

b. Fastest and slowest

Same workload (same instruction count), so compare CPI $\times$ cycle time.

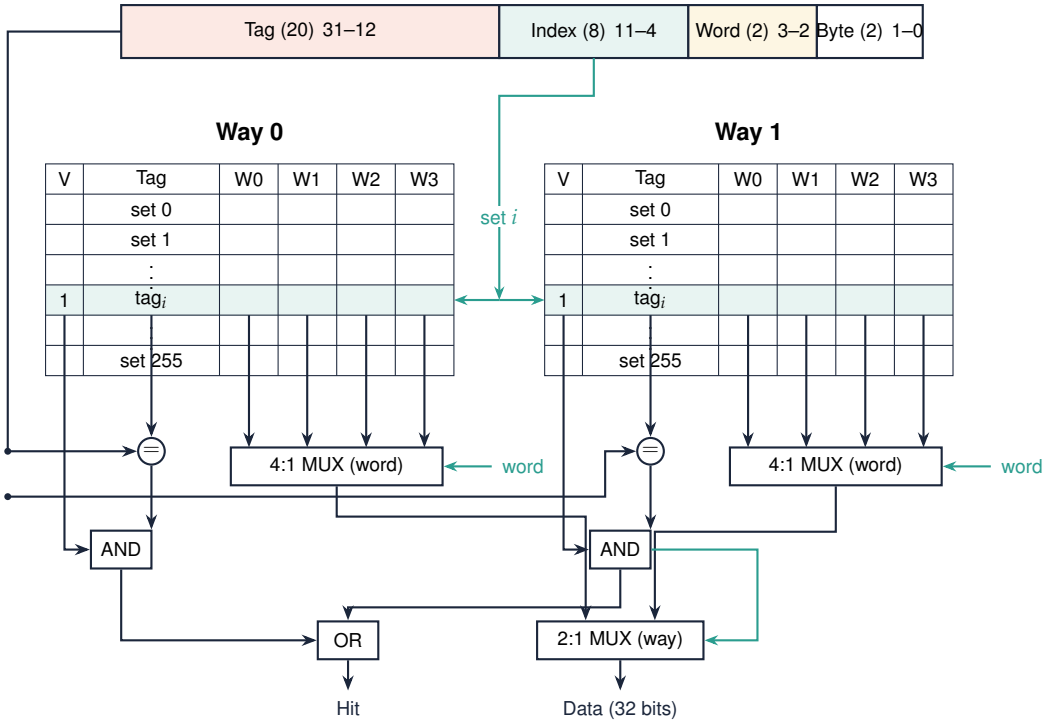
Answer. Fastest: **Processor 3** (576.6 ps/instruction). Slowest: **Processor 1** (840 ps/instruction). P3 is $840/576.6 \approx 1.46\times$ faster than P1, and P2 is $840/802.2 \approx 1.05\times$ faster than P1.

c. Organization and access of cache 3 (2-way set associative, 4-word blocks)

Assume a 32-bit byte address and an 8 KB data capacity, which the question does not give. Then:

$$\text{blocks} = \frac{8\,\text{KB}}{16\,\text{B}} = 512, \qquad \text{sets} = \frac{512}{2} = 256,$$

so index $= 8$ bits, word offset $= 2$, byte offset $= 2$ and tag $= 32 - 12 = 20$ bits.



Access steps:

(i) The 8-bit **index** selects the same set i in both ways.

(ii) The two stored tags of set i are compared *in parallel* with the 20-bit address tag. Each comparator output is ANDed with that entry's valid bit.

(iii) **Hit** = OR of the two AND outputs. The hitting way's signal drives the 2:1 way multiplexor.

(iv) The 2-bit **word offset** selects one of the 4 words of the block (4:1 MUX), and the byte offset selects a byte for byte loads.

(v) On a miss, the 10-cycle penalty brings the 4-word block from memory into the *LRU* way of set i (one LRU bit per set suffices for 2 ways), and the tag and V are updated.

NOTE The capacity of cache 3 is not given in the question. The 8 KB (20/8/2/2 split) is an assumption made only to draw the illustration. For another capacity C, index $= \log_2(C/32)$ and tag $= 28 -$ index.

2018–2019 • Q4(b) CSE 305 • 29/10/2019 • MH34 **MARKS 16**

Suppose you are working on a computer architecture with the following properties: (4+6+6=16)

a. 32-bit address

b. byte addressable RAM

c. 256KB cache

d. 64 words block

e. Random replacement algorithm

f. Write-through scheme

Now answer the following questions:

a. What is the size of the cache?

b. What is the overhead if the cache is: i. direct-mapped ii. 2-way set associative iii. fully associative

c. Consider the current value of $2 is 1000 in decimal. Assuming direct-mapped organization, find the cache address for the following load word instruction: `lw $1, 18($2)`.

SOLUTION

Method

Offset $= \log_2(\text{block bytes})$, index $= \log_2(\text{sets})$, tag $= 32 -$ index $-$ offset. Status per line $= 1$ valid bit only (write-through $\Rightarrow$ no dirty bit; random $\Rightarrow$ no LRU bits).

a. Block $= 64 \times 4 = 256$ B $\Rightarrow$ offset $= 8$ bits; blocks $= 2^{18}/2^8 = 1024$. The cache holds 2^{21} data bits (256 KB) in 1024 blocks; the total including tags is the data plus the overhead below (e.g. 2,112,512 bits $\approx$ 257.9 KB for direct-mapped).

b. Overhead (tag + valid):

Organisation	Sets / index	Tag bits	Overhead bits	% of data
Direct-mapped	1024 / 10	$32 - 10 - 8 = 14$	$1024 \times 15 = 15{,}360$ (1920 B)	0.73%
2-way	512 / 9	$32 - 9 - 8 = 15$	$1024 \times 16 = 16{,}384$ (2048 B)	0.78%
Fully associative	1 / 0	$32 - 8 = 24$	$1024 \times 25 = 25{,}600$ (3200 B)	1.22%

c. Effective address $= 1000 + 18 = 1018 =$ `0x3FA` $=$ `00...0` `0000000011` `11111010`:



Answer. Cache line (index) 3, tag 0, byte offset 250 = word 62, byte 2 of the block.

NOTE 1018 is not a multiple of 4, so in real MIPS `lw $1,18($2)` with $2 = 1000 raises an address-alignment exception; the fields above are what the cache would use for that byte address.

2017–2018 • Q3(a) CSE 305 • 23/09/2018 • MH35 **MARKS 10 + 10**

What are the three sources of cache misses? (10)

The specification of an ideal disk is as follows:
sector size: 1 MB, rotational latency: 10000 rpm, average seek time: 5 ms, transfer rate: 100 MB/s, and controller overhead: 0.5 ms. What is the average read time for the disk? If the average seek time can be reduced to 2 ms, then how much improvement can be possible on average read time? (10)

SOLUTION

Three sources of cache misses (the 3 Cs)

- **Compulsory** (cold-start) misses: the first access to a block, which has never been in the cache. These are reduced by larger blocks or prefetching.
- **Capacity** misses: the cache cannot hold all the blocks the program needs, so a replaced block is later accessed again. These are reduced by a larger cache.
- **Conflict** (collision) misses: in a direct-mapped or set-associative cache, too many blocks compete

for the same set. They would not occur in a fully associative cache of the same size. These are reduced by higher associativity.

Average read time

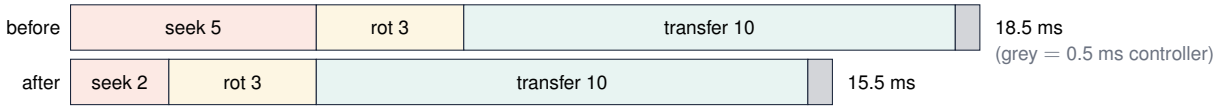
For an idle ideal disk (no queuing delay),

$$T_{\text{read}} = T_{\text{seek}} + \underbrace{\frac{0.5}{\text{rpm}/60}}_{\text{rotational latency}} + \underbrace{\frac{\text{sector size}}{\text{transfer rate}}}_{\text{transfer}} + T_{\text{controller}}.$$

$$T_{\text{rot}} = \frac{0.5}{10000/60}\ \text{s} = 3\ \text{ms}, \qquad T_{\text{transfer}} = \frac{1\ \text{MB}}{100\ \text{MB/s}} = 10\ \text{ms}$$

$$T_{\text{read}} = 5 + 3 + 10 + 0.5 = 18.5\ \text{ms}$$

$$T'_{\text{read}} = 2 + 3 + 10 + 0.5 = 15.5\ \text{ms} \quad (\text{seek } 5 \to 2\ \text{ms})$$



$$\text{Speedup} = \frac{18.5}{15.5} = 1.19, \qquad \text{time saved} = \frac{3}{18.5} = 16.2\%.$$

Answer. Average read time $= 18.5$ ms. With a 2 ms seek it becomes 15.5 ms: 3 ms less (16.2% reduction), i.e. about $1.19\times$ ($\approx 19\%$) faster. The gain is modest because the 10 ms transfer of the large 1 MB sector dominates.

REPEATED ×6 Same question in 6 papers — stacked below, solved once.

2017–2018 • Q3(b) CSE 305 • 23/09/2018 • MH36 — **MARKS 15**

What is meant by spatial and temporal locality? Briefly explain write-through and write-back scheme for cache. How do write-through and write-back schemes handle write-miss?

2016–2017 • Q3(a) CSE 305 • 02/08/2017 • MH36 — **MARKS 5**

What do you mean by the principle of locality of reference? Discuss the role of the principle of locality of reference in designing memory hierarchy.

2015–2016 • Q3(b) CSE 305 • 18/01/2017 • MH36 — **MARKS 10**

Briefly explain write-through and write-back scheme for cache? How does write-though and write-back schemes handle write-miss?

2014–2015 • Q2(a) CSE 305 • 26/01/2016 • MH36 — **MARKS 6 + 3 + 3**

List out the key characteristics of computer systems. What do you mean by the principle of locality of reference? Discuss the role of the principle of locality of reference in designing memory hierarchy.

2013–2014 • Q1(a) CSE 305 • 05/07/2015 • MH36 **MARKS 3 + 3**

State the principle of temporal locality and the principle of spatial locality.

2012–2013 • Q7(a) CSE 305 • 10/05/2014 • MH36 **MARKS 6**

State the advantages of using write back cache.

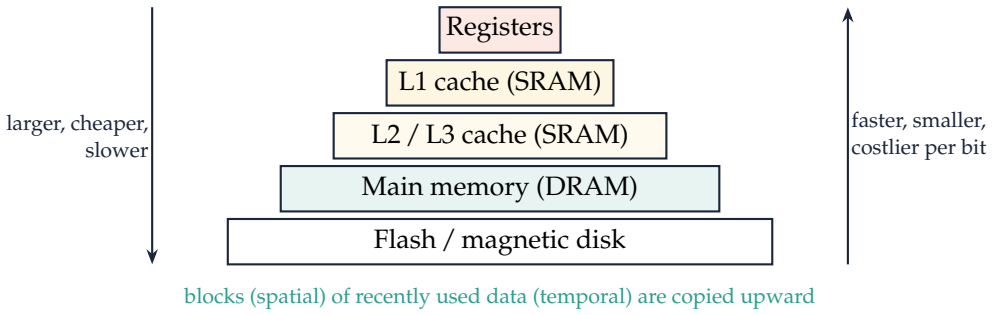
SOLUTION

Principle of locality of reference (2017–18, 2016–17, 2014–15, 2013–14)

Programs access only a small part of their address space at any time.

- **Temporal locality** (locality in time): an item referenced recently is likely to be referenced again soon. Examples: instructions in a loop, loop counters/induction variables, a function called repeatedly.
- **Spatial locality** (locality in space): items whose addresses are near a recently referenced item are likely to be referenced soon. Examples: sequential instruction fetch, walking through an array, fields of a record.

Role of locality in designing the memory hierarchy (2016–17, 2014–15)



- Locality is what makes a hierarchy work. A small, fast memory near the CPU can hold the data in current use, so most references hit there. The user sees memory that is about as fast as the top level and as large and cheap as the bottom level.
- **Temporal locality** means recently accessed items are kept in the upper levels, and LRU-type replacement keeps the ones most likely to be reused.
- **Spatial locality** means data is moved in *blocks* (lines, pages) rather than single words, so the neighbours are already present when they are referenced.
- Design choices follow from it: block size, cache size and associativity, replacement policy, prefetching, and the number of levels.

Key characteristics of computer memory systems (2014–2015 Q2(a))

Location	Internal (registers, cache, main memory) or external (disk, tape, optical).
Capacity	Word size and number of words/bytes.
Unit of transfer	Word (internal) or block (cache line, disk sector, page).
Access method	Sequential (tape), direct (disk), random (RAM), associative (cache/TLB).
Performance	Access time (latency), memory cycle time, transfer rate.
Physical type	Semiconductor, magnetic, optical, magneto-optical.
Physical characteristics	Volatile/non-volatile, erasable/non-erasable.
Organization	Arrangement of bits into words and modules (e.g. interleaving).

Write-through vs write-back (2017–18, 2015–16)

	Write-through	**Write-back**
On a write hit	Update the cache *and* memory, so both are always consistent.	Update the cache only and set the block's **dirty bit**.
Memory updated	On every write. A *write buffer* holds pending writes so the CPU continues, stalling only if the buffer is full.	Only when a dirty block is *replaced*; it is then written back as a whole block.
Cost	Simple, and replacement needs no write-back. Heavy memory traffic: with base CPI 1, 10% stores and 100-cycle writes, CPI $= 1 + 0.1 \times 100 = 11$ without a buffer.	More state (dirty bit) and more complex replacement. Much less traffic.
Write miss	Either **write allocate** (fetch the block into the cache, then write) or **write around**/no-write-allocate (write only to memory). Write around suits programs that write whole blocks before reading them, e.g. initialization.	Usually **write allocate**: fetch the block (after writing back the victim if it is dirty), write the word into the cache, and mark it dirty.

Advantages of a write-back cache (2012–2013 Q7(a))

- Writes complete at cache speed; the CPU never waits for main memory on a write hit.
- Repeated writes to the same block (e.g. a loop variable) cost one memory write when the block is evicted, instead of one per store. This cuts memory/bus traffic and power.
- The write-back transfers a whole block at once, which uses DRAM burst mode and bus bandwidth efficiently.
- Less bus contention, which matters in multiprocessors. It also scales to slow lower levels: virtual memory uses only write-back, because disk writes take millions of cycles.

2017–2018 • Q3(d) CSE 305 • 23/09/2018 • MH37 **MARKS 5**

Assume the miss rate of an instruction cache is 3%, the miss rate of the data cache is 5%, and the frequency of all loads and stores is 40%. If a processor has a CPI of 2 without any memory stalls and the miss penalty is 200 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Also, determine the amount of execution time spent on memory stalls.

SOLUTION

Every instruction is fetched (one I-cache access), and only loads/stores (40%) access the D-cache:

$$\begin{aligned}\text{I-stalls} &= m_I \times \text{penalty} = 0.03 \times 200 = 6 \text{ cycles/instr}\\ \text{D-stalls} &= f_{\text{ld/st}} \times m_D \times \text{penalty} = 0.40 \times 0.05 \times 200 = 4 \text{ cycles/instr}\\ \text{CPI}_{\text{actual}} &= 2 + 6 + 4 = 12\end{aligned}$$

The clock rate and instruction count are unchanged, so

$$\text{Speedup}_{\text{perfect}} = \frac{\text{CPI}_{\text{actual}}}{\text{CPI}_{\text{base}}} = \frac{12}{2} = 6, \qquad \text{stall fraction} = \frac{6+4}{12} = 83.3\%.$$

Answer. The perfect-cache processor is **6**× faster. Memory stalls take **83.3**% of the execution time.

2017–2018 • Q4(b) CSE 305 • 23/09/2018 • MH38 **MARKS 15**

Assume there are three small caches, each consisting of four one-word blocks and uses LRU replacement policy. One cache is fully associative, the second is two-way set-associative, and the third is direct-mapped. Find the number of misses for each cache organization, given the following sequence of block addresses:

- 1, 3, 5, 1, 3, 1, 3, 5, 3, 1, 5, 3

SOLUTION

Method (slides: "Associativity Example")

Direct-mapped: index = block mod 4. Two-way: 2 sets, set = block mod 2. Fully associative: any of 4 ways. On a miss fill an empty way, else replace the **LRU** block of the set. Cells give the block held in each slot after the access.

Fully associative

Blk	H/M	W0	W1	W2	W3
1	×	1			
3	×	1	3		
5	×	1	3	5	
1	✓	1	3	5	
3	✓	1	3	5	
1	✓	1	3	5	
3	✓	1	3	5	
5	✓	1	3	5	
3	✓	1	3	5	
1	✓	1	3	5	
5	✓	1	3	5	
3	✓	1	3	5	
misses 3, hits 9					

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1
1	1	×			1	
3	1	×			1	3
5	1	×			5	3
1	1	×			5	1
3	1	×			3	1
1	1	✓			3	1
3	1	✓			3	1
5	1	×			3	5
3	1	✓			3	5
1	1	×			3	1
5	1	×			5	1
3	1	×			5	3
misses 9, hits 3						

Direct-mapped

Blk	Idx	H/M	0	1	2	3
1	1	×		1		
3	3	×		1		3
5	1	×		5		3
1	1	×		1		3
3	3	✓		1		3
1	1	✓		1		3
3	3	✓		1		3
5	1	×		5		3
3	3	✓		5		3
1	1	×		1		3
5	1	×		5		3
3	3	✓		5		3
misses 7, hits 5						

Answer. Misses: fully associative 3 (only compulsory), two-way 9 (all three odd blocks fight for set 1's two ways), direct-mapped 7.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2017–2018 • Q4(c) CSE 305 • 23/09/2018 • MH39 **MARKS 5**

What is meant by TLB? Why TLB is necessary in a virtual memory system?

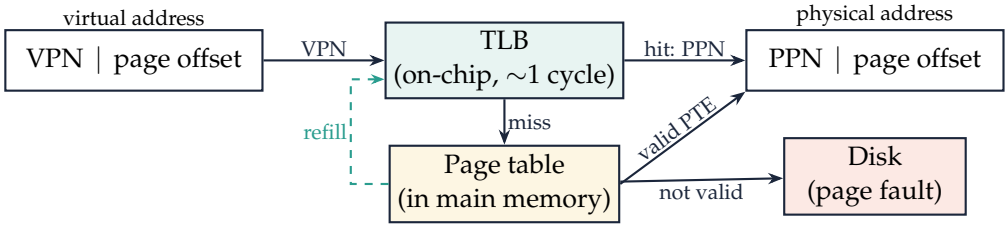
2013–2014 • Q1(d) CSE 305 • 05/07/2015 • MH39 **MARKS 5**

What is the limitation of a processor that does not have a TLB but uses the virtual memory technique?

SOLUTION

What a TLB is

A **Translation Look-aside Buffer** is a small, fast, associative cache of recently used page-table entries inside the CPU (MMU). Each entry holds a tag (the VPN), the physical page number, and valid/dirty/reference bits. Typical figures: 16–512 entries, 0.5–1 cycle on a hit, 10–100 cycles on a miss, miss rate 0.01%–1%.



Why it is needed: the limitation without a TLB

- With paging, every virtual address must be translated, and the page table lives in main memory. Without a TLB, *every* load, store and instruction fetch needs an extra memory access to read the PTE before the real access. That is at least 2 memory accesses per reference, and more with multi-level page tables.

- So the effective memory access time roughly *doubles*. For example, with a 100 ns memory each reference costs $\approx$ 200 ns, and much of the benefit of the cache is lost, because the cache cannot even be indexed (for a physically addressed cache) until translation is done.
- Page-table accesses have good locality, since consecutive references fall in the same page. A small TLB therefore catches almost all translations and makes translation nearly free.

Answer. A TLB is a cache of page-table entries. Without it, a virtual-memory processor must access the page table in memory on every reference, which doubles (or worse) the memory access time and greatly slows execution. The TLB removes this extra access on a hit.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q3(b) CSE 305 • 02/08/2017 • MH40 MARKS 15

Answer the following questions for a memory system that uses 32-bit address at the byte level, and a cache that uses a 64-byte line size.

(i) Assume a direct cache with a tag field in the address of 20 bits. Show the address format and determine the number of addressable units, the number of blocks in main memory, and the number of lines in the cache.

(ii) Assume an associative cache. Show the address format and determine the number of addressable units, the number of blocks of main memory, the number of lines in the cache and the size of the tag.

(iii) Assume a four-way set-associative cache with a tag field in the address of 9 bits. Show the address format and determine the number of addressable units, the number of blocks in main memory, the number of lines in a set, the number of sets in the cache, and the number of lines in the cache.

2014–2015 • Q2(c) CSE 305 • 26/01/2016 • MH40 MARKS 15

Answer the following questions for a memory system that uses 32-bit address at the byte level, and a cache that uses a 64-byte line size.

(i) Assume a direct cache with a tag field in the address of 20 bits. Show the address format and determine the number of addressable units, the number of blocks in main memory, and the number of lines in the cache.

(ii) Assume an associative cache. Show the address format and determine the number of addressable units, the number of blocks of main memory, the number of lines in the cache and the size of the tag.

(iii) Assume a four-way set-associative cache with a tag field in the address of 9 bits. Show the address format and determine the number of addressable units, the number of blocks in main memory, the number of lines in a set, the number of sets in the cache, and the number of lines in the cache.

SOLUTION

Common to all three parts (both papers ask exactly the same question)

- Byte-addressed, 32-bit address: addressable units $= 2^{32}$ bytes (4 GB).

- Line (block) size $= 64 = 2^6$ bytes, so the **word/offset field** is 6 bits.
- Blocks in main memory $= 2^{32}/2^6 = 2^{26} = 67{,}108{,}864$.

(i) Direct-mapped, tag $= 20$ bits

Line field $= 32 - 20 - 6 = 6$ bits.



Answer. Addressable units $= 2^{32}$; blocks in main memory $= 2^{26}$; lines in cache $= 2^6 = 64$ (a 4 KB cache).

(ii) Fully associative

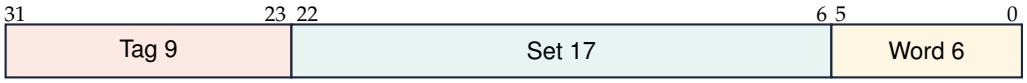
There is no line/set field: the whole block number is the tag.



Answer. Addressable units $= 2^{32}$; blocks in main memory $= 2^{26}$; lines in the cache: *not determined by the address format* (any number, since a block may go in any line); tag size $= 26$ bits.

(iii) Four-way set associative, tag $= 9$ bits

Set field $= 32 - 9 - 6 = 17$ bits.



$$\text{sets} = 2^{17} = 131{,}072, \qquad \text{lines in cache} = 4 \times 2^{17} = 2^{19} = 524{,}288.$$

Answer. Addressable units $= 2^{32}$; blocks in main memory $= 2^{26}$; lines per set $= 4$; sets $= 2^{17}$; lines in cache $= 2^{19}$ (data capacity $2^{19} \times 64$ B $= 32$ MB).

2015–2016 • Q4(a) CSE 305 • 18/01/2017 • MH41 **MARKS 5**

Assume the miss rate of an instruction cache is 3% and the miss rate of the data cache is 5%. If a processor has a CPI of 2 without any memory stalls and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 40%. What is the amount of execution time spent on memory stalls?

SOLUTION

Every instruction is fetched (one I-cache access), and only loads/stores (40%) access the D-cache:

$$\begin{aligned}
\text{I-stalls} &= m_I \times \text{penalty} = 0.03 \times 100 = 3 \text{ cycles/instr} \\
\text{D-stalls} &= f_{\text{ld/st}} \times m_D \times \text{penalty} = 0.40 \times 0.05 \times 100 = 2 \text{ cycles/instr} \\
\text{CPI}_{\text{actual}} &= 2 + 3 + 2 = 7
\end{aligned}$$

The clock rate and instruction count are unchanged, so

$$\text{Speedup}_{\text{perfect}} = \frac{7}{2} = 3.5, \qquad \text{stall fraction} = \frac{3+2}{7} = 71.4\%.$$

Answer. The perfect-cache processor is $\mathbf{3.5}\times$ faster. Memory stalls take $\mathbf{71.4}\%$ of the execution time.

2015–2016 • Q4(c) CSE 305 • 18/01/2017 • MH42 **MARKS 15**

Assume there are three small caches, each consisting if four one-word blocks and uses LRU replacement policy. One cache is fully associative, a second is two-way set-associative, and the third is direct-mapped. Find the number of misses for each cache organization given the following two independent sequences of block addresses:

0, 2, 4, 8, 10, 12, 14, 16, 0

1, 3, 5, 1, 3, 1, 3, 5, 3

SOLUTION

Method (slides: "Associativity Example")

Direct-mapped: index = block mod 4. Two-way: 2 sets, set = block mod 2. Fully associative: any of 4 ways. On a miss fill an empty way, else replace the **LRU** block of the set. Each sequence starts from an empty cache.

Sequence 1: 0, 2, 4, 8, 10, 12, 14, 16, 0

Fully associative

Blk	H/M	W0	W1	W2	W3
0	×	0			
2	×	0	2		
4	×	0	2	4	
8	×	0	2	4	8
10	×	10	2	4	8
12	×	10	12	4	8
14	×	10	12	14	8
16	×	10	12	14	16
0	×	0	12	14	16

misses 9, hits 0

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1
0	0	×	0			
2	0	×	0	2		
4	0	×	4	2		
8	0	×	4	8		
10	0	×	10	8		
12	0	×	10	12		
14	0	×	14	12		
16	0	×	14	16		
0	0	×	0	16		

misses 9, hits 0

Direct-mapped

Blk	Idx	H/M	0	1	2	3
0	0	×	0			
2	2	×	0		2	
4	0	×	4		2	
8	0	×	8		2	
10	2	×	8		10	
12	0	×	12		10	
14	2	×	12		14	
16	0	×	16		14	
0	0	×	0		14	

misses 9, hits 0

Sequence 2: 1, 3, 5, 1, 3, 1, 3, 5, 3

Fully associative

Blk	H/M	W0	W1	W2	W3
1	×	1			
3	×	1	3		
5	×	1	3	5	
1	✓	1	3	5	
3	✓	1	3	5	
1	✓	1	3	5	
3	✓	1	3	5	
5	✓	1	3	5	
3	✓	1	3	5	

misses 3, hits 6

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1
1	1	×			1	
3	1	×			1	3
5	1	×			5	3
1	1	×			5	1
3	1	×			3	1
1	1	✓			3	1
3	1	✓			3	1
5	1	×			3	5
3	1	✓			3	5

misses 6, hits 3

Direct-mapped

Blk	Idx	H/M	0	1	2	3
1	1	×		1		
3	3	×		1		3
5	1	×		5		3
1	1	×		1		3
3	3	✓		1		3
1	1	✓		1		3
3	3	✓		1		3
5	1	×		5		3
3	3	✓		5		3

misses 5, hits 4

Answer. Sequence 1: 9 misses in all three caches (8 distinct blocks $>$ 4 blocks; the final 0 has already been evicted). Sequence 2: fully associative 3, two-way 6, direct-mapped 5 misses.

2015–2016 • Q4(d) CSE 305 • 18/01/2017 • MH43 **MARKS 5**

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache, and a clock rate of 4 GHz. Assume a main memory access time of 100 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 3%. How much faster will the processor be if we add a secondary cache that has a 5 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 1%.

SOLUTION

Penalty (cycles) = time / cycle time, cycle = 0.25 ns: memory = $100/0.25 = 400$ cycles; L2 = $5/0.25 = 20$ cycles. Every L1 miss pays the L2 time, the 1% that also miss L2 pay the memory time.

$$\text{CPI}_{L1} = 1 + 0.03 \times 400 = 13, \qquad \text{CPI}_{L1+L2} = 1 + 0.03 \times 20 + 0.01 \times 400 = 1 + 0.6 + 4 = 5.6$$

Answer. Speed-up $= 13/5.6 = 2.32$.

2013–2014 • Q1(b) CSE 305 • 05/07/2015 • MH44 **MARKS 9**

Assume a two-way set associative cache with four blocks. The current state of the cache is empty. Assuming an LRU replacement policy, how many hits does the address sequence 0, 2, 4, 0, 2, 4, 0, 2, 4 exhibit? Show the state of the cache after every address access.

SOLUTION

Four blocks, two-way ⇒ 2 sets, set = address mod 2. All three addresses are even ⇒ all map to set 0, which has only two ways; on a miss the **LRU** block of the set is replaced.

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1
0	0	×	0			
2	0	×	0	2		
4	0	×	4	2		
0	0	×	4	0		
2	0	×	2	0		
4	0	×	2	4		
0	0	×	0	4		
2	0	×	0	2		
4	0	×	4	2		
misses 9, hits 0						

Answer. 0 hits: with LRU, each block is evicted just before it is needed again (classic LRU thrashing with 3 blocks cycling through 2 ways).

2013–2014 • Q1(e) CSE 305 • 05/07/2015 • MH45 **MARKS 6**

What is the size of the tag field for a direct-mapped cache with 2^n blocks and the following configuration: 32-bit byte address and the block size: 2^m words (Assume 1 word = 4 bytes).

SOLUTION

A 32-bit byte address splits into tag | index | word-in-block | byte-in-word:

Tag $= 32-(n+m+2)$	Index (n)	Word (m)	Byte (2)

- 2^n blocks, direct-mapped $\Rightarrow$ index $= n$ bits.
- 2^m words per block $\Rightarrow$ m bits select the word; 4 bytes per word $\Rightarrow$ 2 bits byte offset.

Answer. Tag $= 32-(n+m+2)$ bits.

2013–2014 • Q2(d) CSE 305 • 05/07/2015 • MH46 **MARKS 7**

A virtual memory system has the following parameters:

Virtual address (bits):	32
Physical DRAM installed:	4 GB
Page size:	8 KB
PTE size (byte):	4

How many page table entries are needed? How much physical memory is needed for storing the page table?

SOLUTION

One PTE per virtual page. Page offset $= \log_2 8\,\text{K} = 13$ bits $\Rightarrow$ VPN $= 32-13 = 19$ bits.

$$\#\text{PTEs} = 2^{19} = 524{,}288, \qquad \text{size} = 2^{19} \times 4\,\text{B} = 2^{21}\,\text{B} = 2\,\text{MB}$$

Answer. 524,288 entries; the page table needs 2 MB (per process; the 4 GB DRAM only sets the PPN width, $32-13 = 19$ bits).

2012–2013 • Q5(a) CSE 305 • 10/05/2014 • MH47 **MARKS 10**

What is the average time to read or write a 512-byte sector for a typical disk rotating at 15,000 RPM? The seek time is 4 ms, the transfer rate is 100 MB/sec, and the controller overhead is 0.2 ms. Assume that the disk is idle so that there is no waiting time.

SOLUTION

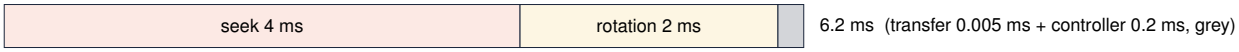
The disk is idle, so the queuing delay is 0 and

$$T = T_{\text{seek}} + \underbrace{\frac{0.5\text{ rev}}{\text{rpm}/60}}_{\text{rotational latency}} + \underbrace{\frac{\text{sector size}}{\text{transfer rate}}}_{\text{transfer}} + T_{\text{controller}}.$$

$$T_{\text{rot}} = \frac{0.5}{15000/60}\,\text{s} = \frac{0.5}{250}\,\text{s} = 2\text{ ms}$$

$$T_{\text{transfer}} = \frac{512\text{ B}}{100\times 10^6\text{ B/s}} = 5.12\,\mu\text{s} \approx 0.005\text{ ms}$$

$$T = 4 + 2 + 0.005 + 0.2 = 6.2\text{ ms}$$



Answer. Average time to read or write the sector ≈ 6.2 ms. Seek and rotational latency dominate; the transfer itself takes only about 5 μs.

2012–2013 • Q5(c) CSE 305 • 10/05/2014 • MH48 **MARKS 5**

Differentiate between NAND and NOR flash memories.

SOLUTION

Both are non-volatile semiconductor (floating-gate) storage, 100–1000× faster than disk. They differ in how the cells are wired:

	NOR flash	**NAND flash**
Bit cell	Like a NOR gate: cells in *parallel* on the bit line.	Like a NAND gate: cells in *series* strings.
Access	Random read/write access to individual bytes/words; code can execute in place.	Block/page-at-a-time access; random access to a single byte is not possible.
Density, cost	Larger cell, so lower density and higher \$/GB.	Denser (more bits/area), so cheaper per GB.
Speed	Fast random reads; slow write/erase.	Slower random reads; faster program/erase of large blocks.
Typical use	Instruction (firmware/boot) memory in embedded systems.	USB keys, SD cards, SSDs, media storage.

Common limitation: flash bits wear out after thousands of write/erase cycles, so neither is a direct replacement for RAM or disk. **Wear leveling** remaps data to less-used blocks.

2012–2013 • Q6(b) CSE 305 • 10/05/2014 • MH49 **MARKS 6**

What is the disadvantages of using virtually addressed cache?

SOLUTION

A virtually addressed cache is indexed and tagged with the *virtual* address, so the TLB is not on the hit path. The cost is that virtual addresses are not unique names for physical data:

- **Aliasing (synonyms).** Two virtual addresses, from different processes sharing a page or the same process mapping a page twice, can refer to the same physical word. The data may then sit in two cache locations; a write to one copy leaves the other stale, so extra hardware or OS restrictions are needed.
- **Homonyms and context switches.** The same virtual address means different data in different processes. The cache must be *flushed* on every context switch, or each tag must be extended with a process ID (ASID). Flushing destroys the cache's warm contents.
- **Protection and page-state checks.** Access rights and dirty/reference bits still come from the page table/TLB, so the TLB is still needed on writes and misses.
- **I/O and coherence use physical addresses.** DMA devices and snooping on other processors see

physical addresses, so a reverse (physical→virtual) translation or duplicated physical tags is needed.

- **Page changes.** When the OS remaps or evicts a page, all cache blocks from that virtual page must be found and invalidated.

Answer. The main disadvantages are aliasing, flushing (or ASID tags) on context switches, and the extra complexity for protection, I/O/coherence and page remapping, all because the cache tags are not physical addresses. The usual compromise is a virtually indexed, physically tagged cache.

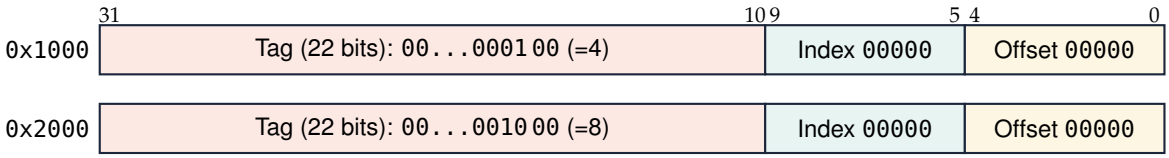
2012–2013 • Q6(d) CSE 305 • 10/05/2014 • MH50 **MARKS 13**

A program accesses 2 cache blocks, one that begins at memory address 0x1000 and another one that begins at memory address 0x2000. Memory accesses alternate between these two blocks and each block is accessed 100 times.
If the program is run on a system with a 1 KB direct-mapped cache with 32 blocks, how many cache misses will occur?
How many of these misses will be compulsory and conflict misses. Mention the reason behind classifying a miss as a compulsory or a conflict miss. Note that there will be no capacity miss as the total amount of data accessed by the program is less than the capacity of the cache.

SOLUTION

Address breakdown

Block size $= 1\,\text{KB}/32 = 32$ B, so the offset is 5 bits, the index $\log_2 32 = 5$ bits, and the tag $32 - 10 = 22$ bits.



Block address = addr/32: `0x1000`/32 = 128 and `0x2000`/32 = 256. Index = 128 mod 32 = 0 and 256 mod 32 = 0. **Both blocks map to cache line 0**, with different tags (4 and 8).

Simulation (alternating A = `0x1000`, B = `0x2000`)

Access	Block	Line	Line 0 before	H/M	Type
1	A	0	empty	×	compulsory (first touch of A)
2	B	0	A	×	compulsory (first touch of B), evicts A
3	A	0	B	×	conflict, evicts B
4	B	0	A	×	conflict, evicts A
⋮	⋮	⋮	⋮	⋮	⋮
200	B	0	A	×	conflict

Each access evicts the block needed next, so every one of the $2 \times 100 = 200$ accesses misses (the hit rate is 0).

Answer. Total misses = **200**: **2 compulsory** + **198 conflict** (0 capacity).

Reason for the classification

- **Compulsory**: the first reference to each block (A once, B once). The block has never been in the cache, so it would miss in any cache, even an infinite one.

- **Conflict**: the other 198 misses are to blocks that were in the cache earlier but were evicted only because A and B compete for the *same* line (same index). Only 64 B of the 1 KB cache is in use, so it is not a capacity problem. A 2-way set-associative (or fully associative) cache of the same size would hold both blocks, and these 198 misses would disappear.

2012–2013 • Q7(b) CSE 305 • 10/05/2014 • MH51 **MARKS 9**

If a cache has a capacity of 16 KB and a line length (block size) of 128 bytes, how many sets does the cache have if it is 2-way, 4-way and 8-way set associative?

SOLUTION

$$\text{Number of lines} = \frac{16\,\text{KB}}{128\,\text{B}} = \frac{2^{14}}{2^{7}} = 2^{7} = 128, \qquad \text{sets} = \frac{\text{lines}}{\text{ways}}.$$

Associativity	Sets	Set-index bits	Lines per set
2-way	$128/2 = 64$	6	2
4-way	$128/4 = 32$	5	4
8-way	$128/8 = 16$	4	8

Answer. 2-way: **64 sets**; 4-way: **32 sets**; 8-way: **16 sets**. Doubling the associativity halves the number of sets; the 7-bit block offset is unchanged.

2012–2013 • Q7(c) CSE 305 • 10/05/2014 • MH52 **MARKS 12**

Consider a processor with following parameters:

Base CPI, no memory stalls	2.0
Processor speed	1 GHz
Main memory access time	100 ns
First-level cache miss rate per instruction	4%
Second level cache, direct-mapped speed	10 cycles
Global miss rate with second-level cache, direct mapped	4%
Second level cache, eight-way set associative speed	20 cycles
Global miss rate with second-level cache, eight-way set associative	1.6%

Calculate the CPI for the processor using (i) only a first-level cache, (ii) both a first-level cache and a second-level direct-mapped cache, (iii) both a first level cache and a second-level eight-way set associative cache.

SOLUTION

Method (slides' multilevel-cache example)

$\text{CPI} = \text{CPI}_{\text{base}} + m_{L1}\,t_{L2} + m_{\text{global}}\,t_{\text{mem}}$: every L1 miss pays the L2 time; the global-miss fraction also pays the memory time. Cycle = 1 ns ⇒ memory = $100/1 = 100$ cycles.

$$\begin{aligned}
\text{(i) L1 only:} \quad & 2 + 0.04 \times 100 = 6.0 \\
\text{(ii) L1 + DM L2:} \quad & 2 + 0.04 \times 10 + 0.04 \times 100 = 2 + 0.4 + 4 = 6.4 \\
\text{(iii) L1 + 8-way L2:} \quad & 2 + 0.04 \times 20 + 0.016 \times 100 = 2 + 0.8 + 1.6 = 4.4
\end{aligned}$$

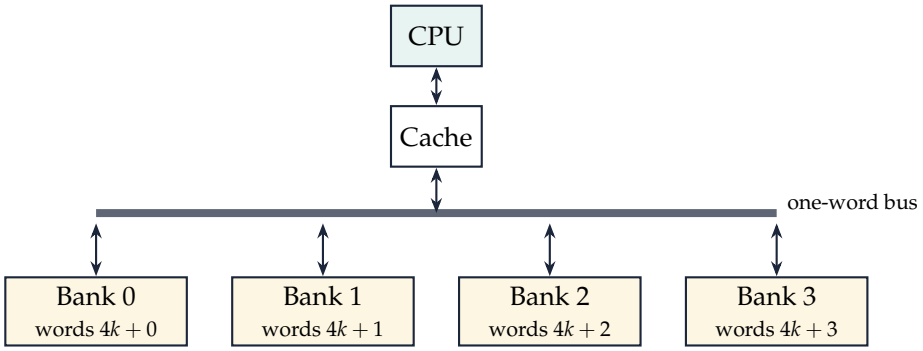
Answer. CPI $= 6.0, 6.4, 4.4$. The direct-mapped L2 is useless here: its global miss rate (4%) equals the L1 miss rate, so every L1 miss still goes to memory and also pays 10 extra cycles. The 8-way L2 cuts the misses to memory to 1.6% and is $6.0/4.4 = 1.36\times$ faster than L1 only.

2012–2013 • Q7(d) CSE 305 • 10/05/2014 • MH53 **MARKS 4**

Briefly discuss interleaved memory organization.

SOLUTION

In an interleaved memory, main memory is split into several independent **banks**. Consecutive words go to consecutive banks: bank = (word address) mod (number of banks). One address is sent to all banks, they perform their (slow) DRAM accesses *in parallel*, and the words are then transferred one after another over a one-word-wide bus.



Example (1 cycle to send the address, 15 cycles per DRAM access, 1 cycle per word transfer, 4-word block):

- One-word-wide memory: $1 + 4 \times 15 + 4 \times 1 = 65$ cycles, i.e. $16/65 \approx 0.25$ B/cycle.
- 4-bank interleaved: $1 + 15 + 4 \times 1 = 20$ cycles, i.e. $16/20 = 0.8$ B/cycle.

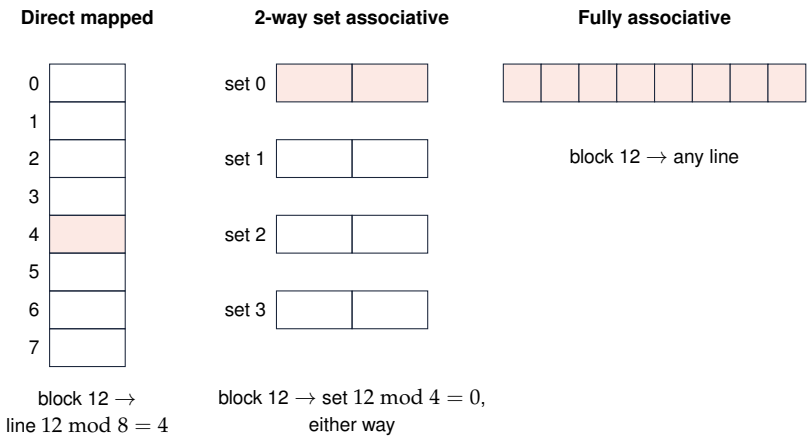
Answer. Interleaving overlaps the access times of several banks, so a block miss costs about one DRAM access time plus the transfers. This raises bandwidth and lowers the miss penalty without the cost of a wide bus (DRAM banking).

2011–2012 • Q5(b) CSE 305 • 27/07/2013 • MH54 **MARKS 6**

What are the 'direct mapped', 'set associative', and 'fully associative' cache schemes?

SOLUTION

The three schemes differ in *where* a memory block may be placed in the cache:



	Direct mapped	***n*-way set associative**	**Fully associative**
Placement	Exactly one line: (block addr) mod (#lines).	Any of the n lines of set (block addr) mod (#sets).	Any line in the cache.
Address fields	Tag | Index | Offset	Tag | Set | Offset	Tag | Offset
Search / compara-tors	1 tag compare.	n compares in parallel.	Compare with every entry (one comparator per entry).
Replacement	No choice.	LRU/random within the set.	LRU/random over all lines.
Pros / cons	Simplest, fastest hit; most conflict misses.	Fewer conflict misses; moderate cost.	No conflict misses; costly and slow for large caches.

Direct mapped is 1-way set associative, and fully associative is one set with all lines.

2011–2012 • Q5(c) CSE 305 • 27/07/2013 • MH55 **MARKS 15**

Assume there are three small coaches [caches], each consisting of 4 one-word blocks. One each is fully associative, another one is 2-way set associative, and the third one is direct mapped. Find the number of misses for each cache organization given the following sequence of block addresses: 0, 4, 0, 2, 6, 8.

SOLUTION

Method (slides: "Associativity Example")

Direct-mapped: index = block mod 4. Two-way: 2 sets, set = block mod 2. Fully associative: any of 4 ways. On a miss fill an empty way, else replace the **LRU** block (LRU assumed, as in the slides).

Fully associative

Blk	H/M	W0	W1	W2	W3
0	×	0			
4	×	0	4		
0	✓	0	4		
2	×	0	4	2	
6	×	0	4	2	6
8	×	0	8	2	6

misses 5, hits 1

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1
0	0	×	0			
4	0	×	0	4		
0	0	✓	0	4		
2	0	×	0	2		
6	0	×	6	2		
8	0	×	6	8		

misses 5, hits 1

Direct-mapped

Blk	Idx	H/M	0	1	2	3
0	0	×	0			
4	0	×	4			
0	0	×	0			
2	2	×	0		2	
6	2	×	0		6	
8	0	×	8		6	

misses 6, hits 0

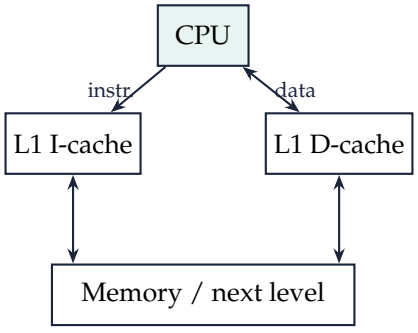
Answer. Misses: fully associative 5, two-way 5, direct-mapped 6.

2011–2012 • Q5(d) CSE 305 • 27/07/2013 • MH56 **MARKS 2 + 2**

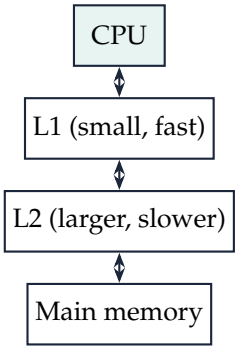
What is a 'split cache'? How is it different from the multilevel cache system?

SOLUTION

NOTE The scan prints "spit cache"; it has been corrected by hand to "split" on the paper.



Split cache (side by side, same level)



Multilevel cache (stacked levels)

- **Split cache**: the cache at one level (usually L1) is divided into a separate *instruction cache* and *data cache*. Both can be accessed in the same cycle (an instruction fetch and a load/store), which removes the structural hazard in a pipeline and doubles the bandwidth. Each half can also be tuned for its own access pattern. Its opposite is a *unified* cache.
- **Multilevel cache**: several caches in *series*. A small, fast L1 focuses on hit time; a larger L2 (and L3) catches L1 misses and focuses on a low miss rate, so fewer misses go to DRAM. This reduces the miss penalty.
- **Difference**: splitting divides one level *horizontally* by the type of content, to gain parallel access. Multilevel adds levels *vertically* by speed and size, to cut the miss penalty. They are combined in practice: split L1 I/D caches backed by a unified L2/L3.

2011–2012 • Q6(b) CSE 305 • 27/07/2013 • MH57 **MARKS 10**

Assume the miss rate of an instruction cache is 2% and the miss rate of a data cache is 4%. If a processor has a CPI of 6 without any memory stalls, determine how much faster a processor would run with a perfect cache that never misses. Assume a main memory access time of 100 ns

(including all the miss handling) and the processor clock rate is 4 GHz.

SOLUTION

Miss penalty in cycles

$$\text{Clock period} = \frac{1}{4\ \text{GHz}} = 0.25\ \text{ns}, \qquad \text{penalty} = \frac{100\ \text{ns}}{0.25\ \text{ns}} = 400\ \text{cycles}.$$

Stall cycles and CPI

Every instruction is fetched (I-cache), and only loads/stores, with frequency f, access the D-cache. Take $f = 36\%$ (see note):

$$\begin{aligned} \text{I-stalls} &= 0.02 \times 400 = 8\ \text{cycles/instr} \\ \text{D-stalls} &= 0.36 \times 0.04 \times 400 = 5.76\ \text{cycles/instr} \\ \text{CPI}_{\text{actual}} &= 6 + 8 + 5.76 = 19.76 \end{aligned}$$

$$\text{Speedup}_{\text{perfect}} = \frac{19.76}{6} \approx 3.29, \qquad \text{stall fraction} = \frac{13.76}{19.76} \approx 69.6\%.$$

Answer. The perfect-cache processor is $\approx$ **3.29**$\times$ faster (with 36% loads/stores).

NOTE The question does not give the load/store frequency. The value 36% is assumed, as in the slide/Patterson–Hennessy example. In general CPI $= 14 + 16f$, so the speedup is $(14 + 16f)/6$; e.g. with $f = 40\%$ the CPI is 20.4 and the speedup 3.4.

2011–2012 • Q6(c) CSE 305 • 27/07/2013 • MH58 **MARKS 5**

Write down the steps to handle a cache miss.

SOLUTION

On a hit the CPU proceeds normally. On a miss the control unit **stalls the pipeline** (it freezes the registers, unlike an interrupt) and fetches the block from the next level:

(i) **Identify the miss.** The tag does not match or $V = 0$. For an instruction-cache miss, the original PC (PC $-$ 4, the faulting fetch address) is sent to memory; for a data miss, the computed data address is sent.

(ii) **Free a line.** Choose the victim: the indexed line in a direct-mapped cache, or the LRU/random way in the set. In a write-back cache, write the victim to memory first if it is dirty.

(iii) **Read the block.** Instruct main memory (or L2) to read the block and wait for the access (the miss penalty).

(iv) **Write the cache entry.** Put the block's data in the line, write the upper address bits into the tag field, and set $V = 1$ (and dirty $= 0$).

(v) **Restart.** For an instruction miss, restart instruction execution at the first step, re-fetching the instruction, which now hits. For a data miss, complete the data access (load/store) and continue.

Answer. Stall, send the miss address to memory, (write back a dirty victim), read the block, write data/tag/valid into the cache, then restart the fetch or complete the data access.

2011–2012 • Q6(d) CSE 305 • 27/07/2013 • MH59 **MARKS 5**

Discuss the components of disk data access time.

SOLUTION

$$T_{\text{access}} = T_{\text{queue}} + T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}} + T_{\text{controller}}$$

- **Queuing delay**: waiting while earlier requests are served. It is 0 for an idle disk.
- **Seek time**: moving the arm/heads to the right track. Manufacturers quote the average over all possible seeks. Locality and OS scheduling make the real average smaller.
- **Rotational latency**: waiting for the sector to come under the head. On average this is half a revolution: $T_{\text{rot}} = \dfrac{0.5}{\text{rpm}/60}$.
- **Transfer time**: $\dfrac{\text{sector size}}{\text{transfer rate}}$, the time to read or write the data bits.
- **Controller overhead**: the disk controller's processing time to set up and complete the transfer.

Example (slides): 512 B sector, 15,000 rpm, 4 ms seek, 100 MB/s, 0.2 ms controller, idle disk. Then $T = 4 + 2 + 0.005 + 0.2 = 6.2$ ms, where seek and rotation dominate.

2011–2012 • Q7(a) CSE 305 • 27/07/2013 • MH60 **MARKS 18**

(i) What do you understand by the 'virtual memory' scheme? (3)

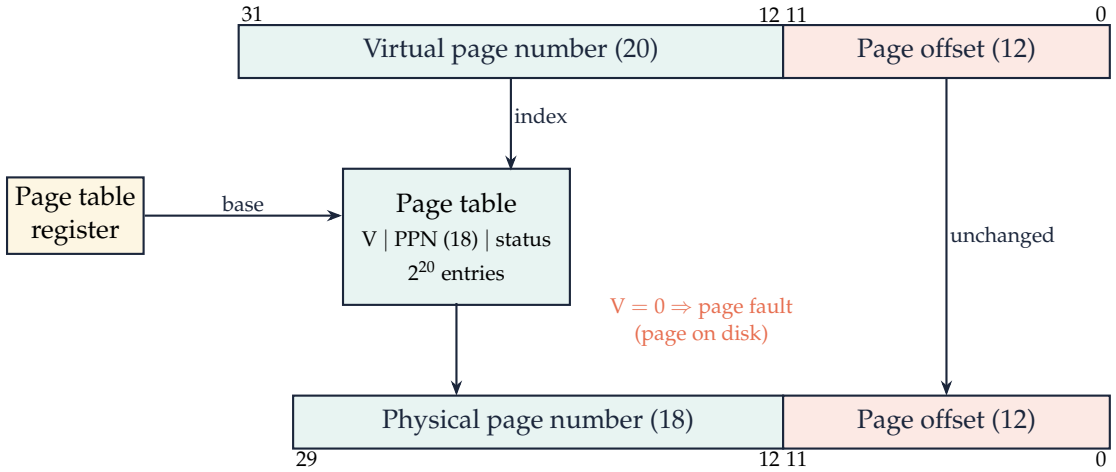
(ii) Explain the purpose of a 'page table' and a 'page table register' in this scheme. (4+2)

(iii) Draw the mapping from a 32-bit virtual address to a 30-bit physical address using a page table. (6)

(iv) With a 32-bit virtual address, 4 kB pages and 4 bytes per page table entry, compute the total size of a page table. (5)

SOLUTION

(i) Virtual memory: main memory is used as a "cache" for secondary storage (disk). Each program gets its own large private virtual address space; the CPU and OS jointly translate virtual addresses to physical addresses page by page. Benefits: programs larger than DRAM, safe sharing and protection between programs, relocation. A VM miss is a *page fault*.
(ii) Page table: an array of PTEs in memory, indexed by the virtual page number; each PTE holds the physical page number (or disk location) plus valid, dirty, reference and protection bits. **Page table register:** a CPU register that holds the physical starting address of the current process's page table; the OS reloads it on a context switch.
(iii) Mapping 32-bit VA → 30-bit PA (4 KB pages assumed, as in the slides and part (iv)): VPN 20 bits, offset 12 bits, PPN 18 bits.



(iv) $2^{32}/2^{12} = 2^{20}$ entries $\times 4$ B:

Answer. Page table size $= 2^{20} \times 4\,\text{B} = 4\,\text{MB}$.

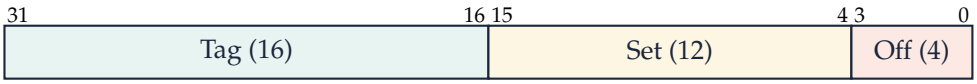
2011–2012 • Q8(a) CSE 305 • 27/07/2013 • MH61 **MARKS 10**

Assuming a cache of 16k blocks, a 4-word block size and a 32 bit address, find the total number of tag bits for a 4-way set associative cache. Also find the total size of the cache. [byte offset = 2 bits, 1 word = 32 bits, and 1k = 2^{10}].

SOLUTION

Fields

Offset = 2 (word in block) +2 (byte) = 4 bits. Sets = $2^{14}/4 = 2^{12} \Rightarrow$ index = 12 bits. Tag = $32 - 12 - 4 = 16$ bits.



Tag bits and total size

$$\text{Tag bits} = 16 \times 2^{14} = 262{,}144; \qquad \text{Total} = 2^{14} \times (128 + 16 + 1) = 2{,}375{,}680 \text{ bits}$$

Answer. Total tag bits = 262,144 (256 Kbit); total cache size = 2,375,680 bits = 2320 Kbit = 290 KB (256 KB data + tags + valid bits).

08

Graphics and Computing GPUs

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q4(a) CSE 209 • 29/04/2026 • GP01 MARKS 10

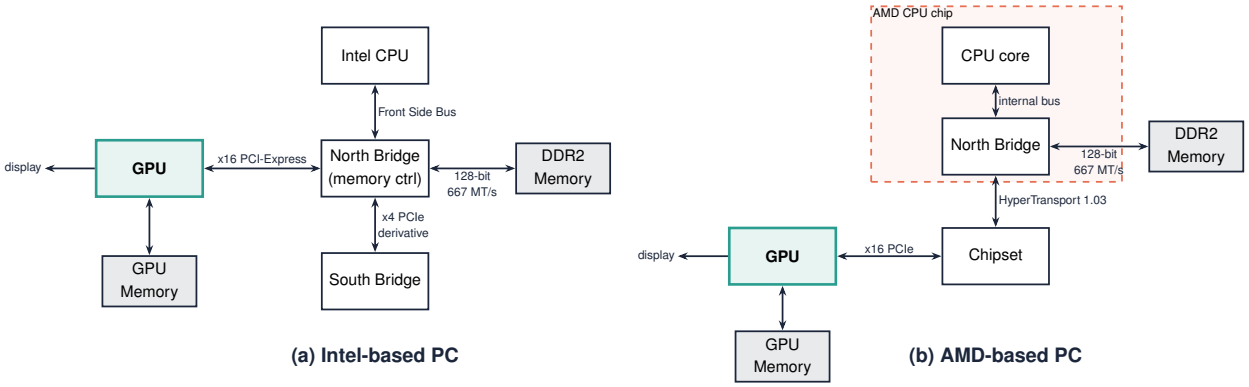
Show with block diagrams how contemporary PCs with intel and AMD CPUs connect a separate GPU. How do the CPU and GPU access each other's memory?

2021–2022 • Q4(b) CSE 305 • 06/04/2023 • GP01 MARKS 10

Show with block diagrams how contemporary PCs with Intel and AMD CPUs connect a separate GPU. How do the CPU and GPU access each other's memory?

SOLUTION

Block diagrams (discrete GPU)



- **Intel:** the CPU talks over the Front Side Bus to the North Bridge, which holds the memory controller for the DDR2 system memory. The GPU sits on a 16-lane *PCI-Express* link to the North Bridge (peak 16 GB/s, 8 GB/s each way) and has its own GDDR *GPU memory*. The South Bridge handles legacy I/O.
- **AMD:** the North Bridge (memory controller) is *integrated on the CPU die*, so system memory hangs directly off the CPU. The CPU reaches a separate chipset over HyperTransport, and the GPU is attached to the chipset by x16 PCI-Express, again with its own local memory.

How CPU and GPU access each other's memory

- Each processor has its own *local* memory and a separate physical address space. Either one can reach the other's memory across PCI-Express, but with much lower bandwidth and higher latency than its own local memory.
- **GPU → system memory:** the GPU issues *virtual* addresses, which an *MMU on the GPU* translates. The GPU page tables are managed by the OS kernel/driver. A system physical page is accessed

by either *coherent* or *non-coherent* PCIe transactions, as chosen by an attribute bit in the GPU page table.

- **CPU → GPU memory:** the GPU's local memory is mapped into an *aperture*, an address range in the PCI-Express address space. CPU loads and stores to that range are forwarded over PCIe to GPU memory.
- (In low-cost *UMA* systems the GPU has no dedicated memory and shares system memory, at lower performance.)

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q4(c) CSE 209 • 29/04/2026 • GP02 **MARKS 10**

What do you mean by kernel, grid, and thread block of the CUDA paradigm? Briefly describe how they map to the streaming processor (SP) cores and streaming multiprocessors (SM) of the unified GPU architecture of NVIDIA.

2021–2022 • Q4(c) CSE 305 • 06/04/2023 • GP02 **MARKS 10**

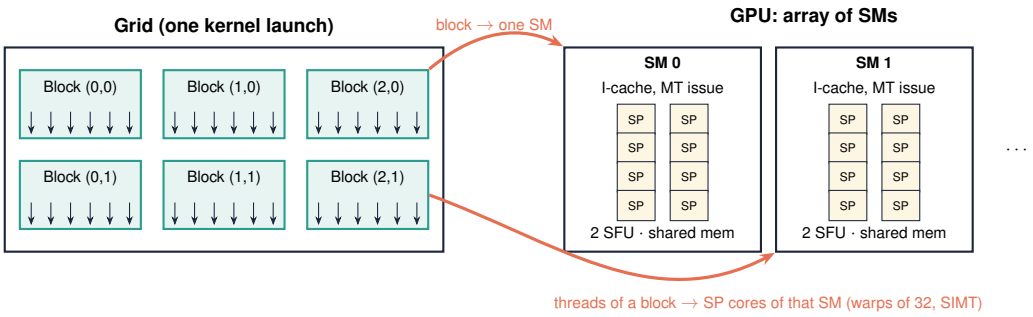
What do you mean by kernel, grid, and thread block of the CUDA paradigm? Briefly describe how they map to the streaming processor (SP) cores and streaming multiprocessors (SM) of the unified GPU architecture of NVIDIA.

SOLUTION

Definitions

- **Kernel:** a C/C++ function written for *one* sequential thread, launched to run in parallel over many threads (SPMD style). Example: `saxpy<<<nblocks, 256>>>(n, a, x, y)`, where each thread computes one $y[i]$.
- **Thread block:** a group of concurrent threads (up to 512 on Tesla) that run the same kernel. They can *cooperate* through per-block *shared memory* and *barrier synchronisation* (`__syncthreads()`). Each thread has an ID (`threadIdx`).
- **Grid:** the set of independent thread blocks (indexed by `blockIdx`) that execute one kernel launch. Blocks must be able to run in any order, in parallel or serially, which gives transparent scalability. Grids of dependent kernels are separated by an inter-kernel barrier.

Mapping onto the unified GPU (e.g. GeForce 8800 / Tesla: 14 SMs × 8 SPs = 112 SP cores)



- **Grid → whole GPU:** a kernel launch creates a grid. The GPU's work-distribution unit hands its thread blocks out to the available SMs. The number of blocks is set by the data size, not the hardware, so the same code runs on GPUs with more or fewer SMs.
- **Thread block → one SM:** each block runs entirely on a single SM, which may hold several blocks at once if registers and shared memory allow. The block's shared memory lives in that SM's on-chip shared memory, and `__syncthreads()` is an SM-local barrier.
- **Thread → SP core:** the SM's multithreaded issue unit groups the block's threads into *warps* of 32 and issues one instruction per warp (SIMT) across the 8 SP cores (plus the 2 SFUs for special functions). Each SP is hardware-multithreaded and keeps the state of many threads (up to 96), so it hides memory latency by switching warps. Per-thread local data lives in SP registers or local memory. Global memory is the DRAM shared by the whole grid.

09

CPU Organization, Control Unit Design and Exceptions

NOTE **Not in the course slides.** None of the ART/RS slide decks covers this material. The questions come from older CSE 305 papers, and the solutions follow standard textbook treatments (Patterson & Hennessy, *Computer Organization and Design*; Hamacher; Stallings; Mano).

2022–2023 • Q8 NOT IN SLIDES CSE 209 • 13/01/2025 • CU01 MARKS 15

Make the necessary changes to incorporate precise exception handling in pipelined version of MIPS hardware. You can make the changes in the Figure given in Figure for Question 8 and have that page stapled securely with your answer script. The question page containing the figure must then be signed by the invigilator.
You do not need to draw any input signal for *Hazard detection unit*. If any output signal is needed for incorporating precise exception handling you must draw that. For any other output signal which isn't necessary for exception handling you do not need to draw it.

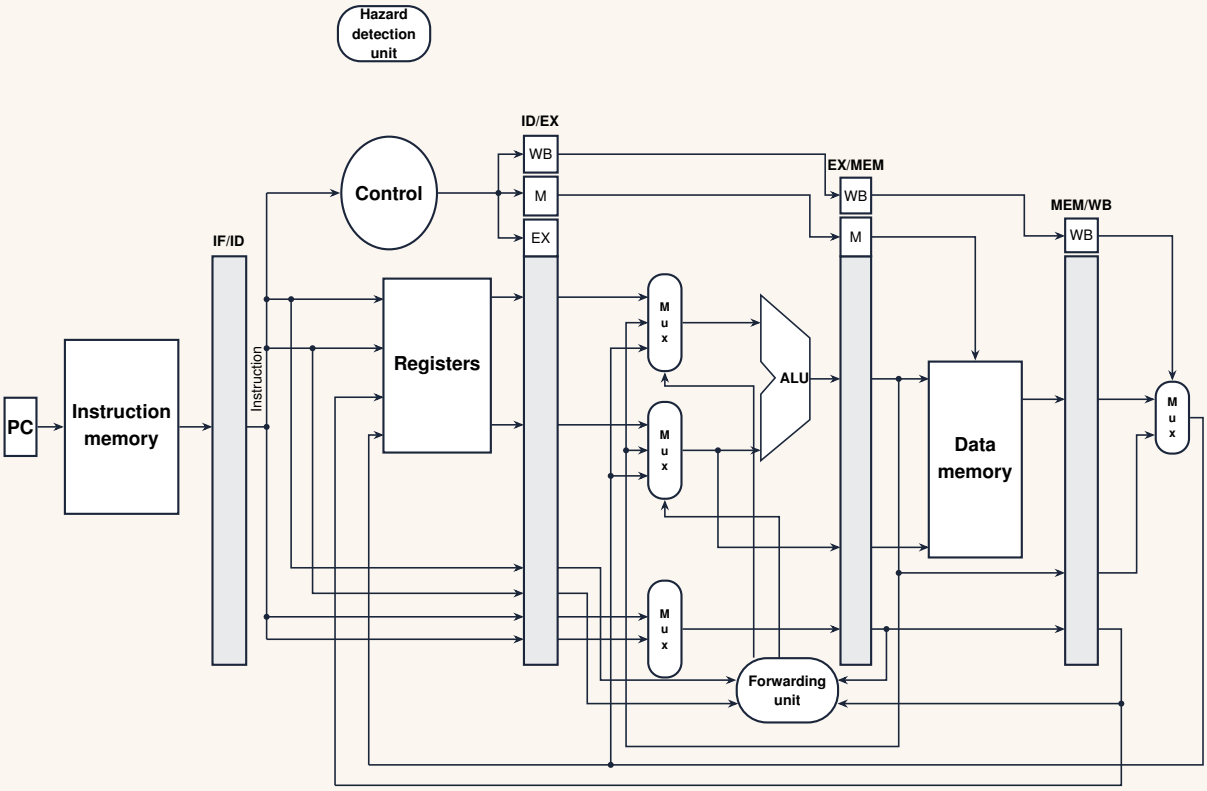


Figure for Question 8

SOLUTION

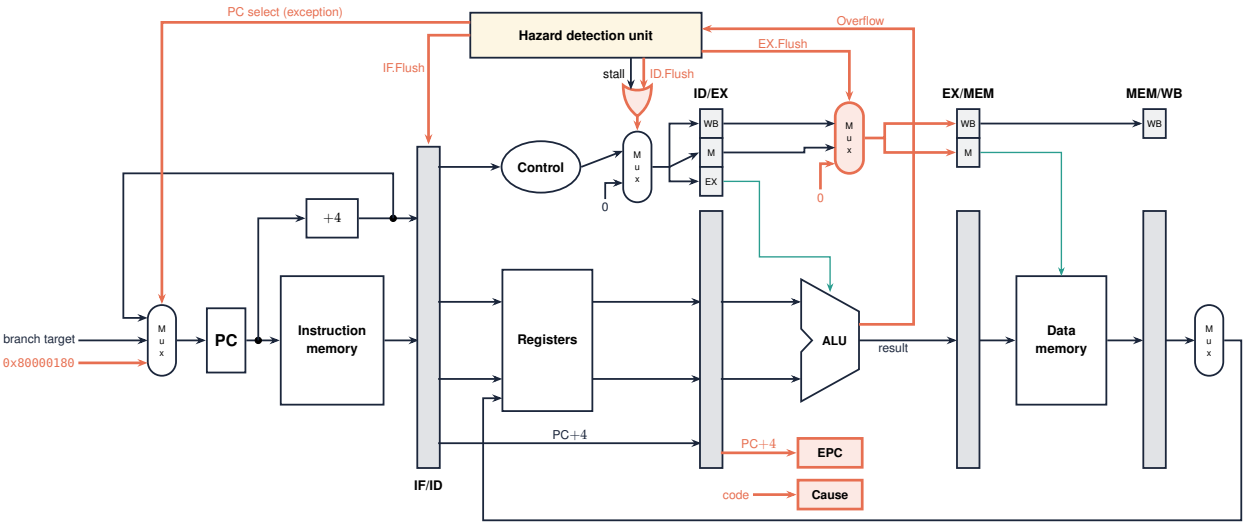
Precise exception: when instruction *I* raises an exception (e.g. arithmetic overflow of `add` in EX, or an undefined opcode in ID), every instruction *before I* completes, while *I* and every instruction *after* it leave no trace (no register or memory write). The address of *I* is saved so that the handler

can restart or report it. Three things are needed:

(i) **Flush** I and the younger instructions behind it: **IF.Flush** turns the instruction in IF/ID into a `nop`. **ID.Flush** (ORed with the stall signal) zeros the control bits entering ID/EX. **EX.Flush** zeros the WB and M control bits of I itself on its way into EX/MEM, so the overflowing `add` cannot write $rd.

(ii) **Record** the cause and the address: **Cause** register ← exception code (overflow = 12, undefined instruction = 10). **EPC** ← PC+4 of I, carried down the pipeline (the handler subtracts 4).

(iii) **Transfer control**: an extra input on the PC mux loads the handler address `0x80000180`.



Coral = hardware and outputs added for exceptions. The forwarding unit and its muxes are unchanged and omitted. Only the hazard-detection outputs needed for exceptions are drawn (plus the existing *stall*).

Output (from the hazard detection / exception logic)	Effect when an exception is detected in EX
IF.Flush	clears IF/ID, so the instruction just fetched becomes a `nop`
ID.Flush (OR stall)	control mux injects 0s into ID/EX, so the instruction in ID becomes a bubble
EX.Flush	muxes zero the WB and M bits of the faulting instruction going to EX/MEM, so no register or memory write happens
EPCWrite, CauseWrite	EPC ← PC+4 of the faulting instruction; Cause ← code
PC select	PC ← `0x80000180` (OS exception handler) on the next clock

Instructions already in MEM and WB are older than the faulting one, so they finish normally. That is what makes the exception *precise*.

2020–2021 • Q11(b) NOT IN SLIDES CSE 305 • 24/09/2022 • CU02 MARKS 2 + 6 = 8

Consider the following figure (Figure 11b) of a processor unit employing a scratchpad memory. How many micro operations does it require to complete an addition operation with two operands, and store the result? Which changes and constraints are required to reduce the number of micro operations to only one?

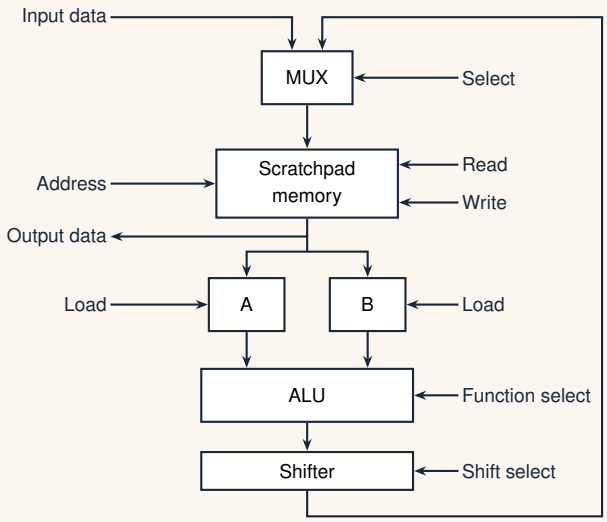


Figure 11(b): Processor unit employing a scratchpad memory

SOLUTION

Micro-operations with the given unit (single-port scratchpad)

The scratchpad has one address input and one data port, so only one word can be read or written per micro-operation. For $M[z] \leftarrow M[x] + M[y]$:

(i) $A \leftarrow M[x]$ (Address $= x$, Read, Load A)

(ii) $B \leftarrow M[y]$ (Address $= y$, Read, Load B)

(iii) $M[z] \leftarrow A + B$ (Function = add, Shift = none, MUX selects the shifter output, Address $= z$, Write)

Answer. 3 micro-operations.

Changes and constraints for a single micro-operation

- Replace the single-port scratchpad with a **multi-port register file**: *two read ports and one write port*, with three separate address inputs (source A, source B, destination). Both operands can then be read in the same cycle.
- Feed the two read buses *directly* to the ALU inputs (or make A and B transparent latches), so no separate "load A" and "load B" steps are needed. Route the shifter output to the write port.
- The control word must carry all three addresses together with the ALU function, shift select and write enable. This gives a wider (horizontal) microinstruction.
- **Timing constraint:** the clock period must cover register read + ALU + shifter + write setup, and the write must occur at the *end* of the cycle (clock edge), after the operands have been read. Reading and writing the same location in one cycle must therefore be well defined, i.e. the old value is read and the new one written.

Then $R_z \leftarrow R_x + R_y$ is *one* micro-operation.

2018–2019 • Q2(c) NOT IN SLIDES CSE 305 • 29/10/2019 • CU03 MARKS 15

Discuss how a TLB miss is handled by MIPS architecture. Consider the pipelined datapath in Figure 2.c. that is capable of handling underfined instruction and overflow exceptions. Now, discuss how TLB miss exception may be handled there. You may update Figure 2.c if you feel necessary. You can make the changes in the question page containing Figure 2.c and have that stapled with your script. The question page must then be signed by the invigilator.

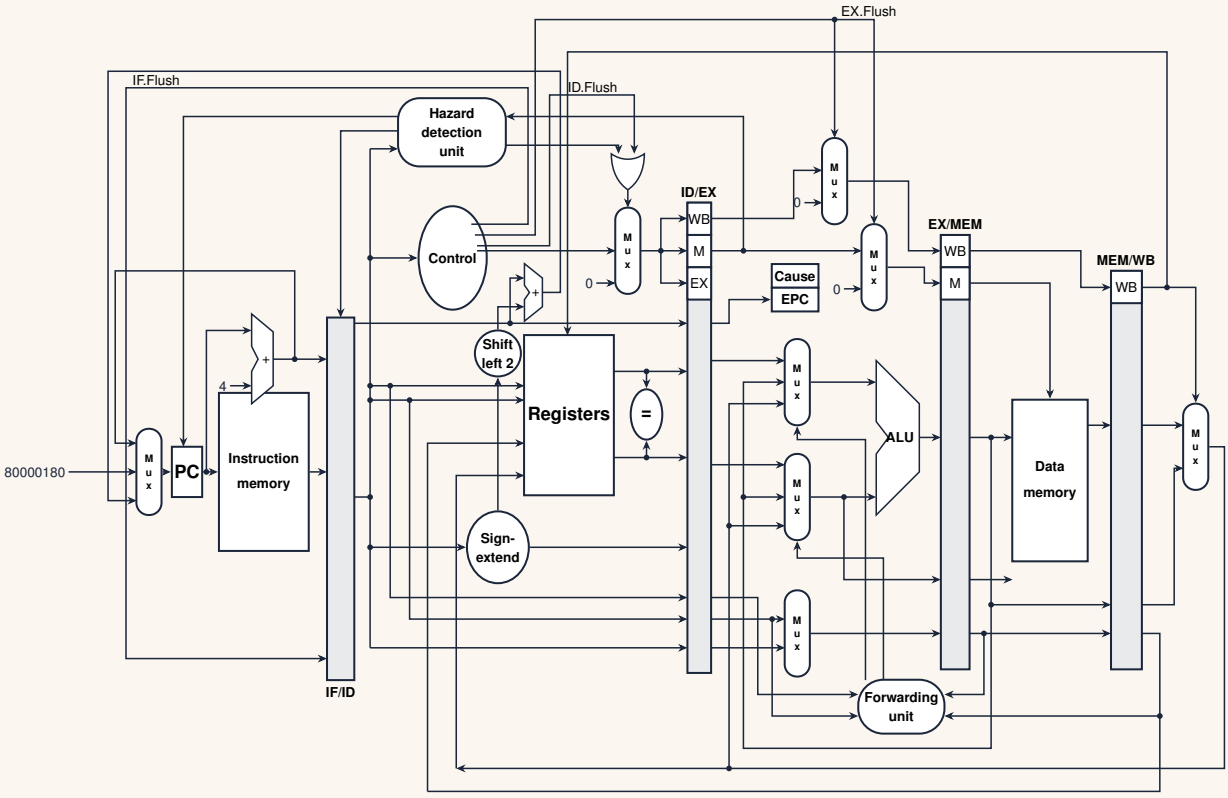


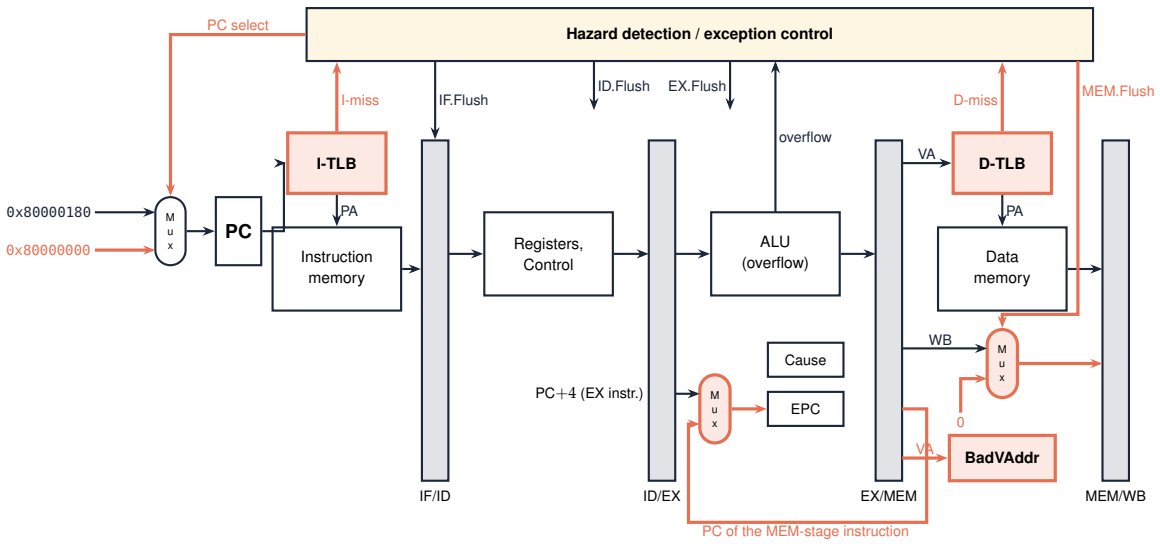
Fig 2.c

SOLUTION

How MIPS handles a TLB miss

- MIPS handles TLB misses in *software*. A miss raises an **exception**: Cause ← TLB-miss code (TLBL for a load or fetch, TLBS for a store). EPC ← address of the faulting instruction. **BadVAddr** (and Context) ← the virtual address that missed. Then the PC jumps to the *TLB-refill handler* (`0x80000000`; other exceptions use `0x80000180`).
- The handler (about a dozen instructions) uses Context to load the PTE from the page table in memory. It writes the PTE into the TLB (`tlbwr`, random entry) and returns with `eret`, which *re-executes* the faulting instruction, which now hits.
- If the PTE is invalid (page not in memory), this is a *page fault*: the OS handler is invoked, the page is brought from disk, and the process is resumed later. The exception must therefore be *precise and restartable*.

Changes to the pipeline of Fig 2.c (coral)



- **I-TLB miss** (detected in IF): the instruction has not done anything yet. Assert IF.Flush, set EPC ← PC and BadVAddr ← PC, Cause ← TLBL, and select the refill vector.
- **D-TLB miss** (detected in MEM for `lw`/`sw`) is new, because Fig 2.c only flushes up to EX. Add **MEM.Flush**: a mux zeros the *WB* control going into MEM/WB, and the memory write enable is gated off, so the `lw`/`sw` has no effect. IF.Flush, ID.Flush and EX.Flush are asserted for the three younger instructions.
- **EPC** must now record the PC of the instruction in MEM. Carry PC(+4) down into EX/MEM and add a mux at the EPC input. **BadVAddr** ← the ALU-computed address held in EX/MEM (or the PC for an I-miss).
- **PC mux** gets the refill vector `0x80000000` as an extra input (or, more simply, `0x80000180` with Cause = TLB miss).
- If several exceptions occur in the same cycle, the *oldest* instruction (deepest stage, MEM first) is serviced, keeping the exception precise. After the refill, `eret` restarts the instruction.

2016–2017 • Q1(a–d) NOT IN SLIDES CSE 305 • 02/08/2017 • CU04 MARKS 10 + 10 + 10 + 5

Consider a CPU which has PC, MAR, MDR, IR, ALU and Instruction Decoder. There are six general purpose registers R0, R1, R2, R3, R4, R5 and a temporary register TEMP inside the CPU. The CPU gets one input directly from the bus and the other input through a register Y. The output of the ALU is sent to the bus through a register Z. The ALU can perform ten arithmetic-logic operations and has a special input for Carry-in.

(a) Draw a block diagram of single bus datapaths inside the CPU following the description above. (10)

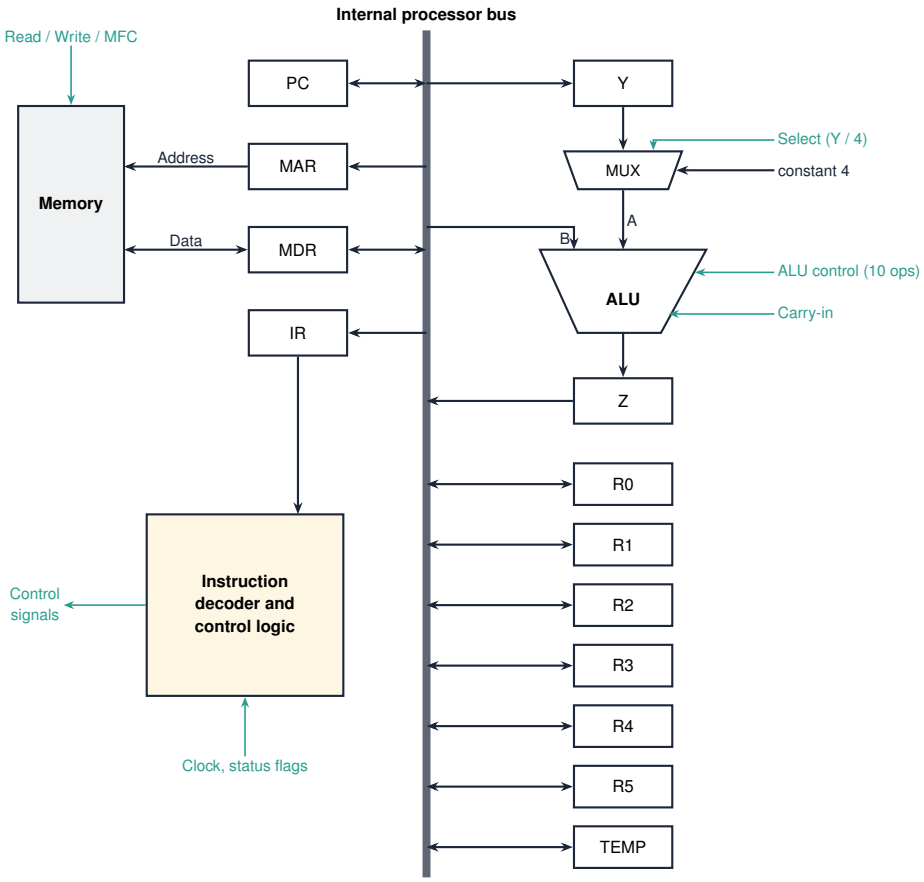
(b) Show step-wise control signals for executing the instruction **Add (R3), R1**, which adds the contents of a memory location pointed to by Register **R3** to Register **R1**. Assume that the instruction is stored in memory and PC contains the address of the instruction. (10)

(c) Design a control word for the CPU above such that the number of bits in the control word is significantly smaller than the number of control signals. (10)

(d) What are the relative merits of horizontal and vertical microinstruction formats? (5)

SOLUTION

(a) Single-bus organisation (Hamacher)



Every register has *in* and *out* gating signals to the single bus. Only one register may drive the bus in a step, while several may load from it.

(b) Control steps for `Add (R3), R1`

Step	Control signals	Action
1	PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}	MAR ← PC; start fetch; Z ← PC +4
2	Z_{out}, PC_{in}, Y_{in}, WMFC	PC ← PC+4 (Y keeps it); wait for memory
3	MDR_{out}, IR_{in}	IR ← instruction
4	$R3_{out}$, MAR_{in}, Read	MAR ← R3; read operand
5	$R1_{out}$, Y_{in}, WMFC	Y ← R1; wait for memory
6	MDR_{out}, SelectY, Add, Z_{in}	Z ← Y + M[R3]
7	Z_{out}, $R1_{in}$, End	R1 ← Z; next instruction

(c) Encoded (vertical-style) control word

Signals that can never be active in the same step are grouped into fields, and each field is binary-encoded with an all-zero “no action” code.

Field	Purpose	Bits	Codes (one per field)
F1	bus source (out)	4	none, PC_{out}, MDR_{out}, Z_{out}, $R0_{out}$..$R5_{out}$, $TEMP_{out}$ (11 codes)
F2	destination group 1	4	none, PC_{in}, IR_{in}, Z_{in}, $R0_{in}$..$R5_{in}$, $TEMP_{in}$ (11)
F3	destination group 2	2	none, MAR_{in}, MDR_{in}, Y_{in}
F4	ALU function	4	10 operations (Add, Sub, AND, ...)
F5	memory	2	none, Read, Write
F6	MUX select	1	Select Y / Select 4
F7	carry-in	1	0 / 1
F8	wait	1	WMFC
F9	end	1	End

Fields F2 and F3 are separate because some steps load two registers at once ($PC_{in} + Y_{in}$ in step 2, $MAR_{in} + Z_{in}$ in step 1).

Answer. Encoded word $= 4+4+2+4+2+1+1+1+1 =$ **20** bits, versus $10+13+10+1+1+2+1+1 =$ **39** individual control signals (10 out, 13 in, 10 ALU, Carry-in, Select, Read/Write, WMFC, End).

(d) Horizontal vs vertical microinstructions

	Horizontal	Vertical
Format	one bit per control signal, little or no encoding	highly encoded fields
Word length	long	short (smaller control store)
Parallelism	many micro-operations per word, fewer microinstructions per routine	limited, so more microinstructions per routine
Speed	fast, no decoding delay	slower, needs field decoders
Flexibility	any combination of signals possible	only combinations allowed by the encoding

2012–2013 • Q3(a–d) NOT IN SLIDES CSE 305 • 10/05/2014 • CU05 MARKS 17 + 8 + 5 + 5

(a) Present and explain a microprogram for the control unit of the multicycle implementation illustrated in Figure 3. (17)
(b) Modify the implementation of Figure 3 so that it can handle the **Underfined Instruction Exception** and the **Overflow Exception**. (8)
[You may modify Figure 3 (separate sheet) and attach the updated figure with your answer script after having it signed by the respected invigilator.]
(c) Which stage should you detect the **overflow exception** in so as to ensure that the offending instruction does not change the content of the destination register? Justify your answer. (5)
(d) Your friend claims that the *PCWriteCond* signal can be safely replaced by *PCSource[0]*. Do you agree with him? Justify your answer. (5)

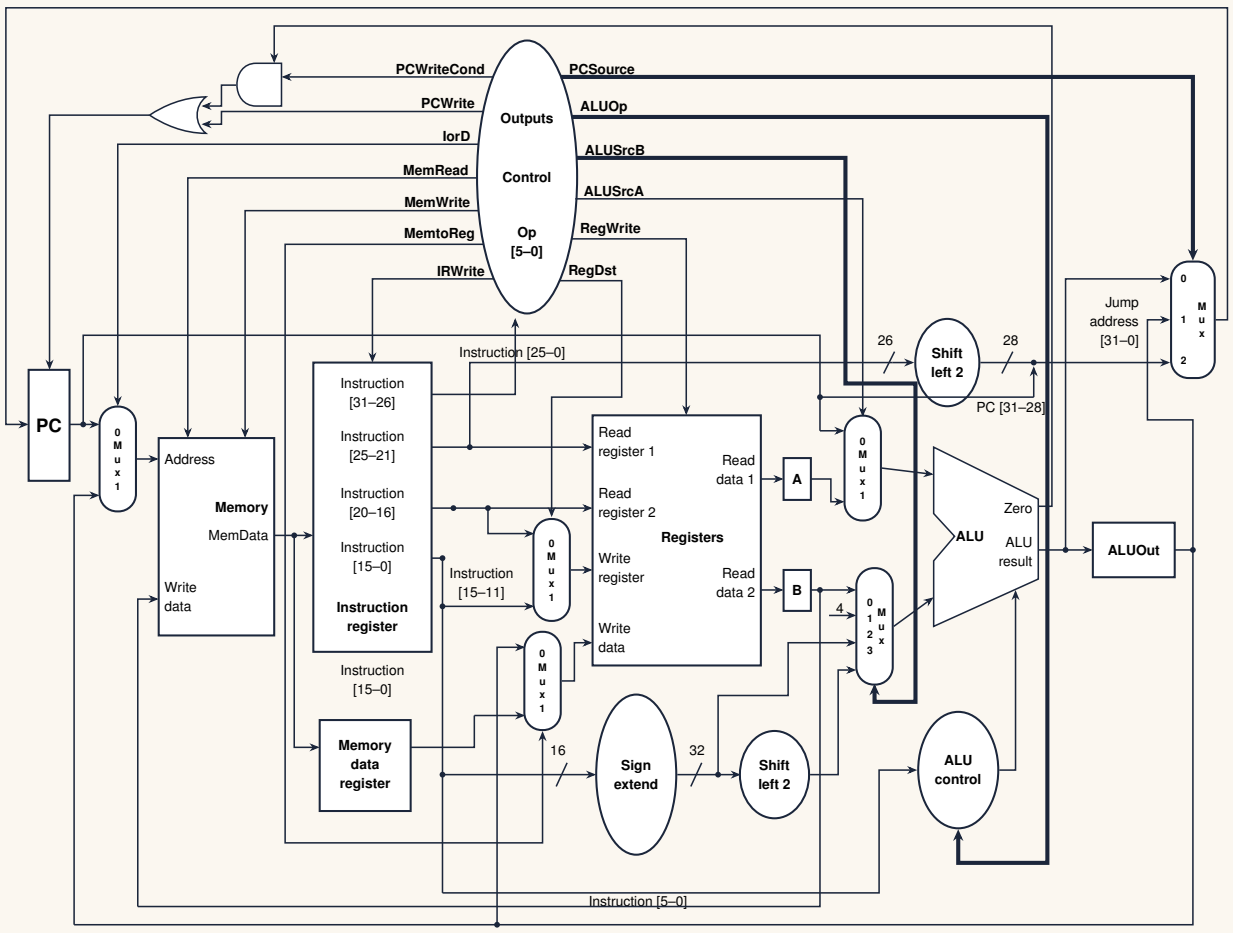


Figure 3

SOLUTION

(a) Microprogram (P&H microinstruction format)

Fields: *ALU control* (Add / Subt / Func code), *SRC1* (PC or A → ALUSrcA), *SRC2* (B, 4, Extend, Extshft → ALUSrcB), *Register control* (Read; Write ALU; Write MDR), *Memory* (Read PC; Read ALU; Write ALU), *PCWrite control* (ALU; ALUOut-cond; Jump address) and *Sequencing* (Seq; Fetch; Dispatch *i*).

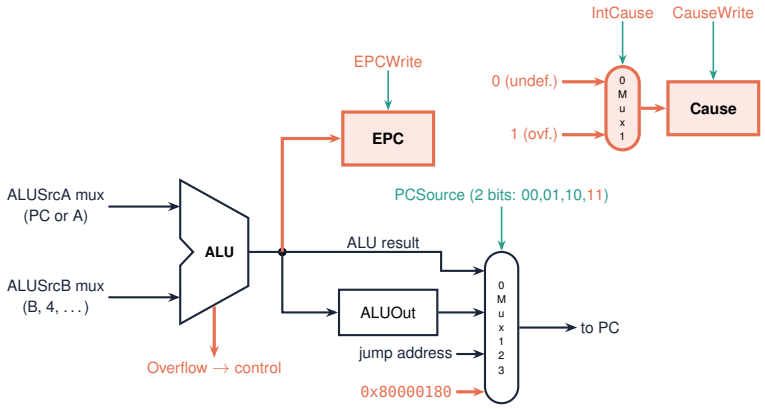
Label	ALU ctrl	SRC1	SRC2	Reg. control	Memory	PCWrite ctrl	Sequencing
Fetch	Add	PC	4		Read PC	ALU	Seq
	Add	PC	Extshft	Read			Dispatch 1
Mem1	Add	A	Extend				Dispatch 2
LW2					Read ALU		Seq
				Write MDR			Fetch
SW2					Write ALU		Fetch
Rformat1	Func code	A	B				Seq
				Write ALU			Fetch
BEQ1	Subt	A	B			ALUOut-cond	Fetch
JUMP1						Jump address	Fetch

Dispatch ROM 1 (on opcode): R-format → Rformat1, `j` → JUMP1, `beq` → BEQ1, `lw/sw` → Mem1. Dispatch ROM 2: `lw` → LW2, `sw` → SW2.

- **Fetch:** IR ← Mem[PC], PC ← PC+4. **Decode:** read rs and rt into A and B, ALUOut ← PC + (sext(imm)≪2), the branch target computed speculatively; then dispatch on the opcode.

- **Mem1** computes A + sext(imm). **LW2** reads memory into MDR, then writes MDR into rt. **SW2** writes B to memory.
- **Rformat1** computes A op B, then writes ALUOut into rd. **BEQ1** subtracts and writes PC ← ALUOut if Zero. **JUMP1** loads the jump address.

(b) Adding the two exceptions



- **Hardware** (coral): an **EPC** register loaded from the ALU output (EPCWrite); a **Cause** register (CauseWrite) loaded through a 2-input mux selected by **IntCause** (0 = undefined instruction, 1 = arithmetic overflow); a 4th input on the PCSource mux, the constant `0x80000180` (PCSource = 11); and the ALU's **Overflow** output sent to the control unit.
- **Microprogram additions.** Dispatch ROM 1 sends any *undefined opcode* to a new microinstruction, and Rformat1 branches to an overflow microinstruction if Overflow = 1. Both do the following in one cycle and then go to Fetch:
  ALU: Subt, SRC1 = PC, SRC2 = 4 (EPC ← PC −4, the address of the offending instruction, because PC was already incremented); EPCWrite; IntCause = 0 or 1; CauseWrite; PCSource = 11, PCWrite.

(c) Where to detect overflow

Detect it at the end of the **execution step of R-type (Rformat1, cycle 3)**, i.e. *before* the R-type completion step ("Write ALU", cycle 4). In the multicycle design the register write happens in a *separate later* cycle. So if the control unit tests Overflow while leaving the execution state and goes to the exception state instead of the write-back state, RegWrite is never asserted, and rd keeps its old value. Detecting it later, in or after write-back, would be too late.

(d) PCWriteCond vs PCSource[0]

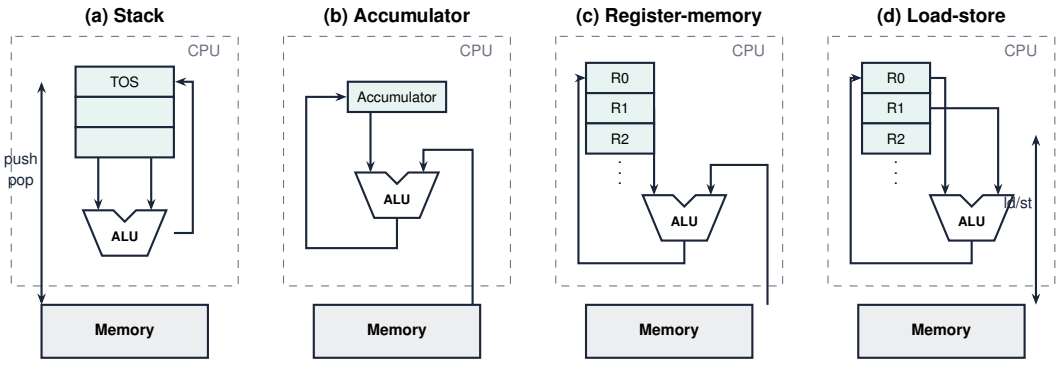
Agree, with a condition. PCWriteCond is asserted only in the `beq` state (BEQ1), and that is also the only state with PCSource = 01. Fetch uses 00 and Jump uses 10, so in every state where the PC can be written, PCSource[0] = 1 exactly when PCWriteCond = 1. The replacement is safe *provided* PCSource is no longer treated as a don't-care in other states: it must be forced to 0 wherever the PC must not be conditionally written. It must also be revisited if new encodings appear: for example, the exception code PCSource = 11 has bit 0 = 1, but there PCWrite = 1 anyway, so it is harmless.

2011–2012 • Q1(b) NOT IN SLIDES CSE 305 • 27/07/2013 • CU06 MARKS 10

Briefly describe ISA classes with examples and block diagrams.

SOLUTION

ISAs are classified by *where the ALU operands come from* (H&P). The examples below all compute `C = A + B`, with A, B and C in memory.



Class	Code for C = A + B	Operands	Examples
Stack	`Push A; Push B; Add; Pop C`	implicit: the top of an operand stack; only push/pop touch memory	JVM, x87 FPU, B5000
Accumulator	`Load A; Add B; Store C`	one implicit operand (the accumulator) and one memory operand	EDSAC, PDP-8, 6502, 8085
Register-memory	`Load R1,A; Add R3,R1,B; Store R3,C`	registers, and one operand may be in memory	IBM 360, x86, (VAX: memory-memory)
Load-store (reg-reg)	`Load R1,A; Load R2,B; Add R3,R1,R2; Store R3,C`	ALU uses registers only; memory is reached only by load/store	MIPS, ARM, RISC-V, SPARC

Stack and accumulator ISAs give compact code but create a bottleneck on one implicit location. GPR (register) ISAs are faster and easier for compilers to use, and load-store gives the simple fixed-format instructions that pipelining favours.

2011–2012 • Q3(c) NOT IN SLIDES CSE 305 • 27/07/2013 • CU07 MARKS 6

Briefly describe the properties of hard-wired and microprogrammed control unit design.

SOLUTION

	Hard-wired control	Microprogrammed control
Implementation	combinational logic (gates, decoders, PLA) driven by opcode, step counter/FSM state and flags	control signals stored as microinstructions in a control memory (ROM), sequenced by a micro-PC
Speed	fast, no control-memory fetch	slower: each step needs a microinstruction read
Design / change	complex to design and debug; any change means redesigning the circuit	systematic, like writing a program; easy to modify or extend (firmware update)
Instruction set	suits small, simple instruction sets (RISC, e.g. MIPS)	suits large, complex instruction sets (CISC, e.g. x86, VAX)
Cost / area	less hardware for simple ISAs, but grows irregularly	regular structure; control store adds area

2011–2012 • Q4(f) NOT IN SLIDES CSE 305 • 27/07/2013 • CU08 MARKS 4 + 3

Why does the MIPS implementation use EPC and Cause registers for processing an exception? What is an imprecise exception?

SOLUTION

EPC and Cause. When an exception occurs, the hardware must save enough information for the OS to service it and possibly resume the program:

- **EPC** (Exception Program Counter) holds the address of the offending (or interrupted) instruction. After the handler finishes, execution can *restart* there (`eret`), or the OS can report the faulting address. Without it, the return address would be lost when the PC is loaded with the handler address `0x80000180`.
- **Cause** records *why* the exception happened (e.g. undefined instruction, arithmetic overflow, TLB miss, I/O interrupt), so that a *single* handler entry point can decode the reason and branch to the right routine. The alternative is a vectored scheme with a separate address per cause.

Imprecise exception. In a pipelined (or out-of-order) machine, an exception is *imprecise* when the saved state does not correspond to a single point in the program. Some instructions *after* the faulting one may already have changed registers or memory, or some *before* it may not have completed, so the EPC does not identify exactly where to restart. Precise exceptions (all earlier instructions complete, faulting and later ones have no effect) make restart easy, and MIPS provides them by flushing the pipeline.

10

Multiprocessors and Parallel Architectures

NOTE **Not in the course slides.** None of the ART/RS slide decks covers this material. The questions come from older CSE 305 papers, and the solutions follow standard textbook treatments (Patterson & Hennessy, *Computer Organization and Design*; Hamacher; Stallings; Mano).

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2022–2023 • Q4(c) **NOT IN SLIDES** CSE 305 • 25/04/2024 • MP01 **MARKS 7**

Briefly explain the following concepts:

(i) Shared Memory Multiprocessor (SMP)

(ii) SISD vs MIMD

2016–2017 • Q3(d) **NOT IN SLIDES** CSE 305 • 02/08/2017 • MP01 **MARKS 10**

Describe SISD, SIMD and MISD architectures with block diagrams.

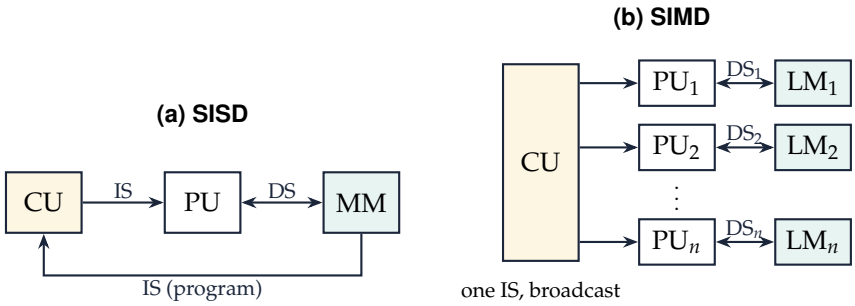
2014–2015 • Q1(a) **NOT IN SLIDES** CSE 305 • 26/01/2016 • MP01 **MARKS 10**

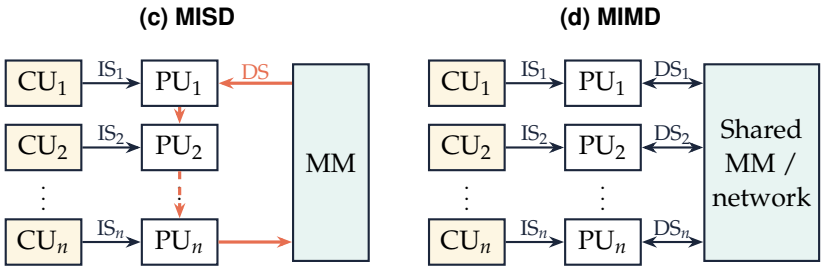
Describe SISD, SIMD, MIMD and MISD architectures with block diagrams.

SOLUTION

Flynn's taxonomy

Flynn (1966) classifies computers by how many **instruction streams** (IS) and how many **data streams** (DS) they handle at the same time. Notation in the diagrams: CU = control unit, PU = processing unit, MM = memory module, LM = local memory.

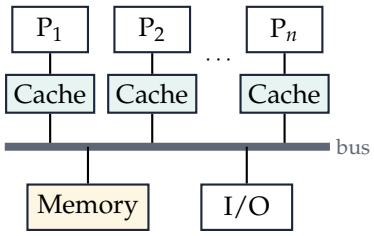




- **SISD** (single instruction, single data): one CU fetches one instruction stream and one PU operates on one data stream — the classical uniprocessor (von Neumann machine; pipelining/superscalar still count as SISD). Example: a single-core MIPS/Intel 8086.
- **SIMD** (single instruction, multiple data): one CU broadcasts each instruction to many PUs that work in lock-step, each on its own data element held in its local memory — data-level parallelism. Examples: array processors (ILLIAC IV), vector units (x86 SSE/AVX), GPU SIMD lanes.
- **MISD** (multiple instruction, single data): several CUs/PUs apply *different* instructions to the *same* data stream, which flows from PU to PU. No commercial machine fits exactly; closest are systolic arrays and redundant fault-tolerant flight computers (same data, different checks).
- **MIMD** (multiple instruction, multiple data): n independent processors, each with its own instruction stream and data stream, cooperating through shared memory or message passing — multicores, SMPs, clusters.

2022–2023 Q4(c)(i) • Shared Memory Multiprocessor (SMP)

An SMP is a MIMD machine in which all processors share **one physical address space**. Each core has private caches, and they all reach a common main memory and I/O through a bus or interconnection network. Processors communicate through ordinary `lw/sw` to shared variables. They coordinate with **locks/synchronization** (e.g. `ll/sc`), and the hardware keeps the caches coherent (snooping or a directory). It is *symmetric* because every processor has equal access to memory and I/O and any processor can run the OS. Memory access is UMA (uniform) or NUMA (non-uniform).



2022–2023 Q4(c)(ii) • SISD vs MIMD

	SISD	MIMD
Streams	1 instruction stream, 1 data stream	n instruction streams, n data streams
Hardware	one CU + one PU	n complete processors (CU+PU each)
Parallelism	only inside one stream (pipelining, ILP)	thread/process-level parallelism across processors
Communication	none needed	shared memory or message passing; needs synchronization & coherence
Example	single-core uniprocessor	multicore CPU, SMP server, cluster

2021–2022 • Q7(a) NOT IN SLIDES CSE 305 • 06/04/2023 • MP02 MARKS 5 + 5 + 5

Assume that you have four different threads as shown in figure (Figure: 7.a) below. Show how the four threads could be combined to execute on the processor more efficiently using the following three multithreading options. You only need to show the first 12 timeslots for each of the following options.

(i) A superscalar with coarse-grained multithreading

(ii) A superscalar with fine-grained multithreading

(iii) A superscalar with simultaneous multithreading

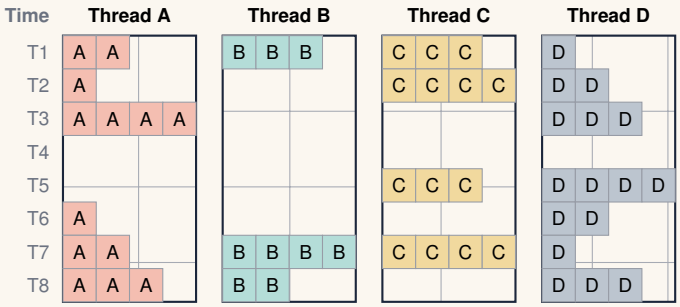


Figure 7.a

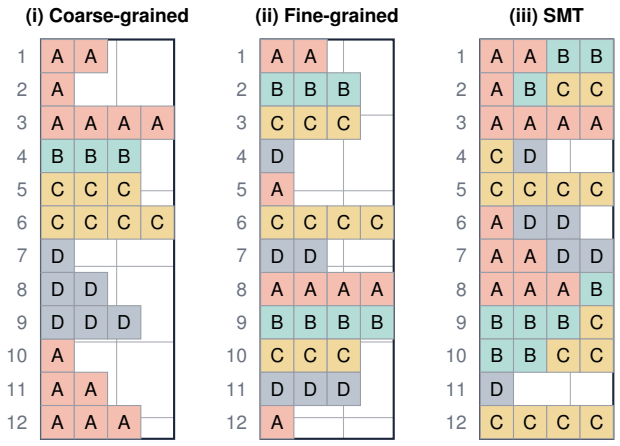
SOLUTION

How to read the figure, and the rules used

Each row is one clock cycle and the four columns are the four issue slots of a 4-wide superscalar. A thread's non-empty row is a group of independent instructions it can issue in that cycle. An **empty row is a stall**, e.g. waiting on a cache miss. Its latency is real: if a thread has g stall rows, it cannot issue its next group until $g+1$ cycles after the previous group finished. On a multithreaded machine those cycles can be given to other threads.

- **Coarse-grained**: one thread runs alone until it reaches a stall (an empty row), then the processor switches to the next ready thread (A→B→C→D→A). The switch is assumed free. Real machines lose a few cycles refilling the pipeline. Only one thread is in the issue slots in any cycle.
- **Fine-grained**: switch thread *every cycle* in round-robin order, skipping a thread that is still stalled. One thread per cycle.
- **SMT**: in every cycle, fill the 4 slots with ready instructions from *any* threads. Fixed priority A > B > C > D is used here. A group that does not fit is split, and its remainder issues next cycle. One thread's next group must wait until its current group has fully issued.

First 12 timeslots



- *Coarse*: A issues T1–T3 and then stalls at T4 ⇒ switch to B (cycle 4). B stalls after one row ⇒ C issues T1–T2 (cycles 5–6) ⇒ D issues T1–T3 (cycles 7–9). A's stall has long since cleared, so A resumes with T6–T8 (cycles 10–12). Then B, C and D continue.
- *Fine*: A, B, C and D take turns. B is skipped from cycle 5 on because its 5-cycle stall is not over. A's 2-cycle stall is hidden by the other threads.
- *SMT*: cycle 1 = A(2)+B(2 of 3). Cycle 2 = A(1)+B(rest)+C(2 of 3). Cycle 3 = A(4). Cycle 4 = C(rest)+D(1). And so on.

Answer. Instructions issued in the first 12 cycles (48 slots): coarse-grained 29, fine-grained 31, SMT 42. All 52 instructions finish in 20, 20 and 16 cycles respectively, so SMT is the most efficient.

2021–2022 • Q7(c) NOT IN SLIDES CSE 305 • 06/04/2023 • MP03 MARKS 10

With illustrative example(s), explain how vector processor differs from array processor.

SOLUTION

Both are SIMD machines: one instruction operates on many data elements. They differ in *how* the elements are processed.

- **Vector processor** — *temporal* parallelism. A few deeply **pipelined** functional units and **vector registers** (e.g. 64 elements × 64 bits). One vector instruction streams all the elements through the same pipelined unit, one new element per clock. Examples: Cray-1, NEC SX, VMIPS, RISC-V "V".
- **Array processor** — *spatial* parallelism. Many identical, simple **processing elements** (PEs), each with its own local memory, are driven in lock-step by one control unit. All PEs do the same operation on their own element *in the same clock*. Examples: ILLIAC IV (64 PEs), Connection Machine CM-2, the SIMD lanes of a GPU.

Example: $C[i] = A[i] + B[i], i = 0 \ldots 63$ **(DAXPY-like loop)**

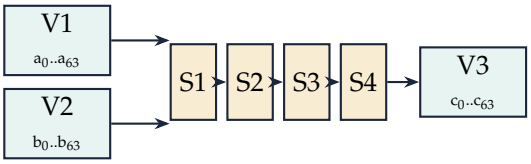
Vector (VMIPS):

```
lv      $v1, 0($s1)    # load 64 A's
lv      $v2, 0($s2)    # load 64 B's
addv.d  $v3,$v1,$v2    # one instr.
sv      $v3, 0($s3)    # store 64 C's
```

Time ≈ pipeline depth + 64 cycles. Only one adder is needed.

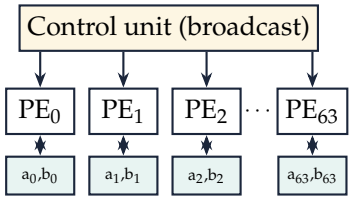
Array (64 PEs): CU broadcasts `LOAD A; LOAD B; ADD; STORE C`. PE_i holds $A[i]$, $B[i]$ in its local memory, and all 64 additions happen in *one* step. Time ≈ 1 add step, but 64 adders are needed.

Vector: one pipelined adder, elements in time



in clock t: S1 holds element t, S4 holds element $t-3$

Array: many PEs, elements in space



	Vector processor	Array processor
Parallelism	temporal (pipelining)	spatial (replicated PEs)
Hardware	few deep pipelined FUs + vector registers	many simple PEs + local memories + interconnect
Element timing	one element per clock through a unit	all elements in the same clock
Memory	shared, highly interleaved/banked memory	distributed local memory per PE
Cost / scaling	cheaper; scales with vector length	expensive; scales with number of PEs

2020–2021 • Q9 NOT IN SLIDES CSE 305 • 24/09/2022 • MP04 MARKS 15

You want to perform two sums in a program: one is a sum of 8 scalar variables and another is a matrix sum of a pair of two-dimensional arrays of size 8 by 8. Assume you are doing these sums with 8 processors.

(a) Assume only the matrix sum is parallelizable, what speed-up will you get with 8 versus 16 processors?

(b) Again assuming only the matrix sum is parallelizable, calculate the speed-up when the matrices grow to 16 by 16.

(c) Devise a recursive algorithm to speed-up the scalar sum and calculate the new speed-up.

SOLUTION

Model (Patterson & Hennessy strong-scaling example)

Let one addition take time t. The scalar sum is counted as 8 additions (each variable added into a running sum), and it is sequential. The matrix sum $C = A + B$ is one independent addition per element, so it divides perfectly among p processors.

$$T_1 = (8 + n^2)\,t, \qquad T_p = \left(8 + \frac{n^2}{p}\right)t, \qquad \text{Speed-up} = \frac{T_1}{T_p}.$$

(a) 8×8 matrices ($n^2 = 64$)

$T_1 = 8 + 64 = 72t$. $p = 8$: $T_8 = 8 + 64/8 = 16t \Rightarrow 72/16 = 4.5$. $p = 16$: $T_{16} = 8 + 64/16 = 12t \Rightarrow 72/12 = 6.0$.

Answer. Speed-up 4.5 with 8 processors (56% of the ideal 8) and 6.0 with 16 processors (only 37.5% of the ideal 16). Doubling the processors gains just 33%, because the serial scalar sum dominates (Amdahl's law).

(b) 16×16 matrices ($n^2 = 256$)

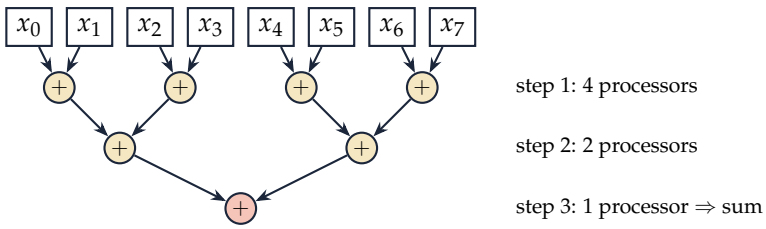
$T_1 = 8 + 256 = 264t$. $p = 8$: $T_8 = 8 + 32 = 40t \Rightarrow 264/40 = 6.6$. $p = 16$: $T_{16} = 8 + 16 = 24t \Rightarrow 264/24 = 11.0$.

Answer. Speed-up 6.6 (82.5% of ideal) with 8 processors and 11.0 (69% of ideal) with 16 processors. A bigger problem keeps the processors busier (strong scaling improves with problem size).

(c) Recursive (tree) reduction for the scalar sum

Split the set in halves, sum each half in parallel on different processors, then add the two partial sums. With s scalars this takes $\lceil \log_2 s \rceil$ addition steps instead of s.

```
int rsum(int x[], int lo, int hi) {   /* sums x[lo..hi-1] */
    if (hi - lo == 1) return x[lo];
    int mid = (lo + hi) / 2;
    int left, right;
    /* in parallel (different processors): */
    left  = rsum(x, lo, mid);
    right = rsum(x, mid, hi);
    return left + right;              /* one addition per level */
}
```



The scalar sum now takes $\log_2 8 = 3t$ instead of $8t$. The serial reference T_1 is unchanged.

	8×8 ($T_1 = 72t$)		16×16 ($T_1 = 264t$)	
	$p = 8$	$p = 16$	$p = 8$	$p = 16$
T_p (serial scalar sum)	$16t$	$12t$	$40t$	$24t$
Speed-up	4.5	6.0	6.6	11.0
T_p (tree scalar sum, $3t$)	$3 + 8 = 11t$	$3 + 4 = 7t$	$3 + 32 = 35t$	$3 + 16 = 19t$
New speed-up	**6.55**	**10.29**	7.54	13.89

Answer. With the recursive reduction, the original case (8 processors, 8×8) improves from 4.5 to $72/11 \approx 6.55$, and with 16 processors from 6.0 to $72/7 \approx 10.3$.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q4(d) **NOT IN SLIDES** CSE 305 • 02/08/2017 • MP05 **MARKS 4**

Consider a bus-based shared memory multiprocessor system. It is constructed using processors with speed of 10^6 fetches/sec. and a bus with a peak bandwidth of 10^5 fetches/sec. The caches are designed to support a hit rate of 90%.

(i) What is the maximum number of processors that can be supported by this system?

(ii) What hit rate is needed to support a 20-processor system?

2014–2015 • Q1(b) NOT IN SLIDES CSE 305 • 26/01/2016 • MP05 MARKS 4

Consider a bus-based shared memory multiprocessor system. It is constructed using processors with speed of 10^6 fetches/sec, and a bus with a peak bandwidth of 10^5 fetches/sec. The caches are designed to support a hit rate of 90%.

(i) What is the maximum number of processors that can be supported by this system?

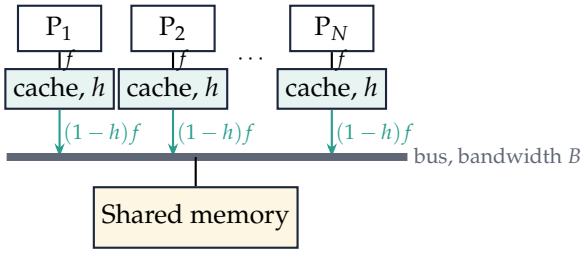
(ii) What hit rate is needed to support a 20-processor system?

SOLUTION

Idea

Only cache **misses** go out on the shared bus. A processor making f fetches/s with hit rate h puts $(1-h)f$ fetches/s on the bus. With N processors, the bus is not overloaded while

$$N(1-h)f \le B \quad \Longrightarrow \quad N_{\max} = \left\lfloor \frac{B}{(1-h)f} \right\rfloor .$$



(i) Here $f = 10^6$, $B = 10^5$, $h = 0.9$. Bus traffic per processor $= (1-0.9) \times 10^6 = 10^5$ fetches/s, so $N_{\max} = 10^5/10^5 = 1$.

Answer. Only **1 processor**: a single processor's misses already use the whole bus bandwidth.

(ii) For $N = 20$: allowed traffic per processor $= 10^5/20 = 5 \times 10^3$ fetches/s, so the miss rate must be $1 - h = 5 \times 10^3/10^6 = 0.005$.

Answer. Hit rate $h \ge 1 - 0.005 = 99.5\%$.

2015–2016 • Q7(a) NOT IN SLIDES CSE 305 • 18/01/2017 • MP06 MARKS 15

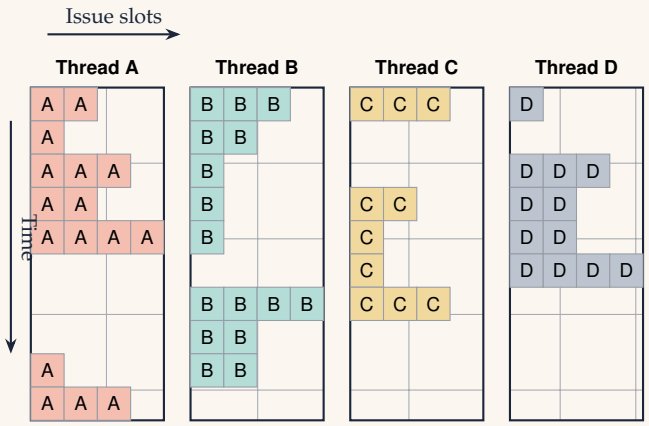
Assume that you have four different threads. Show how the four threads could be combined to execute on the processor more efficiently using following three multithreading options:

(i) A superscalar with coarse-grained multithreading

(ii) A superscalar with fine-grained multithreading

(iii) A superscalar with simultaneous multithreading

How the four threads would execute independently on a superscalar without any multithreading support is given below.



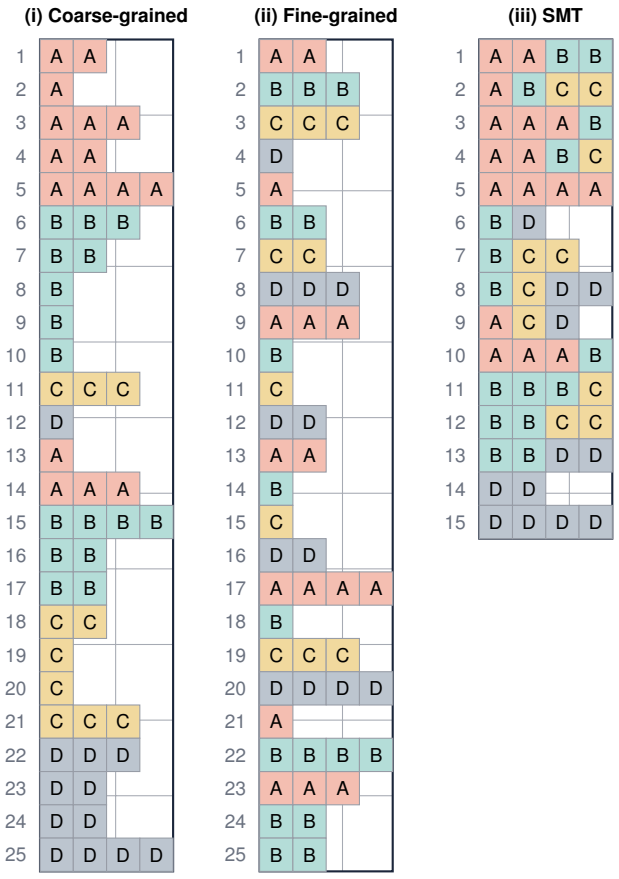
SOLUTION

How to read the figure, and the rules used

Each row is one clock cycle and the four columns are the four issue slots of a 4-wide superscalar. A thread's non-empty row is a group of independent instructions it can issue in that cycle. An **empty row is a stall**, e.g. waiting on a cache miss. Its latency is real: if a thread has g stall rows, it cannot issue its next group until $g + 1$ cycles after the previous group finished. On a multithreaded machine those cycles can be given to other threads.

- **Coarse-grained**: one thread runs alone until it reaches a stall (an empty row), then the processor switches to the next ready thread (A→B→C→D→A). The switch is assumed free. Real machines lose a few cycles refilling the pipeline. Only one thread is in the issue slots in any cycle.
- **Fine-grained**: switch thread *every cycle* in round-robin order, skipping a thread that is still stalled. One thread per cycle.
- **SMT**: in every cycle, fill the 4 slots with ready instructions from *any* threads. Fixed priority A > B > C > D is used here. A group that does not fit is split, and its remainder issues next cycle. One thread's next group must wait until its current group has fully issued.

Complete schedules



- *Coarse*: A rows 1–5, stall → B rows 1–5, stall → C row 1 → D row 1 → A rows 9–10 → B rows 7–9 → C rows 4–7 → D rows 3–6.
- *Fine*: round-robin A,B,C,D, one thread per cycle. The stalls (A: 3 cycles, B/C/D: ≤ 2 cycles) are completely hidden by the other threads. The removed empty rows are the gain.
- *SMT*: several threads share a cycle, so almost every slot is filled.

Answer. All 54 instructions finish in 25 cycles with coarse-grained (54% slot use), 25 cycles with fine-grained (54%) and 15 cycles with SMT (90%), against about 40 cycles (4×10 rows) if the threads ran one after another with no multithreading.

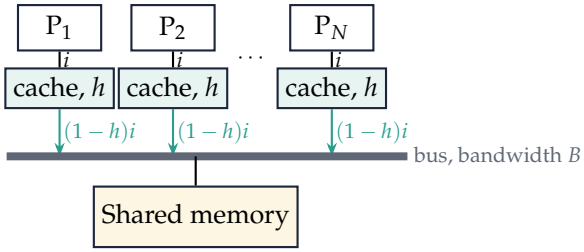
2015–2016 • Q7(b) NOT IN SLIDES CSE 305 • 18/01/2017 • MP07 MARKS 10

Find the maximum number of processors each having clock rate i with cache memories that a bus with Bandwidth B can support.

SOLUTION

Idea

On a bus-based shared-memory multiprocessor, cache hits are served locally. Only cache **misses** use the shared bus, so the bus saturates when the miss traffic of all processors equals its bandwidth.



- Assume each processor makes one memory fetch per clock, so it generates i fetches/s.
- With cache hit rate h (miss rate $1-h$), each processor needs $(1-h)\,i$ bus transfers per second.
- N processors need $N(1-h)\,i$ transfers/s. This must not exceed B: $N(1-h)\,i \le B$.

Answer. $N_{\max} = \left\lfloor \dfrac{B}{(1-h)\,i} \right\rfloor$ (with B in fetches/s). With no cache ($h = 0$) this is only B/i. The cache multiplies the number of processors by $1/(1-h)$, e.g. $10\times$ for $h = 0.9$.

Variants: if a processor makes r references per clock, replace i by $r\,i$. If B is in bytes/s and each miss moves a b-byte block, $N_{\max} = B/\big((1-h)\,i\,b\big)$. Example: $i = 10^6$, $B = 10^5$, $h = 0.9$ gives $N_{\max} = 1$.

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2015–2016 • Q8(b–c) NOT IN SLIDES CSE 305 • 18/01/2017 • MP08 **MARKS 20 + 5**

(b) Suppose we want to sum 64,000 numbers on a shared memory multiprocessor computer with uniform memory access time. Let's assume we have 64 (i) UMA processors, (ii) message passing processors. Write the corresponding C code. (20)

(a) for each processor to sum their subset of numbers

(b) to add the partial sums

(c) Draw the organization of a message passing multiprocessor. (5)

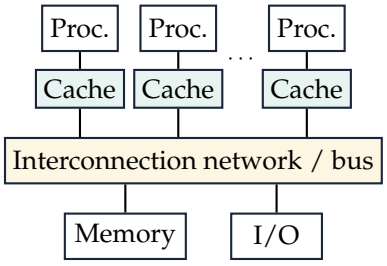
2011–2012 • Q8(b) NOT IN SLIDES CSE 305 • 27/07/2013 • MP08 **MARKS 6 + 6**

Briefly describe the 'shared memory multiprocessor' and the 'message passing multiprocessor' systems.

SOLUTION

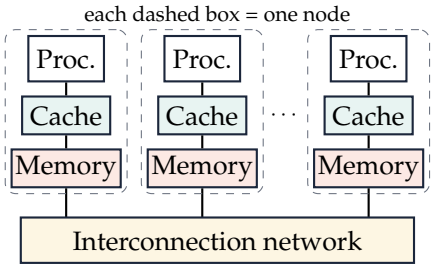
Shared memory multiprocessor (SMP) • 2011–2012 Q8(b)

- All processors share **one physical address space**. They communicate implicitly through shared variables with ordinary loads/stores.
- Every processor has private caches, so the hardware must keep them **coherent** (snooping/directory). Access to shared data is coordinated with **locks/barriers** (`ll`/`sc`).
- **UMA**: every word takes the same time to reach from any processor. **NUMA**: memory is physically distributed, so local accesses are faster.
- Easy to program, but limited scalability (bus/memory contention).



Message-passing multiprocessor • 2011–2012 Q8(b) and 2015–2016 Q8(c)

- Each node is a complete computer with its **own private memory** (separate address spaces). No processor can load another node's memory.
- Nodes communicate explicitly with **send()** and **receive()** messages over an interconnection network. A message also synchronizes the two nodes, so no coherence hardware is needed.
- Scales to thousands of nodes (clusters, warehouse-scale computers), but the programmer must partition the data and the messages cost time.



2015–2016 Q8(b)(i) • 64 UMA processors (shared memory)

`A[64000]` and `sum[64]` are shared. `Pn` $\in \{0..63\}$ is this processor's number, and every processor runs the same code. `synch()` is a barrier.

```
/* (a) each processor sums its own 1000 numbers */
sum[Pn] = 0;
for (i = 1000*Pn; i < 1000*(Pn+1); i += 1)
    sum[Pn] += A[i];

/* (b) add the 64 partial sums: tree reduction, log2(64) = 6 steps */
half = 64;
do {
    synch();                          /* wait for all partial sums  */
    if (half % 2 != 0 && Pn == 0)     /* odd count: P0 takes extra   */
        sum[0] += sum[half-1];
    half = half / 2;
    if (Pn < half) sum[Pn] += sum[Pn+half];
} while (half != 1);                  /* final total is in sum[0]   */
```

2015–2016 Q8(b)(ii) • 64 message-passing processors

The 64,000 numbers are first sent out, 1000 to each node's local array `AN[1000]`. There is no shared memory, so partial sums travel as messages.

```
/* (a) each node sums the numbers in its own memory */
sum = 0;
for (i = 0; i < 1000; i += 1)
    sum += AN[i];
```

```
/* (b) combine partial sums by sending them: halve the set each step */
limit = 64; half = 64;
do {
    half = (half + 1) / 2;            /* send / receive dividing line */
    if (Pn >= half && Pn < limit)
        send(Pn - half, sum);         /* upper half sends its sum     */
    if (Pn < (limit / 2))
        sum = sum + receive();        /* lower half accumulates       */
    limit = half;                     /* upper limit of senders       */
} while (half != 1);                  /* node 0 holds the total       */
```

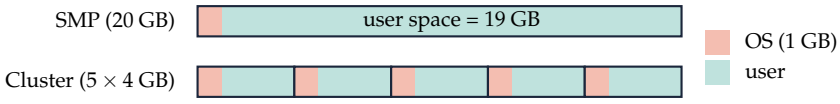
Answer. Both versions take 1000 additions per processor plus $\log_2 64 = 6$ reduction steps. The UMA version needs a barrier `synch()` at each step. In the message-passing version the `send`/`receive` pair itself provides the synchronization.

2013–2014 • Q2(a) NOT IN SLIDES CSE 305 • 05/07/2015 • MP09 MARKS 5

Suppose a single shared memory processor has 20 GB of main memory. Five clustered computers each have 4 GB. The OS occupies 1 GB. How much more space is there for users with shared memory?

SOLUTION

A shared-memory multiprocessor runs **one copy of the OS** for its whole 20 GB. A cluster of 5 computers runs **one OS copy on each** computer, inside that computer's own 4 GB.



SMP user space $= 20 - 1 = 19$ GB. Cluster user space $= 5 \times (4 - 1) = 15$ GB. Difference $= 19 - 15 = 4$ GB, ratio $19/15 \approx 1.27$.

Answer. The shared-memory system has **4 GB more** space for users (about 27% more).

2013–2014 • Q2(b–c) NOT IN SLIDES CSE 305 • 05/07/2015 • MP10 MARKS 8 + 5

(b) Determine the total network bandwidth and dissection bandwidth for ring and fully connected topologies. Assume that the number of processor is P. (8)
(c) Differentiate UMA shared memory multiprocessor from NUMA shared memory multiprocessor. (5)

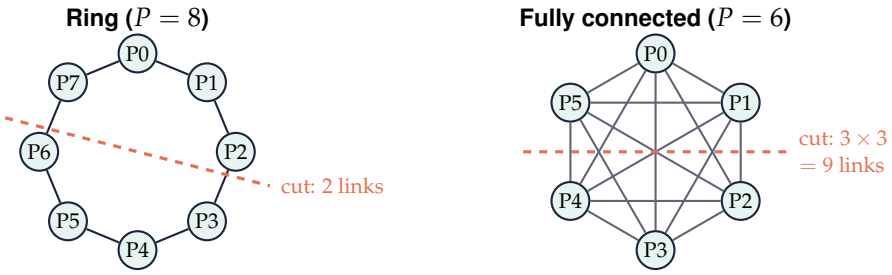
SOLUTION

NOTE "Dissection bandwidth" in the question means **bisection bandwidth**. Let every link have bandwidth b (in each direction), and let P be even.

(b) Definitions

- **Total network bandwidth** = sum of the bandwidths of all links (best case, every link busy at once).
- **Bisection bandwidth** = bandwidth of the links cut when the machine is split into two halves of

$P/2$ nodes, taking the worst (minimum) cut.



Ring. Each node links to its two neighbours, so there are P links. Total $= P\,b$. A cut through the ring splits it at two places, so bisection $= 2b$, which does not grow with P.
Fully connected. Every pair is linked: $\binom{P}{2} = P(P-1)/2$ links. Total $= \frac{P(P-1)}{2}\,b$. After the split, each of the $P/2$ nodes on one side has a link to each of the $P/2$ on the other: bisection $= (P/2)^2\,b$.

Answer. Ring: total $= P\,b$, bisection $= 2b$. Fully connected: total $= P(P-1)b/2$, bisection $= (P/2)^2\,b$. Example $P = 64$: ring $64b$ / $2b$, fully connected $2016b$ / $1024b$. The fully connected network performs far better, but its link count grows as P^2.

(c) UMA vs NUMA shared-memory multiprocessors

	UMA (uniform memory access)	NUMA (non-uniform memory access)
Access time	same for every word, from every processor	local memory fast, remote memory slower
Memory placement	central memory on a common bus/switch	memory physically distributed, one bank per node; still one address space
Scalability	small (a few to tens of processors); bus becomes a bottleneck	large (hundreds of processors)
Programming	simpler; no data-placement concern	OS/programmer should place data near its user for performance
Example	multicore chip / bus-based SMP	multi-socket servers, SGI Origin

2012–2013 • Q7(e) NOT IN SLIDES CSE 305 • 10/05/2014 • MP11 MARKS 4

Differentiate between fine-grain and coarse-grain multithreading.

SOLUTION

Both are forms of **hardware multithreading**: the processor keeps several thread contexts (PC, registers) and switches between them to hide stalls. Only one thread issues in any cycle.

	Fine-grained	Coarse-grained
When to switch	every clock cycle (interleaved, round-robin), skipping stalled threads	only on a costly stall (e.g. L2/last-level cache miss)
Stalls hidden	both short and long stalls	only long stalls; short stalls (data hazards, L1 miss) still waste cycles
Switch cost	must be zero ⇒ per-thread PC + register file, switch every cycle	pipeline must drain/refill on a switch (start-up cost)
Main drawback	slows each individual thread (a ready thread still waits for its turn)	lower throughput gain; startup overhead
Example	Sun Niagara (UltraSPARC T1)	IBM RS64-IV (Northstar)

2012–2013 • Q8(d) NOT IN SLIDES CSE 305 • 10/05/2014 • MP12 MARKS 10

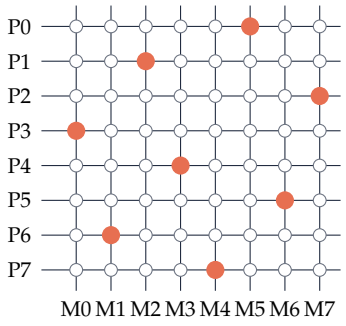
Discuss crossbar network and Omega network with diagrams.

SOLUTION

Both are **multistage/switched interconnection networks**. They connect n processors to n memories (or nodes) through switches, unlike a single shared bus.

Crossbar network

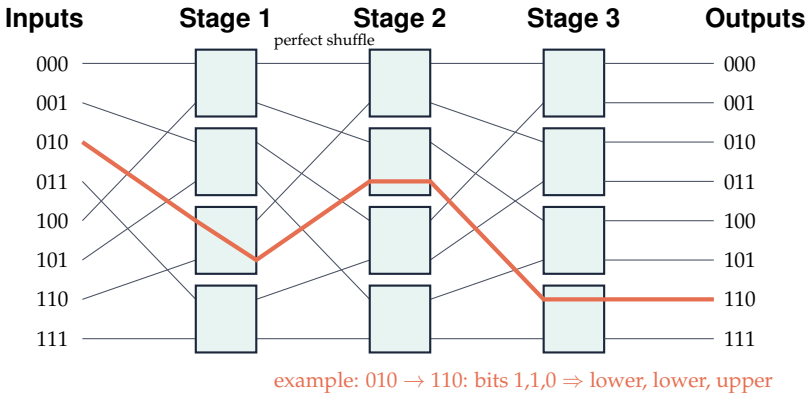
- A grid of n input (processor) lines and n output (memory) lines with a **crosspoint switch** at every intersection: n^2 switches.
- Closing switch (i, j) connects P_i to M_j. Any permutation can be set up at once, so the crossbar is **non-blocking**: every processor can talk to a different memory simultaneously in one hop.
- Conflicts arise only when two processors want the *same* output, which needs arbitration at that column.
- Cost grows as $O(n^2)$, so it is practical only for small n (e.g. Sun Niagara core-to-L2 crossbar).



8×8 crossbar; coral = closed switches (a full permutation set at once)

Omega network

- Built from $\log_2 n$ stages, each with $n/2$ 2×2 **switches**. Each switch can be set *straight*, *cross*, *upper broadcast* or *lower broadcast*.
- Before each stage the lines are wired by a **perfect shuffle**: line i goes to $(2i \bmod n) + \lfloor 2i/n \rfloor$, i.e. the binary address is rotated left one bit.
- **Destination-tag routing**: at stage k the switch looks at bit k (MSB first) of the destination address. A 0 sends the message out of the upper port, a 1 out of the lower port. No routing table is needed.
- Cost is $(n/2)\log_2 n$ switch boxes ($= 2n \log_2 n$ crosspoints) instead of n^2, but it is **blocking**: two messages may need the same internal link at the same time even when their destinations differ.



8×8 Omega network: 3 stages $\times$ 4 switches = 12 switches (a crossbar needs 64 crosspoints).

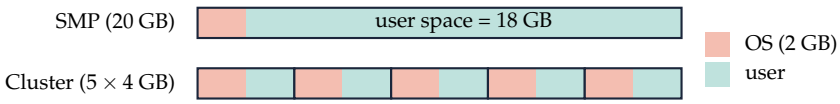
	Crossbar	Omega
Switches (n ports)	n^2 crosspoints	$(n/2)\log_2 n$ 2×2 boxes
Delay	1 switch	$\log_2 n$ switch stages
Blocking	non-blocking (any permutation)	blocking (some permutations conflict)
Use	small n, highest performance	large n, cheaper

2011–2012 • Q8(c) NOT IN SLIDES CSE 305 • 27/07/2013 • MP13 MARKS 8

Suppose a single shared memory processor has 20 GB of main memory. Five clustered computers occupy 4 GB each and the OS occupies 2 GB. How much more space is there for users with shared memory?

SOLUTION

A shared-memory multiprocessor runs **one copy of the OS** for its whole 20 GB. A cluster of 5 computers runs **one OS copy on each** computer, inside that computer's own 4 GB. Both systems have $5 \times 4 = 20$ GB in total, but the cluster pays the OS cost five times.



SMP user space $= 20 - 2 = 18$ GB. Cluster user space $= 5 \times (4 - 2) = 10$ GB. Difference $= 18 - 10 = 8$ GB, ratio $18/10 = 1.8$.

Answer. The shared-memory system has **8 GB more** space for users (80% more). The bigger the OS, the bigger the shared-memory advantage.

11

Memory Systems, Storage, I/O and Buses

NOTE **Not in the course slides.** None of the ART/RS slide decks covers this material. The questions come from older CSE 305 papers, and the solutions follow standard textbook treatments (Patterson & Hennessy, *Computer Organization and Design*; Hamacher; Stallings; Mano).

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2016–2017 • Q3(c) **NOT IN SLIDES** CSE 305 • 02/08/2017 • IO01 **MARKS 5**

The access time of the cache M_1 of a single cache memory system is 6 ns/bit during hit and that of the main memory M_2 is 900 ns/bit during miss. Calculate the block transfer time and the access efficiency of the memory system at the hit ratio of 0.8.

2014–2015 • Q4(d) **NOT IN SLIDES** CSE 305 • 26/01/2016 • IO01 **MARKS 5**

The access time of the cache M_1 of a single cache memory system is 6 ns/bit during hit and that of the main memory M_2 is 900 ns/bit during miss. Calculate the block transfer time and the access efficiency of the memory system at the hit ratio 0.8.

SOLUTION

NOTE The block size is not given, so every time is worked out *per bit*, as in the data. For a block of n bits multiply the block-transfer time by n. The two-level model used is the standard one (Hayes): on a miss the block is first copied from M_2 into M_1 and then read from M_1.

Two-level memory model

$t_{A1} = 6$ ns (hit, from M_1), $t_{A2} = 900$ ns (miss: M_2 access, which includes the block transfer), $H = 0.8$. On a miss, $t_{A2} = t_B + t_{A1}$, where t_B is the block-transfer time from M_2 to M_1.

Block transfer time: $t_B = t_{A2} - t_{A1} = 900 - 6 = 894$ ns per bit.

Average access time: $t_A = H\,t_{A1} + (1-H)\,t_{A2} = 0.8(6) + 0.2(900) = 4.8 + 180 = 184.8$ ns per bit.

Access efficiency: with $r = t_{A2}/t_{A1} = 900/6 = 150$,

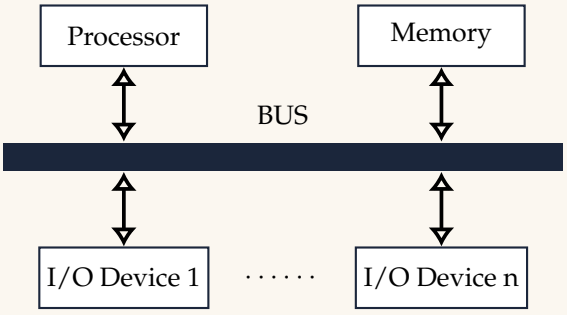
$$e = \frac{t_{A1}}{t_A} = \frac{1}{r + H(1-r)} = \frac{1}{150 + 0.8(1-150)} = \frac{1}{30.8} = \frac{6}{184.8} \approx 0.0325.$$

Answer. Block transfer time $t_B = 894$ ns/bit (894 n ns for an n-bit block). Average access time $= 184.8$ ns/bit. Access efficiency $e \approx 0.032$ (3.25%). Because $r = 150$ is so large, even an 80% hit ratio leaves the system about 30× slower than M_1. H must be close to 1 for good efficiency.

2016–2017 • Q8(b) **NOT IN SLIDES** CSE 305 • 02/08/2017 • IO02 **MARKS 10**

In the following figure, a single bus computer architecture was drawn where, various devices like processor, memory, and I/O devices are connected with each other via a single BUS. Please

note that, these devices do not operate at a similar rate. Then, how can you redesign this single bus computer architecture so that these devices can interact with each other smoothly using this single bus. Just draw your redesigned single bus computer architecture.

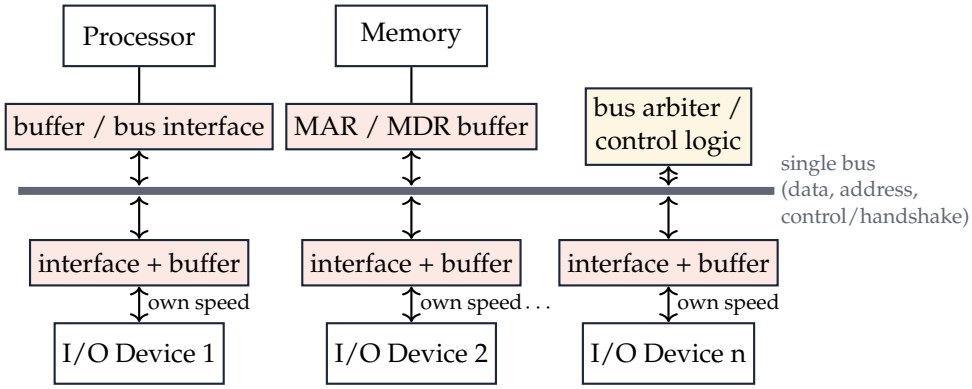


SOLUTION

Idea (Hamacher)

A slow device must not hold the bus for its whole, slow operation. The fix is to give every unit a **buffer register** inside its **interface/controller**, and to run transfers with **handshake control signals** on the bus.

- The processor (or memory) transfers a word to the device's buffer *at bus speed* and releases the bus right away. The slow device (printer, keyboard, disk) then works from its own buffer at its own rate, while the bus is free for other transfers.
- In the other direction, the device fills its buffer at its own speed and raises a *ready* / interrupt signal. The processor then reads the whole buffer in one fast bus transfer.
- Control lines (Read/Write, Ready/Acknowledge handshake, bus request/grant, interrupt) synchronize the units, and a **bus arbiter** decides who uses the bus. Memory may also have a data/address buffer, so the fast processor is not kept waiting.



Answer. Keep the single bus, but connect every device through an interface circuit that holds a **buffer register**, and run all transfers with handshake/control lines and a bus arbiter. Each transfer occupies the bus only for one fast buffer-to-buffer transfer. The slow device then completes the operation from its buffer, so devices of very different speeds share the bus smoothly.

2014–2015 • Q2(b) NOT IN SLIDES CSE 305 • 26/01/2016 • IO03 MARKS 4 + 4

Write short notes on (i) flash storage and (ii) holographic data storage.

SOLUTION

(i) Flash storage

- A **non-volatile** semiconductor memory, a kind of EEPROM. Each cell is a MOS transistor with a *floating gate* that traps charge. Trapped charge (or its absence) stores the bit, and the data stays without power.
- Erasure is done in **blocks** (tens to hundreds of KB). Writes go to erased pages. Each block survives only about 10^3–10^5 program/erase cycles, so controllers use **wear leveling** (remap writes across blocks) and spare blocks.
- **NOR flash**: random access to bytes, suits code storage (BIOS). **NAND flash**: denser and cheaper, accessed in pages, used in SSDs, USB drives, SD cards and phones.
- Compared with disk: about 100–1000× faster (no seek or rotation), shock-proof and low power, but costs more per GB. Compared with DRAM it is slower, especially for writes.

(ii) Holographic data storage

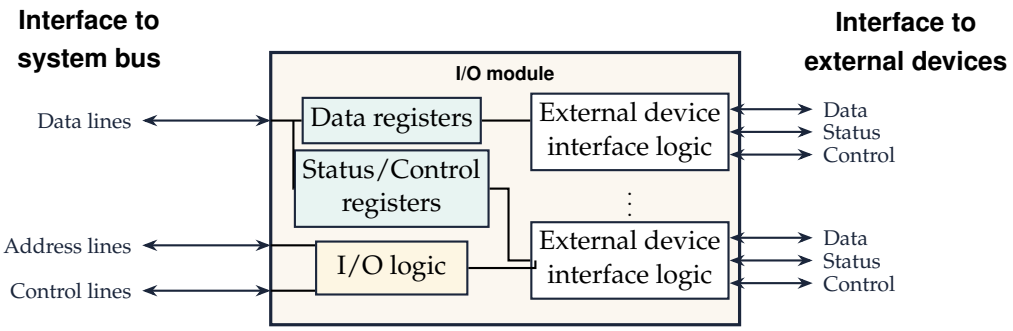
- Stores data as **holograms in the whole volume** of a photosensitive crystal or polymer, not just on its surface, so it is a 3-D optical storage.
- A laser is split into a *signal beam*, which passes through a spatial light modulator that encodes a page of about 10^6 bits as a pattern of dark and light pixels, and a *reference beam*. Their interference pattern is recorded in the medium.
- Lighting the medium with the same reference beam reconstructs the whole page onto a camera (CCD) detector. A full page is read in parallel, giving a very high transfer rate.
- Many pages share the same volume by changing the reference beam's angle or wavelength (**multiplexing**), so densities can reach terabytes per disc.
- Status: mostly experimental (e.g. InPhase Tapestry); suited to long-life archival storage.

2014–2015 • Q4(a–c) NOT IN SLIDES CSE 305 • 26/01/2016 • IO04 MARKS 8 + 10 + 12

(a) Show the components of an I/O module using a block diagram and describe the functions of each component. (8)
(b) Describe the operation of a vector interrupt structure with daisy chain interface for detecting and managing interrupting I/O devices. (10)
(c) Mention the advantages of DMA transfer over programmed I/O and interrupt-driven I/O. Briefly describe a DMA transfer structure with necessary diagrams. (4+8)

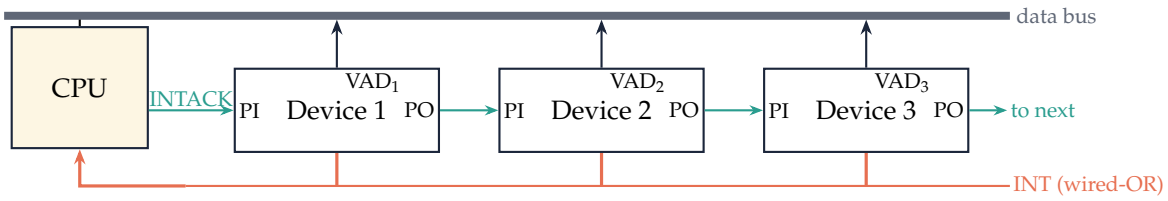
SOLUTION

(a) I/O module (Stallings)



- **Data registers**: buffer the data passing between the bus and the device. They hide the speed difference between the fast bus and the slow device.
- **Status/Control registers**: the processor writes commands (read, write, seek, reset) into the control register and reads the device state (ready, busy, error) from the status register.
- **I/O logic**: decodes the address lines to recognise this module's addresses, decodes the control lines and commands, and generates timing and interrupt signals.
- **External device interface logic**: one per attached device. It exchanges data, status and control signals with the device in that device's own format (e.g. serial/parallel conversion).
- Overall functions: control & timing, processor communication (command decoding, data, status reporting, address recognition), device communication, data buffering and error detection.

(b) Vectored interrupt with daisy chain (Mano)



(i) All devices share one interrupt request line, **INT** (wired-OR). Any device needing service pulls it active.

(ii) The CPU finishes the current instruction. If interrupts are enabled, it answers with **INTACK**. That signal does not go to all devices at once: it enters PI of the first device and is passed along the chain from PO to the next PI.

(iii) A device that receives PI = 1 and has *no* pending request passes it on (PO = 1). The first device that has a pending request blocks it (PO = 0), so devices further along see nothing. That device places its **vector address** VAD on the data bus.

(iv) The CPU uses VAD to jump directly to that device's interrupt service routine, either as the ISR address or an index into the interrupt vector table. No polling is needed.

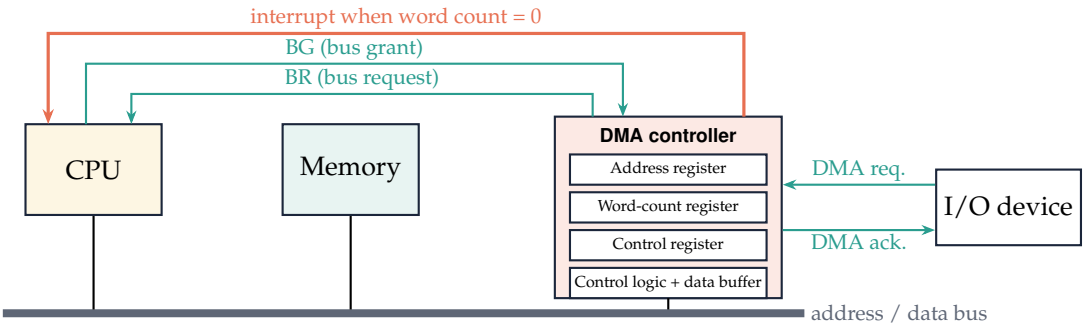
(v) **Priority** is fixed by position: the device nearest the CPU has the highest priority. Several simultaneous requests are therefore resolved automatically, one at a time.

(c) DMA: advantages

In programmed I/O the CPU busy-waits and moves every word itself. In interrupt-driven I/O it no longer waits, but it still executes an ISR and moves every word. With **DMA**, a controller moves an entire block directly between device and memory, and the CPU only sets up the transfer

and handles *one* interrupt at the end. This frees the CPU for other work, makes high-speed block transfers (disk, network) possible, and cuts the overhead from one per word to one per block.

(c) DMA transfer structure



- **Address register**: the memory address of the next word, incremented after each transfer. **Word-count register**: the number of words left, decremented to 0. **Control register**: the transfer direction (read/write) and mode. **Control logic**: drives RD/WR, BR/BG and the interrupt.
- **Operation**: the CPU initialises the registers through the bus. When the device is ready it asserts *DMA request*. The DMA controller raises **BR**. The CPU floats its buses and answers **BG**. The DMA controller then puts the address on the bus, sends *DMA ack* to the device, and the word moves directly between device and memory. This repeats, word by word in *cycle stealing* or the whole block in *burst* mode, until the word count reaches 0. Then the DMA controller drops BR and **interrupts** the CPU.

2013–2014 • Q4(a–d) NOT IN SLIDES CSE 305 • 05/07/2015 • IO05 MARKS 5 + 12 + 6 + 12

(a) Why does RAID 0 improve disk performance? (5)
(b) Write down the steps of handling an interrupt. (12)
(c) What happens if the interrupt enable bit of the Cause register is not set when handling an interrupt? What value could the interrupt mask value take to accomplish the same thing? (3+3)
(d) What is "DMA stale data problem"? How can it occur in a system with caches? (12)

SOLUTION

(a) RAID 0 (striping, no redundancy)

RAID 0 spreads consecutive **strips** of a file round-robin across N disks.

- A large request is served by all N disks **in parallel**, so the transfer rate is about $N\times$ that of one disk.
- Independent small requests that fall on different disks are served *at the same time*, which raises I/O throughput and spreads the load.
- No parity is computed or written, so there is no write penalty and all capacity holds data. The drawback is that it gives *no* fault tolerance: one disk failure loses the array.



(b) Steps of handling an interrupt

(i) A device raises an **interrupt request**, and the matching pending bit (MIPS: Cause.IP) is set.

(ii) The processor **finishes the current instruction**. It then checks that interrupts are enabled (Status.IE = 1) and that this level is not masked.

(iii) Hardware saves the return address in **EPC**, records the reason in the **Cause** register, switches to **kernel mode** and **disables further interrupts** (MIPS sets Status.EXL).

(iv) PC is loaded with the handler address (MIPS `0x80000180`, or the device's vector).

(v) The handler **saves registers** it will use (and EPC/Status, if nested interrupts are to be allowed).

(vi) It **identifies the source** from Cause (or by polling/vector) and may re-enable higher-priority interrupts.

(vii) It **services the device**: transfers the data, clears the device's request and updates OS data structures.

(viii) It **restores the saved registers**.

(ix) It **returns** (`eret`): PC ← EPC, user mode and interrupt enable restored. The interrupted program then continues as if nothing happened.

(c) Interrupt enable bit not set

> **NOTE** In MIPS the interrupt-enable bit (IE, bit 0) and the interrupt mask (bits 15–8) are in the **Status** register. Cause holds only the pending bits. "Cause" in the question is taken to mean Status.

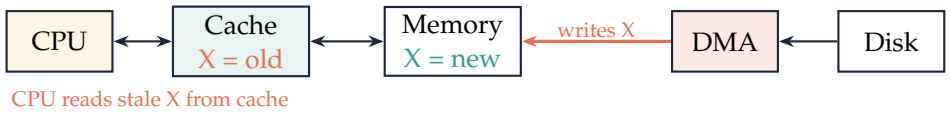
If IE = 0, interrupts are **disabled**. A requesting device's pending bit is still set in Cause, but the processor *ignores* the request, so no exception is taken. The request waits until software sets IE = 1 again. This is exactly what a handler wants while it saves state, because it prevents a second interrupt from overwriting EPC/Cause.

Answer. The same effect is obtained with an interrupt mask of all zeros: Status[15:8] = 00000000_2, i.e. every interrupt level is masked off and no interrupt can be taken.

(d) DMA stale-data problem

DMA moves data between a device and **main memory** without passing through the processor's cache. Memory and cache can then hold different values for the same address, and someone uses the out-of-date (**stale**) copy:

- **DMA input (device → memory)**: the DMA controller writes new data into memory, but the cache still holds the *old* copy of those blocks. The processor reads the cache and gets stale data.
- **DMA output (memory → device), write-back cache**: the processor's latest writes are still *dirty* in the cache and not yet in memory. DMA reads memory and sends stale data to the device.



Solutions: (1) route all I/O through the cache (simple, but it pollutes the cache); (2) have the OS **invalidate** the cache blocks of an input buffer, and **flush** (write back) the dirty blocks of an output buffer, before DMA starts; (3) hardware **snooping**: the cache watches DMA bus writes and invalidates or updates its copies, the same mechanism as multiprocessor coherence. Pages being used for DMA must also stay pinned in physical memory (not paged out) during the transfer.

2012–2013 • Q5(b) NOT IN SLIDES CSE 305 • 10/05/2014 • IO06 MARKS 2 + 18

What is the purpose of using RAID organization? Briefly describe the concept of different RAID levels with diagrams.

SOLUTION

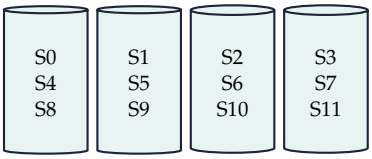
Purpose

RAID (*Redundant Array of Inexpensive/Independent Disks*) groups many small disks so they act as one logical disk, in order to get:

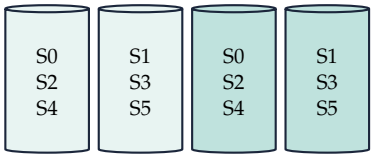
- **performance**: data is striped across disks so they work in parallel, giving a higher transfer rate and more I/Os per second;
- **dependability**: redundant information (mirror or parity) lets the array keep working, and rebuild the data, when a disk fails. Without it, many disks would fail far more often than one.

RAID levels (S = data strip/block, b = bit/byte, P = parity, Q = second check)

RAID 0: striping

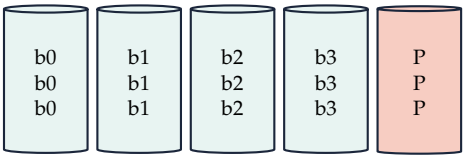


RAID 1: mirroring



mirror copies

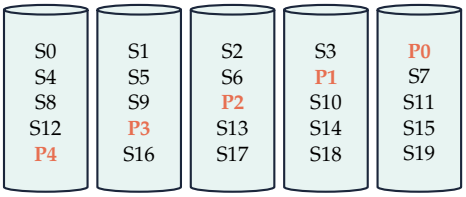
RAID 3: bit/byte-interleaved parity



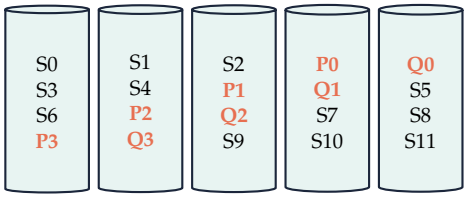
RAID 4: block-interleaved parity



RAID 5: distributed parity



RAID 6: P + Q (dual) parity



Level	Concept	Check disks (N data)
RAID 0	striping only; best performance and capacity; no redundancy, so not really "R"AID	0
RAID 1	mirroring: every write goes to two disks; reads served by either; survives one failure; 100% overhead	N
RAID 2	bit-striped with Hamming ECC disks (corrects single-bit errors); obsolete	$\approx \log_2 N$
RAID 3	bit/byte-interleaved with a single parity disk $P = b_0 \oplus b_1 \oplus \cdots$; every access uses all disks, so high bandwidth for large transfers but one I/O at a time	1
RAID 4	block-interleaved with a dedicated parity disk. Small reads go to one disk in parallel. A small write is a read-modify-write: $P_{\text{new}} = P_{\text{old}} \oplus D_{\text{old}} \oplus D_{\text{new}}$ (4 I/Os). The parity disk becomes the bottleneck	1
RAID 5	like RAID 4, but the parity rotates over all disks. Removes the parity bottleneck, so several small writes can proceed in parallel. The most widely used level	1
RAID 6	two independent check blocks (P + Q, e.g. Reed–Solomon) per stripe; survives *two* disk failures	2

Recovery with parity: when one disk fails, each of its blocks is rebuilt as the XOR of the surviving blocks of that stripe. A failed disk's position is known, so a single parity disk is enough to correct (not just detect) the error.

2012–2013 • Q8(c) NOT IN SLIDES CSE 305 • 10/05/2014 • IO07 MARKS 8

Describe the steps of DMA transfer.

SOLUTION

A DMA controller moves a whole block between an I/O device and memory without the processor touching each word. The block diagram is the one in IO04(c): CPU, memory and DMA controller on the bus, with the BR/BG lines and an interrupt line.

(i) **Set-up by the CPU** (through ordinary I/O writes to the DMA registers): the starting memory address goes into the address register, the number of words into the word-count register, and the direction (read/write) and mode into the control register. Then the device is told to start. The CPU goes back to other work.

(ii) **Device ready**: when the device has a word or block ready (or can accept one), it asserts *DMA request*.

(iii) **Bus request**: the DMA controller raises **BR** (bus request) to the CPU.

(iv) **Bus grant**: at the end of the current bus cycle the CPU puts its address, data and control lines in the high-impedance state and asserts **BG**. The DMA controller is now bus master.

(v) **Transfer**: the DMA controller places the address register's value on the address bus, issues RD/WR, and sends *DMA acknowledge* to the device. The word moves directly between device and memory.

(vi) **Update**: the address register is incremented and the word count decremented. In *cycle-stealing* mode the bus is released after each word, back to step 2. In *burst* mode the transfer continues until the block is done.

(vii) **Completion**: when the word count reaches 0, the controller drops BR, returns the bus to the CPU and sends an **interrupt**. The CPU's handler checks the status register for errors and starts the next transfer if needed.

Answer. CPU initialises address, count and direction → device requests → DMA asks for the bus (BR) → CPU grants it (BG) → words move directly device↔memory, updating address and count → count = 0 → bus released, CPU interrupted. The CPU is involved only at the start and the end.

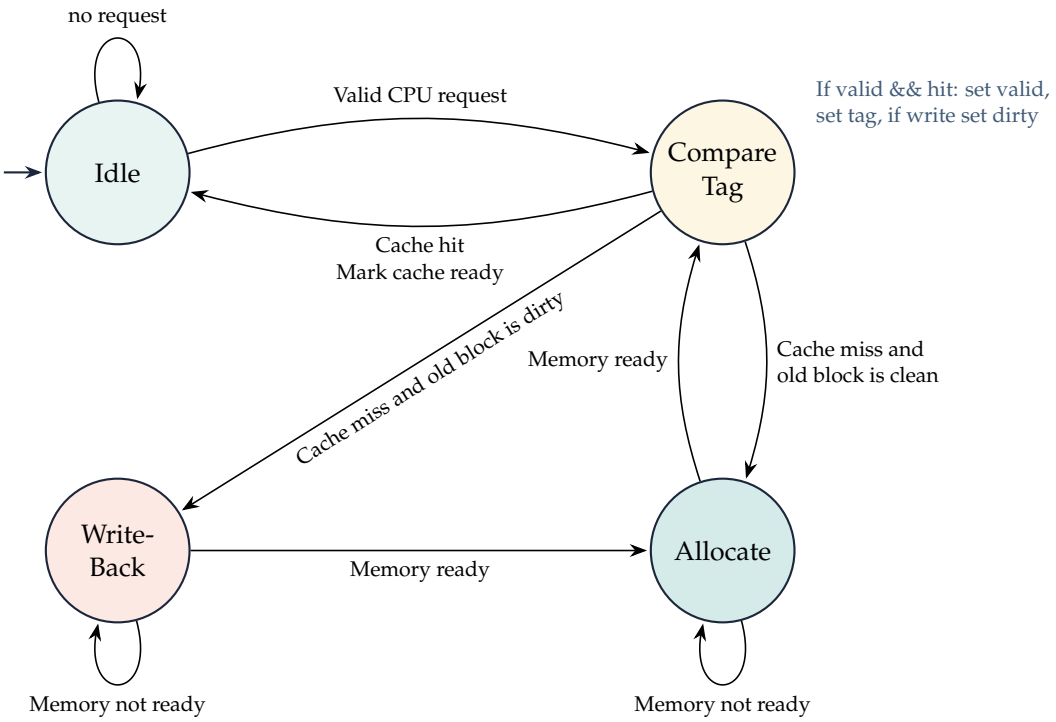
2011–2012 • Q7(b) NOT IN SLIDES CSE 305 • 27/07/2013 • IO08 MARKS 10

Draw the state diagram of a simple cache controller.

SOLUTION

Assumed cache (Patterson & Hennessy)

Direct-mapped, write-back, write-allocate cache with blocking requests and one valid and one dirty bit per block. The controller is a 4-state finite-state machine:



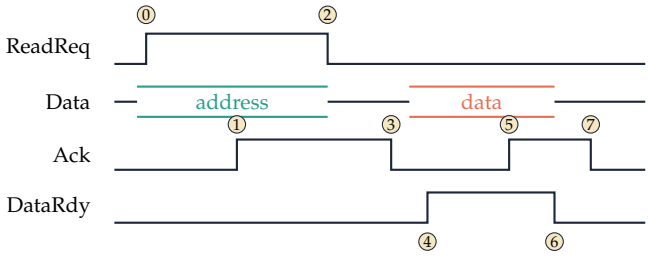
- **Idle**: wait for a valid read or write request from the processor, then go to Compare Tag.
- **Compare Tag**: index the cache and compare the tag (with the valid bit). *Hit*: read or write the word (a write sets the dirty bit), signal *cache ready* to the processor, and return to Idle. *Miss*: update the tag. If the victim block is *dirty*, go to Write-Back. If it is *clean*, go to Allocate.
- **Write-Back**: write the old dirty block to memory, using the old tag to form the address. Stay here until memory is ready, then go to Allocate.
- **Allocate**: read the new block from memory into the cache. Stay here until memory is ready, then go back to Compare Tag, which now hits and completes the request.

2011–2012 • Q7(c) NOT IN SLIDES CSE 305 • 27/07/2013 • IO09 MARKS 5

What are the ‘synchronous’ and the ‘asynchronous’ bus system?

SOLUTION

- **Synchronous bus**: the control lines include a **clock**, and every transfer follows a fixed protocol counted in clock cycles. For example, "address in cycle 1, data valid in cycle 5". The logic is simple and fast, but every device must run at the clock rate, and clock skew limits the bus length and speed. Typical use: processor–memory bus.
- **Asynchronous bus**: there is no clock. Each transfer is coordinated by a **handshaking protocol** on control lines, e.g. *ReadReq*, *DataRdy*, *Ack*. It suits devices of any speed and any bus length, but it is slower and needs more control logic. Typical use: I/O buses (e.g. USB-style, older SCSI).



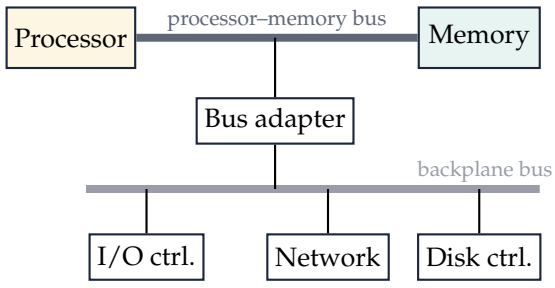
Asynchronous read handshake between an I/O device and memory (P&H). (0) The device puts the address on the bus and raises ReadReq. (1) Memory latches the address and raises Ack. (2) The device sees Ack and drops ReadReq and the address. (3) Memory sees ReadReq low and drops Ack. (4) Memory puts the data on the bus and raises DataRdy. (5) The device reads the data and raises Ack. (6) Memory drops DataRdy and releases the data lines. (7) The device drops Ack, which ends the transfer.

2011–2012 • Q8(d) NOT IN SLIDES CSE 305 • 27/07/2013 • IO10 MARKS 5

What are processor memory bus and back-plane bus?

SOLUTION

- **Processor–memory bus**: a short, high-speed bus that connects only the processor and main memory. It is usually synchronous and proprietary (designed together with the processor), and its length and speed are matched to the memory system, which maximises processor–memory bandwidth. Example: the old front-side bus.
- **Backplane bus**: built into the backplane (the interconnect board) of the chassis, so that processor, memory *and* I/O devices can all plug into one bus. It is usually a standard bus (e.g. PCI/PCIe-style) so third-party cards can be added. It is longer and slower, and has to serve devices of different speeds. In a hierarchy it is attached to the processor–memory bus through a **bus adapter**, and I/O buses hang off it.



12

Performance Benchmarks: SPEC and FLOPS

NOTE **Not in the course slides.** None of the ART/RS slide decks covers this material. The questions come from older CSE 305 papers, and the solutions follow standard textbook treatments (Patterson & Hennessy, *Computer Organization and Design*; Hamacher; Stallings; Mano).

2021–2022 • Q7(b) NOT IN SLIDES CSE 305 • 06/04/2023 • BM01 MARKS 10

Suppose you are using Floating Point Operations Per Second (FLOPS) as a performance metric for comparing processors. Your research partner believes that FLOPS might be a misleading metric for measuring the general performance of a processor, but it could still be a useful performance measure for a specific part of the processor or a specific type of processors. Do you agree or disagree with your research partner's statements? Provide a detailed explanation to support your position.

SOLUTION

Agree with both parts of the statement.

Why FLOPS misleads as a general metric

$\text{FLOPS} = \dfrac{\text{number of floating-point operations}}{\text{execution time}}$ (usually quoted as MFLOPS/GFLOPS/TFLOPS).

- **Ignores non-FP work.** Typical general workloads (compilers, databases, OS, web servers) are dominated by integer, load/store and branch instructions. A machine with a huge FLOPS rating can still be slow on them.
- **Not all FP operations are equal.** An add and a divide or square root count as "one FLOP" each, although they take very different times. FP instruction sets also differ between machines (e.g. FMA counts as 2 FLOPs).
- **Peak vs. sustained.** Vendors quote peak FLOPS (cores $\times$ FP units $\times$ clock), which real programs rarely reach. Memory bandwidth, cache misses and branch stalls dominate instead.
- **Program dependent.** Like MIPS, the FLOPS value changes with the program and compiler, so one machine has no single FLOPS value. Execution time on real benchmarks (e.g. SPEC) is the reliable measure.

Why FLOPS is still useful in specific cases

- **For a specific part of the processor**: it characterises the floating-point unit or SIMD/vector units directly, e.g. to compare FPU designs or to judge how well a kernel uses the FPU (sustained/peak FLOPS).
- **For specific types of processors and workloads**: GPUs, vector processors and supercomputers run FP-dominated scientific, graphics and ML codes (matrix multiply, FFT, simulation). There, FLOPS tracks real performance, which is why the TOP500 ranks systems by LINPACK FLOPS.

Answer. Agree. FLOPS ignores integer, memory and control work and counts unequal operations equally, so it is a poor *general* metric. For FP-dominated hardware (FPU/SIMD units, GPUs, HPC systems) and workloads, it is a meaningful measure of capability.

2020–2021 • Q11(a) NOT IN SLIDES CSE 305 • 24/09/2022 • BM02 MARKS 10

Suppose a hypothetical performance benchmark SPECBM101 runs on a computing system and the following data is obtained.

Description	Execution Time (seconds)	Reference Time (seconds)
Program 1	?	9770
Program 2	527	10490
Program 3	586	12100
Program 4	109	20720
Program 5	470	7020
Program 6	275	6900

If the geometric mean of the benchmark is 28.74 seconds, what is the execution time of Program 1?

SOLUTION

SPEC method

For each program, $\text{SPECratio}_i = \dfrac{\text{Reference time}_i}{\text{Execution time}_i}$ (higher is better; the ratio has no unit). The summary number is the **geometric mean** of the n ratios, $\text{GM} = \sqrt[n]{\prod_{i=1}^{n} \text{SPECratio}_i}$. So $r_1 = \text{GM}^n \big/ \prod_{i \neq 1} r_i$.

NOTE The "geometric mean … 28.74 seconds" is a geometric mean of SPEC *ratios*, so it has no unit. The scan also shows a handwritten "507.63" in the Program 1 cell. It is a student's mark, not given data, so it is recomputed below (it turns out to be correct).

Ratios of programs 2–6:

Program	2	3	4	5	6
Ratio	$\frac{10490}{527} = 19.905$	$\frac{12100}{586} = 20.648$	$\frac{20720}{109} = 190.092$	$\frac{7020}{470} = 14.936$	$\frac{6900}{275} = 25.091$

$$\prod_{i=2}^{6} r_i = 2.928 \times 10^7, \qquad 28.74^6 = 5.635 \times 10^8$$

$$r_1 = \frac{28.74^6}{\prod_{i=2}^{6} r_i} = \frac{5.635 \times 10^8}{2.928 \times 10^7} = 19.25, \qquad t_1 = \frac{\text{Ref}_1}{r_1} = \frac{9770}{19.25} = 507.6 \text{ s}.$$

Answer. Execution time of Program 1 ≈ **507.6** s (SPECratio ≈ 19.25).

2016–2017 • Q5(b) NOT IN SLIDES CSE 305 • 02/08/2017 • BM03 MARKS 10

Lenovo x3650 M5 server containing a 32 cores (Intel Xeon E5-2699v4) obtained following results in SPEC test. Find the spec ratio for the computer.

Program Name	Execution Time	Reference Time	Spec Ratios
perlbench	236	9770	
bzip2	375	9650	
gcc	208	8050	
mcf	133	9120	
gobmk	337	10490	
hmmer	105	9330	
sjeng	340	12100	
libquantum	2	20720	
h264ref	379	22130	
omnetpp	119	6250	
astar	195	7020	
xalancbmk	86	6900	
Spec ratui (SpecInt2006)	–	–	

SOLUTION

SPEC method

For each program, $\text{SPECratio}_i = \dfrac{\text{Reference time}_i}{\text{Execution time}_i}$ (higher is better; the ratio has no unit). The single summary number is the **geometric mean** of the n ratios:

$$\text{GM} = \sqrt[n]{\prod_{i=1}^{n} \text{SPECratio}_i}.$$

The geometric mean is used because it is independent of which reference machine is chosen: the ratio of two machines' GMs equals the GM of their time ratios.

Lenovo x3650 M5, SPECint2006

Program	Exec. time	Ref. time	SPECratio
perlbench	236	9770	41.40
bzip2	375	9650	25.73
gcc	208	8050	38.70
mcf	133	9120	68.57
gobmk	337	10490	31.13
hmmer	105	9330	88.86
sjeng	340	12100	35.59
libquantum	2	20720	10360.00
h264ref	379	22130	58.39
omnetpp	119	6250	52.52
astar	195	7020	36.00
xalancbmk	86	6900	80.23
SPECint2006 $= \sqrt[12]{\prod r_i}$			**73.67**

Using logs: $\frac{1}{12}\sum \ln r_i = \frac{1}{12}(51.594) = 4.2995$, so GM $= e^{4.2995} \approx 73.7$.

Answer. SPECint2006 ratio of the Lenovo x3650 M5 $\approx$ **73.7**. The libquantum outlier (10360) pulls the geometric mean up much less than it would pull an arithmetic mean (which would be ≈ 910).